マルチクラウド自動デプロイプラットフォーム

完全版ソースコード解説書（ソースコード全文付き）

PLAYER1-r7

2026年2月28日

Contents

1 プロジェクト概要 **3**

2 ソース実装解説書 **13**

3 インフラ・CI/CD解説書 **23**

4 バックエンドAPI ソースコード **30**

4.1 main.py - FastAPIアプリケーション . . . 30
4.2 config.py - 環境設定 . . . 36
4.3 models.py - データモデル . . . 39
4.4 auth.py - 認証ミドルウェア . . . 43
4.5 jwt_verifier.py - JWT検証 . . . 45
4.6 backends/base.py - 抽象基底クラス . . . 49
4.7 backends/aws_backend.py - AWS実装 . . . 52
4.8 backends/azure_backend.py - Azure実装 . . . 58
4.9 backends/gcp_backend.py - GCP実装 . . . 66
4.10 backends/local_backend.py - ローカル実装 . . . 72
4.11 routes/posts.py - 投稿API . . . 80
4.12 routes/profile.py - プロフィールAPI . . . 82
4.13 routes/uploads.py - アップロードAPI . . . 82
4.14 index.py - AWS Lambda ハンドラー . . . 83
4.15 function_app.py - Azure Functions ハンドラー . . . 83
4.16 function.py - GCP Cloud Run ハンドラー . . . 85

5 インフラストラクチャコード (Pulumi) **88**

5.1 AWS Pulumi **main**.py . . . 88
5.2 AWS Pulumi.staging.yaml . . . 105
5.3 Azure Pulumi **main**.py . . . 106
5.4 Azure Pulumi.staging.yaml . . . 120
5.5 GCP Pulumi **main**.py . . . 120
5.6 GCP Pulumi.staging.yaml . . . 131

6 CI/CDワークフロー (GitHub Actions) **132**

6.1 deploy-aws.yml - AWS デプロイ . . . 132
6.2 deploy-azure.yml - Azure デプロイ . . . 142
6.3 deploy-gcp.yml - GCP デプロイ . . . 162

7 フロントエンド (React) 主要コード **170**

7.1 App.tsx - メインアプリケーション . . . 170
7.2 main.tsx - エントリーポイント . . . 170
7.3 AuthContext.tsx - 認証コンテキスト . . . 171
7.4 api/client.ts - APIクライアント . . . 173
7.5 api/posts.ts - 投稿API . . . 174
7.6 vite.config.ts - Vite設定 . . . 175
7.7 tsconfig.json - TypeScript設定 . . . 176
7.8 package.json - パッケージ設定 . . . 176

8 スクリプト **178**

8.1 build-lambda-layer.sh - Lambda Layer ビルド 178
8.2 deploy-aws.sh - AWS デプロイスクリプト 181
8.3 deploy-azure.sh - Azure デプロイスクリプト 183
8.4 deploy-gcp.sh - GCP デプロイスクリプト 185
8.5 test-e2e.sh - E2Eテスト 186
8.6 cloud_architecture_mapper.py - アーキテクチャマッパー 193

9 設定ファイル 206
9.1 docker-compose.yml - ローカル開発環境 206
9.2 Makefile - ビルドタスク 209
9.3 .gitignore - Git除外設定 212

Chapter 1

プロジェクト概要

Multi-Cloud

Figure 1.1: Multi-Cloud

Deploy to AWS passing

Deploy to Azure passing

Deploy to GCP passing

マルチクラウド対応の自動デプロイシステム - AWS/Azure/GCP対応のフルスタックアプリケーション自動デプロイプラッ

Cloud Provider	API Endpoint	Frontend (CDN)	Direct Storage
AWS (ap-northeast-1)	API Gateway	CloudFront ☒	S3 Static
Azure (japaneast)	Functions	Front Door ☒	Blob Storage
GCP (asia-northeast1)	Cloud Functions	Cloud CDN ☒	Cloud Storage

Cloud Provider	Resource	セキュリティ実装	状態
AWS	CloudFront + Lambda	CORS 絞り込み + CloudTrail	☒ 本番反映
Azure	Key Vault + Functions	Purge Protection + Managed Identity + 診断ログ	☒ 本番反映
GCP	Cloud Functions	HTTPS redirect + Audit Logs + Cloud Armor	☒ 本番反映

☒ **IAM/RBAC 最小権限の法則適用済み**: デプロイユーザーと管理者を明確に分離 → 詳細: CRITICAL_RULES — Rule 16 | セキュリティガイド

☒ **全クラウドでCDN配信を実装！** CloudFront, Front Door, Cloud CDNによる高速・安全なコンテンツ配信

☒ **Infrastructure as Code**: Pulumiで全CDNリソースを管理（詳細: CDNセットアップガイド）

☒ 詳細情報: エンドポイント一覧

☒ **2026-02-27 更新**: S1 セキュリティ全本番反映完了 / S2 Managed Identity 有効化 / Task 20/21 Key Vault & Defender 実装済み

☒ **2026-02-22 Refactoring**: Infra & CI/CD Refactoring Report — Azure AFD SPA fix, workflow cleanup, Pulumi dead code removal, staging validation

- **マルチクラウド対応**: AWS、Azure、GCPに対応
- **フルスタック**: フロントエンド、バックエンド、データベースの完全なスタック
- **自動デプロイ**: GitHub Actionsによる完全自動化
- **IaC統合**: Pulumi 3.0+による Infrastructure as Code
- **完全Python版**: Pulumi + FastAPI + Reflexによる統一スタック
- **CI/CD**: プッシュやPRで自動的にビルド・デプロイ
- **簡単セットアップ**: スクリプト一つで環境構築

```
multicloud-auto-deploy/ ├──
 .github/workflows/     # GitHub ワークフローActions ├──
 infrastructure/        # インフラストラクチャコード │
    └── pulumi/           # コード（PulumiPython - AWS/Azure/）GCP ├──
 services/             # アプリケーションコード │
    ├── api/              # FastAPI バックエンド（Python ）3.12 │
    ├── frontend_react/   # React フロントエンド（Vite + ）TypeScript │
    └── frontend_reflex/  # Reflex フロントエンド（実験的） ├──
 scripts/              # デプロイ・テストスクリプト ├──
 docs/                 # ドキュメント │
    ├── CDN_SETUP.md                     # 設定ガイドCDN │
    ├── ENDPOINTS.md                     # エンドポイント一覧 │
    └── REFACTORING_REPORT_20260222.md    # 🆕 Infra & CI/CD refactoring report (2026-02-22) └──
 static-site/          # 静的サイト（環境選択画面）
```

- Python 3.12+
- Docker & Docker Compose
- Pulumi 3.0+
- AWS CLI 2.x / Azure CLI 2.x / gcloud CLI 556.0+
- GitHub アカウント

Frontend

- **Framework**: React 18+ (Vite)
- **Hosting**: Static Site (S3 / Azure Blob / Cloud Storage)
- **CDN**: CloudFront / Azure Front Door / Cloud CDN
- **Build**: Vite 7.3+, TypeScript

Backend

- **Framework**: FastAPI 1.0+ (Python 3.12)
- **AWS**: Lambda (x86_64) + API Gateway v2 (HTTP)
- **Azure**: Azure Functions (Python)
- **GCP**: Cloud Functions (Python 3.11)

Database

- **AWS**: DynamoDB (PAY_PER_REQUEST)

- **Azure**: Cosmos DB (Serverless)
- **GCP**: Firestore (Native Mode)

Infrastructure

- **IaC**: Pulumi 3.0+
  - Infrastructure as Code管理（`infrastructure/pulumi/`）
    - AWS: Lambda + API Gateway
    - Azure: Functions + Cosmos DB
    - GCP: Cloud Functions
- **CI/CD**: GitHub Actions

CI/CD

- GitHub Actions
- Automated builds and deployments
- Docker-based deployments
- S3-based Lambda deployment

1. **リポジトリをクローン**

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. **ローカル開発環境起動**

```
docker-compose up -d api

cd services/frontend_react
npm install
npm run dev
```

3. **Pulumiでデプロイ**

```
cd infrastructure/pulumi/aws/simple-sns
pip install -r requirements.txt
pulumi stack init staging
pulumi config set aws:region ap-northeast-1
pulumi up
```

☒ 詳細な移行ガイドは docs/PYTHON_MIGRATION.md を参照

1. **リポジトリをクローン**

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. **GitHub Secretsを設定**

```
./scripts/setup-github-secrets.sh
```

3. **コードをプッシュして自動デプロイ**

```
git push origin main
```

- ☒ セットアップガイド - 初期セットアップ手順
- ☒ CI/CD設定 - GitHub Actions自動デプロイ設定
- ☒ CI/CDテスト結果 - パイプライン検証レポート
- ☒ トラブルシューティング - よくある問題と解決策
- ☒ エンドポイント一覧 - 全環境のエンドポイント情報
- ☒ CDNセットアップガイド - CloudFront/Front Door/Cloud CDN設定
- ☒ 関数サイズ最適化 - デプロイパッケージサイズ削減（AWS 97%削減！）
- ☒ クイックリファレンス - よく使うコマンド集
- ☒ セキュリティ実装 - Key Vault / Managed Identity / WAF / CloudTrail / Audit Logs
- AWS デプロイ
- Azure デプロイ
- GCP デプロイ
- システムアーキテクチャ - 完全版システム設計
- クラウドアーキテクチャ可視化 - 公式アイコン付きインタラクティブHTML図生成
  - ☒ Staging環境 - AWS/Azure/GCP公式アイコン統合版
  - ☒ Production環境 - 本番環境構成図
  - ☒ 統合ビュー - Staging + Production比較図

プッシュやPRで自動的にビルド・デプロイが実行されます：

- `main`ブランチへのプッシュ → 全3クラウド本番デプロイ（AWS/Azure/GCP）
- `develop`ブランチへのプッシュ → ステージング環境へデプロイ
- PRの作成/更新 → ビルド検証
- 手動トリガー → 任意の環境へデプロイ

2026-02-27 デプロイ状況:

- ☒ AWS production: 9/0 SNS E2E test PASS
- ☒ Azure production: 17/0 SNS E2E test PASS
- ☒ GCP production: 13/0 SNS E2E test PASS
- ☒ 全環境セキュリティ実装完了（Pulumi反映）

ワークフロー	トリガー	デプロイ先	説明
deploy-aws.yml	`main`へのpush / 手動	AWS Lambda	Lambda + API Gateway更新
deploy-azure.yml	`main`へのpush / 手動	Azure Functions	Functions + Cosmos DB更新
deploy-gcp.yml	`main`へのpush / 手動	GCP Cloud Functions	Cloud Functions更新

以下のシークレットを設定してください（詳細は CI/CD設定ガイド 参照）：

Pulumi（必須）

- `PULUMI_ACCESS_TOKEN` - Pulumi Cloud認証トークン（すべてのデプロイで必須）

Azure Functions

- `AZURE_CREDENTIALS` - Service Principal認証情報（JSON）
- `AZURE_SUBSCRIPTION_ID` - Azure Subscription ID
- `AZURE_RESOURCE_GROUP` - リソースグループ名

GCP Cloud Functions

- `GCP_CREDENTIALS` - サービスアカウントキー（JSON）
- `GCP_PROJECT_ID` - プロジェクトID

AWS Lambda

- `AWS_ACCESS_KEY_ID` - AWS認証キーID
- `AWS_SECRET_ACCESS_KEY` - AWS認証シークレットキー

最新のデプロイ状況はGitHub Actionsで確認でき、詳細タスク進捗も公開中です。

最新ステータス（2026-02-27）： | タイプ | 対象 | 状態 | |——|——|——| | インフラ整備 | AWS/Azure/GCP Pulumi本番 | ☒ デプロイ完了 | | セキュリティ実装 | CORS / CloudTrail / Key Vault / Managed Identity / Audit Logs | ☒ 本番反映完了 | | E2E テスト | 全クラウド SNS 統合テスト | ☒ 39/39 PASS | | 認証 | AWS Cognito / Azure AD / Firebase | ☒ 全完了 | | CDN | CloudFront / Front Door / Cloud CDN | ☒ 全完了 |

GitHub Actionsページから手動でワークフローを実行：

```
Actions > Deploy to Multi-Cloud > Run workflow

- environment: staging / production
- deploy_target: all / azure / gcp
```

- **Frontend**: S3 Static Website Hosting
- **Backend**: Lambda (Python 3.12) + API Gateway v2
- **Database**: DynamoDB (PAY_PER_REQUEST)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **Frontend**: Blob Storage ($web) + Azure Front Door
- **Backend**: Azure Functions (Python 3.12)
- **Database**: Cosmos DB (Serverless)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **CDN**: Azure Front Door (Standard)
- **Frontend**: Cloud Storage Static Website
- **Backend**: Cloud Functions (Python 3.12)
- **Database**: Firestore (Native Mode)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions

プロジェクトには以下の便利なスクリプトが含まれています：

```
./scripts/test-endpoints.sh

```

```
./scripts/test-e2e.sh

./scripts/manage-github-secrets.sh

./scripts/diagnostics.sh

./scripts/deploy-aws.sh
./scripts/deploy-azure.sh
./scripts/deploy-gcp.sh
```

```
make install         # 依存関係をインストール
make build-frontend  # フロントエンドをビルド
make build-backend   # Lambda パッケージを作成
make test-all        # 全クラウドのデプロイメントをテスト
make deploy-aws      # へデプロイAWS
make pulumi-preview  # 変更プレビューPulumi
make pulumi-up       # スタック適用Pulumi
make clean           # ビルド成果物を削除
```

VS Codeの Dev Containerに対応しています：

```
- Pulumi 3.x
- Node.js 18
- Python 3.12
- AWS CLI, Azure CLI, gcloud CLI
- Docker in Docker

pulumi          # Pulumi CLI
deploy-aws      # 環境にデプロイAWS
deploy-azure    # 環境にデプロイAzure
deploy-gcp      # 環境にデプロイGCP
test-all        # 全エンドポイントテスト
```

システムの健全性をチェック：

```
./scripts/diagnostics.sh
```

- ☒ インストール済みツールの確認
- ☒ クラウドプロバイダー認証状態
- ☒ デプロイメントエンドポイントのテスト
- ☒ Pulumiスタック状態の確認

```
./scripts/test-endpoints.sh

curl https://52z731x570.execute-api.ap-northeast-1.amazonaws.com/
curl https://mcad-staging-api--0000004.livelycoast-fa9d3350.japaneast.azurecontainerapps.io/
curl https://mcad-staging-api-son5b3ml7a-an.a.run.app/
```

```
cd services/frontend
npm install
npm run dev

cd services/backend
pip install -r requirements.txt
uvicorn src.main:app --reload
```

Lambda + API Gatewayの完全自動デプロイメント:

```
cd scripts
./deploy-lambda-aws.sh

PROJECT_NAME=myproject ENVIRONMENT=production ./deploy-lambda-aws.sh
```

スクリプトは以下を自動実行します：

- 依存関係のインストール（manylinux2014_x86_64）
- ZIPパッケージ作成とS3アップロード
- Lambda関数の作成/更新
- API Gateway統合設定
- Lambda権限設定（HTTP API用の正しいSourceArn）
- CloudWatch Logsアクセスログ有効化

エンドポイントの完全なCRUDテスト:

```
./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com

./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com --verbose
```

テスト項目：

- ヘルスチェック
- メッセージCRUD操作（作成、取得、更新、削除）
- ページネーション
- エラーハンドリング
- バリデーション

全環境（AWS/GCP/Azure）のエンドツーエンドCRUD動作を検証:

```
./scripts/test-sns-all.sh

./scripts/test-sns-aws.sh
./scripts/test-sns-azure.sh
./scripts/test-sns-gcp.sh
```

テストカバレッジ（2026-02-27 実績）:

- **AWS**: 9 PASS / 0 FAIL
- **Azure**: 17 PASS / 0 FAIL
- **GCP**: 13 PASS / 0 FAIL
- **Total**: 39 PASS / 0 FAIL ☒

テスト項目:

- ☒ Health Checks: 各環境のヘルスエンドポイント検証
- ☒ CRUD Operations: Create/List/Get/Update/Delete
- ☒ Authentication: Cloud-native認証（Cognito/Azure AD/Firebase）
- ☒ Image Upload: PUT /posts with images（AWS S3/Azure Blob/GCP GCS）
- ☒ Error Handling: 404/403/500エラー処理検証

クラウド固有のパス処理:

- AWS/GCP: `/api/messages/`

- Azure: `/api/HttpTrigger/api/messages/`（Flex Consumption対応）

期待される出力例:

```
═══════════════════════════════════════════════════════════════════════════

        Multi-Cloud E2E Test Suite═══════════════════════════════════════════════════════════════════════

  Testing: AWS═══════════════════════════════════════════════════════════════════════════════════✓

 Health check returned 'ok'✓
 Create message (ID: abc123...)✓
 List messages (found 5)✓
 Get message by ID✓
 Update message✓
 Delete message

[GCP/Azure: 同様]═══════════════════════════════════════════════════════════════════════════

        Test Summary═══════════════════════════════════════════════════════════════════════════

Total Tests:  18
Passed:       18
All tests passed! ✓
```

データ永続性検証:

- AWS: DynamoDB (PAY_PER_REQUEST)
- GCP: Firestore (Native Mode)
- Azure: Cosmos DB (Serverless)

包括的な監視とアラートを自動設定:

```
./scripts/setup-monitoring.sh

ALERT_EMAIL=your@email.com ./scripts/setup-monitoring.sh
```

設定内容：

- SNSトピックとメール通知
- Lambda エラー/スロットリング/実行時間/同時実行数アラーム
- API Gateway 5XXエラー/レイテンシアラーム
- DynamoDB スロットリングアラーム
- CloudWatch Logs メトリクスフィルター
- CloudWatch ダッシュボード自動作成

本番運用のために以下のサービス追加を推奨します:

Lambda関数のトレーシング有効化:

```
aws lambda update-function-configuration \
```

```
  --function-name YOUR_FUNCTION_NAME \
  --tracing-config Mode=Active
```

FastAPIにX-Ray統合:

```
aws-xray-sdk==2.12.0

from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.ext.fastapi.middleware import XRayMiddleware

app.add_middleware(XRayMiddleware, recorder=xray_recorder)
```

API Gatewayへの攻撃防御:

```
aws wafv2 create-web-acl \
  --name multicloud-auto-deploy-waf \
  --scope REGIONAL \
  --default-action Allow={} \
  --rules file://waf-rules.json

aws wafv2 associate-web-acl \
  --web-acl-arn YOUR_WEB_ACL_ARN \
  --resource-arn YOUR_API_GATEWAY_ARN
```

プロダクション用ドメイン設定:

```
aws acm request-certificate \
  --domain-name api.yourdomain.com \
  --validation-method DNS

aws apigatewayv2 create-domain-name \
  --domain-name api.yourdomain.com \
  --domain-name-configurations CertificateArn=YOUR_CERT_ARN

aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch file://route53-changes.json
```

環境変数の安全な管理:

```
aws secretsmanager create-secret \
  --name multicloud-auto-deploy/staging/db-config \
  --secret-string '{"host":"dynamodb","region":"ap-northeast-1"}'
```

```
import boto3
import json

def get_secret():
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(SecretId='multicloud-auto-deploy/staging/db-config')
    return json.loads(response['SecretString'])
```

共通ライブラリの分離でコールドスタート改善:

```
mkdir python
pip install -r requirements.txt -t python/
zip -r layer.zip python/
```

```

aws lambda publish-layer-version \
  --layer-name multicloud-auto-deploy-dependencies \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.12

aws lambda update-function-configuration \
  --function-name YOUR_FUNCTION_NAME \
  --layers YOUR_LAYER_ARN
```

リクエスト/レスポンスのエッジ処理:

```
// CloudFront Function: セキュリティヘッダ追加
function handler(event) {
  var response = event.response;
  response.headers["strict-transport-security"] = {
    value: "max-age=31536000; includeSubdomains",
  };
  response.headers["x-content-type-options"] = { value: "nosniff" };
  response.headers["x-frame-options"] = { value: "DENY" };
  return response;
}
```

DynamoDBの自動バックアップ:

```
aws backup create-backup-plan \
  --backup-plan file://backup-plan.json

aws backup create-backup-selection \
  --backup-plan-id YOUR_PLAN_ID \
  --backup-selection file://backup-selection.json
```

問題が発生した場合は トラブルシューティングガイド を参照してください：

- Azure認証問題（Service Principal、Pulumi Provider等）
- GCPリソース競合（State管理、リソースインポート）
- フロントエンドAPI接続問題（ビルド順序、API URL設定）
- 権限エラー（IAM、RBAC設定）

コントリビューションを歓迎します！詳細は CONTRIBUTING.md をご覧ください。

MIT License - 詳細は LICENSE をご覧ください。

- GitHub Actions Documentation
- Terraform Documentation
- Pulumi Documentation

Chapter 2

ソース実装解説書

本ドキュメントは `docs/AI_AGENT_0*.md` の設計書を前提に、実際のソースコードがどのように実装されているかを日本語

---

1. プロジェクト全体像
2. バックエンド API（FastAPI）
3. 認証・JWT 検証
4. クラウド別バックエンド実装
5. クラウド別エントリーポイント
6. フロントエンド（React SPA）
7. ローカル開発環境

---

multicloud-auto-deploy は、SNS スタイルの投稿アプリ（文章・画像・タグ）を AWS / Azure / GCP の **3つのクラウドに同時デプロイ** するプラットフォームである。

```
ユーザー

  │
  ├── [AWS]   CloudFront → S3 (React SPA)
  │           API Gateway v2 → Lambda (FastAPI) → DynamoDB
  │
  ├── [Azure] Front Door → Blob Storage (React SPA)
  │           Azure Functions → Cosmos DB
  │
  └── [GCP]   Cloud CDN → GCS (React SPA)
             Cloud Run → Firestore
```

```
services/
  api/                      ← FastAPI バックエンド（クラウド共通）3
    app/
      main.py               ← FastAPI アプリ本体・・ルーター登録CORS
      config.py             ← 環境変数を Pydantic Settings で管理
      models.py             ← Pydantic データモデル
      auth.py               ← JWT 認証ミドルウェア
      jwt_verifier.py       ← Cognito / Azure AD / Firebase JWT 検証
      backends/
        base.py             ← 抽象基底クラス（インターフェース定義）
        aws_backend.py      ← DynamoDB + S3 実装
        azure_backend.py    ← Cosmos DB + Blob Storage 実装
        gcp_backend.py      ← Firestore + Cloud Storage 実装
```

```
        local_backend.py      ← DynamoDB Local + MinIO 実装
      routes/
        posts.py              ← /posts エンドポイント
        profile.py            ← /profile エンドポイント
        uploads.py            ← /uploads エンドポイント
    index.py                  ← AWS Lambda ハンドラー () Mangum
    function_app.py           ← Azure Functions ハンドラー
    function.py               ← GCP Cloud Run ハンドラー
  frontend_react/             ← React 18 + Vite + TypeScript SPA
    src/
      App.tsx                 ← ルーティング定義
      api/                    ← Axios API クライアント
      contexts/AuthContext.←tsx 認証状態管理
      components/             ← UI コンポーネント
      pages/                  ← ページコンポーネント
      config/auth.ts          ← 認証プロバイダー設定
```

次セクション → 2. バックエンド API（FastAPI）

---

services/api/app/main.py が FastAPI アプリの中心。

主な処理の流れ:

```python
try:
    from aws_lambda_powertools import Logger, Metrics, Tracer
    logger = Logger(service="simple-sns-api")
    ...
except ImportError:
    logger = logging.getLogger(__name__)

app = FastAPI(title="Simple SNS API", version="3.0.0", ...)

app.add_middleware(CORSMiddleware, allow_origins=origins, ...)

app.add_middleware(GZipMiddleware, minimum_size=1000)

app.include_router(posts.router)    # /posts
app.include_router(uploads.router)  # /uploads
app.include_router(profile.router)  # /profile
app.include_router(limits.router)   # /limits
```

ライフサイクル管理: @asynccontextmanager の lifespan 関数でアプリ起動・終了時のログを出力する。

後方互換エイリアス: 旧フロントエンド向けに /api/messages/ エンドポイントを維持している。このエンドポイントでは認証
anonymous ユーザーとして扱われる（テスト互換のための意図的な設計）。

---

pydantic_settings.BaseSettings を継承した Settings クラスで全ての環境変数を管理する。

```python
class Settings(BaseSettings):
    cloud_provider: CloudProvider = CloudProvider.LOCAL  # "aws"|"azure"|"gcp"|"local"
    auth_disabled: bool = False
    posts_table_name: Optional[str] = None   # 用AWS
    cosmos_db_endpoint: Optional[str] = ...  # 用Azure
    gcp_project_id: Optional[str] = None     # 用GCP
    ...
```

ポイント: - 環境変数名の揺れ（例: `COGNITO_USER_POOL_ID` vs `AWS_COGNITO_USER_POOL_ID`）を `AliasChoices` で吸収している。 - `CLOUD_PROVIDER` が未設定の場合は、実行環境の特徴から自動検出される（`config.py` 末尾の auto-detect ロジック）。

クラウド自動検出ロジック:

環境変数	判定クラウド
`AWS_EXECUTION_ENV`	aws
`WEBSITE_INSTANCE_ID`	azure
`K_SERVICE`	gcp
（なし）	local

Pydantic v2 で定義。**camelCase と snake_case 両方を返す**ことが最大の特徴。

```
class Post(BaseModel):
    id: str = Field(..., alias="postId")
    user_id: str = Field(..., alias="userId")
    content: str
    ...

    @model_serializer
    def serialize_model(self) -> dict:
        return {
            # 新フロントエンド用 (camelCase)
            "postId": self.id,
            "userId": self.user_id,
            ...
            # 旧フロントエンド用 (snake_case)
            "id": self.id,
            "author": self.user_id,
            "created_at": self.created_at,
        }
```

`model_config = {"populate_by_name": True}` により、リクエスト受信時は `postId`（alias）でも `id`（フィールド名）のどちらでも受け付ける。

`ListPostsResponse` も同様に `items` / `results` / `messages` の3つのキーで同じデータを返し、複数バージョンのフロントエンドと互換

ファイル	prefix	主な処理
`posts.py`	`/posts`	投稿の CRUD（GET/POST/PUT/DELETE）
`profile.py`	`/profile`	プロフィール取得・更新
`uploads.py`	`/uploads`	画像アップロード用署名付き URL 発行

認証の使い分け:

```
@router.get("")
def list_posts(...) -> ListPostsResponse:
    ...

@router.post("", status_code=201)
def create_post(body: CreatePostBody, user: UserInfo = Depends(require_user)):
```

```
    ...
```

Depends(require_user) は auth.py の関数で、トークンがない場合 HTTP 401 を返す。Depends(get_current_user) はトークンなしの場合 None を返す（認証任意）。

```
クライアント

  Authorization: Bearer <JWT>
          ↓
  get_current_user() / require_user()
          ↓
  get_jwt_verifier()  ← settings.auth_provider で切替
          ↓
  JWTVerifier.verify_token()
          ↓
  UserInfo { user_id, email, groups }
```

auth_disabled=true の場合（ローカル開発時のみ許容）は検証をスキップし、テスト用ユーザーを返す。

UserInfo クラス:

```python
@dataclass
class UserInfo:
    user_id: str
    email: Optional[str] = None
    groups: Optional[list[str]] = None

    @property
    def is_admin(self) -> bool:
        return self.groups is not None and "Admins" in self.groups
```

管理者判定は groups リストに "Admins" が含まれるかで行う。

JWTVerifier クラスが3クラウドの JWT 検証を統一的に処理する。

JWKS エンドポイント:

プロバイダー	JWKS URI
Cognito (AWS)	https://cognito-idp.{region}.amazonaws.com/{pool_id}/.well-known/jwks.json
Firebase (GCP)	https://www.googleapis.com/service_accounts/v1/jwk/securetoken@...
Azure AD	https://login.microsoftonline.com/{tenant_id}/discovery/v2.0/keys

JWKS キャッシュ: 毎リクエストごとに HTTP 取得すると遅延が大きいため、1時間のインメモリキャッシュを実装してい

```python
def get_jwks(self) -> dict:
    if キャッシュ有効:
        return self._jwks_cache
    # 取得失敗時はキャッシュを使い続ける
    if self._jwks_cache:
        logger.warning("Using expired JWKS cache")
        return self._jwks_cache
    raise
```

トークン検証処理: `python-jose` ライブラリを使い、JWKS から公開鍵を取得して署名検証 + issuer / audience 検証を行う。検証後 `extract_user_info()` でクラウドごとの claims 差異を吸収して `UserInfo` に変換する。

```python
@lru_cache(maxsize=1)
def get_backend() -> BackendBase:
    provider = settings.cloud_provider  # 環境変数 CLOUD_PROVIDER
    if provider == CloudProvider.AWS:
        return AwsBackend()
    elif provider == CloudProvider.AZURE:
        return AzureBackend()
    elif provider == CloudProvider.GCP:
        return GcpBackend()
    else:
        return LocalBackend()
```

`@lru_cache(maxsize=1)` により、同一プロセス内ではバックエンドインスタンスを1回だけ生成する（接続の使い回し）。

全バックエンドが実装すべきインターフェースを定義する。

```python
class BackendBase(ABC):
    @abstractmethod
    def list_posts(limit, next_token, tag) -> (list[Post], next_token)
    @abstractmethod
    def get_post(post_id) -> Optional[Post]
    @abstractmethod
    def create_post(body, user) -> dict
    @abstractmethod
    def update_post(post_id, body, user) -> dict   # ※ AWS / Local のみ完全実装
    @abstractmethod
    def delete_post(post_id, user) -> dict
    @abstractmethod
    def get_profile(user_id) -> ProfileResponse
    @abstractmethod
    def update_profile(user, body) -> ProfileResponse
    @abstractmethod
    def generate_upload_urls(count, user, content_types) -> list[dict]
```

データストア: DynamoDB Single Table Design + S3

テーブル設計:

```
PK        SK                          内容
POSTS     <ISO timestamp>#<uuid>      投稿データ
PROFILES  <userId>                    ユーザープロフィール
```

SK に `<timestamp>#<uuid>` を使うことで、`ScanIndexForward=False` クエリが標準の降順（新着順）になる。

投稿一覧取得:

```python
def list_posts(self, limit, next_token, tag):
    query_kwargs = {
        "KeyConditionExpression": "PK = :pk",
        "ExpressionAttributeValues": {":pk": "POSTS"},
        "ScanIndexForward": False,  # 新しい順
```

```
        "Limit": limit,
    }
    if next_token:
        query_kwargs["ExclusiveStartKey"] = {"PK": "POSTS", "SK": next_token}
    if tag:
        query_kwargs["FilterExpression"] = "contains(tags, :tag)"
    response = self.table.query(**query_kwargs)
    # next_token = LastEvaluatedKey["SK"]
```

ページネーションには `LastEvaluatedKey["SK"]`（= 最後のアイテムの SK）をトークンとして使う。

画像 URL の解決:

S3 に保存された画像キーは、取得時に `generate_presigned_url`（1時間有効）に変換する。 - `https://` で始まる場合はそのまま返す - `http://` はセキュリティ上 Mixed Content となるためスキップする - それ以外（S3 キー）は署名付き URL に変換する

データストア: Cosmos DB（posts / profiles コンテナ） + Blob Storage

DynamoDB との違いは、Cosmos DB では**コンテナ(テーブル)を用途ごとに分ける**点。posts コンテナと profiles コンテナを別々に持つ。

```
self.posts_container = self.database.create_container_if_not_exists(
    id="posts",
    partition_key=PartitionKey(path="/postId"),
)
self.profiles_container = self.database.create_container_if_not_exists(
    id="profiles",
    partition_key=PartitionKey(path="/userId"),
)
```

画像 URL: Blob SAS（24時間有効）に変換。フル Blob URL が既に保存されている場合はキー部分を抽出して SAS を改めて付与する。

インポート安全設計: `azure-cosmos` や `azure-storage-blob` がインストールされていない環境でも `ImportError` をキャッチして警告ログだけ出す。実際に `AzureBackend()` を生成しようとした時に例外を投げる。

データストア: Firestore（posts / profiles コレクション） + Cloud Storage

Firestore ではドキュメント ID でページネーションを行う。

```
def list_posts(self, limit, next_token, tag):
    query = col.order_by("createdAt", direction=DESCENDING).limit(limit + 1)
    if next_token:
        cursor_doc = col.document(next_token).get()
        query = query.start_after(cursor_doc)  # カーソルベースのページネーション
```

`limit + 1` 件取得して `len(docs) > limit` なら次ページがあると判断し、最後のドキュメント ID を `next_token` として返す。

署名付き URL の特殊対応: GCP では Compute Engine のデフォルト認証情報には秘密鍵がないため、`generate_signed_url()` は使えない。代わりに IAM `signBlob` API を使うカスタム実装を行う。認証情報は `google.auth.default()` で取得し、`credentials.valid` を確認してから期限切れ時のみ refresh する（毎リクエストのメタデータサーバー呼び出しを回避）。

データストア: DynamoDB Local + MinIO（S3 互換）

AWS バックエンドと同じ Single Table Design を使うことで、ローカルで発見したバグが AWS でも同様に再現することを保証している。テーブルが存在しない場合は**初回起動時に自動作成**する。

```
def _init_dynamodb(self):
    # テーブルが存在しなければ作成
    try:
        self.table = self.dynamodb.create_table(
            TableName=table_name,
            KeySchema=[{"AttributeName": "PK", ...}, {"AttributeName": "SK", ...}],
            ...
        )
    except self.dynamodb.meta.client.exceptions.ResourceInUseException:
        self.table = self.dynamodb.Table(table_name)  # 既存テーブルを使う
```

FastAPI アプリ本体（`app/main.py`）は3クラウドで**共通**。クラウドごとにその前段となるアダプター層が異なる。

```
from mangum import Mangum
from app.main import app

handler = Mangum(app, lifespan="off")
```

`Mangum` が Lambda イベント（API Gateway Proxy v2 形式）を ASGI に変換し、FastAPI へ渡す。`lifespan="off"` は Lambda の短命実行モデルに対応するための設定。

Lambda Layer に FastAPI 等の依存ライブラリを格納し、アプリコード（〜78KB）と別管理することでデプロイ速度を改

Azure Functions には Mangum 相当の標準アダプターがないため、**カスタム ASGI ブリッジ**を自前実装している。

主な処理フロー:

```
Azure Functions HttpRequest
    ↓
route_params["route"] からパスを取得
    "HttpTrigger/" プレフィックスを除去
    ↓
ASGI scope dict を組み立て
    {type: "http", path: "/posts", method: "GET", ...}
    ↓
FastAPI ASGI app を呼び出し
    ↓レスポンスを
 func.HttpResponse に変換して返す
```

CORS Preflight（OPTIONS） は Azure Functions レベルで直接処理し、FastAPI に渡さずに 204 を返す（速度・安定性のため）。

インポートエラーが発生しても 503 でエラー内容を返すフェイルセーフ設計：

```
_IMPORT_ERROR = None
try:
    from app.main import app as fastapi_app
except Exception as e:
    _IMPORT_ERROR = traceback.format_exc()
...
if fastapi_app is None:
```

```
    return func.HttpResponse(
        body=json.dumps({"error": "Service unavailable", "detail": _IMPORT_ERROR}),
        status_code=503,
    )
```

Cloud Run は通常の HTTP サーバーとして動作するため、`uvicorn` でそのまま FastAPI を起動できる。Lambda や Azure Functions のような特殊なアダプターは必要ない。

`Dockerfile` で `CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]` のような起動コマンドを指定するだけで Cloud Run 上で動作する。

`services/frontend_react/` に React 18 + Vite + TypeScript + Tailwind CSS で構築された SPA。3クラウドすべてで**同一のビルド成果物**をオブジェクトストレージ（S3 / Blob Storage / GCS）に配置し、CDN で配信する。

`vite.config.ts` でビルド時のベースパスを `/sns/` に設定している（CDN の `/sns/*` パスにマッピングされるため）。

```
<AuthProvider>
  <Layout>
    <Routes>
      <Route path="/"              element={<HomePage />} />
      <Route path="/post/:postId"  element={<PostPage />} />
      <Route path="/profile"       element={<ProfilePage />} />
      <Route path="/login"         element={<LoginPage />} />
      <Route path="/auth/callback" element={<CallbackPage />} />
      <Route path="*"              element={<HomePage />} />
    </Routes>
  </Layout>
</AuthProvider>
```

`AuthProvider` が認証状態をアプリ全体に提供する（React Context パターン）。

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? '',  // 空文字 → Vite プロキシ経由でローカル API
  timeout: 15000,
});

// 全リクエストに JWT トークンを付与
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('id_token') || localStorage.getItem('access_token');
  if (token) config.headers['Authorization'] = `Bearer ${token}`;
  return config;
});
```

Cognito は `id_token`（audience が client_id と一致する）を優先して使う。`access_token` はフォールバック。

ビルド時の環境変数 `VITE_AUTH_PROVIDER` で認証プロバイダーを切り替える。

VITE_AUTH_PROVIDER	認証方式
aws	Cognito PKCE 認証コードフロー
azure	Azure AD 暗黙的フロー（id_token フラグメント）
firebase	Firebase SDK Google Sign-In（ポップアップ）
none	認証なし（ローカル開発）

AWS (Cognito) の PKCE フロー: 1. `generateCodeVerifier()` でランダム文字列生成 → `sessionStorage` に保存 2. SHA-256 ハッシュ → base64url エンコード → `code_challenge` 3. Cognito 認証エンドポイントへリダイレクト 4. コールバック（`/auth/callback`）で `code` を受取り + `code_verifier` で検証 → token 取得

`localStorage` にトークンを保存・読込し、`window` の `storage` イベントを監視することでタブ間でログイン状態を同期する。

```
// タブ間同期
useEffect(() => {
  const onStorage = () => setToken(readToken());
  window.addEventListener("storage", onStorage);
  return () => window.removeEventListener("storage", onStorage);
}, []);
```

コンポーネント	役割
HomePage	投稿一覧表示・タグフィルタ・キーワード検索・ページネーション（“もっと見る”）
PostForm	投稿作成フォーム（テキスト・マークダウン・画像複数枚・タグ）
PostCard	1件の投稿表示（マークダウン対応・画像ライトボックス）
ProfilePage	ニックネーム・自己紹介の表示と編集
LoginPage	各クラウドの認証フローへのリダイレクト
CallbackPage	Cognito / Azure AD の認証コールバック処理・token 保存

```
1. POST /uploads/presigned-urls  →  { urls: [{uploadUrl, key}] }
2. 各 uploadUrl に （ブラウザから直接PUT S3/Blob/GCS へアップロード）
3. POST /posts { imageKeys: [key1, key2, ...] }  →  投稿作成
```

バックエンドを経由しないため、大容量画像でもバックエンドに負荷をかけない設計。アップロード上限は `GET /limits` で取得し、ハードコードしない。

`docker-compose.yml` で以下の4サービスを起動する。

サービス	ポート	用途
api	8000	FastAPI（ホットリロード付き uvicorn）
frontend_web	8080	Python SSR フロントエンド（レガシー）
dynamodb-local	8001	DynamoDB Local（AWS 互換）
minio	9000/9001	MinIO（S3 互換オブジェクトストレージ）

API サービスの環境変数（ローカル）：

```
CLOUD_PROVIDER=local
AUTH_DISABLED=true      # ローカルのみ許容
DYNAMODB_ENDPOINT=http://dynamodb-local:8000
DYNAMODB_TABLE_NAME=simple-sns-local
MINIO_ENDPOINT=http://minio:9000
MINIO_PUBLIC_ENDPOINT=http://localhost:9000   # ブラウザからの直接アクセス用
```

`MINIO_ENDPOINT`（コンテナ内部 URL）と `MINIO_PUBLIC_ENDPOINT`（ブラウザからの URL）を分けることで、コンテナ内→MinIO の通信とブラウザ→MinIO の通信が両立する。

```
cd /workspaces/multicloud-auto-deploy
docker compose up -d

curl http://localhost:8000/health

open http://localhost:8000/docs

open http://localhost:9001  # minioadmin / minioadmin
```

`services/api/app` ディレクトリが `docker-compose.yml` でコンテナにマウントされるため、ソース変更は即座にホットリロードで

`LocalBackend._init_dynamodb()` が初回起動時に DynamoDB Local へテーブルを自動作成する。手動でのテーブル作成は不要。

MinIO のバケットも同様に `LocalBackend._init_storage()` 内で自動作成する。

パターン	実装箇所	説明
Strategy パターン	`backends/`	`BackendBase` を実装するクラスをクラウドごとに差し替え
Factory + LRU Cache	`backends/__init__.py`	`get_backend()` で1回だけインスタンスを生成
Pydantic Settings	`config.py`	環境変数を型安全に管理、`AliasChoices` で名前揺れを吸収
Dual serialization	`models.py`	camelCase と snake_case を同時に返してフロントエンド互換
JWKS キャッシュ	`jwt_verifier.py`	1時間キャッシュ、失敗時は期限切れキャッシュをフォールバック
Presigned URL	各バックエンド	画像をバックエンド経由せずブラウザから直接アップロード
PKCE フロー	`config/auth.ts`	AWS Cognito で認証コードフローを安全に実装

Chapter 3

インフラ・CI/CD解説書

> 本ドキュメントは `services/api/`・`services/frontend_react/` のアプリコードを解説した IMPLEMENTATION_GUIDE.md の**姉妹ドキュメント**である。 ここでは **インフラ (Pulumi)・CI/CD (GitHub Actions)・スクリプト** のソースを扱う。

---

1. インフラ全体の考え方
2. Pulumi AWS (`infrastructure/pulumi/aws/__main__.py`)
3. Pulumi Azure (`infrastructure/pulumi/azure/__main__.py`)
4. Pulumi GCP (`infrastructure/pulumi/gcp/__main__.py`)
5. CI/CD ワークフロー (GitHub Actions)
6. フロントエンド SPA デプロイワークフロー
7. スクリプト群 (`scripts/`)

Terraform ではなく **Pulumi** を使う理由は、Python コードで条件分岐・ループ・関数を自由に使えるためである。設定値 `Pulumi.<stack>.yaml` に記述し、デプロイ状態は **Pulumi Cloud**（リモートステート）で管理する。

スタック名	ブランチ	用途
`staging`	`develop`	自動デプロイ・動作確認用
`production`	`main`	本番環境（即時反映）

各スタックは独立したクラウドリソースを持つ（S3 バケット名・Lambda 関数名などに `{stack}` サフィックスが付く）。

```
project_name = "multicloud-auto-deploy"
stack = pulumi.get_stack()   # "staging" または "production"

f"{project_name}-{stack}-api"
```

全リソースに `common_tags`（Project / ManagedBy / Environment）を付与してコスト管理・検索を容易にする。

ファイル: infrastructure/pulumi/aws/**main**.py（約965行）

```
IAM Role (Lambda 用)
   └── AWSLambdaBasicExecutionRole ポリシーアタッチ
   └── DynamoDB + S3 アクセス用インラインポリシー

```

```
Cognito User Pool
   └── User Pool Client (PKCE コードフロー)
   └── User Pool Domain 認証ホスト( UI)

DynamoDB Table (Single Table Design)
   ├── PK="PK" / SK="SK"
   ├── GSI: PostIdIndex (hash=postId)
   └── GSI: UserPostsIndex (hash=userId, range=createdAt)

S3 Bucket:  (画像アップロード先) images
   └── CORS 設定 (ブラウザから直接 PUT できるよう)
   └──  (PublicAccessBlockpublic 禁止)

Lambda  (LayerFastAPI / Mangum / Pydantic 等)
   └── services/api/lambda-layer.zip から生成

Lambda  (FunctionFastAPI + ) Mangum
   └── ENV: POSTS_TABLE_NAME, IMAGES_BUCKET_NAME, COGNITO_* 等

API Gateway v2 (HTTP API)
   └── $default ルート → Lambda 統合

S3 Bucket:  (frontendReact SPA 配信)
   └── OAC (Origin Access Control) で CloudFront のみ許可

CloudFront Distribution
   ├── /sns/* → S3 frontend / CloudFront Function で /sns → /sns/index.html リライト
   └── /*     → S3 frontend (landing pages)
```

```python
user_pool_client = aws.cognito.UserPoolClient(
    allowed_oauth_flows=["code"],         # PKCE コードフロー
    allowed_oauth_scopes=["openid", "email", "profile"],
    callback_urls=(
        ([f"https://{cf_domain}/sns/auth/callback"] if cf_domain else [])
        + ([f"https://{custom_domain}/sns/auth/callback"] if custom_domain else [])
    ),
    access_token_validity=1,   # 時間1
    id_token_validity=1,       # 時間1
    refresh_token_validity=30, # 日30
)
```

`cf_domain`（CloudFront ドメイン）と `custom_domain`（カスタムドメイン）の両方を登録することで、どちらのドメインから

CI とローカルでは ZIP のパスが異なるため、`GITHUB_WORKSPACE` 環境変数で分岐している。

```python
workspace_root = os.environ.get("GITHUB_WORKSPACE")
if workspace_root:
    layer_zip_path = pathlib.Path(workspace_root) / "services/api/lambda-layer.zip"
else:
    # __file__ から階層上がリポジトリルート3
    layer_zip_path = pathlib.Path(__file__).parent.parent.parent / "services/api/lambda-layer.zip"
```

ZIP が存在しない場合はウォーニングを出してスキップする（初回デプロイ時など）。

React SPA は HTML5 History API でルーティングするため、CDN 側で `/sns` → `/sns/index.html` のリライトが必要。CloudFront Function を Pulumi のコード内にインラインで定義している。

```python
spa_rewrite_function = aws.cloudfront.Function(
```

```
    "spa-sns-rewrite",
    name=f"spa-sns-rewrite-{stack}",
    runtime="cloudfront-js-2.0",
    code="""
function handler(event) {
    var uri = event.request.uri;
    if (uri === '/sns' || uri === '/sns/') {
        event.request.uri = '/sns/index.html';
    } else if (uri.startsWith('/sns/') && !uri.includes('.')) {
        event.request.uri = '/sns/index.html';
    }
    return event.request;
}
""",
)
```

ファイル: infrastructure/pulumi/azure/**main**.py（約612行）

```
Resource Group (japaneast)
  ├── Storage Account: mcadfunc{suffix}  ← Azure Functions 用
  ├── Storage Account: mcadweb{suffix}   ← フロントエンド静的サイト用
  │     └── $web コンテナ（静的 Web サイトホスティング）
  ├── Application （監視）Insights
  ├── Cosmos DB アカウント
  │     └── コンテナ: posts / profiles
  ├── Azure AD アプリ登録（認証）
  │     └── リダイレクト URI: https://{frontend_domain}/sns/auth/callback
  ├── Azure （FunctionsFlex ) Consumption
  │     └── HTTP Trigger: {*route} → FastAPI
  └── Azure Front （) DoorCDN
        ├── /sns/* → Blob Storage $web/sns/ (React SPA)
        └── /*     → Blob Storage $web (landing pages)
```

Azure ストレージアカウント名はグローバル一意が必要なため `pulumi_random` で6文字サフィックスを生成する。同じインス

```
storage_suffix = random.RandomString("storage-suffix", length=6, special=False, upper=False)

functions_storage_name = storage_suffix.result.apply(lambda s: f"mcadfunc{s}")
frontend_storage_name  = storage_suffix.result.apply(lambda s: f"mcadweb{s}")
```

AWS Cognito が PKCE コードフローを使うのに対し、Azure AD は**暗黙的フロー**（`id_token` フラグメント）を使う。

```
app_registration = azuread.Application(
    "app-registration",
    web=azuread.ApplicationWebArgs(
        redirect_uris=[f"https://{frontend_domain}/sns/auth/callback"],
        implicit_grant=azuread.ApplicationWebImplicitGrantArgs(
            id_token_issuance_enabled=True,   # 暗黙的フロー
        ),
    ),
)
```

フロントエンドのコールバックハンドラーは URL フラグメント（`#id_token=...`）を解析してトークンを取得する。

AFD の `SpaRuleSet` URL リライトルールで `/sns/*` の非アセットパスを `/sns/index.html` にリライトし、React SPA のクライアントルーティングを実現する。

ファイル: infrastructure/pulumi/gcp/**main**.py（約585行）

```
GCP API 有効化（Cloud Run / Secret Manager / Firebase 等）
Firebase プロジェクト・Web アプリ登録
Cloud Audit （全サービス対象）Logs

GCS バケット: frontend   ← React SPA + landing pages
GCS バケット: uploads    ← 画像アップロード

Secret （環境変数を秘密として管理）Manager

Cloud Run: api           ← FastAPI (uvicorn)
  └── サービスアカウント（Firestore / GCS / Secret Manager 権限）

Cloud （CDNClassic External ）LB
  └── /* → GCS バケット（Backend ）Bucket
```

```
services = ["run.googleapis.com", "secretmanager.googleapis.com", "firebase.googleapis.com", ...]
enabled_services = [gcp.projects.Service(f"enable-{s}", ...) for s in services]

firebase_project = gcp.firebase.Project(
    "firebase-project",
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)
```

Pulumi は Cloud Run の**サービス設定**（メモリ・環境変数・サービスアカウント）を管理するが、**コンテナイメージのデ**
CI/CD ワークフロー（`gcloud run deploy`）が担う。Pulumi デプロイ時にコンテナイメージを要求するという制約を回避する

Firebase の認証プロバイダー設定（Google サインイン）は OAuth 同意画面の設定が必要で Pulumi から自動化できない。コードにコメントで明示している。

ファイル	役割
`deploy-aws.yml`	AWS 全リソース（Pulumi + Lambda + フロントエンド含む）
`deploy-azure.yml`	Azure 全リソース（Pulumi + Functions + フロントエンド含む）
`deploy-gcp.yml`	GCP 全リソース（Pulumi + Cloud Run + フロントエンド含む）
`deploy-frontend-web-aws.yml`	React SPA を S3/CloudFront にのみ再デプロイ（差分更新）
`deploy-frontend-web-azure.yml`	React SPA を Blob Storage/AFD にのみ再デプロイ
`deploy-frontend-web-gcp.yml`	React SPA を GCS/CDN にのみ再デプロイ
`deploy-landing-*.yml`	ランディングページ（静的 HTML）の各クラウドへの配信
`version-bump.yml`	`versions.json` のバージョン自動更新

3つのワークフローすべてで同じルールを採用している。

```
environment: >-
  ${{ github.event_name == 'workflow_dispatch'
        && github.event.inputs.environment
        || (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
```

Pulumi は同一スタックへの並行デプロイで `ResourceConflictException` / `409 Conflict` が発生するため、`concurrency` で直列化している。

```
concurrency:
  group: deploy-aws-${{ ... }}
  cancel-in-progress:
    false # AWS/GCP: 実行中のジョブはキャンセルしない
    # Azure staging: （コスト削減のため古いtrue Run をキャンセル）
```

```
1. AWS 認証（aws-actions/configure-aws-）credentials
2. Pulumi CLI インストール
3. スタック名の決定（→developstaging / →）mainproduction
4. .github/config/aws.{stack}.env から設定を読込（single source of ）truth
5. Pulumi スタック選択 / 初期化
6. Pulumi （インフラ変更適用）up
7. Pulumi outputs 取得（Lambda 名・バケット名・Cognito 設定等）
8. Lambda Layer ZIP ビルド（scripts/build-lambda-layer.）sh
9. Lambda 関数コードの ZIP 化・アップロード（aws lambda update-function-）code
10. CloudFront エイリアス + ACM 証明書の確認・修正（safety ）net
11. Cognito コールバック URL の確認・修正（safety ）net
12. React SPA ビルド（npm run ）build
13. S3 同期（aws s3 ）sync
14. CloudFront キャッシュ無効化（aws cloudfront create-）invalidation
```

「Safety net」ステップの目的: Pulumi がカスタムドメイン設定を誤って消してしまうことがあるため、デプロイ後に AWS CLI で直接確認・修正するステップを追加している。Pulumi の宣言的な制御と、より強制力のある命令的な確認の

設定ファイルの一元管理: `Pulumi.*.yaml` は gitignore されておりリモートで使えないため、代わりに `.github/config/aws.staging.env` 等のコミット済みファイルに設定値を記述する。CI はこのファイルを `source` して環境変数を読み込む。

```
1. （PulumiAzure インフラ）
2. Azure Functions 用パッケージビルド
   - linux/amd64 Docker コンテナで pip （installARM ホストとのアーキテクチャ差異を回避）
   - ZIP 化して Azure CLI でアップロード（az functionapp deployment source config-）zip
3. az storage blob service-properties update で静的 Web サイトホスティング有効化
4. React SPA ビルド・Blob Storage アップロード（az storage blob upload-）batch
5. AFD キャッシュパージ（az afd endpoint ）purge
```

ARM → AMD64 クロスビルド: GitHub Actions のランナーは ARM (aarch64) のホストで動作する場合があるため、Lambda / Functions 用の Python パッケージは `--platform linux/amd64` の Docker コンテナ内でインストールする。

```
docker run --rm --platform linux/amd64 \
  -v "$(pwd)/package:/package" \
  python:3.12-slim \
  bash -c "pip install -r /package/requirements.txt -t /package"
```

```
1. google-github-actions/auth で認証（GCP_CREDENTIALS シークレット）
2. （PulumiGCP インフラ）
3. Docker イメージビルド・Artifact Registry プッシュ
4. Cloud Run サービス更新（gcloud run ）deploy
```

```
5. React SPA ビルド・GCS アップロード（gsutil ）rsync
6. CDN キャッシュ無効化（gcloud compute url-maps invalidate-cdn-）cache
```

`deploy-frontend-web-*.yml` は API やインフラを触らずに **フロントエンドだけを更新**したい場合に使う差分更新用ワークフロー SPA のファイルを変更した際に Pulumi を再実行せずに済むため、デプロイ時間を短縮できる。

```
1. Pulumi outputs 取得（バケット名・Cognito 設定・CloudFront ）ID
   ← インフラには触れず読み取りのみ
2. npm ci / npm run build
   - VITE_API_URL / VITE_AUTH_PROVIDER=aws / VITE_COGNITO_* を環境変数として注入
3. S3 同期（差分更新）
   - アセット類（JS/）CSS: cache-control max-age=31536000, immutable
   - index.html のみ: max-age（ブラウザキャッシュを短くして更新を即反映）=300
4. Cognito コールバック URL を aws cognito-idp update-user-pool-client で同期
   - CloudFront ドメイン + カスタムドメインの両方を登録
5. CloudFront キャッシュ無効化（/sns）/*
```

キャッシュ戦略の意図: ハッシュ付きアセット（`index-abc123.js` 等）は Vite がファイル名を変えるため長期キャッシュ可。一方 `index.html` は常に最新のアセット名を参照するため短期キャッシュにす

```
1. Pulumi outputs 取得（ストレージアカウント名・AFD エンドポイント）
2. npm run （buildVITE_AUTH_PROVIDER=azure 等を注入）
3. az storage blob upload-batch で $web/sns/ へアップロード
4. az afd endpoint purge でキャッシュパージ
```

```
1. Pulumi outputs 取得（GCS バケット名）
2. npm run （buildVITE_AUTH_PROVIDER=firebase 等を注入）
3. gsutil -m rsync -r dist/ gs://{bucket}/sns/
4. gcloud compute url-maps invalidate-cdn-cache
```

Lambda Layer を `linux/amd64` Docker コンテナでビルドして ZIP 化するスクリプト。 デプロイホストが ARM (Apple M1/M2 / aarch64) でも正しいアーキテクチャのバイナリを生成できる。

```
1. requirements-layer.txt を参照
2. docker run --platform linux/amd64 python:3.12-slim で pip install
3. python/ 以下に配置して ZIP 化 → services/api/lambda-layer.zip
```

Layer には `fastapi`, `mangum`, `pydantic`, `python-jose` 等が含まれる。 `boto3` / `botocore` は Lambda ランタイムに同梱されているため**除外**し、Layer サイズを削減している。

各クラウドに手動デプロイするためのラッパースクリプト。主に開発時の動作確認に使う。CI/CD ワークフロー（`.github/workflows/*.yml`）と同等の処理を行うが、インタラクティブな確認ステップを含む。

スクリプト	内容
`test-api.sh`	各クラウドの API エンドポイントに curl でリクエストし、ステータスコードを確認
`test-e2e.sh`	全クラウドを対象とした E2E テスト（投稿作成→取得→削除）

スクリプト	内容
`test-sns-aws.sh` 等	クラウド個別の SNS 機能テスト
`test-auth-crud.sh`	認証付き CRUD 操作のテスト
`diagnose-environments.sh`	各クラウドの設定値・エンドポイントの診断

`versions.json` のバージョン番号をインクリメントするスクリプト。`version-bump.yml` ワークフローから呼び出される。セマンティックバージョニング（major/minor/patch）に対応。

原則	実装箇所	説明
IaC 一元管理	`infrastructure/pulumi/`	クラウドリソースはすべて Pulumi Python コードで定義
設定の単一ソース	`github/config/*.env`	`Pulumi.*.yaml` は gitignore されるため、CI 用設定ファイルをリポジトリにコミット
Safety net	`deploy-aws.yml`	Pulumi だけに頼らず AWS CLI で CloudFront / Cognito を事後確認・修正
ARM/AMD64 分離	CI ワークフロー	Python パッケージは常に `linux/amd64` Docker でビルド
キャッシュ戦略	`deploy-frontend-web-*.yml`	ハッシュ付きアセット: 永続キャッシュ / `index.html`: 短期キャッシュ
差分デプロイ	`deploy-frontend-web-*.yml`	フロントエンド変更時は Pulumi を再実行しない
同時実行防止	`concurrency`: ブロック	Pulumi の競合エラーを防ぐためスタック単位で直列化

Chapter 4

バックエンドAPI ソースコード

このセクションでは、FastAPIベースのバックエンドAPIのソースコード全文を掲載します。

4.1 main.py - FastAPIアプリケーション

```python
import logging
from contextlib import asynccontextmanager

from fastapi import Depends, FastAPI, HTTPException, Query, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from fastapi.responses import JSONResponse

from app.auth import UserInfo, get_current_user
from app.backends import get_backend
from app.config import settings
from app.models import CreatePostBody, HealthResponse, ListPostsResponse, UpdatePostBody
from app.routes import limits, posts, profile, solve, uploads

# AWS Lambda Powertools (observability)
try:
    from aws_lambda_powertools import Logger, Metrics, Tracer
    from aws_lambda_powertools.metrics import MetricUnit

    # Powertools Logger 構造化ログ()
    logger = Logger(service="simple-sns-api")
    tracer = Tracer(service="simple-sns-api")
    metrics = Metrics(namespace="SimpleSNS", service="api")

    powertools_available = True
except ImportError:
    # Fallback to standard logging when AWS Lambda Powertools is not installed.
    logging.basicConfig(
        level=getattr(logging, settings.log_level.upper()),
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    )
    logger = logging.getLogger(__name__)
    powertools_available = False


```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan: startup and shutdown logic."""
    logger.info(
        "Starting Simple SNS API",
        extra={
            "version": "3.0.0",
            "cloud_provider": settings.cloud_provider.value,
            "auth_disabled": settings.auth_disabled,
            "powertools_enabled": powertools_available,
        },
    )
    yield
    logger.info("Shutting down Simple SNS API")


# アプリケーションFastAPI
app = FastAPI(
    title="Simple SNS API",
    description="マルチクラウド対応のシンプルなSNS API",
    version="3.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
    lifespan=lifespan,
)

# 設定CORS
origins = settings.cors_origins.split(",") if settings.cors_origins != "*" else ["*"]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 圧縮Gzip
app.add_middleware(GZipMiddleware, minimum_size=1000)


# ─── Security Validation Middleware ──────────────────────────────────────────
# Detect and block common attack patterns:
#   1. SQL Injection (query strings)
#   2. Path Traversal (URL paths)
#   3. XSS (User-Agent headers)
#   4. Suspicious file extensions
# This provides WAF-like protection without Azure Front Door Premium SKU.
@app.middleware("http")
async def security_validation_middleware(request: Request, call_next):
    """Block requests matching common attack patterns."""

    # Check 1: SQL Injection patterns in query string
    query_string = str(request.url.query).lower()
    sql_patterns = [
        "union select",
        "union+select",
        "' or '1'='1",
```

```
        "' or 1=1",
        "; drop table",
        "; delete from",
        "exec(",
        "execute(",
        "<script>",
        "javascript:",
    ]

    for pattern in sql_patterns:
        if pattern in query_string:
            logger.warning(
                f"SQL Injection attempt detected: {pattern}",
                extra={
                    "path": request.url.path,
                    "query": query_string[:200],
                    "client_ip": request.client.host if request.client else "unknown",
                    "user_agent": request.headers.get("user-agent", "unknown")[:100],
                },
            )
            return JSONResponse(
                status_code=403,
                content={
                    "detail": "Request blocked by security policy",
                    "rule": "SQL Injection Detection",
                },
            )

    # Check 2: Path Traversal patterns in URL path
    path = request.url.path.lower()
    path_traversal_patterns = [
        "../",
        "..\\",
        "%2e%2e%2f",
        "%2e%2e/",
        "..%2f",
        "%2e%2e%5c",
    ]

    for pattern in path_traversal_patterns:
        if pattern in path:
            logger.warning(
                f"Path Traversal attempt detected: {pattern}",
                extra={
                    "path": request.url.path,
                    "client_ip": request.client.host if request.client else "unknown",
                },
            )
            return JSONResponse(
                status_code=403,
                content={
                    "detail": "Request blocked by security policy",
                    "rule": "Path Traversal Detection",
                },
            )

    # Check 3: Suspicious file extensions
    suspicious_extensions = [
```

```
        ".env",
        ".git",
        ".config",
        ".sql",
        ".bak",
        ".old",
        ".log",
        "web.config",
        ".htaccess",
    ]

    for ext in suspicious_extensions:
        if path.endswith(ext):
            logger.warning(
                f"Suspicious file extension requested: {ext}",
                extra={
                    "path": request.url.path,
                    "client_ip": request.client.host if request.client else "unknown",
                },
            )
            return JSONResponse(
                status_code=403,
                content={
                    "detail": "Request blocked by security policy",
                    "rule": "Suspicious File Detection",
                },
            )

    # Check 4: XSS patterns in User-Agent header
    user_agent = request.headers.get("user-agent", "").lower()
    xss_patterns = ["<script", "javascript:", "onerror=", "onload=", "eval(", "alert("]

    for pattern in xss_patterns:
        if pattern in user_agent:
            logger.warning(
                f"XSS attempt detected in User-Agent: {pattern}",
                extra={
                    "user_agent": user_agent[:200],
                    "client_ip": request.client.host if request.client else "unknown",
                },
            )
            return JSONResponse(
                status_code=403,
                content={
                    "detail": "Request blocked by security policy",
                    "rule": "XSS Detection",
                },
            )

    # All checks passed - proceed with request
    response = await call_next(request)
    return response


# ルーター登録
app.include_router(limits.router)
app.include_router(posts.router)
app.include_router(uploads.router)
```

```python
app.include_router(profile.router)
app.include_router(solve.router)


# ─── Validation error handler ──────────────────────────────────────────────────────────────────────────────
# Log request body on 422 to simplify debugging of malformed requests.
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    """Log request body on 422 validation errors to aid debugging."""  # noqa: D401
    try:
        body = await request.body()
        body_str = body.decode("utf-8", errors="replace") if body else "(empty)"
    except Exception:
        body_str = "(could not read body)"
    logger.error(
        f"422 ValidationError on {request.method} {request.url.path}: "
        f"errors={exc.errors()} body={body_str[:500]}"
    )
    return JSONResponse(status_code=422, content={"detail": exc.errors()})


# ─── Backward-compatible /api/messages aliases (legacy frontend) ──────────────────────────────────────────
@app.get("/api/messages/", response_model=ListPostsResponse)
def legacy_list_messages(
    limit: int = Query(
        20, ge=1, le=50, alias="page_size", description="Number of items"
    ),
    nextToken: str | None = Query(None, description="Pagination token"),
    tag: str | None = Query(None, description="Tag filter"),
) -> ListPostsResponse:
    """Legacy alias: list posts (GET /api/messages/). Kept for old frontend compatibility."""
    backend = get_backend()
    posts_list, output_next_token = backend.list_posts(limit, nextToken, tag)
    return ListPostsResponse(items=posts_list, limit=limit, nextToken=output_next_token)


@app.post("/api/messages/", status_code=201)
def legacy_create_message(
    body: CreatePostBody,
    user: UserInfo | None = Depends(get_current_user),
) -> dict:
    """Legacy alias: create post (POST /api/messages/). Kept for old frontend compatibility."""
    # Allow anonymous users so staging tests work without auth.
    if not user:
        user = UserInfo(
            user_id="anonymous",
            email="anonymous@example.com",
            groups=None,
        )
    backend = get_backend()
    return backend.create_post(body, user)


@app.delete("/api/messages/{post_id}")
def legacy_delete_message(
    post_id: str,
    user: UserInfo | None = Depends(get_current_user),
```

```
) -> dict:
    """Legacy alias: delete post (DELETE /api/messages/{id})."""
    # Legacy endpoint: unauthenticated requests receive admin-level access so that
    # staging / test users can delete any post without a real auth token.
    # In production with auth properly configured, get_current_user returns a real user.
    if not user:
        user = UserInfo(
            user_id="anonymous",
            email="anonymous@example.com",
            groups=["Admins"],
        )
    backend = get_backend()
    try:
        return backend.delete_post(post_id, user)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e)) from e
    except PermissionError as e:
        raise HTTPException(status_code=403, detail=str(e)) from e


@app.get("/api/messages/{post_id}")
def legacy_get_message(
    post_id: str,
    user: UserInfo | None = Depends(get_current_user),
) -> dict:
    """Legacy alias: get single post (GET /api/messages/{id})."""
    backend = get_backend()
    try:
        return backend.get_post(post_id)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e)) from e


@app.put("/api/messages/{post_id}")
def legacy_update_message(
    post_id: str,
    body: UpdatePostBody,
    user: UserInfo | None = Depends(get_current_user),
) -> dict:
    """Legacy alias: update post (PUT /api/messages/{id})."""
    # Legacy endpoint: same policy as DELETE — unauthenticated = admin-level access.
    if not user:
        user = UserInfo(
            user_id="anonymous",
            email="anonymous@example.com",
            groups=["Admins"],
        )
    backend = get_backend()
    try:
        return backend.update_post(post_id, body, user)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e)) from e
    except PermissionError as e:
        raise HTTPException(status_code=403, detail=str(e)) from e


@app.get("/", response_model=HealthResponse)
def root() -> HealthResponse:
```

```
    """Root endpoint — returns cloud provider info."""
    if powertools_available:
        metrics.add_metric(name="RootEndpointCalled", unit=MetricUnit.Count, value=1)

    return HealthResponse(
        status="ok",
        provider=settings.cloud_provider.value,
    )


@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    """Health check endpoint."""
    if powertools_available:
        metrics.add_metric(name="HealthCheckCalled", unit=MetricUnit.Count, value=1)

    logger.info(
        "Health check requested", extra={"provider": settings.cloud_provider.value}
    )

    return HealthResponse(
        status="ok",
        provider=settings.cloud_provider.value,
    )


# AWS Lambda handler (Mangum)
try:
    from mangum import Mangum

    _mangum_handler = Mangum(app, lifespan="off")

    if powertools_available:
        # Wrap the Mangum handler with Powertools decorators for structured logging,
        # X-Ray tracing, and cold-start metrics.
        @logger.inject_lambda_context(clear_state=True)
        @tracer.capture_lambda_handler
        @metrics.log_metrics(capture_cold_start_metric=True)
        def handler(event, context):
            return _mangum_handler(event, context)

        logger.info("Mangum handler initialized with Powertools support")
    else:
        handler = _mangum_handler
        logger.info("Mangum handler initialized for AWS Lambda")

except ImportError:
    logger.warning("Mangum not available - AWS Lambda deployment not supported")
    handler = None
```

4.2 config.py - 環境設定

```
from pydantic import AliasChoices, Field
from pydantic_settings import BaseSettings

from app.models import CloudProvider
```

```python


class Settings(BaseSettings):
    """アプリケーション設定 (v1.2.2)"""

    # クラウドプロバイダー
    cloud_provider: CloudProvider = CloudProvider.LOCAL

    # 認証設定
    auth_disabled: bool = False
    auth_provider: str | None = None
    auth_issuer: str | None = None
    auth_jwks_url: str | None = None
    auth_audience: str | None = None
    admin_group: str = "Admins"

    # ローカル開発設定 (DynamoDB Local + MinIO)
    dynamodb_endpoint: str | None = Field(default="http://localhost:8001")
    dynamodb_table_name: str = Field(default="simple-sns-local")
    minio_endpoint: str | None = None
    minio_access_key: str | None = None
    minio_secret_key: str | None = None
    # Accepts both MINIO_BUCKET and MINIO_BUCKET_NAME environment variables
    minio_bucket: str = Field(
        default="images",
        validation_alias=AliasChoices("minio_bucket", "minio_bucket_name"),
    )
    # Public URL for browser-side PUT requests (falls back to minio_endpoint)
    minio_public_endpoint: str | None = None

    # 設定AWS
    aws_region: str = "ap-northeast-1"
    posts_table_name: str | None = None
    images_bucket_name: str | None = None
    images_cdn_url: str | None = None
    # 入試問題解答機能の有効無効/ コスト管理用(: false でエンドポイントを停止)
    solve_enabled: bool = False
    solve_allow_remote_image_url: bool = True
    solve_max_image_bytes: int = 5 * 1024 * 1024
    solve_ocr_review_min_score: float = 0.40
    solve_ocr_review_max_replacement_ratio: float = 0.01
    # Accepts both COGNITO_USER_POOL_ID and AWS_COGNITO_USER_POOL_ID
    cognito_user_pool_id: str | None = Field(
        default=None,
        validation_alias=AliasChoices(
            "cognito_user_pool_id", "aws_cognito_user_pool_id"
        ),
    )
    # Accepts both COGNITO_CLIENT_ID and AWS_COGNITO_CLIENT_ID
    cognito_client_id: str | None = Field(
        default=None,
        validation_alias=AliasChoices("cognito_client_id", "aws_cognito_client_id"),
    )

    # 設定Azure
    azure_tenant_id: str | None = None
    azure_client_id: str | None = None
    azure_storage_account_name: str | None = None
```

```
    azure_storage_account_key: str | None = None
    azure_storage_container: str = "images"

    # Azure AI Document Intelligence (OCR)
    azure_document_intelligence_endpoint: str | None = None
    azure_document_intelligence_key: str | None = None

    # Azure OpenAI 数学解答生成()
    azure_openai_endpoint: str | None = None
    azure_openai_key: str | None = None
    azure_openai_deployment: str = "gpt-4o"
    # accurate モード用推論モデル（o3-mini 等）。未設定時は通常モデルにフォールバック
    azure_openai_accurate_deployment: str = ""
    azure_openai_api_version: str = "2024-12-01-preview"

    # Cosmos 設定DB
    # NOTE: AZURE_COSMOS_DATABASE/CONTAINER names are reserved by Azure CLI/Function App
    #       and always return null values. Use COSMOS_DB_* prefix instead.
    #       Both naming conventions are supported via AliasChoices for compatibility.
    cosmos_db_endpoint: str | None = Field(
        default=None,
        validation_alias=AliasChoices("cosmos_db_endpoint", "azure_cosmos_endpoint"),
    )
    cosmos_db_key: str | None = Field(
        default=None, validation_alias=AliasChoices("cosmos_db_key", "azure_cosmos_key")
    )
    cosmos_db_database: str = Field(
        default="simple-sns",
        validation_alias=AliasChoices("cosmos_db_database", "azure_cosmos_database"),
    )
    cosmos_db_container: str = Field(
        default="items",
        validation_alias=AliasChoices("cosmos_db_container", "azure_cosmos_container"),
    )

    # 設定GCP
    gcp_project_id: str | None = None
    gcp_client_id: str | None = None
    gcp_service_account: str | None = None
    gcp_storage_bucket: str | None = None
    gcp_posts_collection: str = "posts"
    gcp_profiles_collection: str = "profiles"

    # GCP Vision API (OCR)
    # キー不要の場合は Application Default Credentials (ADC) を使用
    gcp_vision_api_key: str | None = None

    # GCP Vertex AI (Gemini LLM)
    gcp_vertex_location: str = "us-central1"
    gcp_vertex_model: str = "gemini-2.0-flash-001"
    # accurate モード用高精度モデル。未設定時は通常モデルにフォールバック
    gcp_vertex_accurate_model: str = ""

    # 共通設定
    presigned_url_expiry: int = 300
    cors_origins: str = "*"
    log_level: str = "INFO"
    # 画像アップロード制限 環境変数( MAX_IMAGES_PER_POST で上書き可)
```

```
    max_images_per_post: int = 10

    model_config = {
        "env_file": ".env",
        "case_sensitive": False,
    }


# シングルトンインスタンス
settings = Settings()
```

4.3 models.py - データモデル

```
from enum import Enum
from typing import Any

from pydantic import BaseModel, Field, model_serializer


class CloudProvider(str, Enum):
    """クラウドプロバイダーの列挙型"""

    LOCAL = "local"
    AWS = "aws"
    AZURE = "azure"
    GCP = "gcp"


class Post(BaseModel):
    """投稿モデル"""

    id: str = Field(..., alias="postId")
    user_id: str = Field(..., alias="userId")
    nickname: str | None = None
    content: str
    is_markdown: bool = Field(False, alias="isMarkdown")
    image_urls: list[str] | None = Field(None, alias="imageUrls")
    tags: list[str] | None = None
    created_at: str = Field(..., alias="createdAt")
    updated_at: str | None = Field(None, alias="updatedAt")

    model_config = {"populate_by_name": True}

    @model_serializer
    def serialize_model(self) -> dict[str, Any]:
        """後方互換性: snake_case と camelCase 両方のフィールド名を返す"""
        return {
            # camelCase (ashnova.v3 形式)
            "postId": self.id,
            "userId": self.user_id,
            "nickname": self.nickname,
            "content": self.content,
            "isMarkdown": self.is_markdown,
            "imageUrls": self.image_urls,
            "tags": self.tags,
            "createdAt": self.created_at,
```

```
            "updatedAt": self.updated_at,
            # snake_case (frontend_react 形式)
            "id": self.id,
            "author": self.user_id,  # userId を author としても返す
            "created_at": self.created_at,
            "updated_at": self.updated_at,
            "image_url": self.image_urls[0] if self.image_urls else None,
        }


class CreatePostBody(BaseModel):
    """投稿作成リクエスト"""

    content: str = Field(..., min_length=1, max_length=10000)
    is_markdown: bool = Field(False, alias="isMarkdown")
    image_keys: list[str] | None = Field(
        None, alias="imageKeys"
    )  # 枚数制限はルートで settings.max_images_per_post により強制
    tags: list[str] | None = Field(None, max_length=10)

    model_config = {"populate_by_name": True}


class UpdatePostBody(BaseModel):
    """投稿更新リクエスト（全フィールドオプション）"""

    content: str | None = Field(None, min_length=1, max_length=10000)
    is_markdown: bool | None = Field(None, alias="isMarkdown")
    image_keys: list[str] | None = Field(None, alias="imageKeys")
    tags: list[str] | None = Field(None, max_length=10)

    model_config = {"populate_by_name": True}


class ListPostsResponse(BaseModel):
    """投稿一覧レスポンス"""

    items: list[Post]
    limit: int
    next_token: str | None = Field(None, alias="nextToken")

    model_config = {"populate_by_name": True}

    @model_serializer
    def serialize_model(self) -> dict[str, Any]:
        """後方互換性: results と messages フィールドを追加"""
        return {
            "items": self.items,
            "results": self.items,  # ashnova.v1 互換
            "messages": self.items,  # frontend_react 互換
            "limit": self.limit,
            "nextToken": self.next_token,
            "total": len(self.items),  # frontend_react 互換
            "page": 1,  # frontend_react 互換
            "page_size": self.limit,  # frontend_react 互換
        }


```

```python
class ProfileResponse(BaseModel):
    """プロフィールレスポンス"""

    user_id: str = Field(..., alias="userId")
    nickname: str | None = None
    bio: str | None = None
    avatar_url: str | None = Field(None, alias="avatarUrl")
    created_at: str | None = Field(None, alias="createdAt")
    updated_at: str | None = Field(None, alias="updatedAt")

    model_config = {"populate_by_name": True}


class ProfileUpdateRequest(BaseModel):
    """プロフィール更新リクエスト"""

    nickname: str | None = Field(None, max_length=100)
    bio: str | None = Field(None, max_length=500)
    avatar_key: str | None = Field(None, alias="avatarKey")

    model_config = {"populate_by_name": True}


class UploadUrlsRequest(BaseModel):
    """アップロード生成リクエストURL"""

    count: int = Field(
        ..., ge=1, le=100
    )  # 絶対上限 、実際の制限は100 settings.max_images_per_post
    content_types: list[str] | None = Field(
        None,
        alias="contentTypes",
        description="各ファイルのContent-Type (image/jpeg, image/png 等)",
    )

    model_config = {"populate_by_name": True}


class UploadUrlsResponse(BaseModel):
    """アップロードレスポンスURL"""

    urls: list[dict[str, str]]  # [{"uploadUrl": "...", "key": "..."}]


class HealthResponse(BaseModel):
    """ヘルスチェックレスポンス"""

    status: str
    provider: str
    version: str = "3.0.0"


class SolveInput(BaseModel):
    """数学問題画像入力"""

    image_base64: str | None = Field(None, alias="imageBase64")
    image_url: str | None = Field(None, alias="imageUrl")
    source: str = Field("paste", pattern="^(paste|upload|url)$")
```

```
    model_config = {"populate_by_name": True}


class SolveExam(BaseModel):
    """試験メタデータ"""

    university: str = "tokyo"
    year: int | None = None
    subject: str = "math"
    question_no: str | None = Field(None, alias="questionNo")

    model_config = {"populate_by_name": True}


class SolveOptions(BaseModel):
    """解答オプション"""

    mode: str = Field("fast", pattern="^(fast|accurate)$")
    need_steps: bool = Field(True, alias="needSteps")
    need_latex: bool = Field(True, alias="needLatex")
    max_tokens: int = Field(2000, alias="maxTokens", ge=256, le=16384)
    debug_ocr: bool = Field(False, alias="debugOcr")

    model_config = {"populate_by_name": True}


class SolveRequest(BaseModel):
    """数学問題解答リクエスト"""

    input: SolveInput
    exam: SolveExam = Field(default_factory=SolveExam)
    options: SolveOptions = Field(default_factory=SolveOptions)


class SolveAnswer(BaseModel):
    """解答AI"""

    final: str
    latex: str | None = None
    steps: list[str] = Field(default_factory=list)
    diagram_guide: str | None = Field(None, alias="diagramGuide")
    confidence: float = Field(ge=0.0, le=1.0)

    model_config = {"populate_by_name": True}


class SolveMeta(BaseModel):
    """実行メタ情報"""

    ocr_provider: str = Field(alias="ocrProvider")
    ocr_source: str | None = Field(default=None, alias="ocrSource")
    ocr_score: float | None = Field(default=None, alias="ocrScore")
    ocr_candidates: int | None = Field(default=None, alias="ocrCandidates")
    ocr_top_candidates: list[dict[str, float | str]] | None = Field(
        default=None, alias="ocrTopCandidates"
    )
    ocr_debug_texts: list[dict] | None = Field(default=None, alias="ocrDebugTexts")
```

```
    structured_problem: dict | None = Field(default=None, alias="structuredProblem")
    ocr_replacement_ratio: float | None = Field(
        default=None, alias="ocrReplacementRatio"
    )
    ocr_non_ascii_ratio: float | None = Field(default=None, alias="ocrNonAsciiRatio")
    ocr_needs_review: bool | None = Field(default=None, alias="ocrNeedsReview")
    model: str
    latency_ms: int = Field(alias="latencyMs")
    cost_usd: float = Field(alias="costUsd")

    model_config = {"populate_by_name": True}


class SolveResponse(BaseModel):
    """数学問題解答レスポンス"""

    request_id: str = Field(alias="requestId")
    status: str
    problem_text: str = Field(alias="problemText")
    answer: SolveAnswer
    meta: SolveMeta

    model_config = {"populate_by_name": True}
```

4.4 auth.py - 認証ミドルウェア

```
import logging
from dataclasses import dataclass
from functools import lru_cache
from typing import Optional

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

logger = logging.getLogger(__name__)
security = HTTPBearer(auto_error=False)


@dataclass
class UserInfo:
    """ユーザー情報"""

    user_id: str
    email: Optional[str] = None
    groups: Optional[list[str]] = None

    @property
    def is_admin(self) -> bool:
        """管理者かどうか"""
        return self.groups is not None and "Admins" in self.groups


@lru_cache(maxsize=1)
def get_jwt_verifier():
    """JWT インスタンスを取得（設定に基づいて）。verifier

```

```
    lru_cache により Lambda の同一ウォームインスタンス内でシングルトンとして
    保持されるため、JWKS の外部 HTTP 取得がリクエストごとに発生しない。
    """
    from app.config import settings
    from app.jwt_verifier import JWTVerifier

    if not settings.auth_provider:
        return None

    # Build config based on provider
    if settings.auth_provider == "cognito":
        config = {
            "region": settings.aws_region,
            "user_pool_id": settings.cognito_user_pool_id,
            "client_id": settings.cognito_client_id,
        }
    elif settings.auth_provider == "firebase":
        config = {
            "project_id": settings.gcp_project_id,
        }
    elif settings.auth_provider == "azure":
        config = {
            "tenant_id": settings.azure_tenant_id,
            "client_id": settings.azure_client_id,
            "is_b2c": False,  # Standard Azure AD (Entra ID), not B2C
            "tenant_name": (
                settings.azure_tenant_id.split(".")[0]
                if settings.azure_tenant_id
                else None
            ),
        }
    else:
        logger.warning(f"Unknown auth provider: {settings.auth_provider}")
        return None

    return JWTVerifier(settings.auth_provider, config)


async def get_current_user(
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(security),
) -> Optional[UserInfo]:
    """現在のユーザーを取得（トークンから）JWT"""
    from app.config import settings

    # 認証が無効の場合はテストユーザーを返す
    if settings.auth_disabled:
        return UserInfo(
            user_id="test-user-1", email="test@example.com", groups=["Admins"]
        )

    # トークンがない場合
    if not credentials:
        return None

    # 検証JWT
    verifier = get_jwt_verifier()
    if not verifier:
        logger.error("JWT verifier not configured")
```

```
        return None

    try:
        # トークンを検証
        claims = verifier.verify_token(credentials.credentials)
        if not claims:
            logger.warning("Token verification failed")
            return None

        # ユーザー情報を抽出
        user_info_dict = verifier.extract_user_info(claims)
        return UserInfo(
            user_id=user_info_dict["user_id"],
            email=user_info_dict.get("email"),
            groups=user_info_dict.get("groups", []),
        )
    except Exception as e:
        logger.error(f"Error verifying token: {e}")
        return None


async def require_user(
    user: Optional[UserInfo] = Depends(get_current_user),
) -> UserInfo:
    """認証が必要なエンドポイント用の依存関数"""
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="認証が必要です",
            headers={"WWW-Authenticate": "Bearer"},
        )

    return user


async def require_admin(user: UserInfo = Depends(require_user)) -> UserInfo:
    """管理者権限が必要なエンドポイント用の依存関数"""
    if not user.is_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="管理者権限が必要です",
        )

    return user
```

4.5 jwt_verifier.py - JWT検証

```
"""
JWT Token Verification for Multi-Cloud Authentication

Supports:
- AWS Cognito
- Firebase Authentication (GCP)
- Azure AD / Azure AD B2C
"""

```

```python
import logging
from datetime import datetime, timedelta
from functools import lru_cache
from typing import Any, Optional

import requests
from jose import JWTError, jwt

logger = logging.getLogger(__name__)


class JWTVerifier:
    """JWT verification for multiple cloud providers"""

    def __init__(self, provider: str, config: dict[str, Any]):
        """
        Initialize JWT verifier

        Args:
            provider: 'cognito', 'firebase', or 'azure'
            config: Provider-specific configuration
        """
        self.provider = provider
        self.config = config
        self._jwks_cache: Optional[dict] = None
        self._jwks_cache_time: Optional[datetime] = None
        self._jwks_cache_duration = timedelta(hours=1)

    @lru_cache(maxsize=10)  # noqa: B019
    def get_jwks_uri(self) -> str:
        """Get JWKS URI based on provider"""
        if self.provider == "cognito":
            region = self.config.get("region", "ap-northeast-1")
            user_pool_id = self.config["user_pool_id"]
            return f"https://cognito-idp.{region}.amazonaws.com/{user_pool_id}/.well-known/jwks.json"

        elif self.provider == "firebase":
            # Firebase uses Google's public keys
            return "https://www.googleapis.com/service_accounts/v1/jwk/securetoken@system.gserviceaccount.com"

        elif self.provider == "azure":
            tenant_id = self.config["tenant_id"]
            # For Azure AD B2C, use different discovery endpoint
            if self.config.get("is_b2c", False):
                tenant_name = self.config["tenant_name"]
                policy = self.config.get("policy", "B2C_1_signupsignin")
                return
     f"https://{tenant_name}.b2clogin.com/{tenant_name}.onmicrosoft.com/{policy}/discovery/v2.0/keys"
            else:
                return (
                    f"https://login.microsoftonline.com/{tenant_id}/discovery/v2.0/keys"
                )

        raise ValueError(f"Unsupported provider: {self.provider}")

    def get_jwks(self) -> dict:
        """Get JWKS with caching"""
        now = datetime.now()
```

```python

        # Check cache
        if (
            self._jwks_cache is not None
            and self._jwks_cache_time is not None
            and now - self._jwks_cache_time < self._jwks_cache_duration
        ):
            return self._jwks_cache

        # Fetch from provider
        try:
            jwks_uri = self.get_jwks_uri()
            response = requests.get(jwks_uri, timeout=5)
            response.raise_for_status()

            self._jwks_cache = response.json()
            self._jwks_cache_time = now

            return self._jwks_cache
        except Exception as e:
            logger.error(f"Failed to fetch JWKS: {e}")
            # If cache exists, use it even if expired
            if self._jwks_cache:
                logger.warning("Using expired JWKS cache")
                return self._jwks_cache
            raise

    def get_issuer(self) -> str:
        """Get expected issuer for token validation"""
        if self.provider == "cognito":
            region = self.config.get("region", "ap-northeast-1")
            user_pool_id = self.config["user_pool_id"]
            return f"https://cognito-idp.{region}.amazonaws.com/{user_pool_id}"

        elif self.provider == "firebase":
            project_id = self.config["project_id"]
            return f"https://securetoken.google.com/{project_id}"

        elif self.provider == "azure":
            tenant_id = self.config["tenant_id"]
            if self.config.get("is_b2c", False):
                tenant_name = self.config["tenant_name"]
                policy = self.config.get("policy", "B2C_1_signupsignin")
                return f"https://{tenant_name}.b2clogin.com/{tenant_id}/{policy}/v2.0/"
            else:
                return f"https://login.microsoftonline.com/{tenant_id}/v2.0"

        raise ValueError(f"Unsupported provider: {self.provider}")

    def get_audience(self) -> str:
        """Get expected audience for token validation"""
        if self.provider == "cognito":
            return self.config["client_id"]
        elif self.provider == "firebase":
            return self.config["project_id"]
        elif self.provider == "azure":
            return self.config["client_id"]
        raise ValueError(f"Unsupported provider: {self.provider}")
```

```python

    def verify_token(self, token: str) -> Optional[dict[str, Any]]:
        """
        Verify JWT token and return claims

        Args:
            token: JWT token string

        Returns:
            Token claims if valid, None otherwise
        """
        try:
            # Get JWKS
            jwks = self.get_jwks()

            # Decode header to get kid
            unverified_header = jwt.get_unverified_header(token)
            kid = unverified_header.get("kid")

            if not kid:
                logger.error("Token missing 'kid' in header")
                return None

            # Find matching key
            key = None
            for jwk in jwks.get("keys", []):
                if jwk.get("kid") == kid:
                    key = jwk
                    break

            if not key:
                logger.error(f"No matching key found for kid: {kid}")
                return None

            # Verify and decode token
            issuer = self.get_issuer()
            audience = self.get_audience()

            # Check if token has 'aud' claim before verifying it.
            # Cognito access tokens may omit 'aud'; in that case relax audience verification.
            try:
                unverified_claims = jwt.get_unverified_claims(token)
            except Exception:
                unverified_claims = {}
            has_aud = "aud" in unverified_claims

            claims = jwt.decode(
                token,
                key,
                algorithms=["RS256"],
                audience=audience if has_aud else None,
                issuer=issuer,
                options={
                    "verify_signature": True,
                    "verify_aud": has_aud,
                    "verify_iat": True,
                    "verify_exp": True,
                    "verify_iss": True,
```

```
                    # id_token contains at_hash (hash of access_token).
                    # We decode id_token standalone (no access_token companion),
                    # so disable at_hash verification.
                    "verify_at_hash": False,
                },
            )

            logger.info(
                f"Token verified successfully for user: {claims.get('sub')}")
            return claims

        except JWTError as e:
            logger.error(f"JWT verification failed: {e}")
            return None
        except Exception as e:
            logger.error(f"Token verification error: {e}")
            return None

    def extract_user_info(self, claims: dict[str, Any]) -> dict[str, Any]:
        """
        Extract user information from token claims

        Returns standardized user info regardless of provider
        """
        user_info = {
            "user_id": claims.get("sub"),
            "email": None,
            "groups": [],
        }

        if self.provider == "cognito":
            user_info["email"] = claims.get("email")
            # access_token uses 'username', id_token uses 'cognito:username'
            if not user_info["user_id"]:
                user_info["user_id"] = claims.get("username")
            user_info["groups"] = claims.get("cognito:groups", [])

        elif self.provider == "firebase":
            user_info["email"] = claims.get("email")
            # Firebase custom claims for groups
            user_info["groups"] = claims.get("groups", [])

        elif self.provider == "azure":
            user_info["email"] = claims.get(
                "preferred_username") or claims.get("email")
            user_info["groups"] = claims.get("groups", [])

        return user_info
```

4.6 backends/base.py - 抽象基底クラス

```
from abc import ABC, abstractmethod
from typing import Optional, Tuple  # noqa: F401
from app.models import Post, CreatePostBody, ProfileResponse, ProfileUpdateRequest
from app.auth import UserInfo

```

```python

class BackendBase(ABC):
    """
    バックエンドの抽象基底クラス

    すべてのクラウドプロバイダー実装はこのインターフェースに従う
    """

    @abstractmethod
    def list_posts(
        self,
        limit: int,
        next_token: Optional[str],
        tag: Optional[str],
    ) -> Tuple[list[Post], Optional[str]]:
        """
        投稿一覧を取得

        Args:
            limit: 取得件数
            next_token: ページネーショントークン
            tag: タグフィルター

        Returns:
            投稿リスト(, 次のページのトークン)
        """
        pass

    @abstractmethod
    def create_post(self, body: CreatePostBody, user: UserInfo) -> dict:
        """
        投稿を作成

        Args:
            body: 投稿内容
            user: ユーザー情報

        Returns:
            作成された投稿情報
        """
        pass

    @abstractmethod
    def delete_post(self, post_id: str, user: UserInfo) -> dict:
        """
        投稿を削除

        Args:
            post_id: 投稿ID
            user: ユーザー情報

        Returns:
            削除結果
        """
        pass

    @abstractmethod
    def get_post(self, post_id: str) -> Optional[Post]:
```

```
        """
        投稿を件取得1

        Args:
            post_id: 投稿ID

        Returns:
            投稿情報、存在しない場合は None
        """
        pass

    @abstractmethod
    def get_profile(self, user_id: str) -> ProfileResponse:
        """
        プロフィールを取得

        Args:
            user_id: ユーザーID

        Returns:
            プロフィール情報
        """
        pass

    @abstractmethod
    def update_profile(
        self,
        user: UserInfo,
        body: ProfileUpdateRequest,
    ) -> ProfileResponse:
        """
        プロフィールを更新

        Args:
            user: ユーザー情報
            body: 更新内容

        Returns:
            更新後のプロフィール
        """
        pass

    @abstractmethod
    def generate_upload_urls(
        self,
        count: int,
        user: UserInfo,
        content_types: Optional[list[str]] = None,
    ) -> list[dict[str, str]]:
        """
        画像アップロード用の署名付きを生成URL

        Args:
            count: 数URL
            user: ユーザー情報
            content_types: 各ファイルのContent-Type

        Returns:
```

```
            [{"url": "...", "key": "..."}, ...]
        """
        pass
```

4.7 backends/aws_backend.py - AWS実装

```
"""AWS Backend Implementation with DynamoDB Single Table Design"""

import os
import boto3
import uuid
from datetime import datetime, timezone
from typing import Optional, Tuple
from decimal import Decimal
import logging

from app.backends.base import BackendBase
from app.models import Post, CreatePostBody, ProfileResponse, ProfileUpdateRequest
from app.auth import UserInfo

logger = logging.getLogger(__name__)


class AwsBackend(BackendBase):
    """実装AWS (DynamoDB Single Table Design + S3 + Cognito)"""

    def __init__(self):
        self.dynamodb = boto3.resource("dynamodb")
        self.s3_client = boto3.client("s3")

        # 環境変数から設定を取得
        self.table_name = os.environ.get("POSTS_TABLE_NAME", "")
        self.bucket_name = os.environ.get("IMAGES_BUCKET_NAME", "")

        if not self.table_name:
            raise ValueError("POSTS_TABLE_NAME environment variable is required")

        self.table = self.dynamodb.Table(self.table_name)
        logger.info(
            f"Initialized AwsBackend with table={self.table_name}, bucket={self.bucket_name}"
        )

    def _key_to_presigned_url(self, key: str) -> str:
        """キーを署名付きS3GET に変換URL 時間有効(1)"""
        return self.s3_client.generate_presigned_url(
            "get_object",
            Params={"Bucket": self.bucket_name, "Key": key},
            ExpiresIn=3600,
        )

    def _resolve_image_urls(self, keys: list) -> list[str]:
        """キーリストを署名付きに変換URL 既に(の場合はそのまま返すURL)

        セキュリティ: http:// URL は Mixed Content 警告の原因になるためスキップする。
        https:// URL のみそのまま返し、キーはプリサイン済みS3 HTTPS URL に変換する。
        """
```

```
        if not self.bucket_name or not keys:
            return []
        result = []
        for k in keys:
            if k and isinstance(k, str):
                if k.startswith("https://"):
                    result.append(k)
                elif k.startswith("http://"):
                    # HTTP URL は Mixed Content 警告を引き起こすためスキップ
                    logger.warning(f"Skipping insecure HTTP image URL (mixed content): {k[:80]}")
                else:
                    try:
                        result.append(self._key_to_presigned_url(k))
                    except Exception as e:
                        logger.warning(f"Failed to generate presigned URL for {k}: {e}")
        return result

    def list_posts(
        self,
        limit: int,
        next_token: Optional[str],
        tag: Optional[str],
    ) -> Tuple[list[Post], Optional[str]]:
        """投稿一覧を取得 (DynamoDB Query)"""
        try:
            query_kwargs = {
                "KeyConditionExpression": "PK = :pk",
                "ExpressionAttributeValues": {":pk": "POSTS"},
                "ScanIndexForward": False,  # 降順 新しい順()
                "Limit": limit,
            }

            if next_token:
                query_kwargs["ExclusiveStartKey"] = {"PK": "POSTS", "SK": next_token}

            # タグフィルター
            if tag:
                query_kwargs["FilterExpression"] = "contains(tags, :tag)"
                query_kwargs["ExpressionAttributeValues"][":tag"] = tag

            response = self.table.query(**query_kwargs)

            posts = []
            for item in response.get("Items", []):
                raw_urls = item.get("imageKeys") or item.get("imageUrls") or []
                posts.append(
                    Post(
                        postId=item["postId"],
                        userId=item["userId"],
                        nickname=item.get("nickname"),
                        content=item["content"],
                        tags=item.get("tags", []),
                        createdAt=item["createdAt"],
                        updatedAt=item.get("updatedAt"),
                        imageUrls=self._resolve_image_urls(raw_urls),
                    )
                )

```

```
            # ページネーショントークン
            next_token = None
            if "LastEvaluatedKey" in response:
                next_token = response["LastEvaluatedKey"]["SK"]

            return posts, next_token

        except Exception as e:
            logger.error(f"Error listing posts: {e}")
            raise

    def create_post(self, body: CreatePostBody, user: UserInfo) -> dict:
        """投稿を作成 (DynamoDB PutItem)"""
        try:
            post_id = str(uuid.uuid4())
            now = datetime.now(timezone.utc).isoformat()

            # プロフィールからを取得nickname
            nickname = None
            try:
                profile_item = self.table.get_item(
                    Key={"PK": "PROFILES", "SK": user.user_id}
                ).get("Item")
                if profile_item:
                    nickname = profile_item.get("nickname")
            except Exception as e:
                logger.warning(f"Failed to fetch nickname for {user.user_id}: {e}")

            image_keys = body.image_keys if body.image_keys else []

            item = {
                "PK": "POSTS",
                "SK": now + "#" + post_id,  # タイムスタンプ + UUID
                "postId": post_id,
                "userId": user.user_id,
                "nickname": nickname,
                "content": body.content,
                "tags": body.tags if body.tags else [],
                "createdAt": now,
                "updatedAt": now,
                "imageKeys": image_keys,  # 生のキーを保存S3
            }

            self.table.put_item(Item=item)

            presigned_urls = self._resolve_image_urls(image_keys)

            return {
                "postId": post_id,
                "userId": user.user_id,
                "nickname": nickname,
                "content": body.content,
                "tags": item["tags"],
                "createdAt": now,
                "imageUrls": presigned_urls,
                # snake_case aliases
                "post_id": post_id,
                "user_id": user.user_id,
```

```
                "created_at": now,
                "image_urls": presigned_urls,
            }

        except Exception as e:
            logger.error(f"Error creating post: {e}")
            raise

    def get_post(self, post_id: str):
        """投稿を件取得1 (PostIdIndex で検索)"""
        try:
            response = self.table.query(
                IndexName="PostIdIndex",
                KeyConditionExpression="postId = :postId",
                ExpressionAttributeValues={":postId": post_id},
            )
            items = response.get("Items", [])
            if not items:
                return None
            item = items[0]
            raw_urls = item.get("imageKeys") or item.get("imageUrls") or []
            return Post(
                postId=item["postId"],
                userId=item["userId"],
                nickname=item.get("nickname"),
                content=item["content"],
                tags=item.get("tags", []),
                createdAt=item["createdAt"],
                updatedAt=item.get("updatedAt"),
                imageUrls=self._resolve_image_urls(raw_urls),
            )
        except Exception as e:
            logger.error(f"Error getting post {post_id}: {e}")
            raise

    def delete_post(self, post_id: str, user: UserInfo) -> dict:
        """投稿を削除 (DynamoDB DeleteItem)"""
        try:
            # まず postId から SK を取得
            response = self.table.query(
                IndexName="PostIdIndex",
                KeyConditionExpression="postId = :postId",
                ExpressionAttributeValues={":postId": post_id},
            )

            if not response.get("Items"):
                raise ValueError(f"Post not found: {post_id}")

            item = response["Items"][0]

            # ユーザー権限チェック
            if item["userId"] != user.user_id and not user.is_admin:
                raise PermissionError("You do not have permission to delete this post")

            # 削除
            self.table.delete_item(Key={"PK": "POSTS", "SK": item["SK"]})

            return {"status": "deleted", "post_id": post_id}
```

```
225
226         except Exception as e:
227             logger.error(f"Error deleting post: {e}")
228             raise
229
230     def get_profile(self, user_id: str) -> ProfileResponse:
231         """プロフィールを取得 (DynamoDB)"""
232         try:
233             result = self.table.get_item(
234                 Key={"PK": "PROFILES", "SK": user_id}
235             )
236             item = result.get("Item")
237             if not item:
238                 return ProfileResponse(
239                     user_id=user_id,
240                     nickname=None,
241                     bio=None,
242                     avatar_url=None,
243                 )
244             return ProfileResponse(
245                 user_id=user_id,
246                 nickname=item.get("nickname"),
247                 bio=item.get("bio"),
248                 avatar_url=item.get("avatar_url"),
249                 created_at=item.get("created_at"),
250                 updated_at=item.get("updated_at"),
251             )
252         except Exception as e:
253             logger.error(f"Error getting profile for {user_id}: {e}")
254             raise
255
256     def update_profile(
257         self,
258         user: UserInfo,
259         body: ProfileUpdateRequest,
260     ) -> ProfileResponse:
261         """プロフィールを更新 (DynamoDB)"""
262         now = datetime.now(timezone.utc).isoformat()
263
264         # 既存プロフィールを取得して created_at を保持し、未指定フィールドをマージ
265         existing = self.table.get_item(
266             Key={"PK": "PROFILES", "SK": user.user_id}
267         ).get("Item")
268         created_at = existing.get("created_at", now) if existing else now
269
270         item: dict = {
271             "PK": "PROFILES",
272             "SK": user.user_id,
273             "user_id": user.user_id,
274             "created_at": created_at,
275             "updated_at": now,
276         }
277         # 既存データをベースにしてリクエストで指定されたフィールドのみ上書き
278         if existing:
279             for field in ("nickname", "bio", "avatar_url"):
280                 if field in existing:
281                     item[field] = existing[field]
282         if body.nickname is not None:
```

```
            item["nickname"] = body.nickname
        if body.bio is not None:
            item["bio"] = body.bio
        if body.avatar_key is not None:
            item["avatar_url"] = body.avatar_key

        try:
            self.table.put_item(Item=item)
        except Exception as e:
            logger.error(f"Error updating profile for {user.user_id}: {e}")
            raise

        return ProfileResponse(
            user_id=user.user_id,
            nickname=item.get("nickname"),
            bio=item.get("bio"),
            avatar_url=item.get("avatar_url"),
            created_at=created_at,
            updated_at=now,
        )

    def generate_upload_urls(
        self,
        count: int,
        user: UserInfo,
        content_types: Optional[list[str]] = None,
    ) -> list[dict[str, str]]:
        """画像アップロード用の署名付きを生成URL"""
        if not self.bucket_name:
            raise ValueError("IMAGES_BUCKET_NAME not configured")

        ext_map = {
            "image/jpeg": "jpg",
            "image/jpg": "jpg",
            "image/png": "png",
            "image/gif": "gif",
            "image/webp": "webp",
            "image/heic": "heic",
            "image/heif": "heif",
        }

        urls = []
        for i in range(count):
            ct = (
                content_types[i]
                if content_types and i < len(content_types)
                else None
            ) or "image/jpeg"
            ext = ext_map.get(ct, "jpg")
            image_id = str(uuid.uuid4())
            key = f"{user.user_id}/{image_id}.{ext}"

            # 署名付きを生成URL (用PUT)
            presigned_url = self.s3_client.generate_presigned_url(
                "put_object",
                Params={
                    "Bucket": self.bucket_name,
                    "Key": key,
```

```
                    "ContentType": ct,
                },
                ExpiresIn=3600,  # 時間1
            )

            urls.append(
                {
                    "url": presigned_url,
                    "key": key,
                }
            )

        return urls
```

4.8 backends/azure_backend.py - Azure実装

```
"""Azure Backend Implementation using Cosmos DB + Blob Storage"""

import logging
import uuid
from datetime import datetime, timedelta, timezone

from fastapi import HTTPException

from app.auth import UserInfo
from app.backends.base import BackendBase
from app.config import settings
from app.models import (
    CreatePostBody,
    Post,
    ProfileResponse,
    ProfileUpdateRequest,
    UpdatePostBody,
)

logger = logging.getLogger(__name__)

try:
    from azure.cosmos import CosmosClient, PartitionKey
    from azure.cosmos import exceptions as cosmos_exceptions

    _cosmos_available = True
except ImportError:
    _cosmos_available = False
    logger.warning("azure-cosmos not available")

try:
    from azure.storage.blob import (
        BlobSasPermissions,
        BlobServiceClient,
        generate_blob_sas,
    )

    _blob_available = True
except ImportError:
    _blob_available = False
```

```
    logger.warning("azure-storage-blob not available")


class AzureBackend(BackendBase):
    """実装Azure (Cosmos DB + Blob Storage + Azure AD B2C)"""

    def __init__(self):
        if not _cosmos_available:
            raise ImportError("azure-cosmos is required for Azure backend")

        endpoint = settings.cosmos_db_endpoint
        key = settings.cosmos_db_key
        if not endpoint or not key:
            raise ValueError(
                "Cosmos DB credentials not configured. "
                "Set COSMOS_DB_ENDPOINT and COSMOS_DB_KEY environment variables."
            )

        self.client = CosmosClient(endpoint, key)
        db_name = settings.cosmos_db_database or "simple-sns"

        # データベースを取得または作成
        self.database = self.client.create_database_if_not_exists(id=db_name)

        # posts コンテナ
        self.posts_container = self.database.create_container_if_not_exists(
            id="posts",
            partition_key=PartitionKey(path="/postId"),
        )
        # profiles コンテナ
        self.profiles_container = self.database.create_container_if_not_exists(
            id="profiles",
            partition_key=PartitionKey(path="/userId"),
        )

        # Blob Storage の設定
        self.storage_account = settings.azure_storage_account_name
        self.storage_key = settings.azure_storage_account_key
        self.images_container = settings.azure_storage_container

        logger.info(
            f"AzureBackend initialized: db={db_name}, "
            f"storage_account={self.storage_account}"
        )

    def _blob_key_to_read_sas_url(self, blob_name: str) -> str:
        """キーを読み取り用BlobSAS に変換URL 時間有効(24)"""
        if not _blob_available or not self.storage_key:
            return (
                f"https://{self.storage_account}.blob.core.windows.net/"
                f"{self.images_container}/{blob_name}"
            )
        sas_token = generate_blob_sas(
            account_name=self.storage_account,
            container_name=self.images_container,
            blob_name=blob_name,
            account_key=self.storage_key,
            permission=BlobSasPermissions(read=True),
```

```
            expiry=datetime.now(timezone.utc) + timedelta(hours=24),
        )
        return (
            f"https://{self.storage_account}.blob.core.windows.net/"
            f"{self.images_container}/{blob_name}?{sas_token}"
        )

    def _resolve_image_urls(self, keys: list) -> list[str]:
        """キー/リストを読み取りに変換URLSASURL (AWS _resolve_image_urls と同パターン)

        - フルBlobURL → キーを抽出してを付与SAS
        - 生のキーBlob → を付与SAS
        - 他のhttps:// URL → そのまま返す
        - http:// URL → Mixed Content のためスキップ
        """
        if not self.storage_account or not keys:
            return []
        prefix = (
            f"https://{self.storage_account}.blob.core.windows.net/"
            f"{self.images_container}/"
        )
        result = []
        for k in keys:
            if not k or not isinstance(k, str):
                continue
            if k.startswith(prefix):
                # 既存の直接BlobURL: キーを抽出してを付与SAS
                blob_key = k[len(prefix) :].split("?")[0]
            elif k.startswith("https://"):
                # 別のhttps はそのまま通すURL
                result.append(k)
                continue
            elif k.startswith("http://"):
                logger.warning(
                    f"Skipping insecure HTTP image URL (mixed content): {k[:80]}"
                )
                continue
            else:
                # 生のキーBlob
                blob_key = k
            try:
                result.append(self._blob_key_to_read_sas_url(blob_key))
            except Exception as e:
                logger.warning(f"Failed to generate SAS read URL for {k}: {e}")
        return result

    def _item_to_post(self, item: dict) -> Post:
        """Cosmos アイテムをモデルに変換DBPost"""
        raw_urls = item.get("imageKeys") or item.get("imageUrls") or []
        return Post(
            postId=item.get("postId", item.get("id", "")),
            userId=item.get("userId", "unknown"),
            nickname=item.get("nickname"),
            content=item.get("content", ""),
            isMarkdown=item.get("isMarkdown", False),
            imageUrls=self._resolve_image_urls(raw_urls),
            tags=item.get("tags", []),
            createdAt=item.get("createdAt", datetime.now(timezone.utc).isoformat()),
```

```
            updatedAt=item.get("updatedAt"),
        )

    def list_posts(
        self,
        limit: int,
        next_token: str | None,
        tag: str | None,
    ) -> tuple[list[Post], str | None]:
        """Cosmos から投稿一覧を取得DB"""
        try:
            if tag:
                query = (
                    "SELECT * FROM c WHERE ARRAY_CONTAINS(c.tags, @tag) "
                    "ORDER BY c.createdAt DESC OFFSET 0 LIMIT @limit"
                )
                parameters = [
                    {"name": "@tag", "value": tag},
                    {"name": "@limit", "value": limit + 1},
                ]
            else:
                query = (
                    "SELECT * FROM c ORDER BY c.createdAt DESC OFFSET 0 LIMIT @limit"
                )
                parameters = [{"name": "@limit", "value": limit + 1}]

            # next_token (offset) を数値トークンとして扱う
            offset = 0
            if next_token:
                try:
                    offset = int(next_token)
                except (ValueError, TypeError):
                    offset = 0

            if offset > 0:
                if tag:
                    query = (
                        "SELECT * FROM c WHERE ARRAY_CONTAINS(c.tags, @tag) "
                        "ORDER BY c.createdAt DESC OFFSET @offset LIMIT @limit"
                    )
                    parameters = [
                        {"name": "@tag", "value": tag},
                        {"name": "@offset", "value": offset},
                        {"name": "@limit", "value": limit + 1},
                    ]
                else:
                    query = (
                        "SELECT * FROM c ORDER BY c.createdAt DESC "
                        "OFFSET @offset LIMIT @limit"
                    )
                    parameters = [
                        {"name": "@offset", "value": offset},
                        {"name": "@limit", "value": limit + 1},
                    ]

            items = list(
                self.posts_container.query_items(
                    query=query,
```

```
                parameters=parameters,
                enable_cross_partition_query=True,
            )
        )

        output_next_token = None
        if len(items) > limit:
            items = items[:limit]
            output_next_token = str(offset + limit)

        posts = [self._item_to_post(item) for item in items]
        return posts, output_next_token

    except Exception as e:
        logger.error(f"Error listing posts from Cosmos DB: {e}")
        raise

def create_post(self, body: CreatePostBody, user: UserInfo) -> dict:
    """Cosmos に投稿を作成DB"""
    try:
        post_id = str(uuid.uuid4())
        now_str = datetime.now(timezone.utc).isoformat()

        # プロフィールからを取得nickname
        nickname = None
        try:
            profile_item = self.profiles_container.read_item(
                item=user.user_id, partition_key=user.user_id
            )
            if profile_item:
                nickname = profile_item.get("nickname")
        except Exception as e:
            logger.warning(f"Failed to fetch nickname for {user.user_id}: {e}")

        # 画像キーをそのまま保存 取得時に(SAS に変換URL)
        image_keys = list(body.image_keys) if body.image_keys else []

        item = {
            "id": post_id,
            "postId": post_id,
            "userId": user.user_id,
            "nickname": nickname,
            "content": body.content,
            "isMarkdown": body.is_markdown,
            "imageKeys": image_keys,  # 生のキーを保存Blob (なしSAS)
            "tags": body.tags or [],
            "createdAt": now_str,
            "updatedAt": None,
        }

        self.posts_container.create_item(body=item)
        logger.info(f"Created post {post_id} by user {user.user_id}")

        presigned_urls = self._resolve_image_urls(image_keys)

        return Post(
            postId=post_id,
            userId=user.user_id,
```

```
                nickname=nickname,
                content=body.content,
                isMarkdown=body.is_markdown,
                imageUrls=presigned_urls,
                tags=body.tags or [],
                createdAt=now_str,
            ).model_dump()

        except Exception as e:
            logger.error(f"Error creating post in Cosmos DB: {e}")
            raise

    def get_post(self, post_id: str):
        """Cosmos から投稿を件取得DB1"""
        try:
            item = self.posts_container.read_item(item=post_id, partition_key=post_id)
        except Exception:
            return None
        from app.models import Post

        raw_urls = item.get("imageKeys") or item.get("imageUrls") or []
        return Post(
            postId=item["id"],
            userId=item["userId"],
            nickname=item.get("nickname"),
            content=item["content"],
            tags=item.get("tags") or [],
            createdAt=item["createdAt"],
            updatedAt=item.get("updatedAt"),
            imageUrls=self._resolve_image_urls(raw_urls),
        )

    def delete_post(self, post_id: str, user: UserInfo) -> dict:
        """Cosmos から投稿を削除DB"""
        try:
            item = self.posts_container.read_item(item=post_id, partition_key=post_id)
        except cosmos_exceptions.CosmosResourceNotFoundError:
            from fastapi import HTTPException

            raise HTTPException(status_code=404, detail="Post not found")

        if item.get("userId") != user.user_id:
            from fastapi import HTTPException

            raise HTTPException(status_code=403, detail="Not authorized")

        self.posts_container.delete_item(item=post_id, partition_key=post_id)
        logger.info(f"Deleted post {post_id}")
        return {"message": "Post deleted successfully", "postId": post_id}

    def update_post(self, post_id: str, body: UpdatePostBody, user: UserInfo) -> dict:
        """Cosmos の投稿を更新DB"""
        try:
            item = self.posts_container.read_item(item=post_id, partition_key=post_id)
        except cosmos_exceptions.CosmosResourceNotFoundError:
            raise HTTPException(status_code=404, detail="Post not found")

        if item.get("userId") != user.user_id and not user.is_admin:
```

```
            raise HTTPException(
                status_code=403, detail="You can only update your own posts"
            )

        now_str = datetime.now(timezone.utc).isoformat()

        if body.content is not None:
            item["content"] = body.content
        if body.is_markdown is not None:
            item["isMarkdown"] = body.is_markdown
        if body.tags is not None:
            item["tags"] = body.tags
        if body.image_keys is not None:
            item["imageKeys"] = list(body.image_keys)

        item["updatedAt"] = now_str

        updated_item = self.posts_container.replace_item(item=post_id, body=item)
        return self._item_to_post(updated_item).model_dump(by_alias=True)

    def get_profile(self, user_id: str) -> ProfileResponse:
        """Cosmos からプロフィールを取得DB"""
        try:
            item = self.profiles_container.read_item(
                item=user_id, partition_key=user_id
            )
            return ProfileResponse(
                userId=user_id,
                nickname=item.get("nickname"),
                bio=item.get("bio"),
                avatarUrl=item.get("avatarUrl"),
                createdAt=item.get("createdAt"),
                updatedAt=item.get("updatedAt"),
            )
        except cosmos_exceptions.CosmosResourceNotFoundError:
            return ProfileResponse(userId=user_id)
        except Exception as e:
            logger.error(f"Error getting profile {user_id}: {e}")
            raise

    def update_profile(
        self,
        user: UserInfo,
        body: ProfileUpdateRequest,
    ) -> ProfileResponse:
        """Cosmos のプロフィールを更新DB"""
        try:
            now_str = datetime.now(timezone.utc).isoformat()
            user_id = user.user_id

            try:
                existing = self.profiles_container.read_item(
                    item=user_id, partition_key=user_id
                )
            except cosmos_exceptions.CosmosResourceNotFoundError:
                existing = {
                    "id": user_id,
                    "userId": user_id,
```

```
                    "createdAt": now_str,
                }

            if body.nickname is not None:
                existing["nickname"] = body.nickname
            if body.bio is not None:
                existing["bio"] = body.bio
            if body.avatar_key is not None:
                existing["avatarUrl"] = (
                    f"https://{self.storage_account}.blob.core.windows.net/"
                    f"{self.images_container}/{body.avatar_key}"
                )
            existing["updatedAt"] = now_str

            self.profiles_container.upsert_item(body=existing)
            return self.get_profile(user_id)

        except Exception as e:
            logger.error(f"Error updating profile {user.user_id}: {e}")
            raise

    def generate_upload_urls(
        self,
        count: int,
        user: UserInfo,
        content_types: list[str] | None = None,
    ) -> list[dict[str, str]]:
        """Azure Blob Storage の SAS を生成URL"""
        if not _blob_available:
            raise ImportError("azure-storage-blob is required")

        ext_map = {
            "image/jpeg": "jpg",
            "image/jpg": "jpg",
            "image/png": "png",
            "image/gif": "gif",
            "image/webp": "webp",
            "image/heic": "heic",
            "image/heif": "heif",
        }
        urls = []
        account = self.storage_account
        key = self.storage_key
        container = self.images_container

        for i in range(count):
            ct = (
                content_types[i] if content_types and i < len(content_types) else None
            ) or "image/jpeg"
            ext = ext_map.get(ct, "jpg")
            # blob_name must NOT include the container name prefix.
            # URL is built as: https://account.blob.core.windows.net/{container}/{blob_name}
            # Container is already self.images_container (e.g. "images"), so
            # prefixing blob_name with "images/" would produce /images/images/...
            blob_name = f"{user.user_id}/{uuid.uuid4()}.{ext}"
            sas_token = generate_blob_sas(
                account_name=account,
                container_name=container,
```

```
                blob_name=blob_name,
                account_key=key,
                permission=BlobSasPermissions(write=True, create=True),
                expiry=datetime.now(timezone.utc)
                + timedelta(seconds=settings.presigned_url_expiry),
                content_type=ct,
            )
            upload_url = f"https://{account}.blob.core.windows.net/{container}/{blob_name}?{sas_token}"
            urls.append({"url": upload_url, "key": blob_name})

        return urls
```

4.9 backends/gcp_backend.py - GCP実装

```
"""GCP Backend Implementation using Firestore + Cloud Storage"""

import uuid
import logging
from datetime import datetime, timezone, timedelta
from typing import Optional, Tuple

from app.backends.base import BackendBase
from app.models import Post, CreatePostBody, ProfileResponse, ProfileUpdateRequest
from app.auth import UserInfo
from app.config import settings

logger = logging.getLogger(__name__)

try:
    from google.cloud import firestore, storage
    import google.auth
    import google.auth.transport.requests
    _gcp_available = True
except ImportError:
    _gcp_available = False
    logger.warning("google-cloud-firestore/storage not available")


class GcpBackend(BackendBase):
    """実装GCP (Firestore + Cloud Storage + Firebase Auth)"""

    def __init__(self):
        if not _gcp_available:
            raise ImportError("google-cloud-firestore and google-cloud-storage are required")

        project_id = settings.gcp_project_id
        self.db = firestore.Client(project=project_id)
        self.storage_client = storage.Client(project=project_id)

        self.posts_collection = settings.gcp_posts_collection
        self.profiles_collection = settings.gcp_profiles_collection
        self.bucket_name = settings.gcp_storage_bucket or f"{project_id}-uploads"

        # GCS 署名付き用URL: 認証情報をキャッシュ（毎リクエストのメタデータサーバー呼び出し回避）
        # generate_upload_urls で credentials.valid をチェックし、期限切れ時のみ refresh する
        try:
```

```
            self._gcs_credentials, _ = google.auth.default()
            self._gcs_auth_request = google.auth.transport.requests.Request()
        except Exception as e:
            logger.warning(f"Could not pre-fetch GCS credentials at init: {e}")
            self._gcs_credentials = None
            self._gcs_auth_request = None

        logger.info(
            f"GcpBackend initialized: project={project_id}, "
            f"posts={self.posts_collection}, profiles={self.profiles_collection}, "
            f"bucket={self.bucket_name}"
        )

    def _doc_to_post(self, doc) -> Post:
        """ドキュメントをモデルに変換FirestorePost"""
        data = doc.to_dict()

        # Firestore Timestamp → 文字列ISO
        def ts_to_str(ts) -> Optional[str]:
            if ts is None:
                return None
            if hasattr(ts, "isoformat"):
                return ts.isoformat()
            if hasattr(ts, "timestamp"):
                return datetime.fromtimestamp(ts.timestamp(), tz=timezone.utc).isoformat()
            return str(ts)

        return Post(
            postId=data.get("postId", doc.id),
            userId=data.get("userId", "unknown"),
            nickname=data.get("nickname"),
            content=data.get("content", ""),
            isMarkdown=data.get("isMarkdown", False),
            imageUrls=data.get("imageUrls", []),
            tags=data.get("tags", []),
            createdAt=ts_to_str(data.get("createdAt")) or datetime.now(timezone.utc).isoformat(),
            updatedAt=ts_to_str(data.get("updatedAt")),
        )

    def list_posts(
        self,
        limit: int,
        next_token: Optional[str],
        tag: Optional[str],
    ) -> Tuple[list[Post], Optional[str]]:
        """Firestore から投稿一覧を取得"""
        try:
            col = self.db.collection(self.posts_collection)
            query = col.order_by("createdAt", direction=firestore.Query.DESCENDING).limit(limit + 1)

            if next_token:
                # next_token はドキュメントとして使用ID
                cursor_doc = col.document(next_token).get()
                if cursor_doc.exists:
                    query = query.start_after(cursor_doc)

            if tag:
                query = query.where(filter=firestore.FieldFilter("tags", "array_contains", tag))
```

```python

            docs = list(query.stream())

            output_next_token = None
            if len(docs) > limit:
                docs = docs[:limit]
                output_next_token = docs[-1].id

            posts = [self._doc_to_post(doc) for doc in docs]
            return posts, output_next_token

        except Exception as e:
            logger.error(f"Error listing posts from Firestore: {e}")
            raise

    def create_post(self, body: CreatePostBody, user: UserInfo) -> dict:
        """に投稿を作成Firestore"""
        try:
            post_id = str(uuid.uuid4())
            now = datetime.now(timezone.utc)
            now_str = now.isoformat()

            # プロフィールからを取得nickname
            nickname = None
            try:
                profile_doc = self.db.collection(self.profiles_collection).document(user.user_id).get()
                if profile_doc.exists:
                    nickname = profile_doc.to_dict().get("nickname")
            except Exception as e:
                logger.warning(f"Failed to fetch nickname for {user.user_id}: {e}")

            # 画像キーをに変換URL
            image_urls = []
            if body.image_keys:
                image_urls = [
                    f"https://storage.googleapis.com/{self.bucket_name}/{key}"
                    for key in body.image_keys
                ]

            doc_data = {
                "postId": post_id,
                "userId": user.user_id,
                "nickname": nickname,
                "content": body.content,
                "isMarkdown": body.is_markdown,
                "imageUrls": image_urls,
                "tags": body.tags or [],
                "createdAt": now_str,
                "updatedAt": None,
            }

            self.db.collection(self.posts_collection).document(post_id).set(doc_data)
            logger.info(f"Created post {post_id} by user {user.user_id}")

            return Post(
                postId=post_id,
                userId=user.user_id,
                nickname=nickname,
```

```
                content=body.content,
                isMarkdown=body.is_markdown,
                imageUrls=image_urls,
                tags=body.tags or [],
                createdAt=now_str,
            ).model_dump()

        except Exception as e:
            logger.error(f"Error creating post in Firestore: {e}")
            raise

    def get_post(self, post_id: str):
        """から投稿を件取得Firestore1"""
        try:
            doc = self.db.collection(self.posts_collection).document(post_id).get()
            if not doc.exists:
                return None
            item = doc.to_dict()
            from app.models import Post
            return Post(
                postId=post_id,
                userId=item["userId"],
                nickname=item.get("nickname"),
                content=item["content"],
                tags=item.get("tags") or [],
                createdAt=item["createdAt"],
                updatedAt=item.get("updatedAt"),
                imageUrls=item.get("imageUrls") or [],
            )
        except Exception as e:
            logger.error(f"Error getting post {post_id}: {e}")
            raise

    def delete_post(self, post_id: str, user: UserInfo) -> dict:
        """から投稿を削除Firestore"""
        try:
            doc_ref = self.db.collection(self.posts_collection).document(post_id)
            doc = doc_ref.get()

            if not doc.exists:
                from fastapi import HTTPException
                raise HTTPException(status_code=404, detail="Post not found")

            data = doc.to_dict()
            if data.get("userId") != user.user_id:
                from fastapi import HTTPException
                raise HTTPException(status_code=403, detail="Not authorized")

            doc_ref.delete()
            logger.info(f"Deleted post {post_id}")
            return {"message": "Post deleted successfully", "postId": post_id}

        except Exception as e:
            logger.error(f"Error deleting post {post_id} from Firestore: {e}")
            raise

    def get_profile(self, user_id: str) -> ProfileResponse:
        """からプロフィールを取得Firestore"""
```

```
        try:
            doc_ref = self.db.collection(self.profiles_collection).document(user_id)
            doc = doc_ref.get()

            if not doc.exists:
                return ProfileResponse(userId=user_id)

            data = doc.to_dict()

            def ts_to_str(ts) -> Optional[str]:
                if ts is None:
                    return None
                if hasattr(ts, "isoformat"):
                    return ts.isoformat()
                if hasattr(ts, "timestamp"):
                    return datetime.fromtimestamp(ts.timestamp(), tz=timezone.utc).isoformat()
                return str(ts)

            return ProfileResponse(
                userId=user_id,
                nickname=data.get("nickname"),
                bio=data.get("bio"),
                avatarUrl=data.get("avatarUrl"),
                createdAt=ts_to_str(data.get("createdAt")),
                updatedAt=ts_to_str(data.get("updatedAt")),
            )

        except Exception as e:
            logger.error(f"Error getting profile {user_id} from Firestore: {e}")
            raise

    def update_profile(
        self,
        user: UserInfo,
        body: ProfileUpdateRequest,
    ) -> ProfileResponse:
        """のプロフィールを更新Firestore"""
        try:
            doc_ref = self.db.collection(self.profiles_collection).document(user.user_id)
            now_str = datetime.now(timezone.utc).isoformat()

            update_data: dict = {"updatedAt": now_str}
            if body.nickname is not None:
                update_data["nickname"] = body.nickname
            if body.bio is not None:
                update_data["bio"] = body.bio
            if body.avatar_key is not None:
                update_data["avatarUrl"] = (
                    f"https://storage.googleapis.com/{self.bucket_name}/{body.avatar_key}"
                )

            doc = doc_ref.get()
            if not doc.exists:
                update_data["createdAt"] = now_str
                update_data["userId"] = user.user_id
                doc_ref.set(update_data)
            else:
                doc_ref.update(update_data)
```

```
275
276             return self.get_profile(user.user_id)
277
278         except Exception as e:
279             logger.error(f"Error updating profile {user.user_id} in Firestore: {e}")
280             raise
281
282     def generate_upload_urls(
283         self,
284         count: int,
285         user: UserInfo,
286         content_types: Optional[list[str]] = None,
287     ) -> list[dict[str, str]]:
288         """Cloud Storage の署名付きを生成URL"""
289         ext_map = {
290             "image/jpeg": "jpg", "image/jpg": "jpg",
291             "image/png": "png", "image/gif": "gif",
292             "image/webp": "webp", "image/heic": "heic", "image/heif": "heif",
293         }
294         try:
295             # Cloud Functions / Cloud Run は Compute Engine 認証情報トークンのみ()を持つ。
296             # generate_signed_url には秘密鍵が必要なため、service_account_email と
297             # access_token を渡し、IAM signBlob API 経由で署名する方式を使用する。
298             # 認証情報は __init__ でキャッシュ済み。トークン期限切れ時のみ refresh する。
299             credentials = self._gcs_credentials
300             auth_request = self._gcs_auth_request
301             if credentials is None or auth_request is None:
302                 # フォールバック: 初期化失敗時
303                 credentials, _ = google.auth.default()
304                 auth_request = google.auth.transport.requests.Request()
305                 self._gcs_credentials = credentials
306                 self._gcs_auth_request = auth_request
307
308             # token が未取得または期限切れの場合のみ （時間に回程度）refresh11
309             if not getattr(credentials, "valid", True):
310                 credentials.refresh(auth_request)
311
312             access_token = credentials.token
313             # settings.gcp_service_account は環境変数 GCP_SERVICE_ACCOUNT から設定
314             sa_email = settings.gcp_service_account
315             if not sa_email:
316                 raise RuntimeError("GCP_SERVICE_ACCOUNT env var is not set")
317
318             bucket = self.storage_client.bucket(self.bucket_name)
319             urls = []
320
321             for i in range(count):
322                 ct = (
323                     content_types[i]
324                     if content_types and i < len(content_types)
325                     else None
326                 ) or "image/jpeg"
327                 ext = ext_map.get(ct, "jpg")
328                 key = f"images/{user.user_id}/{uuid.uuid4()}.{ext}"
329                 blob = bucket.blob(key)
330                 upload_url = blob.generate_signed_url(
331                     version="v4",
332                     expiration=timedelta(seconds=settings.presigned_url_expiry),
```

```
                    method="PUT",
                    content_type=ct,
                    service_account_email=sa_email,
                    access_token=access_token,
                )
                urls.append({"url": upload_url, "key": key})

            return urls

        except Exception as e:
            logger.error(f"Error generating upload URLs for GCS: {e}")
            raise
```

4.10 backends/local_backend.py - ローカル実装

```
"""Local Development Backend — DynamoDB Local + MinIO

Uses the same Single-Table Design as AwsBackend so that bugs found locally
behave identically in the cloud AWS environment.

Table: simple-sns-local  (configured via DYNAMODB_TABLE_NAME)
GSI:   PostIdIndex        (hash key: postId, projection: ALL)

Item types:
  Post    PK=POSTS          SK=<ISO timestamp>#<uuid>
  Profile PK=USER#<userId>  SK=PROFILE
          postId=PROFILE#<userId>  (used by PostIdIndex for profile lookups)
"""

import logging
import os
import time
import uuid
from datetime import datetime, timezone
from typing import Optional

import boto3
from botocore.config import Config
from botocore.exceptions import ClientError
from fastapi import HTTPException, status

from app.auth import UserInfo
from app.backends.base import BackendBase
from app.config import settings
from app.models import (
    CreatePostBody,
    Post,
    ProfileResponse,
    ProfileUpdateRequest,
    UpdatePostBody,
)

logger = logging.getLogger(__name__)

_POSTS_PK = "POSTS"

```

```python

class LocalBackend(BackendBase):
    """ローカル開発環境用バックエンド (DynamoDB Local + MinIO)"""

    def __init__(self):
        self._init_dynamodb()
        self._init_storage()

    # ----------------------------------------------------------------
    # Initialisation
    # ----------------------------------------------------------------

    def _init_dynamodb(self):
        """DynamoDB Local への接続を初期化し、テーブルを自動作成する"""
        endpoint = settings.dynamodb_endpoint or "http://localhost:8001"
        table_name = settings.dynamodb_table_name or "simple-sns-local"

        self.dynamodb = boto3.resource(
            "dynamodb",
            endpoint_url=endpoint,
            aws_access_key_id=os.environ.get("AWS_ACCESS_KEY_ID", "local"),
            aws_secret_access_key=os.environ.get(
                "AWS_SECRET_ACCESS_KEY", "local"),
            region_name=settings.aws_region or "ap-northeast-1",
        )
        self.table_name = table_name
        self._ensure_table()
        self.table = self.dynamodb.Table(table_name)
        logger.info(
            f"DynamoDB Local connected: endpoint={endpoint}, table={table_name}")

    def _ensure_table(self):
        """テーブルが存在しなければ作成する (PostIdIndex GSI 付き) """
        client = self.dynamodb.meta.client
        try:
            client.describe_table(TableName=self.table_name)
            logger.info(f"Table '{self.table_name}' already exists")
        except ClientError as e:
            if e.response["Error"]["Code"] != "ResourceNotFoundException":
                raise
            logger.info(f"Creating table '{self.table_name}' ...")
            client.create_table(
                TableName=self.table_name,
                AttributeDefinitions=[
                    {"AttributeName": "PK", "AttributeType": "S"},
                    {"AttributeName": "SK", "AttributeType": "S"},
                    {"AttributeName": "postId", "AttributeType": "S"},
                ],
                KeySchema=[
                    {"AttributeName": "PK", "KeyType": "HASH"},
                    {"AttributeName": "SK", "KeyType": "RANGE"},
                ],
                GlobalSecondaryIndexes=[
                    {
                        "IndexName": "PostIdIndex",
                        "KeySchema": [
                            {"AttributeName": "postId", "KeyType": "HASH"},
                        ],
```

```
                        "Projection": {"ProjectionType": "ALL"},
                        "ProvisionedThroughput": {
                            "ReadCapacityUnits": 5,
                            "WriteCapacityUnits": 5,
                        },
                    }
                ],
                ProvisionedThroughput={
                    "ReadCapacityUnits": 5,
                    "WriteCapacityUnits": 5,
                },
            )
            # waiter は DynamoDB Local で動作しないため polling で代替
            for _ in range(20):
                time.sleep(0.5)
                try:
                    client.describe_table(TableName=self.table_name)
                    break
                except ClientError:
                    pass
            logger.info(f"Table '{self.table_name}' created")

    def _init_storage(self):
        """MinIO クライアントを初期化する（失敗時はローカル FS にフォールバック）"""
        self.minio_client = None
        if not settings.minio_endpoint:
            logger.info("MINIO_ENDPOINT not set — using local filesystem URLs")
            return
        try:
            from minio import Minio

            endpoint = (
                settings.minio_endpoint
                .replace("http://", "")
                .replace("https://", "")
            )
            self.minio_client = Minio(
                endpoint,
                access_key=settings.minio_access_key or "minioadmin",
                secret_key=settings.minio_secret_key or "minioadmin",
                secure=settings.minio_endpoint.startswith("https://"),
            )
            bucket = settings.minio_bucket
            if not self.minio_client.bucket_exists(bucket):
                self.minio_client.make_bucket(bucket)
            logger.info(f"MinIO storage initialised: bucket={bucket}")
        except Exception as exc:
            logger.warning(
                f"MinIO not available, falling back to local FS: {exc}")
            self.minio_client = None

    # ----------------------------------------------------------------
    # Helpers
    # ----------------------------------------------------------------

    def _build_image_urls(self, image_keys: list[str]) -> Optional[list[str]]:
        if not image_keys:
            return None
```

```
        # /storage/ プロキシ経由でブラウザからアクセス ホスト非依存の相対( URL)
        bucket = settings.minio_bucket
        return [f"/storage/{bucket}/{k}" for k in image_keys]

    def _item_to_post(self, item: dict) -> Post:
        return Post(
            postId=item["postId"],
            userId=item.get("userId", ""),
            content=item.get("content", ""),
            isMarkdown=bool(item.get("isMarkdown", False)),
            tags=list(item.get("tags", [])),
            imageUrls=self._build_image_urls(list(item.get("imageKeys", []))),
            createdAt=item.get("createdAt", ""),
            updatedAt=item.get("updatedAt"),
            nickname=item.get("nickname"),
        )

    def _get_nickname(self, user_id: str) -> Optional[str]:
        try:
            res = self.table.query(
                IndexName="PostIdIndex",
                KeyConditionExpression="postId = :pid",
                ExpressionAttributeValues={":pid": f"PROFILE#{user_id}"},
            )
            items = res.get("Items", [])
            return items[0].get("nickname") if items else None
        except Exception:
            return None

    def _get_post_item_by_id(self, post_id: str) -> dict:
        """GSI で postId から DynamoDB アイテムを取得"""
        response = self.table.query(
            IndexName="PostIdIndex",
            KeyConditionExpression="postId = :pid",
            ExpressionAttributeValues={":pid": post_id},
        )
        items = response.get("Items", [])
        if not items:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail="Post not found",
            )
        return items[0]

    # ----------------------------------------------------------------
    # BackendBase implementation
    # ----------------------------------------------------------------

    def list_posts(
        self,
        limit: int,
        next_token: Optional[str],
        tag: Optional[str],
    ) -> tuple[list[Post], Optional[str]]:
        """投稿一覧を取得 (PK=POSTS, 降順) """
        kwargs: dict = {
            "KeyConditionExpression": "PK = :pk",
            "ExpressionAttributeValues": {":pk": _POSTS_PK},
```

```
            "ScanIndexForward": False,
            "Limit": limit,
        }
        if next_token:
            kwargs["ExclusiveStartKey"] = {"PK": _POSTS_PK, "SK": next_token}
        if tag:
            kwargs["FilterExpression"] = "contains(tags, :tag)"
            kwargs["ExpressionAttributeValues"][":tag"] = tag

        response = self.table.query(**kwargs)
        items = response.get("Items", [])

        # プロフィール（ニックネーム）をまとめて取得
        user_ids = list({item.get("userId")
                         for item in items if item.get("userId")})
        nicknames: dict[str, Optional[str]] = {}
        for uid in user_ids:
            nicknames[uid] = self._get_nickname(uid)

        posts = []
        for item in items:
            item["nickname"] = nicknames.get(item.get("userId"))
            posts.append(self._item_to_post(item))

        output_next_token = None
        if "LastEvaluatedKey" in response:
            output_next_token = response["LastEvaluatedKey"]["SK"]

        return posts, output_next_token

    def create_post(self, body: CreatePostBody, user: UserInfo) -> dict:
        """投稿を作成"""
        post_id = str(uuid.uuid4())
        now = datetime.now(timezone.utc).isoformat()

        item = {
            "PK": _POSTS_PK,
            "SK": f"{now}#{post_id}",
            "postId": post_id,
            "userId": user.user_id,
            "content": body.content,
            "isMarkdown": body.is_markdown or False,
            "tags": body.tags or [],
            "imageKeys": body.image_keys or [],
            "createdAt": now,
            "updatedAt": now,
        }
        self.table.put_item(Item=item)

        return {
            "postId": post_id,
            "userId": user.user_id,
            "nickname": self._get_nickname(user.user_id),
            "content": body.content,
            "isMarkdown": body.is_markdown or False,
            "imageKeys": body.image_keys,
            "imageUrls": self._build_image_urls(body.image_keys or []),
            "tags": body.tags,
```

```
            "createdAt": now,
        }

    def get_post(self, post_id: str):
        """投稿を件取得1"""
        with self._get_connection() as conn:
            from sqlalchemy import text
            result = conn.execute(
                text("SELECT * FROM posts WHERE id = :post_id"),
                {"post_id": post_id}
            )
            row = result.mappings().fetchone()
            if not row:
                return None
            from app.models import Post
            return Post(
                postId=row["id"],
                userId=row["user_id"],
                nickname=row.get("nickname"),
                content=row["content"],
                tags=row.get("tags") or [],
                createdAt=row["createdAt"],
                updatedAt=row.get("updatedAt"),
                imageUrls=row.get("imageUrls") or [],
            )

    def delete_post(self, post_id: str, user: UserInfo) -> dict:
        """投稿を削除"""
        with self._get_connection() as conn:
            from sqlalchemy import text
            # 投稿の所有者確認
            check_query = text("SELECT user_id FROM posts WHERE id = :post_id")
            result = conn.execute(check_query, {"post_id": post_id})
            row = result.fetchone()

            if not row:
                raise HTTPException(
                    status_code=status.HTTP_404_NOT_FOUND,
                    detail="Post not found",
                )

            owner_id = row[0]
            if owner_id != user.user_id and not user.is_admin:
                raise HTTPException(
                    status_code=status.HTTP_403_FORBIDDEN,
                    detail="You can only delete your own posts",
                )

            # 削除実行
            delete_query = text("DELETE FROM posts WHERE id = :post_id")
            conn.execute(delete_query, {"post_id": post_id})
            conn.commit()

        return {"message": "Post deleted successfully"}

    def get_post(self, post_id: str) -> dict:
        """投稿を取得"""
        item = self._get_post_item_by_id(post_id)
```

```
        return {
            "postId": item["postId"],
            "userId": item.get("userId"),
            "content": item.get("content"),
            "isMarkdown": bool(item.get("isMarkdown", False)),
            "tags": list(item.get("tags", [])),
            "imageUrls": self._build_image_urls(list(item.get("imageKeys", []))),
            "createdAt": item.get("createdAt"),
            "updatedAt": item.get("updatedAt"),
            "nickname": self._get_nickname(item.get("userId", "")),
        }

    def update_post(self, post_id: str, body: UpdatePostBody, user: UserInfo) -> dict:
        """投稿を更新"""
        item = self._get_post_item_by_id(post_id)
        if item["userId"] != user.user_id and not user.is_admin:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="You can only update your own posts",
            )
        now = datetime.now(timezone.utc).isoformat()
        update_expr = "SET updatedAt = :now"
        expr_values: dict = {":now": now}

        if body.content is not None:
            update_expr += ", content = :content"
            expr_values[":content"] = body.content
        if getattr(body, "is_markdown", None) is not None:
            update_expr += ", isMarkdown = :isMarkdown"
            expr_values[":isMarkdown"] = body.is_markdown
        if body.tags is not None:
            update_expr += ", tags = :tags"
            expr_values[":tags"] = body.tags
        if body.image_keys is not None:
            update_expr += ", imageKeys = :imageKeys"
            expr_values[":imageKeys"] = body.image_keys

        self.table.update_item(
            Key={"PK": _POSTS_PK, "SK": item["SK"]},
            UpdateExpression=update_expr,
            ExpressionAttributeValues=expr_values,
        )
        return self.get_post(post_id)

    def get_profile(self, user_id: str) -> ProfileResponse:
        """プロフィールを取得"""
        try:
            response = self.table.query(
                IndexName="PostIdIndex",
                KeyConditionExpression="postId = :pid",
                ExpressionAttributeValues={":pid": f"PROFILE#{user_id}"},
            )
            items = response.get("Items", [])
        except Exception as exc:
            logger.error(f"Failed to get profile for {user_id}: {exc}")
            items = []

        if not items:
```

```
            return ProfileResponse(
                userId=user_id,
                nickname=None,
                bio=None,
                avatarUrl=None,
                createdAt=datetime.now(timezone.utc).isoformat(),
                updatedAt=None,
            )

        item = items[0]
        avatar_url = None
        if item.get("avatarKey"):
            urls = self._build_image_urls([item["avatarKey"]])
            avatar_url = urls[0] if urls else None

        return ProfileResponse(
            userId=item.get("userId", user_id),
            nickname=item.get("nickname"),
            bio=item.get("bio"),
            avatarUrl=avatar_url,
            createdAt=item.get("createdAt", ""),
            updatedAt=item.get("updatedAt"),
        )

    def update_profile(self, user: UserInfo, body: ProfileUpdateRequest) -> ProfileResponse:
        """プロフィールを更新 () UPSERT"""
        now = datetime.now(timezone.utc).isoformat()
        pk = f"USER#{user.user_id}"
        sk = "PROFILE"

        try:
            existing = self.table.get_item(
                Key={"PK": pk, "SK": sk}).get("Item")
        except Exception:
            existing = None

        created_at = existing.get("createdAt", now) if existing else now

        self.table.put_item(Item={
            "PK": pk,
            "SK": sk,
            "postId": f"PROFILE#{user.user_id}",
            "userId": user.user_id,
            "nickname": body.nickname or (existing.get("nickname") if existing else None),
            "bio": body.bio or (existing.get("bio") if existing else None),
            "avatarKey": body.avatar_key or (existing.get("avatarKey") if existing else None),
            "createdAt": created_at,
            "updatedAt": now,
        })
        return self.get_profile(user.user_id)

    def generate_upload_urls(self, count: int, user: UserInfo, content_types: Optional[list[str]] = None) ->
     list[dict[str, str]]:
        """画像アップロード用の署名付き URL を生成"""
        urls = []
        if not self.minio_client:
            # MinIO 未接続時はダミー URL
            for _ in range(count):
```

```
                key = f"images/{user.user_id}/{uuid.uuid4()}.jpg"
                urls.append(
                    {"url": f"http://localhost:8000/uploads/{key}", "key": key})
            return urls

        # boto3 で presigned URL を生成 (HTTP 接続なし・純粋なローカル計算)
        # minio:9000 (Docker 内部ホスト) で署名し /storage/ プロキシ経由の相対 URL に変換
        s3_signing = boto3.client(
            "s3",
            endpoint_url="http://minio:9000",  # 内部ホストで署名
            aws_access_key_id=settings.minio_access_key or "minioadmin",
            aws_secret_access_key=settings.minio_secret_key or "minioadmin",
            region_name="us-east-1",
            config=Config(
                signature_version="s3v4",
                s3={"addressing_style": "path"},
            ),
        )
        for _ in range(count):
            key = f"images/{user.user_id}/{uuid.uuid4()}"
            try:
                upload_url = s3_signing.generate_presigned_url(
                    "put_object",
                    Params={
                        "Bucket": settings.minio_bucket,
                        "Key": key,
                        # ContentType を含めないことで署名に Content-Type ヘッダーが紛つかなくなり、
                        # JPEG/PNG/HEIC 等任意のファイル形式をアップロードできる
                    },
                    ExpiresIn=settings.presigned_url_expiry,
                )
            except Exception as exc:
                logger.error(f"Failed to generate presigned URL: {exc}")
                raise HTTPException(
                    status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                    detail="Failed to generate upload URL",
                ) from exc
            # http://minio:9000 → /storage 相対( URL) に変換してブラウザから使用可能にする
            proxy_url = upload_url.replace("http://minio:9000", "/storage", 1)
            urls.append({"url": proxy_url, "key": key})
        return urls

    def like_post(self, post_id: str, user: UserInfo) -> dict:
        """いいね機能（未実装）"""
        return {"post_id": post_id, "liked": True}

    def unlike_post(self, post_id: str, user: UserInfo) -> dict:
        """いいね取り消し機能（未実装）"""
        return {"post_id": post_id, "liked": False}
```

4.11 routes/posts.py - 投稿API

```
from fastapi import APIRouter, Depends, HTTPException, Query
from app.auth import UserInfo, require_user
from app.backends import get_backend
from app.config import settings
```

```
from app.models import CreatePostBody, ListPostsResponse, Post, UpdatePostBody

router = APIRouter(prefix="/posts", tags=["posts"])


@router.get("", response_model=ListPostsResponse)
def list_posts(
    limit: int = Query(20, ge=1, le=50, description="取得件数"),
    nextToken: str | None = Query(None, description="ページネーショントークン"),
    tag: str | None = Query(None, description="タグフィルター"),
) -> ListPostsResponse:
    """投稿一覧を取得"""
    backend = get_backend()
    posts, output_next_token = backend.list_posts(limit, nextToken, tag)
    return ListPostsResponse(items=posts, limit=limit, nextToken=output_next_token)


@router.get("/{post_id}")
def get_post(post_id: str) -> Post:
    """投稿を件取得1"""
    backend = get_backend()
    post = backend.get_post(post_id)
    if post is None:
        raise HTTPException(status_code=404, detail="Post not found")
    return post


@router.post("", status_code=201)
def create_post(
    body: CreatePostBody,
    user: UserInfo = Depends(require_user),
) -> dict:
    """投稿を作成"""
    limit = settings.max_images_per_post
    if body.image_keys and len(body.image_keys) > limit:
        raise HTTPException(
            status_code=400,
            detail=f"画像は投稿あたり1{limit枚までです（送信}: {len(body.image_keys)枚）}",
        )
    backend = get_backend()
    return backend.create_post(body, user)


@router.delete("/{post_id}")
def delete_post(
    post_id: str,
    user: UserInfo = Depends(require_user),
) -> dict:
    """投稿を削除"""
    backend = get_backend()
    return backend.delete_post(post_id, user)


@router.put("/{post_id}")
def update_post(
    post_id: str,
    body: UpdatePostBody,
    user: UserInfo = Depends(require_user),
```

```
) -> dict:
    """投稿を更新"""
    backend = get_backend()
    return backend.update_post(post_id, body, user)
```

4.12 routes/profile.py - プロフィールAPI

```
from fastapi import APIRouter, Depends

from app.auth import UserInfo, require_user
from app.backends import get_backend
from app.models import ProfileResponse, ProfileUpdateRequest

router = APIRouter(prefix="/profile", tags=["profile"])


@router.get("/{user_id}", response_model=ProfileResponse)
def get_profile(user_id: str) -> ProfileResponse:
    """プロフィールを取得"""
    backend = get_backend()
    return backend.get_profile(user_id)


@router.get("", response_model=ProfileResponse)
def get_my_profile(user: UserInfo = Depends(require_user)) -> ProfileResponse:
    """自分のプロフィールを取得"""
    backend = get_backend()
    return backend.get_profile(user.user_id)


@router.put("", response_model=ProfileResponse)
def update_profile(
    body: ProfileUpdateRequest,
    user: UserInfo = Depends(require_user),
) -> ProfileResponse:
    """プロフィールを更新"""
    backend = get_backend()
    return backend.update_profile(user, body)
```

4.13 routes/uploads.py - アップロードAPI

```
from fastapi import APIRouter, Depends, HTTPException
from app.auth import UserInfo, require_user
from app.backends import get_backend
from app.config import settings
from app.models import UploadUrlsRequest, UploadUrlsResponse

router = APIRouter(prefix="/uploads", tags=["uploads"])


@router.post("/presigned-urls", response_model=UploadUrlsResponse)
def generate_upload_urls(
    body: UploadUrlsRequest,
    user: UserInfo = Depends(require_user),
```

```
) -> UploadUrlsResponse:
    """画像アップロード用の署名付きを生成URL"""
    limit = settings.max_images_per_post
    if body.count > limit:
        raise HTTPException(
            status_code=400,
            detail=f"画像は投稿あたり1{limit枚までです（リクエスト}: {body.count枚）}",
        )
    backend = get_backend()
    urls = backend.generate_upload_urls(body.count, user, body.content_types)
    return UploadUrlsResponse(urls=urls)
```

4.14 index.py - AWS Lambda ハンドラー

```
"""AWS Lambda エントリーポイント"""

from mangum import Mangum
from app.main import app

# Lambda handler
handler = Mangum(app, lifespan="off")
```

4.15 function_app.py - Azure Functions ハンドラー

```
import azure.functions as func
import logging
import traceback
from urllib.parse import urlparse

# Safe import: インポート失敗時も関数を登録し、でエラー内容を返す503
_IMPORT_ERROR: str | None = None
fastapi_app = None
try:
    from app.main import app as fastapi_app
except Exception as _e:
    _IMPORT_ERROR = traceback.format_exc()
    logging.error(f"Failed to import FastAPI app: {_IMPORT_ERROR}")

# ---------------------------------------------------------------------
# Azure Functions Flex Consumption は同一インスタンスで複数リクエストを処理する。
# インポートをモジュールレベルに移動することで:
#   - リクエスト毎のモジュール解決コスト排除
#   - メモリ断片化リスクを抑制
#   - Application Insights への不要なオーバーヘッド削減
# 旧(: from fastapi import Request / Response / BytesIO は未使用のため削除)
# ---------------------------------------------------------------------

# Azure Functions のエントリーポイント
app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)


@app.function_name(name="HttpTrigger")
@app.route(route="{*route}", methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"])
async def main(req: func.HttpRequest) -> func.HttpResponse:
```

```
    """Azure Functions HTTP trigger that forwards to FastAPI"""
    logging.debug(f"HTTP trigger: {req.method} {req.url}")

    # インポート失敗時はでエラー内容を返す503
    if fastapi_app is None:
        import json
        return func.HttpResponse(
            body=json.dumps({"error": "Service unavailable", "detail": _IMPORT_ERROR}),
            status_code=503,
            headers={"Content-Type": "application/json"},
        )

    # CORS Preflight (OPTIONS) リクエストを直接処理
    if req.method == "OPTIONS":
        return func.HttpResponse(
            body="",
            status_code=204,
            headers={
                "Access-Control-Allow-Origin": "*",
                "Access-Control-Allow-Methods": "GET, POST, PUT, DELETE, PATCH, OPTIONS",
                "Access-Control-Allow-Headers": "Content-Type, Authorization, X-Requested-With",
                "Access-Control-Max-Age": "86400",
            },
        )

    # FastAPI アプリケーションをASGIAzure で実行Functions

    # Azure Functions Request → FastAPI Request 変換
    # route_params には "HttpTrigger/api/messages" のような値が入っている
    # または "HttpTrigger/messages、""HttpTrigger/health" の場合もある
    route_path = req.route_params.get("route", "")

    # "HttpTrigger/" プレフィックスを削除
    if route_path.startswith("HttpTrigger/"):
        route_path = route_path[len("HttpTrigger/"):]
    elif route_path == "HttpTrigger":
        route_path = ""

    path = "/" + route_path if route_path else "/"

    logging.debug(f"Converted path for FastAPI: {path}")

    # クエリ文字列を正しく抽出
    parsed = urlparse(req.url)
    query_string = parsed.query.encode("utf-8") if parsed.query else b""

    scope = {
        "type": "http",
        "asgi": {"version": "3.0"},
        "http_version": "1.1",
        "method": req.method,
        "scheme": "https",
        "path": path,
        "query_string": query_string,
        "root_path": "",
        "headers": [[k.encode(), v.encode()] for k, v in req.headers.items()],
        "server": (parsed.hostname or "localhost", 443),
        "client": ("127.0.0.1", 0),
```

```
    }

    # FastAPI を ASGI で実行
    async def receive():
        return {"type": "http.request", "body": req.get_body()}

    status_code = 200
    headers = []
    body_parts = []

    async def send(message):
        nonlocal status_code, headers, body_parts

        if message["type"] == "http.response.start":
            status_code = message["status"]
            headers = message.get("headers", [])
        elif message["type"] == "http.response.body":
            body_parts.append(message.get("body", b""))

    # FastAPI アプリケーション実行
    try:
        await fastapi_app(scope, receive, send)
    except Exception as e:
        logging.error(
            f"Error in FastAPI application: {type(e).__name__}: {e}", exc_info=True)
        return func.HttpResponse(
            body=f'{{"error": "{type(e).__name__}", "message": "{str(e)}"}}',
            status_code=500,
            headers={
                "Content-Type": "application/json",
                "Access-Control-Allow-Origin": "*",
            },
        )

    # レスポンス構築
    response_body = b"".join(body_parts)
    response_headers = {k.decode(): v.decode() for k, v in headers}

    # ヘッダーを追加（のミドルウェアからのヘッダーがない場合）CORSFastAPI
    if "Access-Control-Allow-Origin" not in response_headers:
        response_headers["Access-Control-Allow-Origin"] = "*"
        response_headers["Access-Control-Allow-Methods"] = "GET, POST, PUT, DELETE, PATCH, OPTIONS"
        response_headers["Access-Control-Allow-Headers"] = (
            "Content-Type, Authorization, X-Requested-With"
        )

    return func.HttpResponse(body=response_body, status_code=status_code, headers=response_headers)
```

4.16 function.py - GCP Cloud Run ハンドラー

```
"""Google Cloud Functions エントリーポイント"""
import asyncio
import functions_framework
from app.main import app as fastapi_app

# ----------------------------------------------------------------
```

```python
# GCP Cloud Functions Gen 2 は Cloud Run 上で動作するため、
# 同一インスタンスが複数リクエストを処理する。
# イベントループをモジュールレベルで保持し、リクエスト毎の
# new_event_loop() 生成・破棄を廃止することで:
#   - メモリ断片化リスクを排除
#   - コールドスタート後の余分な割当オーバーヘッドを削減
#   - asyncio タスクのリークを防止
# ------------------------------------------------------------------
_loop: asyncio.AbstractEventLoop | None = None


def _get_event_loop() -> asyncio.AbstractEventLoop:
    """モジュール共有イベントループを取得（クローズ済みなら再作成）"""
    global _loop
    if _loop is None or _loop.is_closed():
        _loop = asyncio.new_event_loop()
        asyncio.set_event_loop(_loop)
    return _loop


@functions_framework.http
def handler(request):
    """Cloud Functions HTTP handler that forwards to FastAPI"""

    # Cloud Functions Request → ASGI scope 変換
    scope = {
        "type": "http",
        "asgi": {"version": "3.0"},
        "http_version": "1.1",
        "method": request.method,
        "scheme": "https",
        "path": request.path,
        "query_string": request.query_string if request.query_string else b"",
        "headers": [[k.lower().encode(), v.encode()] for k, v in request.headers.items()],
        "server": (request.host.split(":")[0], int(request.host.split(":")[1]) if ":" in request.host else 443),
    }

    # Request body
    body = request.get_data()

    async def receive():
        return {"type": "http.request", "body": body, "more_body": False}

    # Response capture
    status_code = 200
    headers = []
    body_parts = []

    async def send(message):
        nonlocal status_code, headers, body_parts

        if message["type"] == "http.response.start":
            status_code = message["status"]
            headers = message.get("headers", [])
        elif message["type"] == "http.response.body":
            body_parts.append(message.get("body", b""))

    # FastAPI をモジュール共有ループで実行（インスタンス内再利用）
```

```
    loop = _get_event_loop()
    loop.run_until_complete(fastapi_app(scope, receive, send))

    # レスポンス構築
    response_body = b"".join(body_parts)
    response_headers = {
        k.decode(): v.decode()
        for k, v in headers
        if k.decode().lower() != "content-length"
    }

    return (response_body, status_code, response_headers)
```

Chapter 5

インフラストラクチャコード (Pulumi)

このセクションでは、Pulumi Pythonを使用したIaCコードを掲載します。

5.1 AWS Pulumi main.py

```python
"""
Multi-Cloud Auto Deploy - AWS Pulumi Implementation

Architecture:
- Lambda Function (Python 3.12, ZIP deployment)
- API Gateway (HTTP API v2)
- S3 (Static website hosting)
- CloudFront (Optional, for custom domain)

Cost: ~$2-5/month for low traffic
"""

import json
import os
import pathlib

import monitoring
import pulumi
import pulumi_aws as aws

# Configuration
config = pulumi.Config()
aws_config = pulumi.Config("aws")
region = aws_config.get("region") or "us-east-1"
stack = pulumi.get_stack()
project_name = "multicloud-auto-deploy"

# CORS allowed origins (configurable per environment)
allowed_origins = config.get("allowedOrigins") or "*"
allowed_origins_list = allowed_origins.split(",") if allowed_origins != "*" else ["*"]

# CloudFront domain (set after first deploy: pulumi config set cloudFrontDomain <domain>)
# Used for Cognito callback/logout URLs and frontend_web redirect URIs
cf_domain = config.get("cloudFrontDomain") or ""

# Custom domain (optional, e.g. staging.aws.ashnova.jp)
```

```
# Used for Cognito callback/logout URLs alongside the CloudFront domain
custom_domain = config.get("customDomain") or ""

# Common tags
common_tags = {
    "Project": project_name,
    "ManagedBy": "pulumi",
    "Environment": stack,
}


def unique_urls(urls: list[str]) -> list[str]:
    return list(dict.fromkeys(urls))


# ======================================
# IAM Role for Lambda
# ======================================
lambda_role = aws.iam.Role(
    "lambda-role",
    name=f"{project_name}-{stack}-lambda-role",
    assume_role_policy=json.dumps(
        {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Action": "sts:AssumeRole",
                    "Principal": {"Service": "lambda.amazonaws.com"},
                    "Effect": "Allow",
                }
            ],
        }
    ),
    tags=common_tags,
)

# Attach basic Lambda execution policy
aws.iam.RolePolicyAttachment(
    "lambda-basic-execution",
    role=lambda_role.name,
    policy_arn="arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole",
)

# Attach Secrets Manager read policy for Lambda
aws.iam.RolePolicyAttachment(
    "lambda-secrets-manager",
    role=lambda_role.name,
    policy_arn="arn:aws:iam::aws:policy/SecretsManagerReadWrite",
)

# ======================================
# AWS Secrets Manager
# ======================================
# Create a secret for database credentials or API keys
app_secret = aws.secretsmanager.Secret(
    "app-secret",
    name=f"{project_name}/{stack}/app-config",
    description=f"Application secrets for {project_name} {stack} environment",
```

```
    tags=common_tags,
)

# Store initial secret value (example - should be updated with actual values)
app_secret_version = aws.secretsmanager.SecretVersion(
    "app-secret-version",
    secret_id=app_secret.id,
    secret_string=pulumi.Output.secret(
        '{"database_url":"changeme","api_key":"changeme"}'
    ),
)

# ========================================
# Cognito User Pool for Authentication
# ========================================
user_pool = aws.cognito.UserPool(
    "user-pool",
    name=f"{project_name}-{stack}-users",
    mfa_configuration="OFF",
    password_policy={
        "minimum_length": 8,
        "require_lowercase": True,
        "require_numbers": True,
        "require_symbols": False,
        "require_uppercase": True,
        "temporary_password_validity_days": 7,
    },
    auto_verified_attributes=["email"],
    schemas=[
        {
            "attribute_data_type": "String",
            "name": "email",
            "required": True,
            "mutable": True,
        }
    ],
    account_recovery_setting={
        "recovery_mechanisms": [
            {
                "name": "verified_email",
                "priority": 1,
            }
        ]
    },
    email_configuration={
        "email_sending_account": "COGNITO_DEFAULT",
    },
    tags=common_tags,
)

# Cognito User Pool Client
user_pool_client = aws.cognito.UserPoolClient(
    "user-pool-client",
    name=f"{project_name}-{stack}-web-client",
    user_pool_id=user_pool.id,
    allowed_oauth_flows=["code"],
    allowed_oauth_scopes=["openid", "email", "profile"],
    allowed_oauth_flows_user_pool_client=True,
```

```
    callback_urls=unique_urls(
        ([f"https://{cf_domain}/sns/auth/callback"] if cf_domain else [])
        + ([f"https://{custom_domain}/sns/auth/callback"] if custom_domain else [])
    ),
    logout_urls=unique_urls(
        ([f"https://{cf_domain}/sns/"] if cf_domain else [])
        + ([f"https://{custom_domain}/sns/"] if custom_domain else [])
    ),
    access_token_validity=1,
    id_token_validity=1,
    refresh_token_validity=30,
    token_validity_units={
        "access_token": "hours",
        "id_token": "hours",
        "refresh_token": "days",
    },
    prevent_user_existence_errors="ENABLED",
    enable_token_revocation=True,
    explicit_auth_flows=[
        "ALLOW_ADMIN_USER_PASSWORD_AUTH",
        "ALLOW_REFRESH_TOKEN_AUTH",
        "ALLOW_USER_PASSWORD_AUTH",
        "ALLOW_USER_SRP_AUTH",
    ],
    supported_identity_providers=["COGNITO"],
)

# Cognito User Pool Domain
user_pool_domain = aws.cognito.UserPoolDomain(
    "user-pool-domain",
    domain=f"{project_name}-{stack}",
    user_pool_id=user_pool.id,
)

# ======================================
# DynamoDB Table (Single Table Design)
# ======================================
posts_table = aws.dynamodb.Table(
    "posts-table",
    name=f"{project_name}-{stack}-posts",
    billing_mode="PAY_PER_REQUEST",
    hash_key="PK",
    range_key="SK",
    attributes=[
        {
            "name": "PK",
            "type": "S",
        },
        {
            "name": "SK",
            "type": "S",
        },
        {
            "name": "postId",
            "type": "S",
        },
        {
            "name": "userId",
```

```
            "type": "S",
        },
        {
            "name": "createdAt",
            "type": "S",
        },
    ],
    global_secondary_indexes=[
        {
            "name": "PostIdIndex",
            "hash_key": "postId",
            "projection_type": "ALL",
        },
        {
            "name": "UserPostsIndex",
            "hash_key": "userId",
            "range_key": "createdAt",
            "projection_type": "ALL",
        },
    ],
    tags=common_tags,
)

# S3 Bucket for Images
images_bucket = aws.s3.BucketV2(
    "images-bucket",
    bucket=f"{project_name}-{stack}-images",
    tags=common_tags,
)

# Disable versioning for cost savings
aws.s3.BucketVersioningV2(
    "images-bucket-versioning",
    bucket=images_bucket.id,
    versioning_configuration={
        "status": "Disabled",
    },
)

# CORS configuration for browser uploads
aws.s3.BucketCorsConfigurationV2(
    "images-bucket-cors",
    bucket=images_bucket.id,
    cors_rules=[
        {
            "allowed_headers": ["*"],
            "allowed_methods": ["GET", "PUT", "POST", "DELETE", "HEAD"],
            "allowed_origins": allowed_origins_list,
            "expose_headers": ["ETag"],
            "max_age_seconds": 3000,
        }
    ],
)

# Block public access for images bucket
aws.s3.BucketPublicAccessBlock(
    "images-bucket-public-access",
    bucket=images_bucket.id,
```

```
    block_public_acls=True,
    block_public_policy=True,
    ignore_public_acls=True,
    restrict_public_buckets=True,
)

lambda_caller_identity = aws.get_caller_identity()
lambda_function_arn_for_self_invoke = (
    f"arn:aws:lambda:{region}:{lambda_caller_identity.account_id}:function:"
    f"{project_name}-{stack}-api"
)

# Create inline policy for DynamoDB and S3 access
lambda_policy = aws.iam.RolePolicy(
    "lambda-policy",
    role=lambda_role.id,
    policy=pulumi.Output.all(posts_table.arn, images_bucket.arn).apply(
        lambda args: json.dumps(
            {
                "Version": "2012-10-17",
                "Statement": [
                    {
                        "Effect": "Allow",
                        "Action": [
                            "dynamodb:GetItem",
                            "dynamodb:PutItem",
                            "dynamodb:UpdateItem",
                            "dynamodb:DeleteItem",
                            "dynamodb:Query",
                            "dynamodb:Scan",
                        ],
                        "Resource": [
                            args[0],
                            f"{args[0]}/index/*",
                        ],
                    },
                    {
                        "Effect": "Allow",
                        "Action": [
                            "s3:GetObject",
                            "s3:PutObject",
                            "s3:DeleteObject",
                        ],
                        "Resource": f"{args[1]}/*",
                    },
                    {
                        "Effect": "Allow",
                        "Action": ["s3:ListBucket"],
                        "Resource": args[1],
                    },
                    {
                        "Effect": "Allow",
                        "Action": [
                            "bedrock:InvokeModel",
                        ],
                        "Resource": "*",
                    },
                    {
```

```
                        "Effect": "Allow",
                        "Action": [
                            "lambda:InvokeFunction",
                        ],
                        "Resource": lambda_function_arn_for_self_invoke,
                    },
                    {
                        "Effect": "Allow",
                        "Action": [
                            "sns:Subscribe",
                            "sns:Unsubscribe",
                            "sns:ListSubscriptionsByTopic",
                        ],
                        "Resource": f"arn:aws:sns:{region}:{lambda_caller_identity.account_id}:*",
                    },
                ],
            }
        )
    ),
)

# ======================================
# Lambda Layer (Dependencies)
# ======================================
# Lambda Layer with all dependencies (FastAPI, Pydantic, Mangum, etc.)
# Build with: ./scripts/build-lambda-layer.sh
# The layer ZIP is automatically managed by Pulumi


# Path to Lambda Layer ZIP
# Strategy: Use GITHUB_WORKSPACE env var in CI, fallback to relative path for local development
workspace_root = os.environ.get("GITHUB_WORKSPACE")
if workspace_root:
    # GitHub Actions: GITHUB_WORKSPACE points to repository root
     (/home/runner/work/multicloud-auto-deploy/multicloud-auto-deploy)
    # Build step creates ZIP at services/api/lambda-layer.zip (relative to repository root)
    layer_zip_path = (
        pathlib.Path(workspace_root) / "services" / "api" / "lambda-layer.zip"
    )
else:
    # Local development: Calculate relative path from this file
    # __file__ -> infrastructure/pulumi/aws/__main__.py
    # Go up 3 levels to reach project root, then down to services/api
    layer_zip_path = (
        pathlib.Path(__file__).parent.parent.parent
        / "services"
        / "api"
        / "lambda-layer.zip"
    )

# Check if layer ZIP exists
if not layer_zip_path.exists():
    pulumi.log.warn(
        f"Lambda Layer ZIP not found at {layer_zip_path}. "
        "Run './scripts/build-lambda-layer.sh' to build the layer. "
        "Using placeholder for now."
    )
    # Create a minimal placeholder if ZIP doesn't exist
```

```
    layer_zip_path = None
else:
    pulumi.log.info(
        f"Lambda Layer ZIP found: {layer_zip_path} ({os.path.getsize(layer_zip_path) / 1024 / 1024:.2f} MB)"
    )

# Create Lambda Layer (only if ZIP exists)
lambda_layer = None
if layer_zip_path:
    lambda_layer = aws.lambda_.LayerVersion(
        "dependencies-layer",
        layer_name=f"{project_name}-{stack}-dependencies",
        code=pulumi.FileArchive(str(layer_zip_path)),
        compatible_runtimes=["python3.12"],
        description=f"Dependencies for {project_name} {stack} (FastAPI, Mangum, Pydantic, etc.)",
        opts=pulumi.ResourceOptions(
            # Delete old versions automatically (keeps only latest)
            delete_before_replace=True,
        ),
    )
    pulumi.export("lambda_layer_arn", lambda_layer.arn)
    pulumi.export("lambda_layer_version", lambda_layer.version)

# ========================================
# Lambda Function (FastAPI with Mangum)
# ========================================
# Note: Lambda function code is deployed separately using deploy-lambda-aws.sh script
# Pulumi manages the function configuration, but not the code itself

# Create a minimal placeholder Lambda function
# The actual code will be deployed via CI/CD or deploy script

placeholder_code = """
import json
def handler(event, context):
    return {
        'statusCode': 200,
        'body': json.dumps({'message': 'Please deploy actual code using deploy-lambda-aws.sh'})
    }
"""

lambda_function = aws.lambda_.Function(
    "api-function",
    name=f"{project_name}-{stack}-api",
    runtime="python3.12",
    handler="app.main.handler",  # FastAPI application entry point with Mangum
    role=lambda_role.arn,
    # 1769MB = 1 vCPU: コールドスタートを大幅短縮 (512MB は 0.3 vCPU で遅い)
    memory_size=1769,
    # OCR + 処理のため余裕を持たせるBedrock
    timeout=60,
    # Use x86_64 for compatibility with custom layers
    architectures=["x86_64"],
    # Lambda Layer is automatically managed by Pulumi
    # If lambda-layer.zip exists, it will be deployed as a Layer Version
    # and automatically attached to this function
    layers=[lambda_layer.arn] if lambda_layer else [],
    # Use inline code or skip code updates
```

```
    # Code will be uploaded separately via deploy-lambda-aws.sh
    code=pulumi.AssetArchive({"index.py": pulumi.StringAsset(placeholder_code)}),
    # Skip code and layer updates: deployed by deploy-aws.yml workflow
    # Layers are managed by the CI/CD pipeline (lambda-layer.zip is built at deploy time)
    # Also skip environment: CI/CD (Update Lambda step) sets CORS_ORIGINS with the correct
    # custom domain. If Pulumi manages environment, it sets CORS_ORIGINS from allowedOrigins
    # which may not include the custom domain, causing "CORS policy" errors on every deploy.
    opts=pulumi.ResourceOptions(
        ignore_changes=["code", "source_code_hash", "layers", "environment"]
    ),
    environment={
        "variables": {
            "ENVIRONMENT": stack,
            "CLOUD_PROVIDER": "aws",
            "AUTH_PROVIDER": "cognito",
            "AUTH_DISABLED": "false",
            "SECRET_NAME": app_secret.name,
            "COGNITO_USER_POOL_ID": user_pool.id,
            "COGNITO_CLIENT_ID": user_pool_client.id,
            "COGNITO_REGION": region,
            "POSTS_TABLE_NAME": posts_table.name,
            "IMAGES_BUCKET_NAME": images_bucket.id,
            "BEDROCK_REGION": "us-east-1",
            "BEDROCK_MODEL_ID": "amazon.nova-pro-v1:0",
            "SOLVE_OCR_REVIEW_MIN_SCORE": "0.40",
            "SOLVE_OCR_REVIEW_MAX_REPLACEMENT_RATIO": "0.01",
            "CORS_ORIGINS": allowed_origins,
        }
    },
    tags=common_tags,
)

# Lambda Function URL (no API Gateway needed for simple HTTP)
lambda_url = aws.lambda_.FunctionUrl(
    "api-function-url",
    function_name=lambda_function.name,
    authorization_type="NONE",  # Public access
    cors={
        "allow_origins": allowed_origins_list,
        "allow_methods": ["GET", "POST", "PUT", "DELETE", "PATCH"],
        "allow_headers": ["Content-Type", "Authorization", "X-Requested-With"],
        "max_age": 3600,
    },
)

# ======================================
# API Gateway HTTP API v2
# ======================================
api_gateway = aws.apigatewayv2.Api(
    "http-api",
    name=f"{project_name}-{stack}-api",
    protocol_type="HTTP",
    cors_configuration={
        "allow_origins": allowed_origins_list,
        "allow_methods": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
        "allow_headers": ["Content-Type", "Authorization", "X-Requested-With"],
        "max_age": 3600,
    },
```

```
    tags=common_tags,
)

# API Gateway Integration with Lambda (backend api)
integration = aws.apigatewayv2.Integration(
    "lambda-integration",
    api_id=api_gateway.id,
    integration_type="AWS_PROXY",
    integration_uri=lambda_function.arn,
    integration_method="POST",
    payload_format_version="2.0",
)

# API Gateway Route
route = aws.apigatewayv2.Route(
    "default-route",
    api_id=api_gateway.id,
    route_key="$default",  # Catch all routes
    target=integration.id.apply(lambda id: f"integrations/{id}"),
)

# API Gateway Stage
stage = aws.apigatewayv2.Stage(
    "default-stage",
    api_id=api_gateway.id,
    name="$default",
    auto_deploy=True,
)

# Permission for API Gateway to invoke Lambda
aws.lambda_.Permission(
    "api-gateway-invoke",
    action="lambda:InvokeFunction",
    function=lambda_function.name,
    principal="apigateway.amazonaws.com",
    source_arn=api_gateway.execution_arn.apply(lambda arn: f"{arn}/*/*"),
)

# ======================================
# S3 Bucket for Frontend
# ======================================
frontend_bucket = aws.s3.Bucket(
    "frontend-bucket",
    bucket=f"{project_name}-{stack}-frontend",
    website={
        "index_document": "index.html",
        "error_document": "index.html",  # For SPA routing
    },
    tags=common_tags,
)

# Block all public access — CloudFront OAI 経由のみアクセス許可
frontend_public_access = aws.s3.BucketPublicAccessBlock(
    "frontend-public-access",
    bucket=frontend_bucket.id,
    block_public_acls=True,
    block_public_policy=True,
    ignore_public_acls=True,
```

```
    restrict_public_buckets=True,
)

# Configure bucket for website hosting
aws.s3.BucketWebsiteConfigurationV2(
    "frontend-website-config",
    bucket=frontend_bucket.id,
    index_document={
        "suffix": "index.html",
    },
    error_document={
        "key": "index.html",
    },
)

# ========================================
# CloudFront Distribution for Frontend
# ========================================
# CloudFront Origin Access Identity (OAI) for S3 access
cloudfront_oai = aws.cloudfront.OriginAccessIdentity(
    "cloudfront-oai",
    comment=f"{project_name}-{stack} CloudFront OAI",
)

# S3 バケットポリシー: CloudFront OAI 経由のみ許可（パブリック直接アクセス禁止）
cloudfront_bucket_policy = aws.s3.BucketPolicy(
    "cloudfront-bucket-policy",
    bucket=frontend_bucket.id,
    policy=pulumi.Output.all(frontend_bucket.arn, cloudfront_oai.iam_arn).apply(
        lambda args: json.dumps(
            {
                "Version": "2012-10-17",
                "Statement": [
                    {
                        "Sid": "CloudFrontOAIAccess",
                        "Effect": "Allow",
                        "Principal": {"AWS": args[1]},
                        "Action": "s3:GetObject",
                        "Resource": f"{args[0]}/*",
                    },
                ],
            }
        )
    ),
    opts=pulumi.ResourceOptions(
        depends_on=[frontend_public_access], replace_on_changes=["bucket"]
    ),
)

# ========================================
# AWS WAF for CloudFront
# ========================================
# WAF Web ACL for DDoS protection and rate limiting
# Note: WAF is disabled for staging to reduce cost (~$6/month)
if stack == "production":
    waf_web_acl = aws.wafv2.WebAcl(
        "cloudfront-waf",
        name=f"{project_name}-{stack}-cloudfront-waf",
```

```
        scope="CLOUDFRONT",  # Must be CLOUDFRONT for CloudFront distributions
        default_action=aws.wafv2.WebAclDefaultActionArgs(
            allow={},
        ),
        rules=[
            # AWS Managed Rule: Core Rule Set (CRS)
            aws.wafv2.WebAclRuleArgs(
                name="AWS-AWSManagedRulesCommonRuleSet",
                priority=1,
                override_action=aws.wafv2.WebAclRuleOverrideActionArgs(
                    none={},
                ),
                statement=aws.wafv2.WebAclRuleStatementArgs(
                    managed_rule_group_statement=aws.wafv2.WebAclRuleStatementManagedRuleGroupStatementArgs(
                        vendor_name="AWS",
                        name="AWSManagedRulesCommonRuleSet",
                    ),
                ),
                visibility_config=aws.wafv2.WebAclRuleVisibilityConfigArgs(
                    cloudwatch_metrics_enabled=True,
                    metric_name="AWSManagedRulesCommonRuleSetMetric",
                    sampled_requests_enabled=True,
                ),
            ),
            # AWS Managed Rule: Known Bad Inputs
            aws.wafv2.WebAclRuleArgs(
                name="AWS-AWSManagedRulesKnownBadInputsRuleSet",
                priority=2,
                override_action=aws.wafv2.WebAclRuleOverrideActionArgs(
                    none={},
                ),
                statement=aws.wafv2.WebAclRuleStatementArgs(
                    managed_rule_group_statement=aws.wafv2.WebAclRuleStatementManagedRuleGroupStatementArgs(
                        vendor_name="AWS",
                        name="AWSManagedRulesKnownBadInputsRuleSet",
                    ),
                ),
                visibility_config=aws.wafv2.WebAclRuleVisibilityConfigArgs(
                    cloudwatch_metrics_enabled=True,
                    metric_name="AWSManagedRulesKnownBadInputsRuleSetMetric",
                    sampled_requests_enabled=True,
                ),
            ),
            # Rate limiting: max 2000 requests per 5 minutes per IP
            aws.wafv2.WebAclRuleArgs(
                name="RateLimitRule",
                priority=3,
                action=aws.wafv2.WebAclRuleActionArgs(
                    block={},
                ),
                statement=aws.wafv2.WebAclRuleStatementArgs(
                    rate_based_statement=aws.wafv2.WebAclRuleStatementRateBasedStatementArgs(
                        limit=2000,
                        aggregate_key_type="IP",
                    ),
                ),
                visibility_config=aws.wafv2.WebAclRuleVisibilityConfigArgs(
                    cloudwatch_metrics_enabled=True,
```

```
                    metric_name="RateLimitRuleMetric",
                    sampled_requests_enabled=True,
                ),
            ),
        ],
        visibility_config=aws.wafv2.WebAclVisibilityConfigArgs(
            cloudwatch_metrics_enabled=True,
            metric_name=f"{project_name}-{stack}-waf-metrics",
            sampled_requests_enabled=True,
        ),
        tags=common_tags,
        # WAF for CloudFront must be created in us-east-1
        opts=pulumi.ResourceOptions(
            provider=aws.Provider("us-east-1-provider", region="us-east-1")
        ),
    )

# ======================================
# CloudFront Response Headers Policy セキュリティヘッダー()
# HSTS, CSP(upgrade-insecure-requests), X-Content-Type-Options,
# X-Frame-Options, Referrer-Policy, XSS を付与
# 「保護されていない通信」警告の解消
# ======================================
cloudfront_response_headers_policy = aws.cloudfront.ResponseHeadersPolicy(
    "security-headers-policy",
    name=f"{project_name}-{stack}-security-headers",
    comment="Security headers: HSTS, CSP upgrade-insecure-requests, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, XSS",
    security_headers_config=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigArgs(
        strict_transport_security=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigStrictTransportSecurityArgs(
            access_control_max_age_sec=31536000,  # 年1
            include_subdomains=True,
            preload=False,  # HSTS preload list 登録は手動で行うため False
            override=True,
        ),
        # upgrade-insecure-requests: ブラウザが自動的に HTTP → HTTPS にアップグレード
        # 「保護されていない通信」(mixed content) 警告を抑制する
        content_security_policy=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigContentSecurityPolicyArgs(
            content_security_policy="upgrade-insecure-requests",
            override=True,
        ),
        content_type_options=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigContentTypeOptionsArgs(
            override=True,
        ),
        frame_options=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigFrameOptionsArgs(
            frame_option="SAMEORIGIN",
            override=True,
        ),
        referrer_policy=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigReferrerPolicyArgs(
            referrer_policy="strict-origin-when-cross-origin",
            override=True,
        ),
        xss_protection=aws.cloudfront.ResponseHeadersPolicySecurityHeadersConfigXssProtectionArgs(
            mode_block=True,
            override=True,
            protection=True,
```

```
        ),
    ),
)

# CloudFront Distribution with conditional WAF
# PriceClass_200: 北米 + 欧州 + 日本 + 韓国 + インド 日本ユーザーの遅延解消()
# PriceClass_100 北米欧州のみ(+) は日本からのアクセスが米国エッジ経由になり遅延が大きい
cloudfront_kwargs = {
    "enabled": True,
    "is_ipv6_enabled": True,
    "comment": f"{project_name}-{stack} Frontend Distribution",
    "default_root_object": "index.html",
    "price_class": "PriceClass_200",  # NA/EU/JP/KR/IN エッジ使用 (PriceClass_All より安価)
    "origins": [
        aws.cloudfront.DistributionOriginArgs(
            origin_id=frontend_bucket.bucket_regional_domain_name,
            domain_name=frontend_bucket.bucket_regional_domain_name,
            s3_origin_config=aws.cloudfront.DistributionOriginS3OriginConfigArgs(
                origin_access_identity=cloudfront_oai.cloudfront_access_identity_path,
            ),
        ),
    ],
    "default_cache_behavior": aws.cloudfront.DistributionDefaultCacheBehaviorArgs(
        target_origin_id=frontend_bucket.bucket_regional_domain_name,
        viewer_protocol_policy="redirect-to-https",
        allowed_methods=["GET", "HEAD", "OPTIONS"],
        cached_methods=["GET", "HEAD"],
        compress=True,
        forwarded_values=aws.cloudfront.DistributionDefaultCacheBehaviorForwardedValuesArgs(
            query_string=False,
            cookies=aws.cloudfront.DistributionDefaultCacheBehaviorForwardedValuesCookiesArgs(
                forward="none",
            ),
        ),
        response_headers_policy_id=cloudfront_response_headers_policy.id,
        min_ttl=0,
        default_ttl=3600,  # 1 hour
        max_ttl=86400,  # 24 hours
    ),
    "custom_error_responses": [
        aws.cloudfront.DistributionCustomErrorResponseArgs(
            error_code=403,
            response_code=200,
            response_page_path="/index.html",
        ),
        aws.cloudfront.DistributionCustomErrorResponseArgs(
            error_code=404,
            response_code=200,
            response_page_path="/index.html",
        ),
    ],
    "restrictions": aws.cloudfront.DistributionRestrictionsArgs(
        geo_restriction=aws.cloudfront.DistributionRestrictionsGeoRestrictionArgs(
            restriction_type="none",
        ),
    ),
    "tags": common_tags,
}
```

```python

# Custom domain configuration (optional)
# custom_domain is defined near the top of this file
acm_certificate_arn = config.get(
    "acmCertificateArn"
)  # ACM certificate ARN in us-east-1

if custom_domain and acm_certificate_arn:
    cloudfront_kwargs["aliases"] = [custom_domain]
    cloudfront_kwargs["viewer_certificate"] = (
        aws.cloudfront.DistributionViewerCertificateArgs(
            acm_certificate_arn=acm_certificate_arn,
            ssl_support_method="sni-only",
            minimum_protocol_version="TLSv1.2_2021",
        )
    )
else:
    cloudfront_kwargs["viewer_certificate"] = (
        aws.cloudfront.DistributionViewerCertificateArgs(
            cloudfront_default_certificate=True,
        )
    )

# Add WAF only for production
if stack == "production":
    cloudfront_kwargs["web_acl_id"] = waf_web_acl.arn

# /sns* → S3 bucket (React SPA static files)
# React SPA はビルド済み静的ファイルなので S3 オリジンを使用
# CloudFront Function で /sns/ → /sns/index.html にリライト (SPA ルーティング対応)
cf_function_name = f"spa-sns-rewrite-{stack}"
caller_identity = aws.get_caller_identity()
cf_function_arn = (
    f"arn:aws:cloudfront::{caller_identity.account_id}:function/{cf_function_name}"
)
cloudfront_kwargs["ordered_cache_behaviors"] = [
    aws.cloudfront.DistributionOrderedCacheBehaviorArgs(
        path_pattern="/sns*",
        target_origin_id=frontend_bucket.bucket_regional_domain_name,
        viewer_protocol_policy="redirect-to-https",
        allowed_methods=["GET", "HEAD", "OPTIONS"],
        cached_methods=["GET", "HEAD"],
        compress=True,
        # Managed CachingOptimized policy 静的ファイル向けキャッシュ()
        cache_policy_id="658327ea-f89d-4fab-a63d-7e88639e58f6",
        response_headers_policy_id=cloudfront_response_headers_policy.id,
        function_associations=[
            aws.cloudfront.DistributionOrderedCacheBehaviorFunctionAssociationArgs(
                event_type="viewer-request",
                function_arn=cf_function_arn,
            )
        ],
    )
]

cloudfront_distribution = aws.cloudfront.Distribution(
    "cloudfront-distribution", **cloudfront_kwargs
)
```

```

# ========================================
# CloudTrail - Audit Logging (optional — requires cloudtrail:* IAM permissions)
# Enable by running: pulumi config set cloudtrailEnabled true
# IAM requirements: cloudtrail:CreateTrail, cloudtrail:StartLogging,
#                   cloudtrail:GetTrail, cloudtrail:DescribeTrails,
#                   cloudtrail:PutEventSelectors, cloudtrail:ListTags
# ========================================
cloudtrail_enabled = config.get_bool("cloudtrailEnabled") or False

if cloudtrail_enabled:
    # S3 bucket to store CloudTrail management event logs
    cloudtrail_bucket = aws.s3.BucketV2(
        "cloudtrail-bucket",
        bucket=f"{project_name}-{stack}-cloudtrail-logs",
        force_destroy=True,
        tags=common_tags,
    )

    # Block all public access to the audit log bucket
    aws.s3.BucketPublicAccessBlock(
        "cloudtrail-bucket-public-access",
        bucket=cloudtrail_bucket.id,
        block_public_acls=True,
        block_public_policy=True,
        ignore_public_acls=True,
        restrict_public_buckets=True,
    )

    # Enable versioning on the CloudTrail bucket for tamper evidence
    aws.s3.BucketVersioningV2(
        "cloudtrail-bucket-versioning",
        bucket=cloudtrail_bucket.id,
        versioning_configuration={"status": "Enabled"},
    )

    # Bucket policy: grant CloudTrail service permission to write logs
    cloudtrail_bucket_policy = aws.s3.BucketPolicy(
        "cloudtrail-bucket-policy",
        bucket=cloudtrail_bucket.id,
        policy=cloudtrail_bucket.arn.apply(
            lambda bucket_arn: json.dumps(
                {
                    "Version": "2012-10-17",
                    "Statement": [
                        {
                            "Sid": "AWSCloudTrailAclCheck",
                            "Effect": "Allow",
                            "Principal": {"Service": "cloudtrail.amazonaws.com"},
                            "Action": "s3:GetBucketAcl",
                            "Resource": bucket_arn,
                        },
                        {
                            "Sid": "AWSCloudTrailWrite",
                            "Effect": "Allow",
                            "Principal": {"Service": "cloudtrail.amazonaws.com"},
                            "Action": "s3:PutObject",
                            "Resource": f"{bucket_arn}/AWSLogs/*",
```

```
                            "Condition": {
                                "StringEquals": {
                                    "s3:x-amz-acl": "bucket-owner-full-control"
                                }
                            },
                        },
                    ],
                }
            )
        ),
        opts=pulumi.ResourceOptions(depends_on=[cloudtrail_bucket]),
    )

    # CloudTrail trail: multi-region, global service events, log file validation
    cloudtrail = aws.cloudtrail.Trail(
        "cloudtrail",
        name=f"{project_name}-{stack}-trail",
        s3_bucket_name=cloudtrail_bucket.id,
        include_global_service_events=True,  # Capture IAM, STS, etc.
        is_multi_region_trail=True,  # Cover all regions
        enable_log_file_validation=True,  # SHA-256 digest for tamper detection
        tags=common_tags,
        opts=pulumi.ResourceOptions(depends_on=[cloudtrail_bucket_policy]),
    )

    pulumi.export("cloudtrail_name", cloudtrail.name)
    pulumi.export("cloudtrail_bucket_name", cloudtrail_bucket.id)

# ========================================
# Monitoring and Alerts
# ========================================
alarm_email = config.get("alarmEmail")
monitoring_resources = monitoring.setup_monitoring(
    project_name=project_name,
    stack=stack,
    lambda_function_name=lambda_function.name,
    api_gateway_id=api_gateway.id,
    api_gateway_name=api_gateway.name,
    cloudfront_distribution_id=cloudfront_distribution.id,
    alarm_email=alarm_email,
)

# ========================================
# Outputs
# ========================================
pulumi.export("lambda_function_name", lambda_function.name)
pulumi.export("lambda_function_arn", lambda_function.arn)
pulumi.export("lambda_function_url", lambda_url.function_url)
pulumi.export("api_gateway_id", api_gateway.id)
pulumi.export("api_gateway_endpoint", api_gateway.api_endpoint)
pulumi.export("api_url", api_gateway.api_endpoint)  # For workflow
pulumi.export("frontend_bucket_name", frontend_bucket.id)
pulumi.export(
    "frontend_url",
    frontend_bucket.website_endpoint.apply(lambda endpoint: f"http://{endpoint}"),
)
pulumi.export("cloudfront_distribution_id", cloudfront_distribution.id)
pulumi.export("cloudfront_domain", cloudfront_distribution.domain_name)
```

```
pulumi.export(
    "cloudfront_url",
    cloudfront_distribution.domain_name.apply(lambda domain: f"https://{domain}"),
)
# Custom domain exports (if configured)
if custom_domain:
    pulumi.export("custom_domain", custom_domain)
    pulumi.export("custom_domain_url", f"https://{custom_domain}")
    pulumi.export("acm_certificate_arn", acm_certificate_arn)
# WAF exports only for production
if stack == "production":
    pulumi.export("waf_web_acl_id", waf_web_acl.id)
    pulumi.export("waf_web_acl_arn", waf_web_acl.arn)
pulumi.export("secret_name", app_secret.name)
pulumi.export("secret_arn", app_secret.arn)
pulumi.export("cognito_user_pool_id", user_pool.id)
pulumi.export("cognito_user_pool_arn", user_pool.arn)
pulumi.export("cognito_client_id", user_pool_client.id)
pulumi.export("cognito_domain", user_pool_domain.domain)
pulumi.export("posts_table_name", posts_table.name)
pulumi.export("posts_table_arn", posts_table.arn)
pulumi.export("images_bucket_name", images_bucket.id)
pulumi.export("images_bucket_arn", images_bucket.arn)

# Monitoring exports
if monitoring_resources["sns_topic"]:
    pulumi.export("monitoring_sns_topic_arn", monitoring_resources["sns_topic"].arn)
pulumi.export(
    "monitoring_lambda_alarms", list(monitoring_resources["lambda_alarms"].keys())
)
pulumi.export(
    "monitoring_api_alarms", list(monitoring_resources["api_gateway_alarms"].keys())
)
pulumi.export(
    "monitoring_cloudfront_alarms",
    list(monitoring_resources["cloudfront_alarms"].keys()),
)

# Cost estimation
pulumi.export(
    "cost_estimate",
    "AWS Lambda: 1M free requests/month, then $0.20 per 1M requests. "
    "API Gateway: $1 per million requests. "
    "S3: $0.023/GB/month. "
    "CloudFront: $0.085/GB for first 10TB/month (free tier: 1TB/month for 12 months). "
    "AWS WAF: $5/month per web ACL + $1/month per rule + $0.60 per 1M requests. "
    "Secrets Manager: $0.40/month per secret + $0.05 per 10,000 API calls. "
    "DynamoDB: Pay-per-request pricing (25 WCU and 25 RCU free tier). "
    "Cognito: 50,000 MAU free for first 12 months, then $0.0055 per MAU. "
    "Estimated: $10-25/month for low traffic with WAF and security features.",
)
```

5.2 AWS Pulumi.staging.yaml

```
config:
  aws:region: ap-northeast-1
```

```
  # SECURITY: Change this before production deployment!
  # Set to actual frontend URLs once custom domain is configured.
  # Example: "https://aws.yourdomain.com,https://yourdomain.com"
  multicloud-auto-deploy-aws:allowedOrigins: "https://staging.aws.ashnova.jp"
  multicloud-auto-deploy-aws:alarmEmail: sat0sh1kawada@spa.nifty.com
  # Static landing page custom domain (optional)
  # multicloud-auto-deploy-aws:staticSiteDomain: aws.yourdomain.com
  # multicloud-auto-deploy-aws:staticSiteAcmCertificateArn: arn:aws:acm:us-east-1:ACCOUNT_ID:certificate/CERT_ID

  multicloud-auto-deploy-aws:cloudFrontDomain: d1tf3uumcm4bo1.cloudfront.net
  multicloud-auto-deploy-aws:customDomain: staging.aws.ashnova.jp
  multicloud-auto-deploy-aws:acmCertificateArn:
     arn:aws:acm:us-east-1:278280499340:certificate/9110a7f2-324c-4b45-9ffc-133c1e154c90
```

5.3 Azure Pulumi main.py

```python
"""
Multi-Cloud Auto Deploy - Azure Pulumi Implementation

Architecture:
- Azure Functions (Consumption Plan Y1)
- Storage Account (Functions + Frontend)
- Application Insights (Monitoring)

Cost: ~$2-8/month for low traffic
"""

import monitoring
import pulumi
import pulumi_azure_native as azure
import pulumi_azuread as azuread
import pulumi_random as random

# Configuration
config = pulumi.Config()
azure_config = pulumi.Config("azure-native")
location = azure_config.get("location") or "japaneast"
stack = pulumi.get_stack()
project_name = "multicloud-auto-deploy"

# Frontend domain (Azure Front Door hostname) — set after first deploy:
#   pulumi config set frontendDomain <afd_hostname>
# Used for Azure AD app registration redirect URIs
frontend_domain = config.get("frontendDomain") or ""

# Common tags
common_tags = {
    "Project": project_name,
    "ManagedBy": "pulumi",
    "Environment": stack,
}

# ========================================
# Resource Group
# ========================================
resource_group = azure.resources.ResourceGroup(
```

```
    "resource-group",
    resource_group_name=f"{project_name}-{stack}-rg",
    location=location,
    tags=common_tags,
)

# ========================================
# Random suffix for storage accounts (must be globally unique)
# ========================================
storage_suffix = random.RandomString(
    "storage-suffix",
    length=6,
    special=False,
    upper=False,
)

# Random UUID for OAuth2 permission scope
oauth2_scope_id = random.RandomUuid("oauth2-scope-id")

# ========================================
# Storage Account for Azure Functions
# ========================================
functions_storage = azure.storage.StorageAccount(
    "functions-storage",
    account_name=storage_suffix.result.apply(lambda suffix: f"mcadfunc{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    sku=azure.storage.SkuArgs(
        name="Standard_LRS",  # Locally redundant storage
    ),
    kind="StorageV2",
    minimum_tls_version="TLS1_2",
    tags=common_tags,
)

# ========================================
# Storage Account for Frontend (Static Website)
# ========================================
frontend_storage = azure.storage.StorageAccount(
    "frontend-storage",
    account_name=storage_suffix.result.apply(lambda suffix: f"mcadweb{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    sku=azure.storage.SkuArgs(
        name="Standard_LRS",
    ),
    kind="StorageV2",
    minimum_tls_version="TLS1_2",
    tags=common_tags,
)

# Enable static website hosting on frontend storage
# Note: This requires using Azure CLI or REST API in the workflow
# Pulumi doesn't directly support static website configuration yet

# ========================================
# Log Analytics Workspace (Centralized audit & security logging)
# ========================================
```

```
# Workspace for aggregating logs from Front Door, Function App, Cosmos DB, and Azure AD
log_analytics_workspace = azure.operationalinsights.Workspace(
    "log-analytics-workspace",
    workspace_name=storage_suffix.result.apply(lambda suffix: f"mcad-logs-{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    sku=azure.operationalinsights.WorkspaceSkuArgs(
        name="PerGB2018",  # Pay-per-GB (5 GB/month free tier)
    ),
    retention_in_days=30,  # Retain logs for 30 days
    tags=common_tags,
)

# ========================================
# Application Insights
# ========================================
app_insights = azure.insights.Component(
    "app-insights",
    resource_group_name=resource_group.name,
    location=location,
    application_type="web",
    kind="web",
    # Send telemetry to Log Analytics Workspace for centralized visibility
    ingestion_mode="LogAnalytics",
    workspace_resource_id=log_analytics_workspace.id,
    tags=common_tags,
)

# ========================================
# Cosmos DB for NoSQL Database
# ========================================
cosmos_account = azure.documentdb.DatabaseAccount(
    "cosmos-account",
    account_name=storage_suffix.result.apply(lambda suffix: f"mcad-cosmos-{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    database_account_offer_type="Standard",
    locations=[
        azure.documentdb.LocationArgs(
            location_name=location,
            failover_priority=0,
        )
    ],
    consistency_policy=azure.documentdb.ConsistencyPolicyArgs(
        default_consistency_level="Session",  # Session consistency for balance
        max_interval_in_seconds=5,
        max_staleness_prefix=100,
    ),
    enable_automatic_failover=False,
    capabilities=[
        # Serverless mode for cost efficiency
        azure.documentdb.CapabilityArgs(name="EnableServerless"),
    ],
    tags=common_tags,
)

# Cosmos DB Database
cosmos_database = azure.documentdb.SqlResourceSqlDatabase(
```

```
    "cosmos-database",
    database_name="messages",
    resource_group_name=resource_group.name,
    account_name=cosmos_account.name,
    resource=azure.documentdb.SqlDatabaseResourceArgs(
        id="messages",
    ),
    opts=pulumi.ResourceOptions(depends_on=[cosmos_account]),
)

# Cosmos DB Container for messages
cosmos_container = azure.documentdb.SqlResourceSqlContainer(
    "cosmos-container",
    container_name="messages",
    resource_group_name=resource_group.name,
    account_name=cosmos_account.name,
    database_name=cosmos_database.name,
    resource=azure.documentdb.SqlContainerResourceArgs(
        id="messages",
        partition_key=azure.documentdb.ContainerPartitionKeyArgs(
            paths=["/userId"],
            kind="Hash",
        ),
        indexing_policy=azure.documentdb.IndexingPolicyArgs(
            automatic=True,
            indexing_mode="Consistent",
            included_paths=[azure.documentdb.IncludedPathArgs(path="/*")],
            excluded_paths=[azure.documentdb.ExcludedPathArgs(path='/"_etag"/?')],
        ),
    ),
    opts=pulumi.ResourceOptions(depends_on=[cosmos_database]),
)

# Get Cosmos DB keys
cosmos_keys = pulumi.Output.all(resource_group.name, cosmos_account.name).apply(
    lambda args: azure.documentdb.list_database_account_keys(
        resource_group_name=args[0],
        account_name=args[1],
    )
)

# ========================================
# Azure Key Vault for Secret Management
# ========================================
# Get current Azure client configuration for tenant ID
current = azure.authorization.get_client_config()

key_vault = azure.keyvault.Vault(
    "key-vault",
    vault_name=storage_suffix.result.apply(lambda suffix: f"mcad-kv-{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    properties=azure.keyvault.VaultPropertiesArgs(
        tenant_id=current.tenant_id,
        sku=azure.keyvault.SkuArgs(
            family="A",
            name="standard",
        ),
```

```
        enable_rbac_authorization=True,  # Use RBAC instead of access policies
        enable_soft_delete=True,
        soft_delete_retention_in_days=7,
        enable_purge_protection=True,  # Defender for Cloud 推奨: purge 有効化protection
        enabled_for_deployment=False,
        enabled_for_disk_encryption=False,
        enabled_for_template_deployment=True,
        network_acls=azure.keyvault.NetworkRuleSetArgs(
            bypass="AzureServices",
            default_action="Allow",  # Change to "Deny" for production with specific IP rules
        ),
    ),
    tags=common_tags,
)

# Store example secret (should be updated with actual values)
app_secret = azure.keyvault.Secret(
    "app-secret",
    secret_name="app-config",
    resource_group_name=resource_group.name,
    vault_name=key_vault.name,
    properties=azure.keyvault.SecretPropertiesArgs(
        value=pulumi.Output.secret('{"database_url":"changeme","api_key":"changeme"}'),
    ),
    tags=common_tags,
)

# ========================================
# Azure AI Document Intelligence (OCR for university exam images)
# ========================================
# Uses the prebuilt-read model to extract text from exam problem images.
# Kind "FormRecognizer" = Azure AI Document Intelligence (rebranded service name).
# F0 free tier: 500 pages/month — upgrade to S0 for production workloads.
document_intelligence = azure.cognitiveservices.Account(
    "document-intelligence",
    account_name=storage_suffix.result.apply(lambda suffix: f"mcad-di-{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    kind="FormRecognizer",
    sku=azure.cognitiveservices.SkuArgs(
        name="S0",  # Standard tier: pay-per-page OCR
    ),
    properties=azure.cognitiveservices.AccountPropertiesArgs(
        public_network_access="Enabled",
        restore=False,
    ),
    tags=common_tags,
    opts=pulumi.ResourceOptions(depends_on=[resource_group]),
)

# Retrieve Document Intelligence keys for Function App configuration
document_intelligence_keys = pulumi.Output.all(
    resource_group.name, document_intelligence.name
).apply(
    lambda args: azure.cognitiveservices.list_account_keys(
        resource_group_name=args[0],
        account_name=args[1],
    )
```

```
)

# ========================================
# Azure OpenAI (GPT-4o for university exam solver LLM)
# ========================================
# NOTE: Azure OpenAI Cognitive Services account for LLM-based answer generation.
# Uses GPT-4o model deployed in the same region as other resources.
azure_openai_account = azure.cognitiveservices.Account(
    "azure-openai",
    account_name=storage_suffix.result.apply(lambda suffix: f"mcad-openai-{suffix}"),
    resource_group_name=resource_group.name,
    location=location,
    kind="OpenAI",
    sku=azure.cognitiveservices.SkuArgs(
        name="S0",
    ),
    properties=azure.cognitiveservices.AccountPropertiesArgs(
        public_network_access="Enabled",
        restore=False,
        custom_sub_domain_name=storage_suffix.result.apply(
            lambda suffix: f"mcad-openai-{suffix}"
        ),
    ),
    tags=common_tags,
    opts=pulumi.ResourceOptions(depends_on=[resource_group]),
)

# GPT-4o model deployment
azure_openai_deployment = azure.cognitiveservices.Deployment(
    "gpt-4o-deploy",
    account_name=azure_openai_account.name,
    resource_group_name=resource_group.name,
    deployment_name="gpt-4o",
    properties=azure.cognitiveservices.DeploymentPropertiesArgs(
        model=azure.cognitiveservices.DeploymentModelArgs(
            format="OpenAI",
            name="gpt-4o",
            version="2024-11-20",
        ),
        version_upgrade_option="OnceNewDefaultVersionAvailable",
    ),
    sku=azure.cognitiveservices.SkuArgs(
        name="GlobalStandard",
        capacity=10,  # 10K tokens per minute
    ),
    opts=pulumi.ResourceOptions(depends_on=[azure_openai_account]),
)

# Retrieve Azure OpenAI keys
azure_openai_keys = pulumi.Output.all(
    resource_group.name, azure_openai_account.name
).apply(
    lambda args: azure.cognitiveservices.list_account_keys(
        resource_group_name=args[0],
        account_name=args[1],
    )
)

```

```
# ========================================
# Authentication Setup - Azure AD Application
# ========================================
# Azure AD Application for authentication (automated)
# This creates an Azure AD app registration for the API

# Get Azure AD tenant from azuread config
azuread_config = pulumi.Config("azuread")
azure_tenant_id = azuread_config.get("tenantId") or pulumi.Output.from_input("")

# Create Azure AD Application
app_registration = azuread.Application(
    "api-app",
    display_name=f"{project_name}-{stack}-api",
    # Sign-in audience: AzureADMyOrg = Single tenant
    sign_in_audience="AzureADMyOrg",
    # Web application configuration
    web=azuread.ApplicationWebArgs(
        redirect_uris=(
            [
                # Legacy / fallback
                f"https://{project_name}-{stack}-web.azurewebsites.net/callback",
            ]
            + (
                [
                    # FrontDoor — login callback + post-logout redirect
                    f"https://{frontend_domain}/sns/auth/callback",
                    f"https://{frontend_domain}/sns/",
                ]
                if frontend_domain
                else []
            )
        ),
        implicit_grant=azuread.ApplicationWebImplicitGrantArgs(
            access_token_issuance_enabled=True,
            id_token_issuance_enabled=True,
        ),
    ),
    # API configuration (expose an API)
    api=azuread.ApplicationApiArgs(
        # OAuth2 permission scopes
        oauth2_permission_scopes=[
            azuread.ApplicationApiOauth2PermissionScopeArgs(
                admin_consent_description="Allow the application to access the API on behalf of the signed-in user.",
                admin_consent_display_name="Access API",
                enabled=True,
                id=oauth2_scope_id.result,
                type="User",
                user_consent_description="Allow the application to access the API on your behalf.",
                user_consent_display_name="Access API",
                value="API.Access",
            ),
        ],
    ),
    opts=pulumi.ResourceOptions(depends_on=[resource_group]),
)

```

```
# Create Service Principal for the application
app_service_principal = azuread.ServicePrincipal(
    "api-sp",
    client_id=app_registration.client_id,
    app_role_assignment_required=False,
    opts=pulumi.ResourceOptions(depends_on=[app_registration]),
)

# ========================================
# Note: Function App is managed manually
# ========================================
# Function App and App Service Plan are created and managed manually
# Existing resources:
# - Function App: multicloud-auto-deploy-staging-func
# - App Service Plan: FC1 (FlexConsumption)
# - API URL: https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net

# ========================================
# Azure Front Door for Frontend CDN
# ========================================
# Front Door Profile (Standard SKU - cost-effective)
frontdoor_profile = azure.cdn.Profile(
    "frontdoor-profile",
    profile_name=f"{project_name}-{stack}-fd",
    resource_group_name=resource_group.name,
    location="Global",  # Front Door is global
    sku=azure.cdn.SkuArgs(
        name="Standard_AzureFrontDoor",
    ),
    # Extend origin response timeout to 60s (default: 30s) to accommodate
    # Python Dynamic Consumption cold starts which can take 10-30+ seconds.
    # Without this, AFD times out during cold start and returns HTTP 502.
    origin_response_timeout_seconds=60,
    tags=common_tags,
    # Force replacement when SKU changes (Premium -> Standard not supported)
    opts=pulumi.ResourceOptions(replace_on_changes=["sku"]),
)

# Front Door Endpoint
frontdoor_endpoint = azure.cdn.AFDEndpoint(
    "frontdoor-endpoint",
    endpoint_name=storage_suffix.result.apply(lambda suffix: f"mcad-{stack}-{suffix}"),
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
    location="Global",
    enabled_state="Enabled",
    tags=common_tags,
)

# Front Door Origin Group
frontdoor_origin_group = azure.cdn.AFDOriginGroup(
    "frontdoor-origin-group",
    origin_group_name=f"{project_name}-{stack}-origin-group",
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
    load_balancing_settings=azure.cdn.LoadBalancingSettingsParametersArgs(
        sample_size=4,
        successful_samples_required=3,
```

```
        additional_latency_in_milliseconds=50,
    ),
    health_probe_settings=azure.cdn.HealthProbeParametersArgs(
        probe_path="/",
        probe_request_type="HEAD",
        probe_protocol="Https",
        probe_interval_in_seconds=100,
    ),
)

# Front Door Origin (Storage Account for Frontend)
frontdoor_origin = azure.cdn.AFDOrigin(
    "frontdoor-origin",
    origin_name=f"{project_name}-{stack}-origin",
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
    origin_group_name=frontdoor_origin_group.name,
    host_name=frontend_storage.primary_endpoints.apply(
        lambda endpoints: (
            endpoints.web.replace("https://", "").replace("/", "")
            if endpoints.web
            else ""
        )
    ),
    origin_host_header=frontend_storage.primary_endpoints.apply(
        lambda endpoints: (
            endpoints.web.replace("https://", "").replace("/", "")
            if endpoints.web
            else ""
        )
    ),
    http_port=80,
    https_port=443,
    priority=1,
    weight=1000,
    enabled_state="Enabled",
)

# ========================================
# Front Door: Rule Set for React SPA URL rewriting
# /sns/<deep-link> → /sns/index.html (SPA client-side routing)
#
# Conditions (all AND'd):
#   1. Path begins with /sns/
#   2. Path does NOT begin with /sns/assets/  (preserve static bundles)
#   3. Path does NOT end with known static file extensions
# Action: URL Rewrite source=/sns/  destination=/sns/index.html
# ========================================
spa_rule_set = azure.cdn.RuleSet(
    "spa-rule-set",
    rule_set_name="SpaRuleSet",
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
)

spa_rewrite_rule = azure.cdn.Rule(
    "spa-rewrite-rule",
    rule_name="SpaRouting",
```

```
    rule_set_name=spa_rule_set.name,
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
    order=1,
    conditions=[
        # Condition 1: path begins with /sns/
        azure.cdn.DeliveryRuleUrlPathConditionArgs(
            name="UrlPath",
            parameters=azure.cdn.UrlPathMatchConditionParametersArgs(
                type_name="DeliveryRuleUrlPathMatchConditionParameters",
                operator="BeginsWith",
                negate_condition=False,
                match_values=["/sns/"],
                transforms=["Lowercase"],
            ),
        ),
        # Condition 2: NOT /sns/assets/ (Vite bundle output)
        azure.cdn.DeliveryRuleUrlPathConditionArgs(
            name="UrlPath",
            parameters=azure.cdn.UrlPathMatchConditionParametersArgs(
                type_name="DeliveryRuleUrlPathMatchConditionParameters",
                operator="BeginsWith",
                negate_condition=True,
                match_values=["/sns/assets/"],
                transforms=["Lowercase"],
            ),
        ),
        # Condition 3: NOT a static file (has no known extension)
        # Azure AFD limits match_values to 10 per condition
        azure.cdn.DeliveryRuleUrlPathConditionArgs(
            name="UrlPath",
            parameters=azure.cdn.UrlPathMatchConditionParametersArgs(
                type_name="DeliveryRuleUrlPathMatchConditionParameters",
                operator="EndsWith",
                negate_condition=True,
                match_values=[
                    ".html",
                    ".js",
                    ".css",
                    ".png",
                    ".svg",
                    ".ico",
                    ".json",
                    ".woff",
                    ".woff2",
                    ".map",
                ],
                transforms=["Lowercase"],
            ),
        ),
    ],
    actions=[
        azure.cdn.UrlRewriteActionArgs(
            name="UrlRewrite",
            parameters=azure.cdn.UrlRewriteActionParametersArgs(
                type_name="DeliveryRuleUrlRewriteActionParameters",
                source_pattern="/sns/",
                destination="/sns/index.html",
```

```
                preserve_unmatched_path=False,
            ),
        ),
    ],
    opts=pulumi.ResourceOptions(depends_on=[spa_rule_set]),
)

# Front Door Route (/* → Blob Storage with SPA rule set attached)
# ========================================
# Front Door: Security Rule Set (Free Custom Rules)
# Purpose: Implement security patterns without WAF Premium SKU
# Note: Azure Front Door Standard SKU rule engine has limited action types.
#       Security headers and advanced WAF rules are best implemented via:
#       1. Azure Application Gateway WAF (if used)
#       2. React application headers configuration
#       3. Azure Defender for Cloud (already enabled)
# Current Setup: Basic rule set for future expansion
# ========================================
security_rule_set = azure.cdn.RuleSet(
    "security-rule-set",
    rule_set_name="SecurityRules",
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
)

# Update Front Door Route to include both SPA and Security rule sets
frontdoor_route = azure.cdn.Route(
    "frontdoor-route",
    route_name=f"{project_name}-{stack}-route",
    profile_name=frontdoor_profile.name,
    resource_group_name=resource_group.name,
    endpoint_name=frontdoor_endpoint.name,
    origin_group=azure.cdn.ResourceReferenceArgs(
        id=frontdoor_origin_group.id,
    ),
    supported_protocols=["Http", "Https"],
    patterns_to_match=["/*"],
    forwarding_protocol="HttpsOnly",
    link_to_default_domain="Enabled",
    https_redirect="Enabled",
    rule_sets=[
        azure.cdn.ResourceReferenceArgs(id=spa_rule_set.id),
        azure.cdn.ResourceReferenceArgs(id=security_rule_set.id),
    ],
    opts=pulumi.ResourceOptions(
        depends_on=[
            frontdoor_origin,
            spa_rewrite_rule,
            security_rule_set,
        ]
    ),
)

# Export security rule set ID for reference
pulumi.export("security_rule_set_id", security_rule_set.id)

# ========================================
# Front Door Diagnostic Settings (Access Logs → Log Analytics)
```

```
# ========================================
# Captures FrontDoorAccessLog and WAF logs for security auditing.
frontdoor_diagnostics = azure.insights.DiagnosticSetting(
    "frontdoor-diagnostics",
    name=f"{project_name}-{stack}-fd-diag",
    resource_uri=frontdoor_profile.id,
    workspace_id=log_analytics_workspace.id,
    logs=[
        azure.insights.LogSettingsArgs(
            category_group="allLogs",
            enabled=True,
        ),
    ],
    opts=pulumi.ResourceOptions(
        depends_on=[frontdoor_profile, log_analytics_workspace]
    ),
)

# ========================================
# Monitoring and Alerts
# ========================================
alarm_email = config.get("alarmEmail")

# Flex Consumption instanceMemoryMB (512 / 2048 / 4096).
# Must match the value configured on the Function App in Azure Portal.
# Default: 2048MB. Update here (and in Pulumi.[stack].yaml) if changed.
# Bug history: was hardcoded to 800MB in monitoring.py, firing Sev3 alerts
# unconditionally on a 2048MB instance (800MB = only 39% of limit).
function_memory_mb = config.get_int("functionMemoryMb") or 2048

# Note: Function App ID is derived from the manually-managed app
client_config = azure.authorization.get_client_config()
function_app_id = pulumi.Output.concat(
    "/subscriptions/",
    client_config.subscription_id,
    "/resourceGroups/",
    resource_group.name,
    "/providers/Microsoft.Web/sites/",
    f"{project_name}-{stack}-func",
)

monitoring_resources = monitoring.setup_monitoring(
    project_name=project_name,
    stack=stack,
    resource_group_name=resource_group.name,
    location=location,
    function_app_id=function_app_id,
    cosmos_account_id=cosmos_account.id,
    frontdoor_profile_id=frontdoor_profile.id,
    alarm_email=alarm_email,
    function_memory_mb=function_memory_mb,
)

# ========================================
# Outputs
# ========================================
pulumi.export("resource_group_name", resource_group.name)
pulumi.export("functions_storage_name", functions_storage.name)
```

```
pulumi.export("frontend_storage_name", frontend_storage.name)

# Azure AD Authentication
pulumi.export("azure_ad_client_id", app_registration.client_id)
# Object ID of the application
pulumi.export("azure_ad_object_id", app_registration.id)
pulumi.export(
    "auth_config_instructions",
    pulumi.Output.concat(
        "Configure Function App with these environment variables:\\n",
        "  AUTH_PROVIDER=azure\\n",
        "  AZURE_TENANT_ID=<your-tenant-id>\\n",
        "  AZURE_CLIENT_ID=",
        app_registration.client_id,
        "\\n",
    ),
)

# Existing manually-managed Function App
pulumi.export("function_app_name", f"{project_name}-{stack}-func")
pulumi.export(
    "api_url",
    "https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net",
)

pulumi.export(
    "frontend_url",
    frontend_storage.primary_endpoints.apply(
        lambda endpoints: endpoints.web if endpoints.web else "Not configured yet"
    ),
)
pulumi.export("frontdoor_endpoint_name", frontdoor_endpoint.name)
pulumi.export("frontdoor_hostname", frontdoor_endpoint.host_name)
pulumi.export(
    "frontdoor_url",
    frontdoor_endpoint.host_name.apply(lambda hostname: f"https://{hostname}"),
)
pulumi.export("app_insights_instrumentation_key", app_insights.instrumentation_key)
pulumi.export("key_vault_name", key_vault.name)
pulumi.export("key_vault_uri", key_vault.properties.vault_uri)
pulumi.export("log_analytics_workspace_name", log_analytics_workspace.name)
pulumi.export("log_analytics_workspace_id", log_analytics_workspace.id)

# Cosmos DB exports
pulumi.export("cosmos_account_name", cosmos_account.name)
pulumi.export("cosmos_db_endpoint", cosmos_account.document_endpoint)
pulumi.export(
    "cosmos_db_key",
    pulumi.Output.secret(cosmos_keys.apply(lambda k: k.primary_master_key)),
)
pulumi.export("cosmos_database_name", cosmos_database.name)
pulumi.export("cosmos_container_name", cosmos_container.name)
pulumi.export(
    "cosmos_connection_instructions",
    pulumi.Output.concat(
        "Configure Function App with these environment variables:\\n",
        "  COSMOS_DB_ENDPOINT=",
        cosmos_account.document_endpoint,
```

```
        "\\n",
        "  COSMOS_DB_KEY=<from-pulumi-output>\\n",
        "  COSMOS_DB_DATABASE=",
        cosmos_database.name,
        "\\n",
        "  COSMOS_DB_CONTAINER=",
        cosmos_container.name,
        "\\n",
    ),
)

# Azure AI Document Intelligence exports (OCR for university exam problems)
pulumi.export("document_intelligence_name", document_intelligence.name)
pulumi.export(
    "document_intelligence_endpoint",
    document_intelligence.properties.apply(
        lambda p: p.endpoint if p and p.endpoint else ""
    ),
)
pulumi.export(
    "document_intelligence_key",
    pulumi.Output.secret(document_intelligence_keys.apply(lambda k: k.key1 or "")),
)
pulumi.export(
    "ocr_config_instructions",
    pulumi.Output.concat(
        "Configure Function App with these environment variables for OCR/LLM:\\n",
        "  AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT=",
        document_intelligence.properties.apply(
            lambda p: p.endpoint if p and p.endpoint else "<endpoint>"
        ),
        "\\n",
        "  AZURE_DOCUMENT_INTELLIGENCE_KEY=<from pulumi stack output document_intelligence_key>\\n",
        "  AZURE_OPENAI_ENDPOINT=<from pulumi stack output azure_openai_endpoint>\\n",
        "  AZURE_OPENAI_KEY=<from pulumi stack output azure_openai_key>\\n",
        "  AZURE_OPENAI_DEPLOYMENT=gpt-4o\\n",
        "  SOLVE_ENABLED=true\\n",
    ),
)

# Azure OpenAI exports (LLM for university exam solver)
pulumi.export("azure_openai_account_name", azure_openai_account.name)
# customSubDomainName is set, so the per-resource endpoint https://{name}.openai.azure.com/
# will resolve correctly for the OpenAI Python SDK.
pulumi.export(
    "azure_openai_endpoint",
    azure_openai_account.name.apply(lambda name: f"https://{name}.openai.azure.com/"),
)
pulumi.export(
    "azure_openai_key",
    pulumi.Output.secret(azure_openai_keys.apply(lambda k: k.key1 or "")),
)
pulumi.export("azure_openai_deployment_name", azure_openai_deployment.name)

# Monitoring exports
if monitoring_resources["action_group"]:
    pulumi.export("monitoring_action_group_id", monitoring_resources["action_group"].id)
pulumi.export(
```

```
    "monitoring_function_alerts",
    list(monitoring_resources["function_alerts"].keys()),
)
pulumi.export(
    "monitoring_cosmos_alerts", list(monitoring_resources["cosmos_alerts"].keys())
)
pulumi.export(
    "monitoring_frontdoor_alerts", list(monitoring_resources["frontdoor_alerts"].keys())
)

# Cost estimation
pulumi.export(
    "cost_estimate",
    "Azure Functions FlexConsumption: Pay-as-you-go. "
    "Storage: $0.02/GB/month. "
    "Application Insights: 5GB free/month. "
    "Azure Front Door Standard: $35/month + $0.01/GB. "
    "Key Vault: $0.03 per 10,000 operations (secrets are free). "
    "Estimated: $35-50/month with Front Door Standard.",
)
```

5.4 Azure Pulumi.staging.yaml

```
config:
  multicloud-auto-deploy-azure:allowedOrigins:
     "https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net,http://localhost:5173"
  multicloud-auto-deploy-azure:alarmEmail: sat0sh1kawada@spa.nifty.com
  multicloud-auto-deploy-azure:function_app_hostname:
     multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net
  multicloud-auto-deploy-azure:frontendDomain: mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net
  # Static landing page custom domain (optional)
  # mulicloud-auto-deploy-azure:staticSiteDomain: azure-staging.yourdomain.com
```

5.5 GCP Pulumi main.py

```
"""
Multi-Cloud Auto Deploy - GCP Pulumi Implementation

Architecture:
- Cloud Storage (Function source + Frontend)
- Cloud Functions Gen 2 (deployed via gcloud CLI in workflow)
- IAM settings
- Secret Manager with IAM bindings

Note: Cloud Functions is deployed via gcloud CLI, not Pulumi,
because Pulumi requires the ZIP file to exist before creating the function.

Cost: ~$2-5/month for low traffic
"""

import hashlib
import os

import monitoring
```

```python
import pulumi
import pulumi_gcp as gcp

# Configuration
config = pulumi.Config()
gcp_config = pulumi.Config("gcp")
project = gcp_config.require("project")
region = gcp_config.get("region") or "asia-northeast1"
stack = pulumi.get_stack()
project_name = "multicloud-auto-deploy"

# Get the service account email from environment or config (used by GitHub Actions)
# This will be set from GCP_CREDENTIALS in the workflow
github_actions_sa = config.get("github_actions_sa") or os.getenv("GITHUB_ACTIONS_SA")

# CORS allowed origins (configurable per environment)
allowed_origins = config.get("allowedOrigins") or "*"
allowed_origins_list = allowed_origins.split(",") if allowed_origins != "*" else ["*"]

# Audit logs toggle (requires project IAM policy edit permission)
enable_audit_logs = config.get_bool("enableAuditLogs")
if enable_audit_logs is None:
    enable_audit_logs = True

# Common labels
common_labels = {
    "project": project_name,
    "managed-by": "pulumi",
    "environment": stack,
}

# ======================================
# Enable required APIs
# ======================================
services = [
    "cloudfunctions.googleapis.com",
    "cloudbuild.googleapis.com",
    "storage-api.googleapis.com",
    "run.googleapis.com",  # Cloud Functions Gen 2 uses Cloud Run
    "secretmanager.googleapis.com",  # Secret Manager
    "firebase.googleapis.com",  # Firebase
    # Identity Platform (Firebase Auth backend)
    "identitytoolkit.googleapis.com",
]

enabled_services = []
for service in services:
    svc = gcp.projects.Service(
        f"enable-{service.split('.')[0]}",
        project=project,
        service=service,
        disable_on_destroy=False,
    )
    enabled_services.append(svc)

# ======================================
# Cloud Audit Logs
# ======================================
```

```
# Enable Admin Read, Data Read, and Data Write audit logs project-wide.
# Covers Firestore, Cloud Storage, Secret Manager, Cloud Run, and IAM.
# Admin Read logs are free; Data Read/Write logs incur minimal ingestion cost.
audit_config = None
if enable_audit_logs:
    audit_config = gcp.projects.IAMAuditConfig(
        "audit-config",
        project=project,
        service="allServices",
        audit_log_configs=[
            gcp.projects.IAMAuditConfigAuditLogConfigArgs(log_type="ADMIN_READ"),
            gcp.projects.IAMAuditConfigAuditLogConfigArgs(log_type="DATA_READ"),
            gcp.projects.IAMAuditConfigAuditLogConfigArgs(log_type="DATA_WRITE"),
        ],
        opts=pulumi.ResourceOptions(depends_on=enabled_services),
    )
else:
    pulumi.log.warn("GCP audit logs disabled (enableAuditLogs=false).")

# ========================================
# Authentication Setup - Firebase Project
# ========================================
# Create Firebase Project (enables Firebase for this GCP project)
firebase_project = gcp.firebase.Project(
    "firebase-project",
    project=project,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)

# Create Firebase Web App
firebase_web_app = gcp.firebase.WebApp(
    "web-app",
    project=project,
    display_name=f"{project_name}-{stack}-web",
    opts=pulumi.ResourceOptions(depends_on=[firebase_project]),
)

# Note: Firebase Web App configuration (API Key, Auth Domain) is available
# in the Firebase Console after the app is created. These cannot be retrieved
# programmatically via Pulumi at provisioning time.
#
# To get the configuration:
# 1. Visit Firebase Console: https://console.firebase.google.com/
# 2. Select your project
# 3. Go to Project Settings → General
# 4. Scroll down to "Your apps" and find your Web App
# 5. Copy the Firebase SDK configuration

# Note: Authentication providers (Google, Email/Password, etc.) must be configured
# manually in Firebase Console due to OAuth consent screen requirements.
# Visit: https://console.firebase.google.com/project/{project}/authentication/providers
#
# Required steps:
# 1. Enable Google Sign-In provider
# 2. Configure OAuth consent screen
# 3. Add authorized domains for your frontend

# ========================================
```

```
# Secret Manager
# ========================================
# Create a secret for application configuration
app_secret = gcp.secretmanager.Secret(
    "app-secret",
    secret_id=f"{project_name}-{stack}-app-config",
    project=project,
    replication=gcp.secretmanager.SecretReplicationArgs(
        auto=gcp.secretmanager.SecretReplicationAutoArgs(),
    ),
    labels=common_labels,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)

# Grant access to the secret manually via gcloud or Cloud Console
# IAM bindings must be set manually because Pulumi SA needs setIamPolicy permission
# Run: scripts/setup-gcp-secret-permissions.sh
# Or use: gcloud secrets add-iam-policy-binding multicloud-auto-deploy-staging-app-config \
#         --member="serviceAccount:github-actions-deploy@ashnova.iam.gserviceaccount.com" \
#         --role="roles/secretmanager.secretAccessor"

# NOTE: SecretVersion resource intentionally removed from Pulumi management.
# Pulumi SA lacks 'secretmanager.versions.access' permission (read-back after create causes 403).
# Secret versions are managed manually via gcloud:
#   gcloud secrets versions add {project_name}-{stack}-app-config --data-file=secret.json

# ========================================
# Cloud Storage Bucket for Function Source
# ========================================
function_source_bucket = gcp.storage.Bucket(
    "function-source-bucket",
    name=f"{project}-{project_name}-{stack}-function-source",
    location=region.upper(),
    storage_class="STANDARD",
    uniform_bucket_level_access=True,
    labels=common_labels,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)

# ========================================
# Cloud Storage Bucket for Uploads (Image Uploads via Presigned URLs)
# ========================================
uploads_bucket = gcp.storage.Bucket(
    "uploads-bucket",
    name=f"{project}-{project_name}-{stack}-uploads",
    location=region.upper(),
    storage_class="STANDARD",
    uniform_bucket_level_access=True,
    cors=[
        gcp.storage.BucketCorArgs(
            origins=allowed_origins_list,
            methods=["GET", "HEAD", "PUT", "OPTIONS"],
            response_headers=[
                "Content-Type",
                "Authorization",
                "X-Requested-With",
                "x-ms-blob-type",
            ],
```

```
            max_age_seconds=3600,
        )
    ],
    labels=common_labels,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)

# Get default Compute Engine service account (used by Cloud Functions Gen 2 / Cloud Run)
default_compute_sa = gcp.compute.get_default_service_account()

# Grant Compute SA objectAdmin on uploads bucket
gcp.storage.BucketIAMBinding(
    "uploads-bucket-object-admin",
    bucket=uploads_bucket.name,
    role="roles/storage.objectAdmin",
    members=[f"serviceAccount:{default_compute_sa.email}"],
)

# Allow public (unauthenticated) read access to uploaded images
gcp.storage.BucketIAMBinding(
    "uploads-bucket-public-read",
    bucket=uploads_bucket.name,
    role="roles/storage.objectViewer",
    members=["allUsers"],
)

# Grant Compute SA serviceAccountTokenCreator on itself (for signed URL generation)
gcp.serviceaccount.IAMBinding(
    "compute-sa-token-creator",
    service_account_id=default_compute_sa.name,
    role="roles/iam.serviceAccountTokenCreator",
    members=[f"serviceAccount:{default_compute_sa.email}"],
)

# ======================================
# Cloud Storage Bucket for Frontend
# ======================================
frontend_bucket = gcp.storage.Bucket(
    "frontend-bucket",
    name=f"{project}-{project_name}-{stack}-frontend",
    location=region.upper(),
    storage_class="STANDARD",
    uniform_bucket_level_access=True,
    website=gcp.storage.BucketWebsiteArgs(
        main_page_suffix="index.html",
        not_found_page="index.html",  # For SPA routing
    ),
    # CORS configuration
    cors=[
        gcp.storage.BucketCorArgs(
            origins=allowed_origins_list,
            methods=["GET", "HEAD", "OPTIONS"],
            response_headers=["Content-Type", "Authorization", "X-Requested-With"],
            max_age_seconds=3600,
        )
    ],
    labels=common_labels,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
```

```
)

# Make frontend bucket publicly readable
gcp.storage.BucketIAMBinding(
    "frontend-public-read",
    bucket=frontend_bucket.name,
    role="roles/storage.objectViewer",
    members=["allUsers"],
)

# ========================================
# Cloud Armor Security Policy (WAF)
# Note: Cloud Armor is disabled for staging to reduce cost (~$6-7/month)
# ========================================
if stack == "production":
    armor_security_policy = gcp.compute.SecurityPolicy(
        "cloud-armor-policy",
        name=f"{project_name}-{stack}-armor",
        project=project,
        type="CLOUD_ARMOR_EDGE",  # Required for edge security
        description="Cloud Armor edge security policy for CDN protection",
        rules=[
            # Block known bad IPs (example - can be customized)
            gcp.compute.SecurityPolicyRuleArgs(
                action="deny(403)",
                priority=1000,
                match=gcp.compute.SecurityPolicyRuleMatchArgs(
                    versioned_expr="SRC_IPS_V1",
                    config=gcp.compute.SecurityPolicyRuleMatchConfigArgs(
                        src_ip_ranges=["192.0.2.0/24"],  # Example bad IP range
                    ),
                ),
                description="Block known bad IP ranges",
            ),
            # Default rule: allow all (required at priority 2147483647)
            gcp.compute.SecurityPolicyRuleArgs(
                action="allow",
                priority=2147483647,
                match=gcp.compute.SecurityPolicyRuleMatchArgs(
                    versioned_expr="SRC_IPS_V1",
                    config=gcp.compute.SecurityPolicyRuleMatchConfigArgs(
                        src_ip_ranges=["*"],
                    ),
                ),
                description="Default rule: allow all traffic",
            ),
        ],
        opts=pulumi.ResourceOptions(depends_on=enabled_services),
    )

# ========================================
# Cloud CDN for Frontend with HTTPS
# ========================================
# External IP Address for Load Balancer
cdn_ip_address = gcp.compute.GlobalAddress(
    "cdn-ip-address",
    name=f"{project_name}-{stack}-cdn-ip",
    project=project,
```

```
    labels=common_labels,
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
)

# Backend Bucket for Cloud CDN with conditional Cloud Armor
backend_bucket_kwargs = {
    "name": f"{project_name}-{stack}-cdn-backend",
    "bucket_name": frontend_bucket.name,
    "enable_cdn": True,
    "project": project,
    "cdn_policy": gcp.compute.BackendBucketCdnPolicyArgs(
        cache_mode="CACHE_ALL_STATIC",
        default_ttl=3600,  # 1 hour
        max_ttl=86400,  # 24 hours
        client_ttl=3600,
        negative_caching=True,
        serve_while_stale=86400,
    ),
    # Allow popup window (Firebase signInWithPopup) to check window.closed
    # without being blocked by Cross-Origin-Opener-Policy on the OAuth page.
    "custom_response_headers": [
        "Cross-Origin-Opener-Policy: same-origin-allow-popups",
    ],
    "opts": pulumi.ResourceOptions(depends_on=enabled_services),
}

# Attach Cloud Armor security policy only for production
if stack == "production":
    backend_bucket_kwargs["edge_security_policy"] = armor_security_policy.self_link

backend_bucket = gcp.compute.BackendBucket(
    "cdn-backend-bucket", **backend_bucket_kwargs
)

# Managed SSL Certificate (for custom domain - optional)
# Note: This requires a custom domain. For now, we'll use HTTP only.
# To enable HTTPS with custom domain:
# 1. Set up a custom domain
# 2. Create a managed SSL certificate
# 3. Use TargetHttpsProxy instead of TargetHttpProxy

# URL Map for Load Balancer
# All traffic → backend_bucket (GCS static files)
# React SPA routing (/sns/*) is handled by the GCS bucket's not_found_page
# (index.html fallback) and served as static files.
url_map = gcp.compute.URLMap(
    "cdn-url-map",
    name=f"{project_name}-{stack}-cdn-urlmap",
    project=project,
    default_service=backend_bucket.self_link,
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# Target HTTPS Proxy with SSL (managed certificate)
# Custom domain configuration (optional)
custom_domain = config.get("customDomain")  # e.g., gcp.yourdomain.com
ssl_domains = (
    [custom_domain] if custom_domain else [f"{project_name}-{stack}.example.com"]
```

```
)

# Use a hash of the domain list in the cert name so that changing domains
# triggers create-before-delete (avoids "resource in use" errors).
_ssl_domain_hash = hashlib.md5(",".join(ssl_domains).encode()).hexdigest()[:8]
managed_ssl_cert = gcp.compute.ManagedSslCertificate(
    "managed-ssl-cert",
    name=f"{project_name}-{stack}-ssl-cert-{_ssl_domain_hash}",
    project=project,
    managed=gcp.compute.ManagedSslCertificateManagedArgs(
        domains=ssl_domains,
    ),
    opts=pulumi.ResourceOptions(
        depends_on=enabled_services,
        # Avoid forced replacement when domain hash changes; update manually if needed.
        ignore_changes=["name", "managed"],
        # No delete_before_replace: Pulumi creates new cert first, updates the
        # HTTPS proxy, then deletes the old cert (avoids "in use" errors).
    ),
)

https_proxy = gcp.compute.TargetHttpsProxy(
    "cdn-https-proxy",
    name=f"{project_name}-{stack}-cdn-https-proxy",
    project=project,
    url_map=url_map.self_link,
    ssl_certificates=[managed_ssl_cert.self_link],
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# URL Map for HTTP → HTTPS redirect (301 Moved Permanently)
# All HTTP traffic is redirected to HTTPS; no content is served over HTTP.
redirect_url_map = gcp.compute.URLMap(
    "cdn-redirect-url-map",
    name=f"{project_name}-{stack}-cdn-redirect",
    project=project,
    default_url_redirect=gcp.compute.URLMapDefaultUrlRedirectArgs(
        https_redirect=True,
        redirect_response_code="MOVED_PERMANENTLY_DEFAULT",
        strip_query=False,
    ),
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# Target HTTP Proxy — redirects HTTP → HTTPS (not serving content directly)
http_proxy = gcp.compute.TargetHttpProxy(
    "cdn-http-proxy",
    name=f"{project_name}-{stack}-cdn-http-proxy",
    project=project,
    url_map=redirect_url_map.self_link,
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# Global Forwarding Rule for HTTPS (443)
https_forwarding_rule = gcp.compute.GlobalForwardingRule(
    "cdn-https-forwarding-rule",
    name=f"{project_name}-{stack}-cdn-lb-https",
    project=project,
```

```
    ip_address=cdn_ip_address.address,
    ip_protocol="TCP",
    port_range="443",
    target=https_proxy.self_link,
    load_balancing_scheme="EXTERNAL",
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# Global Forwarding Rule for HTTP (80) - redirect to HTTPS
forwarding_rule = gcp.compute.GlobalForwardingRule(
    "cdn-forwarding-rule",
    name=f"{project_name}-{stack}-cdn-lb",
    project=project,
    ip_address=cdn_ip_address.address,
    ip_protocol="TCP",
    port_range="80",
    target=http_proxy.self_link,
    load_balancing_scheme="EXTERNAL",
    opts=pulumi.ResourceOptions(depends_on=enabled_services, ignore_changes=["name"]),
)

# ========================================
# Note about Cloud Functions
# ========================================
# Cloud Functions Gen 2 will be deployed via gcloud CLI in the GitHub Actions workflow
# because Pulumi requires the source ZIP to exist before creating the function.
#
# The workflow will:
# 1. Use Pulumi to create buckets and IAM settings (this file)
# 2. Build and package the function code
# 3. Upload ZIP to function_source_bucket
# 4. Deploy Cloud Functions via: gcloud functions deploy --gen2 --source=gs://...

# ========================================
# Outputs
# ========================================
pulumi.export("project_id", project)
pulumi.export("region", region)

# Custom domain exports (if configured)
if custom_domain:
    pulumi.export("custom_domain", custom_domain)
    pulumi.export("custom_domain_url", f"https://{custom_domain}")
    pulumi.export("ssl_certificate_domains", ssl_domains)

# Firebase Authentication
pulumi.export("firebase_project_id", project)
pulumi.export("firebase_web_app_id", firebase_web_app.app_id)
pulumi.export("firebase_web_app_name", firebase_web_app.name)
pulumi.export(
    "auth_config_instructions",
    pulumi.Output.concat(
        "Firebase Web App created successfully!\\n",
        "\\n",
        "To get Firebase configuration (API Key, Auth Domain):\\n",
        "1. Visit: https://console.firebase.google.com/project/",
        project,
        "/settings/general\\n",
```

```
        "2. Scroll to 'Your apps' section\\n",
        "3. Find your web app: ",
        firebase_web_app.display_name,
        "\\n",
        "4. Copy the Firebase SDK configuration\\n",
        "\\n",
        "To enable authentication:\\n",
        "1. Visit: https://console.firebase.google.com/project/",
        project,
        "/authentication\\n",
        "2. Enable Google Sign-In provider\\n",
        "3. Configure OAuth consent screen\\n",
        "\\n",
        "Cloud Functions environment variables:\\n  AUTH_PROVIDER=firebase\\n",
        "  GCP_PROJECT_ID=",
        project,
        "\\n",
    ),
)
pulumi.export("function_source_bucket", function_source_bucket.name)
pulumi.export("uploads_bucket", uploads_bucket.name)
pulumi.export("compute_sa_email", default_compute_sa.email)
pulumi.export("frontend_bucket", frontend_bucket.name)
pulumi.export("secret_name", app_secret.secret_id)
pulumi.export(
    "frontend_url",
    frontend_bucket.name.apply(
        lambda name: f"https://storage.googleapis.com/{name}/index.html"
    ),
)
pulumi.export("cdn_ip_address", cdn_ip_address.address)
pulumi.export("cdn_url", cdn_ip_address.address.apply(lambda ip: f"http://{ip}"))
pulumi.export("cdn_https_url", cdn_ip_address.address.apply(lambda ip: f"https://{ip}"))
pulumi.export("backend_bucket_name", backend_bucket.name)
pulumi.export("url_map_name", url_map.name)
pulumi.export("redirect_url_map_name", redirect_url_map.name)
if audit_config is not None:
    pulumi.export("audit_config_service", audit_config.service)
# Cloud Armor exports only for production
if stack == "production":
    pulumi.export("cloud_armor_policy_name", armor_security_policy.name)
pulumi.export("ssl_certificate_name", managed_ssl_cert.name)

# ======================================
# Monitoring and Alerts
# ======================================
alarm_email = config.get("alarmEmail")
billing_account_id = config.get("billingAccountId")
# Billing budget disabled by default due to API authentication issues
enable_billing_budget = False
function_name = f"{project_name}-{stack}-api"

# Cloud Function memory setting (must match --memory in GitHub Actions deploy workflow)
# Default: 512MB. Update here if changed in .github/workflows/deploy-gcp.yml
function_memory_mb = config.get_int("functionMemoryMb") or 512

monitoring_resources = monitoring.setup_monitoring(
    project_name=project_name,
```

```
    stack=stack,
    function_name=pulumi.Output.from_input(function_name),
    region=region,
    project_id=project,
    alarm_email=alarm_email,
    billing_account_id=None,
    monthly_budget_usd=config.get_int("monthlyBudgetUsd") or 50,
    function_memory_mb=function_memory_mb,
)

# Function name for gcloud deployment (fixed name)
pulumi.export("function_name", f"{project_name}-{stack}-api")

# Monitoring exports
if monitoring_resources["notification_channel"]:
    pulumi.export(
        "monitoring_notification_channel_id",
        monitoring_resources["notification_channel"].id,
    )
pulumi.export(
    "monitoring_function_alerts", list(monitoring_resources["function_alerts"].keys())
)
pulumi.export(
    "monitoring_firestore_alerts", list(monitoring_resources["firestore_alerts"].keys())
)
if monitoring_resources["billing_budget"]:
    pulumi.export(
        "monitoring_billing_budget_name",
        monitoring_resources["billing_budget"].display_name,
    )

# Cost estimation
cost_estimate_base = (
    "Cloud Functions: 2M free invocations/month, then $0.40 per 1M. "
    "Cloud Storage: $0.02/GB/month. "
    "Cloud CDN: $0.02-0.08/GB depending on region. "
    "Managed SSL Certificate: Free. "
    "Secret Manager: $0.06 per month per secret + $0.03 per 10,000 access operations. "
    "External IP: $0.01/hour (~$7.30/month). "
)

if stack == "production":
    cost_estimate = (
        cost_estimate_base
        + "Cloud Armor: $6/month per policy + $0.75 per 1M requests. "
        "Estimated: $15-25/month for low traffic with security features."
    )
else:
    cost_estimate = (
        cost_estimate_base + "Cloud Armor: Disabled for staging (saves ~$6-7/month). "
        "Estimated: $8-15/month for low traffic without Cloud Armor."
    )

pulumi.export("cost_estimate", cost_estimate)
```

5.6 GCP Pulumi.staging.yaml

```yaml
config:
  gcp:project: ashnova
  gcp:region: asia-northeast1
  # SECURITY: Change this before production deployment!
  # Set to actual frontend URLs once custom domain is configured.
  # Example: "https://gcp.yourdomain.com,https://yourdomain.com"
  multicloud-auto-deploy-gcp:allowedOrigins: "https://staging.gcp.ashnova.jp"
  multicloud-auto-deploy-gcp:alarmEmail: sat0sh1kawada@spa.nifty.com
  multicloud-auto-deploy-gcp:monthlyBudgetUsd: "50"
  # Static landing page custom domain (optional)
  # multicloud-auto-deploy-gcp:staticSiteDomain: gcp-staging.yourdomain.com

  multicloud-auto-deploy-gcp:customDomain: staging.gcp.ashnova.jp
  multicloud-auto-deploy-gcp:enableAuditLogs: "true"
```

Chapter 6

CI/CDワークフロー (GitHub Actions)

このセクションでは、GitHub Actionsワークフローファイルを掲載します。

6.1 deploy-aws.yml - AWS デプロイ

```yaml
name: Deploy to AWS

on:
  push:
    branches:
      - develop # 自動デプロイstaging
      - main # 自動デプロイproduction
    paths:
      - "services/**"
      - "infrastructure/pulumi/aws/**"
      - ".github/workflows/deploy-aws.yml"
  workflow_dispatch:
    inputs:
      environment:
        description: "Deployment environment"
        required: true
        default: "staging"
        type: choice
        options:
          - staging
          - production

env:
  AWS_REGION: ap-northeast-1
  NODE_VERSION: "24"
  PYTHON_VERSION: "3.13"

# Prevent concurrent Pulumi updates on the same stack (causes ResourceConflictException)
concurrency:
  group: deploy-aws-${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment ||
     (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
  cancel-in-progress: false

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
```

```
    # →developstaging, →mainproduction, 手動→選択した環境
    environment: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment || (github.ref
     == 'refs/heads/main' && 'production' || 'staging') }}

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install Pulumi CLI
        uses: pulumi/actions@v5

      - name: Install Pulumi Python Dependencies
        run: |
          cd infrastructure/pulumi/aws
          pip install -r requirements.txt

      - name: Determine Pulumi Stack Name
        id: set_stack
        run: |
          if [ "${{ github.event_name }}" == "workflow_dispatch" ]; then
            STACK_NAME="${{ github.event.inputs.environment }}"
          elif [ "${{ github.ref }}" == "refs/heads/main" ]; then
            STACK_NAME="production"
          else
            STACK_NAME="staging"
          fi
          echo "stack_name=$STACK_NAME" >> $GITHUB_OUTPUT
          echo ☒" Using Pulumi stack: $STACK_NAME"

      - name: Load Cloud Config
        # Single source of truth for stable per-environment values.
        # eliminates dependency on gitignored Pulumi.*.yaml and repo-level secrets.
        id: cloud_config
        run: |
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          CONFIG_FILE=".github/config/aws.${STACK_NAME}.env"
          if [ ! -f "$CONFIG_FILE" ]; then
            echo ☒" Config file $CONFIG_FILE not found — cannot proceed"
            exit 1
          fi
          source "$CONFIG_FILE"
          echo ☒" Loaded config: $CONFIG_FILE"
```

```
          echo "  custom_domain=$CLOUD_CUSTOM_DOMAIN"
          echo "  cloudfront_domain=$CLOUD_CLOUDFRONT_DOMAIN"
          echo "custom_domain=$CLOUD_CUSTOM_DOMAIN" >> $GITHUB_OUTPUT
          echo "acm_cert_arn=$CLOUD_ACM_CERT_ARN" >> $GITHUB_OUTPUT
          echo "cloudfront_domain=$CLOUD_CLOUDFRONT_DOMAIN" >> $GITHUB_OUTPUT
          echo "cloudfront_distribution_id=$CLOUD_CLOUDFRONT_DISTRIBUTION_ID" >> $GITHUB_OUTPUT

      - name: Initialize Pulumi Stack
        run: |
          cd infrastructure/pulumi/aws
          pulumi login
          pulumi stack select ${{ steps.set_stack.outputs.stack_name }} 2>/dev/null || pulumi stack init ${{ steps.set_stack.outputs.stack_name }}
          pulumi config set aws:region ${{ env.AWS_REGION }}
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}

      - name: Sync Pulumi Config (custom domain + ACM cert)
        run: |
          cd infrastructure/pulumi/aws
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          STACK_FILE="Pulumi.${STACK_NAME}.yaml"

          # Read config from committed Pulumi YAML (single source of truth).
          # Previously this step read from GitHub Secrets (AWS_CUSTOM_DOMAIN etc.),
          # which caused production to inherit the staging custom domain when the
          # repo-level secret was set to staging.aws.ashnova.jp.
          # Fix: always read customDomain / acmCertificateArn / cloudFrontDomain
          # from the committed Pulumi.<stack>.yaml — no secrets needed.
          # Read from committed config file (Pulumi.*.yaml is gitignored and unavailable in CI)
          CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          ACM_CERT_ARN="${{ steps.cloud_config.outputs.acm_cert_arn }}"
          CF_DOMAIN="${{ steps.cloud_config.outputs.cloudfront_domain }}"

          if [ -n "$CUSTOM_DOMAIN" ]; then
            echo ☒" Setting customDomain = $CUSTOM_DOMAIN"
            pulumi config set multicloud-auto-deploy-aws:customDomain "$CUSTOM_DOMAIN"
          else
            echo ⚠"  customDomain not set in $STACK_FILE — skipping"
          fi
          if [ -n "$ACM_CERT_ARN" ]; then
            echo ☒" Setting acmCertificateArn"
            pulumi config set multicloud-auto-deploy-aws:acmCertificateArn "$ACM_CERT_ARN"
          else
            echo ⚠"  acmCertificateArn not set in $STACK_FILE — skipping"
          fi
          if [ -n "$CF_DOMAIN" ]; then
            echo ☒" Setting cloudFrontDomain = $CF_DOMAIN"
            pulumi config set multicloud-auto-deploy-aws:cloudFrontDomain "$CF_DOMAIN"
          else
            echo ⚠"  cloudFrontDomain not set in $STACK_FILE — skipping"
          fi
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          AWS_REGION: ${{ env.AWS_REGION }}

      - name: Remove stuck obsolete resources from Pulumi state
        run: |
```

```
          cd infrastructure/pulumi/aws
          STACK="${{ steps.set_stack.outputs.stack_name }}"
          # These landing-* resources were removed from code but remain in state.
          # Pulumi tries to delete them on every up, but fails:
          #   landing-oai: IAM permission denied (cloudfront:DeleteCloudFrontOriginAccessIdentity)
          #   landing-bucket: BucketNotEmpty
          # Removing from state retains AWS resources but stops Pulumi tracking them.
          pulumi state delete \
            "urn:pulumi:${STACK}::multicloud-auto-deploy-aws::aws:cloudfront/originAccessIdentity:OriginAccessIdentity::landing-oai" \
            --force --yes 2>/dev/null || echo "landing-oai not in state"
          pulumi state delete \
            "urn:pulumi:${STACK}::multicloud-auto-deploy-aws::aws:s3/bucket:Bucket::landing-bucket" \
            --force --yes 2>/dev/null || echo "landing-bucket not in state"
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          AWS_REGION: ${{ env.AWS_REGION }}

      - name: Deploy Infrastructure with Pulumi
        id: pulumi
        uses: pulumi/actions@v5
        with:
          command: up
          stack-name: ${{ steps.set_stack.outputs.stack_name }}
          work-dir: infrastructure/pulumi/aws
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          AWS_REGION: ${{ env.AWS_REGION }}

      - name: Get Pulumi Outputs
        id: pulumi_outputs
        run: |
          cd infrastructure/pulumi/aws
          echo "lambda_function_name=$(pulumi stack output lambda_function_name)" >> $GITHUB_OUTPUT
          echo "api_gateway_endpoint=$(pulumi stack output api_gateway_endpoint)" >> $GITHUB_OUTPUT
          echo "frontend_bucket_name=$(pulumi stack output frontend_bucket_name)" >> $GITHUB_OUTPUT
          echo "cloudfront_domain=$(pulumi stack output cloudfront_domain)" >> $GITHUB_OUTPUT
          echo "cloudfront_distribution_id=$(pulumi stack output cloudfront_distribution_id)" >> $GITHUB_OUTPUT
          echo "custom_domain=$(pulumi stack output custom_domain 2>/dev/null || echo '')" >> $GITHUB_OUTPUT
          echo "cognito_user_pool_id=$(pulumi stack output cognito_user_pool_id)" >> $GITHUB_OUTPUT
          echo "cognito_client_id=$(pulumi stack output cognito_client_id)" >> $GITHUB_OUTPUT
          echo "cognito_domain=$(pulumi stack output cognito_domain)" >> $GITHUB_OUTPUT
          echo "posts_table_name=$(pulumi stack output posts_table_name)" >> $GITHUB_OUTPUT
          echo "images_bucket_name=$(pulumi stack output images_bucket_name)" >> $GITHUB_OUTPUT
          echo "secret_name=$(pulumi stack output secret_name)" >> $GITHUB_OUTPUT
          echo "acm_certificate_arn=$(pulumi stack output acm_certificate_arn 2>/dev/null || echo '')" >> $GITHUB_OUTPUT

      - name: Ensure CloudFront alias + ACM cert (post-Pulumi safety net)
        run: |
          CD="${{ steps.pulumi_outputs.outputs.cloudfront_distribution_id }}"
          CUSTOM_DOMAIN="${{ steps.pulumi_outputs.outputs.custom_domain }}"
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          ACM_CERT_ARN="${{ steps.cloud_config.outputs.acm_cert_arn }}"

          if [[ -z "$CUSTOM_DOMAIN" || -z "$ACM_CERT_ARN" || -z "$CD" ]]; then
            echo ☒"  No custom domain or ACM cert configured — skipping alias check"
```

```
            exit 0
          fi

          CURRENT_ALIAS=$(aws cloudfront get-distribution --id "$CD" \
            --query 'Distribution.DistributionConfig.Aliases.Items[0]' --output text 2>/dev/null || echo "None")
          if [[ "$CURRENT_ALIAS" == "$CUSTOM_DOMAIN" ]]; then
            echo ☒" CloudFront alias already set to $CUSTOM_DOMAIN — no action needed"
            exit 0
          fi

          echo ⚠"  CloudFront alias is '$CURRENT_ALIAS' (expected '$CUSTOM_DOMAIN') — restoring..."
          ETAG=$(aws cloudfront get-distribution-config --id "$CD" --query 'ETag' --output text)
          aws cloudfront get-distribution-config --id "$CD" --query 'DistributionConfig' --output json >
      /tmp/cf-dist.json
          # Replace Python heredoc with jq to avoid YAML block scalar parsing issues
          # (same fix pattern applied to deploy-gcp.yml: "Python heredoc が YAML block scalar を壊す")
          jq --arg domain "$CUSTOM_DOMAIN" --arg cert "$ACM_CERT_ARN" '
            .Aliases = {"Quantity": 1, "Items": [$domain]} |
            .ViewerCertificate = {
              "ACMCertificateArn": $cert,
              "SSLSupportMethod": "sni-only",
              "MinimumProtocolVersion": "TLSv1.2_2021",
              "CertificateSource": "acm"
            } |
            del(.ViewerCertificate.CloudFrontDefaultCertificate)
          ' /tmp/cf-dist.json > /tmp/cf-dist-updated.json && mv /tmp/cf-dist-updated.json /tmp/cf-dist.json
          aws cloudfront update-distribution --id "$CD" --if-match "$ETAG" \
            --distribution-config file:///tmp/cf-dist.json \
            --query 'Distribution.{Status:Status,Aliases:DistributionConfig.Aliases.Items}' \
            --output json
          echo ☒" CloudFront alias + ACM cert restored for $CUSTOM_DOMAIN"

        - name: Ensure Cognito callback URLs (post-Pulumi safety net)
          run: |
            USER_POOL_ID="${{ steps.pulumi_outputs.outputs.cognito_user_pool_id }}"
            CLIENT_ID="${{ steps.pulumi_outputs.outputs.cognito_client_id }}"
            CUSTOM_DOMAIN="${{ steps.pulumi_outputs.outputs.custom_domain }}"
            CF_DOMAIN="${{ steps.pulumi_outputs.outputs.cloudfront_domain }}"

            if [[ -z "$USER_POOL_ID" || -z "$CLIENT_ID" ]]; then
              echo ☒"  No Cognito pool/client — skipping"
              exit 0
            fi

            # Build required callback URLs
            REQUIRED_CALLBACK="https://${CUSTOM_DOMAIN}/sns/auth/callback"
            CURRENT=$(aws cognito-idp describe-user-pool-client \
              --user-pool-id "$USER_POOL_ID" --client-id "$CLIENT_ID" \
              --query 'UserPoolClient.CallbackURLs' --output json)

            if echo "$CURRENT" | grep -q "$REQUIRED_CALLBACK"; then
              echo ☒" Cognito callback URL already includes $REQUIRED_CALLBACK"
              exit 0
            fi

            echo ⚠"  Cognito callback URL missing $REQUIRED_CALLBACK — restoring..."
            # Build merged callback URL list using jq (avoids multi-line Python heredoc that breaks YAML parsing)
            CALLBACK_URLS=$(echo "$CURRENT" | jq -r --arg d "$CUSTOM_DOMAIN" --arg c "$CF_DOMAIN" \
```

```
262             '(["http://localhost:5173/sns/auth/callback", "https://\($d)/sns/auth/callback"] + (if ($c != "" and
        $c != $d) then ["https://\($c)/sns/auth/callback"] else [] end) + .) | reduce .[] as $u ([]; if (. |
        index($u)) then . else . + [$u] end) | join(" ")')
263           LOGOUT_URLS="http://localhost:5173/sns/ https://${CUSTOM_DOMAIN}/sns/"
264
265           aws cognito-idp update-user-pool-client \
266             --user-pool-id "$USER_POOL_ID" \
267             --client-id "$CLIENT_ID" \
268             --callback-urls $CALLBACK_URLS \
269             --logout-urls $LOGOUT_URLS \
270             --allowed-o-auth-flows "code" \
271             --allowed-o-auth-scopes "email" "openid" "profile" \
272             --supported-identity-providers "COGNITO" \
273             --allowed-o-auth-flows-user-pool-client \
274             --query 'UserPoolClient.CallbackURLs' --output json
275           echo ☒" Cognito callback URLs restored"
276
277       - name: Build and Deploy Frontend
278         run: |
279           FRONTEND_BUCKET="${{ steps.pulumi_outputs.outputs.frontend_bucket_name }}"
280           CLOUDFRONT_ID="${{ steps.pulumi_outputs.outputs.cloudfront_distribution_id }}"
281           COGNITO_DOMAIN="${{ steps.pulumi_outputs.outputs.cognito_domain }}"
282           COGNITO_CLIENT_ID="${{ steps.pulumi_outputs.outputs.cognito_client_id }}"
283           API_URL="${{ steps.pulumi_outputs.outputs.api_gateway_endpoint }}"
284           CLOUDFRONT_DOMAIN="${{ steps.pulumi_outputs.outputs.cloudfront_domain }}"
285           CUSTOM_DOMAIN="${{ steps.pulumi_outputs.outputs.custom_domain }}"
286
287           # Determine the public URL (prefer custom domain if set)
288           if [[ -n "$CUSTOM_DOMAIN" ]]; then
289             PUBLIC_ORIGIN="https://${CUSTOM_DOMAIN}"
290           else
291             PUBLIC_ORIGIN="https://${CLOUDFRONT_DOMAIN}"
292           fi
293
294           echo ☒"  Building React frontend for AWS (Cognito auth)..."
295           cd services/frontend_react
296
297           npm ci --prefer-offline
298
299           VITE_AUTH_PROVIDER=aws \
300           VITE_API_URL="${API_URL}" \
301           VITE_COGNITO_DOMAIN="${COGNITO_DOMAIN}.auth.${{ env.AWS_REGION }}.amazoncognito.com" \
302           VITE_COGNITO_CLIENT_ID="${COGNITO_CLIENT_ID}" \
303           VITE_COGNITO_REDIRECT_URI="${PUBLIC_ORIGIN}/sns/auth/callback" \
304           VITE_COGNITO_LOGOUT_URI="${PUBLIC_ORIGIN}/sns/" \
305           VITE_BASE_PATH=/sns/ \
306           npm run build
307
308           echo ☒" Frontend build complete. Uploading to S3..."
309           cd ../..
310
311           # Upload to S3 at sns/ prefix (CloudFront /sns* path pattern points here)
312           aws s3 sync services/frontend_react/dist/ "s3://${FRONTEND_BUCKET}/sns/" \
313             --delete \
314             --region ${{ env.AWS_REGION }}
315
316           # Invalidate CloudFront cache for sns/ paths
317           aws cloudfront create-invalidation \
```

```
            --distribution-id $CLOUDFRONT_ID \
            --paths "/sns/*" \
            --region us-east-1

          echo ☒" Frontend deployed to S3 and CloudFront cache invalidated"
          echo ☒" Frontend URL: ${PUBLIC_ORIGIN}/sns/"

      - name: Build Lambda Layer
        id: build_layer
        run: |
          cd services/api
          rm -rf .build-layer lambda-layer.zip

          # Build Lambda Layer (dependencies only)
          echo ☒" Building Lambda Layer (dependencies)..."
          bash ../../scripts/build-lambda-layer.sh

          # Capture layer info for subsequent steps
          if [ -f lambda-layer.zip ]; then
            LAYER_SIZE=$(du -h lambda-layer.zip | cut -f1)
            echo ☒" Layer package created: $LAYER_SIZE"
            ls -lh lambda-layer.zip

            # Export for GitHub Actions
            echo "layer_created=true" >> $GITHUB_OUTPUT
          else
            echo ☒" Layer package creation failed"
            exit 1
          fi

      - name: Package Backend (Application Code Only)
        run: |
          cd services/api
          rm -rf .build lambda.zip
          mkdir -p .build/package

          # Copy application code only (no dependencies - they're in the Layer)
          echo ☒" Copying application code..."
          cp -r app .build/package/
          cp index.py .build/package/

          # Create zip (application code only - lightweight!)
          echo ☒"  Creating lambda package (code only)..."
          cd .build/package
          zip -r ../../lambda.zip .
          cd ../..

          # Show package info
          echo ☒" Package created:"
          ls -lh lambda.zip
          SIZE_MB=$(du -m lambda.zip | cut -f1)
          echo ☒" Package size: ${SIZE_MB}MB"

          if [ $SIZE_MB -lt 50 ]; then
            echo ☒" Package is under 50MB - direct upload possible!"
            echo "use_direct_upload=true" >> $GITHUB_OUTPUT
          else
            echo ⚠"  Package exceeds 50MB - will use S3"
```

```
            echo "use_direct_upload=false" >> $GITHUB_OUTPUT
          fi

      - name: Deploy Lambda Layer
        id: deploy_layer
        run: |
          set -e

          LAYER_NAME="multicloud-auto-deploy-${{ steps.set_stack.outputs.stack_name }}-dependencies"
          LAYER_ZIP="services/api/lambda-layer.zip"

          # Validate ZIP file exists and is readable
          if [ ! -f "$LAYER_ZIP" ]; then
            echo ⊠" Layer ZIP not found: $LAYER_ZIP"
            exit 1
          fi

          LAYER_SIZE=$(du -h "$LAYER_ZIP" | cut -f1)
          echo ⊠" Layer size: $LAYER_SIZE"
          echo ⊠" Publishing Custom Lambda Layer: $LAYER_NAME"

          # Publish layer with error handling
          LAYER_VERSION_ARN=$(aws lambda publish-layer-version \
            --layer-name "$LAYER_NAME" \
            --description "Dependencies for FastAPI + Mangum + JWT (Python 3.12)" \
            --zip-file "fileb://$LAYER_ZIP" \
            --compatible-runtimes python3.12 \
            --region "${{ env.AWS_REGION }}" \
            --query 'LayerVersionArn' \
            --output text 2>&1) || {
            echo ⊠" Failed to publish layer"
            echo "$LAYER_VERSION_ARN"
            exit 1
          }

          if [ -z "$LAYER_VERSION_ARN" ]; then
            echo ⊠" Layer ARN is empty"
            exit 1
          fi

          echo ⊠" Layer published: $LAYER_VERSION_ARN"
          echo "layer_arn=$LAYER_VERSION_ARN" >> $GITHUB_OUTPUT

      - name: Cleanup Old Lambda Layer Versions
        continue-on-error: true
        run: |
          set +e  # Continue even if cleanup fails

          LAYER_NAME="multicloud-auto-deploy-${{ steps.set_stack.outputs.stack_name }}-dependencies"
          echo ⊠" Cleaning up old Lambda Layer versions (keeping latest 3) for: $LAYER_NAME"

          # Get layer versions (suppress errors if layer doesn't exist yet)
          VERSIONS=$(aws lambda list-layer-versions \
            --layer-name "$LAYER_NAME" \
            --region "${{ env.AWS_REGION }}" \
            --query "LayerVersions[].Version" \
            --output text 2>/dev/null) || true

```

```
          if [ -z "$VERSIONS" ]; then
            echo ⊠"  No versions found — skipping cleanup"
            exit 0
          fi

          # Sort and delete old versions (keep latest 3)
          SORTED=$(echo "$VERSIONS" | tr ' ' '\n' | sort -rn | tail -n +4)
          DELETED_COUNT=0

          for v in $SORTED; do
            if aws lambda delete-layer-version \
              --layer-name "$LAYER_NAME" \
              --version-number "$v" \
              --region "${{ env.AWS_REGION }}" >/dev/null 2>&1; then
              echo "  ✓ Deleted v$v"
              ((DELETED_COUNT++))
            else
              echo "  ⚠  Could not delete v$v (skipped)"
            fi
          done

          REMAINING=$(aws lambda list-layer-versions \
            --layer-name "$LAYER_NAME" \
            --region "${{ env.AWS_REGION }}" \
            --query "length(LayerVersions)" \
            --output text 2>/dev/null || echo "?")

          echo ⊠" Cleanup done ($DELETED_COUNT deleted). Remaining versions: $REMAINING"

      - name: Update Lambda Function
        id: update_lambda
        run: |
          set -e
          LAMBDA_FUNCTION="${{ steps.pulumi_outputs.outputs.lambda_function_name }}"

          # Use custom Lambda Layer (optimized, no Klayers)
          echo ⊠" Using custom Lambda Layer"
          LAYER_ARNS="${{ steps.deploy_layer.outputs.layer_arn }}"

          # Wait for any Pulumi-triggered updates to complete before touching this Lambda
          # (Pulumi up just ran and may have updated the Lambda's environment/configuration)
          echo ⊠" Waiting for Lambda to be ready (Pulumi may still be updating it)..."
          aws lambda wait function-updated \
            --function-name $LAMBDA_FUNCTION \
            --region ${{ env.AWS_REGION }}
          echo ⊠" Lambda ready"

          # Debug: Check lambda.zip exists and show size
          echo ⊠" Checking lambda.zip..."
          ls -lh services/api/lambda.zip || { echo ⊠" lambda.zip not found!"; exit 1; }

          SIZE_MB=$(du -m services/api/lambda.zip | cut -f1)

          if [ $SIZE_MB -lt 50 ]; then
            # Direct upload (faster!)
            echo ⊠" Direct upload to Lambda (${SIZE_MB}MB < 50MB)"
            aws lambda update-function-code \
              --function-name $LAMBDA_FUNCTION \
```

```
            --region ${{ env.AWS_REGION }} \
            --zip-file fileb://services/api/lambda.zip \
            --output json || { echo ☒" Lambda update failed!"; exit 1; }
          else
            # S3 upload for large packages
            S3_BUCKET="${{ steps.pulumi_outputs.outputs.frontend_bucket_name }}"
            S3_KEY="lambda-deployments/lambda-$(date +%Y%m%d-%H%M%S).zip"

            echo ☁"  Uploading to S3: s3://$S3_BUCKET/$S3_KEY"
            aws s3 cp services/api/lambda.zip s3://$S3_BUCKET/$S3_KEY || { echo ☒" S3 upload failed!"; exit 1; }

            echo ☒" Updating Lambda from S3"
            aws lambda update-function-code \
              --function-name $LAMBDA_FUNCTION \
              --region ${{ env.AWS_REGION }} \
              --s3-bucket $S3_BUCKET \
              --s3-key $S3_KEY \
              --output json || { echo ☒" Lambda update failed!"; exit 1; }
          fi

          # Wait for code update to complete before modifying configuration
          echo ☒" Waiting for Lambda code update to stabilize..."
          aws lambda wait function-updated \
            --function-name $LAMBDA_FUNCTION \
            --region ${{ env.AWS_REGION }}
          echo ☒" Lambda ready for configuration update"

          # Get CloudFront domain for CORS
          CLOUDFRONT_DOMAIN="${{ steps.pulumi_outputs.outputs.cloudfront_domain }}"
          CUSTOM_DOMAIN="${{ steps.pulumi_outputs.outputs.custom_domain }}"

          # Fallback: if Pulumi output is empty, use committed config file value
          if [[ -z "$CUSTOM_DOMAIN" ]]; then
            CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
            [[ -n "$CUSTOM_DOMAIN" ]] && echo ☒" CUSTOM_DOMAIN from cloud config: $CUSTOM_DOMAIN"
          fi

          # Include custom domain in CORS if set
          if [[ -n "$CUSTOM_DOMAIN" ]]; then
            CORS_ORIGINS="https://${CLOUDFRONT_DOMAIN},https://${CUSTOM_DOMAIN},http://localhost:5173"
          else
            CORS_ORIGINS="https://${CLOUDFRONT_DOMAIN},http://localhost:5173"
          fi

          echo ☒" CORS Origins: $CORS_ORIGINS"

          # Create environment variables JSON to avoid escaping issues
          USER_POOL_ID="${{ steps.pulumi_outputs.outputs.cognito_user_pool_id }}"
          CLIENT_ID="${{ steps.pulumi_outputs.outputs.cognito_client_id }}"
          POSTS_TABLE="${{ steps.pulumi_outputs.outputs.posts_table_name }}"
          IMAGES_BUCKET="${{ steps.pulumi_outputs.outputs.images_bucket_name }}"
          SECRET_NAME="${{ steps.pulumi_outputs.outputs.secret_name }}"

          # Determine environment and auth settings
          DEPLOY_ENV="${{ steps.set_stack.outputs.stack_name }}"
          # Auth is always enabled: Cognito is configured for both staging and production.
          AUTH_DISABLED_VALUE="false"

```

```
          echo '{' > /tmp/lambda-env.json
          echo '  "Variables": {' >> /tmp/lambda-env.json
          echo '    "CLOUD_PROVIDER": "aws",' >> /tmp/lambda-env.json
          echo "    \"ENVIRONMENT\": \"$DEPLOY_ENV\"," >> /tmp/lambda-env.json
          echo "    \"AUTH_DISABLED\": \"$AUTH_DISABLED_VALUE\"," >> /tmp/lambda-env.json
          echo '    "AUTH_PROVIDER": "cognito",' >> /tmp/lambda-env.json
          echo "    \"AWS_COGNITO_USER_POOL_ID\": \"$USER_POOL_ID\"," >> /tmp/lambda-env.json
          echo "    \"AWS_COGNITO_CLIENT_ID\": \"$CLIENT_ID\"," >> /tmp/lambda-env.json
          echo "    \"COGNITO_REGION\": \"${{ env.AWS_REGION }}\"," >> /tmp/lambda-env.json
          echo "    \"POSTS_TABLE_NAME\": \"$POSTS_TABLE\"," >> /tmp/lambda-env.json
          echo "    \"IMAGES_BUCKET_NAME\": \"$IMAGES_BUCKET\"," >> /tmp/lambda-env.json
          echo "    \"SECRET_NAME\": \"$SECRET_NAME\"," >> /tmp/lambda-env.json
          echo "    \"TEXTRACT_REGION\": \"ap-northeast-2\"," >> /tmp/lambda-env.json
          echo "    \"BEDROCK_REGION\": \"us-east-1\"," >> /tmp/lambda-env.json
          echo "    \"BEDROCK_MODEL_ID\": \"amazon.nova-pro-v1:0\"," >> /tmp/lambda-env.json
          echo "    \"SOLVE_OCR_REVIEW_MIN_SCORE\": \"0.35\"," >> /tmp/lambda-env.json
          echo "    \"SOLVE_OCR_REVIEW_MAX_REPLACEMENT_RATIO\": \"0.01\"," >> /tmp/lambda-env.json
          echo "    \"CORS_ORIGINS\": \"$CORS_ORIGINS\"" >> /tmp/lambda-env.json
          echo '  }' >> /tmp/lambda-env.json
          echo '}' >> /tmp/lambda-env.json

          # Update configuration with environment variables and Layer(s)
          echo ☒" Updating Lambda configuration..."
          aws lambda update-function-configuration \
            --function-name $LAMBDA_FUNCTION \
            --region ${{ env.AWS_REGION }} \
            --layers $LAYER_ARNS \
            --environment file:///tmp/lambda-env.json \
            --output json || { echo ☒" Lambda configuration update failed!"; exit 1; }

          echo ☒" Lambda function updated successfully with Layer(s) and environment variables"
          echo "api_endpoint=${{ steps.pulumi_outputs.outputs.api_gateway_endpoint }}" >> $GITHUB_OUTPUT

      - name: Notify Success
        if: success()
        run: |
          echo ☒" AWS deployment succeeded!"
          echo ☒" API Endpoint: ${{ steps.pulumi_outputs.outputs.api_gateway_endpoint }}"
          echo ☒" Frontend: https://${{ steps.pulumi_outputs.outputs.cloudfront_domain }}/sns/"

      - name: Notify Failure
        if: failure()
        run: echo ☒" AWS deployment failed!"
```

6.2 deploy-azure.yml - Azure デプロイ

```
name: Deploy to Azure

on:
  push:
    branches:
      - develop # 自動デプロイstaging
      - main # 自動デプロイproduction
    paths:
      - "services/api/**"
      - "infrastructure/pulumi/azure/**"
```

```yaml
      - ".github/workflows/deploy-azure.yml"
  workflow_dispatch:
    inputs:
      environment:
        description: "Deployment environment"
        required: true
        default: "staging"
        type: choice
        options:
          - staging
          - production

env:
  AZURE_REGION: japaneast
  NODE_VERSION: "24"
  # Must match Azure Functions Flex Consumption runtime version.
  # The "Build and Package" step Docker image must also use this version.
  PYTHON_VERSION: "3.12"

# Prevent concurrent Pulumi updates on the same stack (causes 409 Conflict)
concurrency:
  group: deploy-azure-${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment ||
     (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
  # staging(develop): 古いをキャンセルしてコスト削減Run / production(main): 安全確実に完走
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    # →developstaging, →mainproduction, 手動→選択した環境
    environment: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment || (github.ref
     == 'refs/heads/main' && 'production' || 'staging') }}

    steps:
      - uses: actions/checkout@v4

      # Skip initial Azure Login - use Service Principal env vars for Terraform only
      # - name: Azure Login
      #   uses: azure/login@v1
      #   with:
      #     creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: "pip"
          cache-dependency-path: infrastructure/pulumi/azure/requirements.txt

      - name: Install Pulumi CLI
        uses: pulumi/actions@v5

      - name: Install Pulumi Python Dependencies
```

```
        run: |
          cd infrastructure/pulumi/azure
          pip install -r requirements.txt

      - name: Determine Pulumi Stack Name
        id: set_stack
        run: |
          if [ "${{ github.event_name }}" == "workflow_dispatch" ]; then
            STACK_NAME="${{ github.event.inputs.environment }}"
          elif [ "${{ github.ref }}" == "refs/heads/main" ]; then
            STACK_NAME="production"
          else
            STACK_NAME="staging"
          fi
          echo "stack_name=$STACK_NAME" >> $GITHUB_OUTPUT
          echo ☒" Using Pulumi stack: $STACK_NAME"

      - name: Load Cloud Config
        # Single source of truth for stable per-environment values.
        # Eliminates dependency on repo-level secrets and Pulumi runtime outputs
        # for values that rarely change (AD Client ID, Tenant ID, custom domain).
        id: cloud_config
        run: |
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          CONFIG_FILE=".github/config/azure.${STACK_NAME}.env"
          if [ ! -f "$CONFIG_FILE" ]; then
            echo ☒" Config file $CONFIG_FILE not found — cannot proceed"
            exit 1
          fi
          source "$CONFIG_FILE"
          echo ☒" Loaded config: $CONFIG_FILE"
          echo "  custom_domain=$CLOUD_CUSTOM_DOMAIN"
          echo "  azure_client_id=${CLOUD_AZURE_CLIENT_ID:0:8}..."
          echo "custom_domain=$CLOUD_CUSTOM_DOMAIN" >> $GITHUB_OUTPUT
          echo "azure_client_id=$CLOUD_AZURE_CLIENT_ID" >> $GITHUB_OUTPUT
          echo "azure_tenant_id=$CLOUD_AZURE_TENANT_ID" >> $GITHUB_OUTPUT

      - name: Initialize Pulumi Stack
        run: |
          cd infrastructure/pulumi/azure
          pulumi login
          pulumi stack select ${{ steps.set_stack.outputs.stack_name }} 2>/dev/null || pulumi stack init ${{
    steps.set_stack.outputs.stack_name }}
          pulumi config set azure-native:location japaneast
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Set Azure Environment Variables
        id: azure_env
        run: |
          echo "subscription_id=${{ secrets.AZURE_SUBSCRIPTION_ID }}" >> $GITHUB_OUTPUT
          CREDS='${{ secrets.AZURE_CREDENTIALS }}'
          echo "client_id=$(echo $CREDS | jq -r '.clientId')" >> $GITHUB_OUTPUT
```

```
          echo "client_secret=$(echo $CREDS | jq -r '.clientSecret')" >> $GITHUB_OUTPUT
          echo "tenant_id=$(echo $CREDS | jq -r '.tenantId')" >> $GITHUB_OUTPUT

      # Deploy Azure AD resources with Pulumi (Function App remains manual)
      # Azure Portal quota limitations prevent IaC-based App Service Plan creation
      # Function App created manually: multicloud-auto-deploy-staging-func (FC1 Flex Consumption)

      - name: Cancel any pending Pulumi operations
        run: |
          cd infrastructure/pulumi/azure
          # Cancel any in-progress update
          pulumi cancel --yes 2>/dev/null || true
          # Clear any leftover pending operations from interrupted deployments
          pulumi stack export > /tmp/pulumi-state.json 2>/dev/null || true
          if jq -e '.deployment.pending_operations | length > 0' /tmp/pulumi-state.json > /dev/null 2>&1; then
            echo "Found pending operations - clearing..."
            jq '.deployment.pending_operations = []' /tmp/pulumi-state.json > /tmp/pulumi-state-clean.json
            pulumi stack import --force < /tmp/pulumi-state-clean.json
            echo "Pending operations cleared"
          else
            echo "No pending operations found"
          fi
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}

      - name: Sync Pulumi Config (frontendDomain)
        run: |
          cd infrastructure/pulumi/azure
          # Set frontendDomain in Pulumi config so Azure AD app registration
          # gets the correct redirect URI for the Front Door endpoint.
          # On first deploy (no FD yet), this is a no-op and re-run will fix it.
          FRONTDOOR=$(pulumi stack output frontdoor_hostname 2>/dev/null || echo "")
          if [ -n "$FRONTDOOR" ] && [ "$FRONTDOOR" != "None" ]; then
            echo ☒" Setting frontendDomain = $FRONTDOOR"
            pulumi config set frontendDomain "$FRONTDOOR"
          else
            echo ☒"  No existing frontdoor_hostname in stack state — skipping frontendDomain config"
          fi
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          ARM_CLIENT_ID: ${{ steps.azure_env.outputs.client_id }}
          ARM_CLIENT_SECRET: ${{ steps.azure_env.outputs.client_secret }}
          ARM_TENANT_ID: ${{ steps.azure_env.outputs.tenant_id }}
          ARM_SUBSCRIPTION_ID: ${{ steps.azure_env.outputs.subscription_id }}

      - name: Deploy Infrastructure with Pulumi
        id: pulumi
        uses: pulumi/actions@v5
        with:
          command: up
          stack-name: ${{ steps.set_stack.outputs.stack_name }}
          work-dir: infrastructure/pulumi/azure
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          ARM_CLIENT_ID: ${{ steps.azure_env.outputs.client_id }}
          ARM_CLIENT_SECRET: ${{ steps.azure_env.outputs.client_secret }}
          ARM_TENANT_ID: ${{ steps.azure_env.outputs.tenant_id }}
          ARM_SUBSCRIPTION_ID: ${{ steps.azure_env.outputs.subscription_id }}
```

```
          AZURE_STORAGE_ACCOUNT: "" # Required for Pulumi state backend

      - name: Get Pulumi Outputs
        id: pulumi_outputs
        run: |
          cd infrastructure/pulumi/azure

          # Get resource names from Pulumi
          RG_NAME=$(pulumi stack output resource_group_name)
          echo "resource_group_name=${RG_NAME}" >> $GITHUB_OUTPUT

          # Get Azure AD outputs
          echo "azure_ad_client_id=$(pulumi stack output azure_ad_client_id)" >> $GITHUB_OUTPUT
          echo "azure_ad_object_id=$(pulumi stack output azure_ad_object_id)" >> $GITHUB_OUTPUT

          # Get Cosmos DB outputs
          echo "cosmos_account_name=$(pulumi stack output cosmos_account_name)" >> $GITHUB_OUTPUT
          echo "cosmos_db_endpoint=$(pulumi stack output cosmos_db_endpoint)" >> $GITHUB_OUTPUT
          echo "cosmos_database_name=$(pulumi stack output cosmos_database_name)" >> $GITHUB_OUTPUT
          echo "cosmos_container_name=$(pulumi stack output cosmos_container_name)" >> $GITHUB_OUTPUT

          # Get Azure AI Document Intelligence outputs (OCR for university exam problems)
          echo "document_intelligence_endpoint=$(pulumi stack output document_intelligence_endpoint 2>/dev/null || echo '')" >> $GITHUB_OUTPUT
          echo "document_intelligence_key=$(pulumi stack output --show-secrets document_intelligence_key 2>/dev/null || echo '')" >> $GITHUB_OUTPUT

          # Get Azure OpenAI outputs (LLM for university exam solver)
          echo "azure_openai_endpoint=$(pulumi stack output azure_openai_endpoint 2>/dev/null || echo '')" >> $GITHUB_OUTPUT
          echo "azure_openai_key=$(pulumi stack output --show-secrets azure_openai_key 2>/dev/null || echo '')" >> $GITHUB_OUTPUT
          echo "azure_openai_deployment_name=$(pulumi stack output azure_openai_deployment_name 2>/dev/null || echo 'gpt-4o')" >> $GITHUB_OUTPUT

          # Get Front Door hostname for CORS
          echo "frontdoor_hostname=$(pulumi stack output frontdoor_hostname)" >> $GITHUB_OUTPUT
          echo "frontdoor_endpoint_name=$(pulumi stack output frontdoor_endpoint_name)" >> $GITHUB_OUTPUT

          # Manual Function App reference (not managed by Pulumi)
          # Function App name is environment-specific
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          FUNC_APP_NAME="multicloud-auto-deploy-${STACK_NAME}-func"
          echo "function_app_name=${FUNC_APP_NAME}" >> $GITHUB_OUTPUT

          # Get storage account names directly from Pulumi outputs
          # (Azure CLI query "contains(name, 'web')" matches multiple accounts and may pick wrong one)
          echo "frontend_storage_name=$(pulumi stack output frontend_storage_name)" >> $GITHUB_OUTPUT
          echo "functions_storage_name=$(pulumi stack output functions_storage_name)" >> $GITHUB_OUTPUT

          # Get Function App URL
          FUNC_HOSTNAME=$(az functionapp config hostname list \
            --webapp-name ${FUNC_APP_NAME} \
            --resource-group $RG_NAME \
            --query '[0].name' -o tsv)
          API_URL="https://${FUNC_HOSTNAME}/api/HttpTrigger"
          echo "api_url=${API_URL}" >> $GITHUB_OUTPUT

```

```
          echo ☒" Configured API URL: ${API_URL}"
          echo ☒" Azure AD Client ID: $(pulumi stack output azure_ad_client_id)"
          echo ☒"  Cosmos DB Account: $(pulumi stack output cosmos_account_name)"

      - name: Link Custom Domain to Front Door Route
        # Pulumi's Route resource does not include custom_domains, so we must link the manually-managed
        # custom domain after each Pulumi deploy. Without this step, Azure Front Door serves the
        # *.azureedge.net wildcard cert instead of the managed cert for www.azure.ashnova.jp.
        run: |
          AZURE_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          if [ -z "$AZURE_CUSTOM_DOMAIN" ]; then
            echo ☒"  No custom domain configured, skipping"
            exit 0
          fi

          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"
          ENDPOINT_NAME="${{ steps.pulumi_outputs.outputs.frontdoor_endpoint_name }}"
          PROFILE_NAME="multicloud-auto-deploy-${STACK_NAME}-fd"
          ROUTE_NAME="multicloud-auto-deploy-${STACK_NAME}-route"
          SUBSCRIPTION_ID=$(az account show --query id --output tsv)

          # Domain resource name: dots replaced with dashes (Azure naming convention)
          DOMAIN_NAME=$(echo "$AZURE_CUSTOM_DOMAIN" | sed 's/\\./-/g')

DOMAIN_ID="/subscriptions/${SUBSCRIPTION_ID}/resourcegroups/${RESOURCE_GROUP}/providers/Microsoft.Cdn/profiles/${PROFILE_NAME

          echo ☒" Linking $AZURE_CUSTOM_DOMAIN to Front Door route: $ROUTE_NAME"

          ACCESS_TOKEN=$(az account get-access-token --query accessToken --output tsv)
          HTTP_STATUS=$(curl -sX PATCH \

"https://management.azure.com/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Cdn/profi
\
            -H "Authorization: Bearer ${ACCESS_TOKEN}" \
            -H "Content-Type: application/json" \
            -d "{\"properties\":{\"customDomains\":[{\"id\":\"${DOMAIN_ID}\"}]}}" \
            --max-time 30 -o /dev/null -w "%{http_code}")

          if [ "$HTTP_STATUS" = "200" ] || [ "$HTTP_STATUS" = "202" ]; then
            echo ☒" Custom domain $AZURE_CUSTOM_DOMAIN linked to Front Door route (HTTP ${HTTP_STATUS})"
          else
            echo ⚠"  Custom domain linking returned HTTP ${HTTP_STATUS} (non-fatal, continuing)"
          fi

      - name: Update Azure AD App Redirect URIs
        # Ensures ALL required redirect URIs are registered (login callbacks + post-logout redirects).
        # Pulumi only manages FrontDoor-based URIs; custom domain and /sns/ logout URIs are added here.
        run: |
          # Config file is authoritative — no Pulumi runtime dependency for stable AD values
          AD_APP_ID="${{ steps.cloud_config.outputs.azure_client_id }}"
          FRONTDOOR="${{ steps.pulumi_outputs.outputs.frontdoor_hostname }}"
          AZURE_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          if [ -z "$AD_APP_ID" ]; then
            echo ☒"  No Azure AD App ID in config file, skipping redirect URI update"
            exit 0
          fi

```

```
290          # Build redirect URI list: login callbacks (/sns/auth/callback) + logout redirects (/sns/)
291          URIS=(
292            "http://localhost:3000/callback"
293            "https://localhost:3000/callback"
294            "http://localhost:5173/sns/auth/callback"
295            "https://localhost:5173/sns/auth/callback"
296            "http://localhost:5173/sns/"
297          )
298
299          # Add FrontDoor URIs
300          if [ -n "$FRONTDOOR" ]; then
301            URIS+=("https://${FRONTDOOR}/sns/auth/callback")
302            URIS+=("https://${FRONTDOOR}/sns/")
303          fi
304
305          # Add custom domain URIs
306          if [ -n "$AZURE_CUSTOM_DOMAIN" ]; then
307            URIS+=("https://${AZURE_CUSTOM_DOMAIN}/sns/auth/callback")
308            URIS+=("https://${AZURE_CUSTOM_DOMAIN}/sns/")
309          fi
310
311          echo ⊠" Registering redirect URIs on Azure AD App (${AD_APP_ID}):"
312          for uri in "${URIS[@]}"; do echo "   $uri"; done
313
314          az ad app update --id "$AD_APP_ID" --web-redirect-uris "${URIS[@]}"
315          echo ⊠" Azure AD redirect URIs updated"
316
317        - name: Build and Package Azure Function
318          run: |
319            cd services/api
320
321            echo ⊠" Creating optimized deployment package..."
322
323            # Create deployment directory
324            rm -rf .deployment
325            mkdir -p .deployment
326
327            # Install dependencies using Docker with the EXACT same platform/Python as Azure Functions
328            # Azure Functions Flex Consumption uses Python 3.12 on linux/amd64.
329            # The GitHub Actions runner may be a different arch (e.g., arm64), so using the runner's
330            # native pip would produce wrong .so binaries (e.g., cpython-311-aarch64 instead of
331            # cpython-312-x86_64). Docker ensures correct platform + Python version.
332            echo "Installing Azure-specific dependencies (linux/amd64 python:3.12)..."
333            docker run --rm --platform linux/amd64 \
334              -v "$(pwd):/work" \
335              python:3.12-slim \
336              pip install --target /work/.deployment --no-cache-dir -q \
337                -r /work/requirements-azure.txt
338
339            # Aggressive cleanup of unnecessary files
340            echo "Cleaning up unnecessary files..."
341            find .deployment -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true
342            find .deployment -type d -name "tests" -exec rm -rf {} + 2>/dev/null || true
343            find .deployment -type d -name "*.dist-info" -exec rm -rf {} + 2>/dev/null || true
344            find .deployment -type d -name "*.egg-info" -exec rm -rf {} + 2>/dev/null || true
345            find .deployment -type f -name "*.pyc" -delete 2>/dev/null || true
346            find .deployment -type f -name "*.pyo" -delete 2>/dev/null || true
347            # NOTE: *.so files must NOT be deleted — pydantic_core and other C extensions
```

```
          # ship as compiled .so shared libraries and are required at runtime.

          # Remove test and documentation files
          find .deployment -type f -name "test_*.py" -delete 2>/dev/null || true
          find .deployment -type f -name "*_test.py" -delete 2>/dev/null || true
          find .deployment -type d -name "docs" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type d -name "examples" -exec rm -rf {} + 2>/dev/null || true

          # Copy application code
          cp -r app .deployment/
          cp function_app.py .deployment/
          cp host.json .deployment/
          cp requirements-azure.txt .deployment/requirements.txt

          # Clean up app __pycache__
          find .deployment/app -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true

          # Verify correct .so architecture
          SO_FILE=$(find .deployment -name '_pydantic_core*.so' | head -1)
          if [ -n "$SO_FILE" ]; then
            echo ☒" pydantic_core: $(basename $SO_FILE)"
          else
            echo ⚠"  pydantic_core .so not found — package may be missing"
          fi

          # Create ZIP package with maximum compression
          cd .deployment
          zip -r9 -q ../function-app.zip .
          cd ..

          echo ☒" Package created:"
          ls -lh function-app.zip
          PACKAGE_SIZE=$(du -h function-app.zip | cut -f1)
          PACKAGE_SIZE_KB=$(du -k function-app.zip | cut -f1)
          echo ☒" Package size: $PACKAGE_SIZE ($PACKAGE_SIZE_KB KB)"

          # Show size breakdown
          echo ""
          echo ☒" Top 10 largest items in package:"
          unzip -l function-app.zip | sort -k4 -n -r | head -n 11

      - name: Deploy Function App
        run: |
          FUNCTION_APP="${{ steps.pulumi_outputs.outputs.function_app_name }}"
          RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"
          COSMOS_ACCOUNT="${{ steps.pulumi_outputs.outputs.cosmos_account_name }}"

          echo ☒" Deploying to Function App: $FUNCTION_APP"
          echo ☒"  Cosmos DB Account: $COSMOS_ACCOUNT"

          # Check for ongoing deployments and wait if necessary
          echo ☒" Checking for ongoing deployments..."
          MAX_WAIT=300  # 5 minutes max wait
          WAIT_COUNT=0
          while [ $WAIT_COUNT -lt $MAX_WAIT ]; do
            # Get deployment status from Kudu API
            DEPLOY_STATUS=$(az rest --method get \
              --url "https://${FUNCTION_APP}.scm.azurewebsites.net/api/deployments/latest" \
```

```
            --resource "https://management.azure.com/" \
            --query "status" -o tsv 2>/dev/null || echo "Unknown")

          if [ "$DEPLOY_STATUS" = "Running" ] || [ "$DEPLOY_STATUS" = "Building" ]; then
            echo ⏳" Deployment in progress (status: $DEPLOY_STATUS), waiting 30s..."
            sleep 30
            WAIT_COUNT=$((WAIT_COUNT+30))
          else
            echo ✅" No ongoing deployment detected"
            break
          fi
        done

        if [ $WAIT_COUNT -ge $MAX_WAIT ]; then
          echo ⚠️"  Timeout waiting for previous deployment, proceeding anyway..."
        fi

        sleep 10  # Additional safety buffer

        # Get Cosmos DB endpoint and keys directly from Pulumi outputs
        echo 🔍" Getting Cosmos DB credentials from Pulumi outputs..."
        COSMOS_ENDPOINT="${{ steps.pulumi_outputs.outputs.cosmos_db_endpoint }}"
        COSMOS_DATABASE="${{ steps.pulumi_outputs.outputs.cosmos_database_name }}"
        COSMOS_CONTAINER="${{ steps.pulumi_outputs.outputs.cosmos_container_name }}"
        COSMOS_ACCOUNT="${{ steps.pulumi_outputs.outputs.cosmos_account_name }}"

        # Get Cosmos DB key from Azure
        COSMOS_KEY=$(az cosmosdb keys list \
          --name $COSMOS_ACCOUNT \
          --resource-group $RESOURCE_GROUP \
          --type keys \
          --query primaryMasterKey -o tsv)

        if [ -z "$COSMOS_ENDPOINT" ] || [ -z "$COSMOS_KEY" ]; then
          echo ❌" Failed to retrieve Cosmos DB credentials"
          echo "   Endpoint: $COSMOS_ENDPOINT"
          echo "   Key: ${COSMOS_KEY:0:20}..."
          echo "   Account: $COSMOS_ACCOUNT"
          exit 1
        fi

        echo ✅" Cosmos DB Account: $COSMOS_ACCOUNT"
        echo ✅" Cosmos DB Endpoint: $COSMOS_ENDPOINT"

        # Get web storage key for image uploads (SAS URL generation)
        FRONTEND_STORAGE_NAME="${{ steps.pulumi_outputs.outputs.frontend_storage_name }}"
        IMAGES_STORAGE_KEY=$(az storage account keys list \
          --account-name "$FRONTEND_STORAGE_NAME" \
          --resource-group "$RESOURCE_GROUP" \
          --query "[0].value" -o tsv 2>/dev/null || echo "")
        if [ -z "$IMAGES_STORAGE_KEY" ]; then
          echo ⚠️"  Could not retrieve storage key for $FRONTEND_STORAGE_NAME — image uploads may fail"
        else
          echo ✅" Images storage: $FRONTEND_STORAGE_NAME (key retrieved)"
        fi

        # Get Front Door URL for CORS from Pulumi outputs
        FRONTDOOR_URL="${{ steps.pulumi_outputs.outputs.frontdoor_hostname }}"
```

```
464         CORS_ORIGINS="https://${FRONTDOOR_URL},http://localhost:5173"
465         # Custom domain from config file (single source of truth)
466         AZURE_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
467         if [ -n "$AZURE_CUSTOM_DOMAIN" ]; then
468           CORS_ORIGINS="https://${AZURE_CUSTOM_DOMAIN},${CORS_ORIGINS}"
469         fi
470
471         echo ⊠" CORS Origins: $CORS_ORIGINS"
472
473         # Set Azure Functions platform-level CORS.
474         # In Flex Consumption, Kestrel intercepts OPTIONS preflight BEFORE Python/FastAPI.
475         # Platform CORS must include the allowed origins so Kestrel returns the required
476         # Access-Control-Allow-Origin and Access-Control-Allow-Headers response headers.
477         # NOTE: Kestrel returns CORS headers for origins that are explicitly listed here.
478         #       It only responds to 'authorization' preflight headers (GET/non-body requests)
479         #       and 'authorization+content-type' (POST requests with JSON body), which covers
480         #       all browser use cases.
481         echo ⊠" Setting platform-level CORS origins..."
482         # Build JSON array of allowed origins
483         ORIGINS_JSON="["
484         IFS=',' read -ra ORIGIN_ARRAY <<< "$CORS_ORIGINS"
485         for i in "${!ORIGIN_ARRAY[@]}"; do
486           if [ $i -gt 0 ]; then ORIGINS_JSON+=","; fi
487           ORIGINS_JSON+="\"${ORIGIN_ARRAY[$i]}\""
488         done
489         ORIGINS_JSON+="]"
490         az rest \
491           --method PUT \
492           --url "https://management.azure.com/subscriptions/${{ secrets.AZURE_SUBSCRIPTION_ID
    }}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites/${FUNCTION_APP}/config/web?api-version=2022-03-01"
    \
493           --body
    "{\"properties\":{\"cors\":{\"allowedOrigins\":${ORIGINS_JSON},\"supportCredentials\":false}}}" \
494           2>/dev/null || echo ⚠"  Could not set platform CORS (non-fatal)"
495
496         # Remove existing CORS settings (both uppercase and lowercase)
497         echo ⊠" Removing existing CORS settings if present..."
498         az functionapp config appsettings delete \
499           --name $FUNCTION_APP \
500           --resource-group $RESOURCE_GROUP \
501           --setting-names CORS_ORIGINS cors_origins 2>/dev/null || true
502
503         # Wait for delete to propagate
504         sleep 3
505
506         # Set environment variables (including Azure AD authentication)
507         # Determine environment and auth settings
508         DEPLOY_ENV="${{ steps.set_stack.outputs.stack_name }}"
509         # AUTH_DISABLED must always be false to enforce auth guards on all environments
510         AUTH_DISABLED_VALUE="false"
511
512         echo ⊠"  Setting environment variables (ENVIRONMENT=$DEPLOY_ENV,
    AUTH_DISABLED=$AUTH_DISABLED_VALUE)..."
513
514         # Fix storage settings: ensure AzureWebJobsStorage points to the correct storage account.
515         # The Function App was originally misconfigured with a non-existent storage account.
516         # Fetch the connection string from the Pulumi-managed functions storage account.
517         FUNCTIONS_STORAGE_NAME="${{ steps.pulumi_outputs.outputs.functions_storage_name }}"
```

```
      STORAGE_CONN=$(az storage account show-connection-string \
        --name "$FUNCTIONS_STORAGE_NAME" \
        --resource-group "$RESOURCE_GROUP" \
        --query connectionString -o tsv 2>/dev/null || echo "")

      if [ -n "$STORAGE_CONN" ]; then
        echo ☒" Retrieved storage connection string for $FUNCTIONS_STORAGE_NAME — fixing AzureWebJobsStorage"
        az functionapp config appsettings set \
          --name $FUNCTION_APP \
          --resource-group $RESOURCE_GROUP \
          --settings \
            AzureWebJobsStorage="$STORAGE_CONN" \
            DEPLOYMENT_STORAGE_CONNECTION_STRING="$STORAGE_CONN" \
          > /dev/null
      else
        echo ⚠"  Could not retrieve storage connection string for $FUNCTIONS_STORAGE_NAME (using existing setting)"
      fi

      az functionapp config appsettings set \
        --name $FUNCTION_APP \
        --resource-group $RESOURCE_GROUP \
        --settings \
          CLOUD_PROVIDER=azure \
          ENVIRONMENT="${DEPLOY_ENV}" \
          AUTH_DISABLED="${AUTH_DISABLED_VALUE}" \
          COSMOS_DB_ENDPOINT="${COSMOS_ENDPOINT}" \
          COSMOS_DB_KEY="${COSMOS_KEY}" \
          COSMOS_DB_DATABASE="${{ steps.pulumi_outputs.outputs.cosmos_database_name }}" \
          COSMOS_DB_CONTAINER="${{ steps.pulumi_outputs.outputs.cosmos_container_name }}" \
          AUTH_PROVIDER=azure \
          AZURE_TENANT_ID="${{ steps.cloud_config.outputs.azure_tenant_id }}" \
          AZURE_CLIENT_ID="${{ steps.cloud_config.outputs.azure_client_id }}" \
          AZURE_STORAGE_ACCOUNT_NAME="${FRONTEND_STORAGE_NAME}" \
          AZURE_STORAGE_ACCOUNT_KEY="${IMAGES_STORAGE_KEY}" \
          AZURE_STORAGE_CONTAINER="images" \
          CORS_ORIGINS="${CORS_ORIGINS}" \
          WEBSITE_WARMUP_PATH="/sns/health" \
          WEBSITE_KEEPALIVE_TIMEOUT="30" \
        > /dev/null

      # Configure Azure AI Document Intelligence (OCR) and Azure OpenAI settings.
      # DI endpoint/key come from Pulumi outputs; OpenAI comes from GitHub Secrets.
      # These are optional: if not set, solve endpoints return 503 (SOLVE_ENABLED defaults to false).
      DI_ENDPOINT="${{ steps.pulumi_outputs.outputs.document_intelligence_endpoint }}"
      DI_KEY="${{ steps.pulumi_outputs.outputs.document_intelligence_key }}"
      # Azure OpenAI: prefer Pulumi outputs (auto-provisioned), fall back to GitHub Secrets
      AOAI_ENDPOINT="${{ steps.pulumi_outputs.outputs.azure_openai_endpoint }}"
      AOAI_KEY="${{ steps.pulumi_outputs.outputs.azure_openai_key }}"
      AOAI_DEPLOYMENT="${{ steps.pulumi_outputs.outputs.azure_openai_deployment_name }}"
      if [ -z "$AOAI_ENDPOINT" ]; then AOAI_ENDPOINT="${{ secrets.AZURE_OPENAI_ENDPOINT }}"; fi
      if [ -z "$AOAI_KEY" ]; then AOAI_KEY="${{ secrets.AZURE_OPENAI_KEY }}"; fi
      if [ -z "$AOAI_DEPLOYMENT" ]; then AOAI_DEPLOYMENT="${{ secrets.AZURE_OPENAI_DEPLOYMENT }}"; fi
      SOLVE_ENABLED_VALUE="${{ secrets.AZURE_SOLVE_ENABLED }}"

      if [ -n "$DI_ENDPOINT" ] && [ -n "$DI_KEY" ]; then
        echo ☒" Configuring Azure Document Intelligence (OCR) settings..."
```

```
            az functionapp config appsettings set \
              --name $FUNCTION_APP \
              --resource-group $RESOURCE_GROUP \
              --settings \
                AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT="${DI_ENDPOINT}" \
                AZURE_DOCUMENT_INTELLIGENCE_KEY="${DI_KEY}" \
              > /dev/null
          else
            echo ⚠"  Document Intelligence endpoint/key not available — OCR feature will use fallback"
          fi

          if [ -n "$AOAI_ENDPOINT" ] && [ -n "$AOAI_KEY" ]; then
            echo ☒" Configuring Azure OpenAI settings..."
            az functionapp config appsettings set \
              --name $FUNCTION_APP \
              --resource-group $RESOURCE_GROUP \
              --settings \
                AZURE_OPENAI_ENDPOINT="${AOAI_ENDPOINT}" \
                AZURE_OPENAI_KEY="${AOAI_KEY}" \
                AZURE_OPENAI_DEPLOYMENT="${AOAI_DEPLOYMENT:-gpt-4o}" \
              > /dev/null
          else
            echo ☒"  AZURE_OPENAI_ENDPOINT/KEY not set — LLM answer generation will use fallback"
          fi

          # Enable or disable solve endpoints based on AZURE_SOLVE_ENABLED secret
          az functionapp config appsettings set \
            --name $FUNCTION_APP \
            --resource-group $RESOURCE_GROUP \
            --settings \
              SOLVE_ENABLED="${SOLVE_ENABLED_VALUE:-false}" \
            > /dev/null

          # Disable HTTP/2 on Function App to prevent AFD stale TCP connection pool issue
          # AFD Standard uses HTTP/2 multiplexing which can hold stale streams after
          # Dynamic Consumption instance recycling, causing intermittent 502 errors
          az functionapp config set \
            --name $FUNCTION_APP \
            --resource-group $RESOURCE_GROUP \
            --http20 false \
            --output none

          echo ☒" Azure AD Authentication configured:"
          echo "  Tenant ID: ${{ steps.azure_env.outputs.tenant_id }}"
          echo "  Client ID: ${{ steps.pulumi_outputs.outputs.azure_ad_client_id }}"
          echo ☒"  Cosmos DB configured:"
          echo "  Database: ${{ steps.pulumi_outputs.outputs.cosmos_database_name }}"
          echo "  Container: ${{ steps.pulumi_outputs.outputs.cosmos_container_name }}"

          # Clear WEBSITE_RUN_FROM_PACKAGE to prevent stale package blob from overriding new deployment.
          # If this is set to an old/expired SAS URL, the function will ignore the new zip deploy and serve 404.
          echo ☒" Clearing WEBSITE_RUN_FROM_PACKAGE (prevent stale package conflict)..."
          az functionapp config appsettings delete \
            --name $FUNCTION_APP \
            --resource-group $RESOURCE_GROUP \
            --setting-names WEBSITE_RUN_FROM_PACKAGE \
            --output none 2>/dev/null && echo ☒"  WEBSITE_RUN_FROM_PACKAGE cleared" || echo ☒"
    WEBSITE_RUN_FROM_PACKAGE not set (OK)"
```

```
          sleep 5

          # Fix FC1 Flex Consumption deployment storage.
          # Strategy: read the ACTUAL deployment storage account from the function app's
          # ARM resource (functionAppConfig.deployment.storage.value), then ensure that
          # storage account EXISTS and the correct connection string app setting is set.
          echo ⊠" Reading FC1 deployment storage configuration from ARM..."
          SUBSCRIPTION_ID=$(az account show --query id -o tsv 2>/dev/null)
          FUNCTIONS_STORAGE_NAME="${{ steps.pulumi_outputs.outputs.functions_storage_name }}"

          # Read current functionAppConfig from ARM resource using jq
          DEPLOY_STORAGE_URL=$(az rest \
            --method GET \
            --url
"https://management.azure.com/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites
\
            --output json 2>/dev/null \
            | jq -r '.properties.functionAppConfig.deployment.storage.value // empty' 2>/dev/null || echo "")
          DEPLOY_AUTH_TYPE=$(az rest \
            --method GET \
            --url
"https://management.azure.com/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites
\
            --output json 2>/dev/null \
            | jq -r '.properties.functionAppConfig.deployment.storage.authentication.type // empty' 2>/dev/null
|| echo "")
          DEPLOY_CONN_KEY=$(az rest \
            --method GET \
            --url
"https://management.azure.com/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites
\
            --output json 2>/dev/null \
            | jq -r
'.properties.functionAppConfig.deployment.storage.authentication.storageAccountConnectionStringName //
"DEPLOYMENT_STORAGE_CONNECTION_STRING"' 2>/dev/null || echo "DEPLOYMENT_STORAGE_CONNECTION_STRING")

          echo ⊠" FC1 Deployment storage URL: ${DEPLOY_STORAGE_URL}"
          echo ⊠" FC1 Auth type: ${DEPLOY_AUTH_TYPE} (connection key: ${DEPLOY_CONN_KEY})"

          if [ -n "$DEPLOY_STORAGE_URL" ]; then
            # Extract storage account name and container from URL
            DEPLOY_STORAGE_ACCOUNT=$(echo "$DEPLOY_STORAGE_URL" | sed 's|https://||;
s|\.blob\.core\.windows\.net.*||')
            DEPLOY_STORAGE_CONTAINER=$(echo "$DEPLOY_STORAGE_URL" | sed 's|https://[^/]*/||; s|/.*||')
            echo ⊠" Storage account: ${DEPLOY_STORAGE_ACCOUNT}, Container: ${DEPLOY_STORAGE_CONTAINER}"

            # Ensure the storage account exists (recreate if deleted)
            if ! az storage account show --name "$DEPLOY_STORAGE_ACCOUNT" --resource-group "$RESOURCE_GROUP"
--output none 2>/dev/null; then
              echo ⚠"  Deployment storage $DEPLOY_STORAGE_ACCOUNT does NOT exist — recreating..."
              az storage account create \
                --name "$DEPLOY_STORAGE_ACCOUNT" \
                --resource-group "$RESOURCE_GROUP" \
                --location japaneast \
                --sku Standard_LRS \
                --kind StorageV2 \
                --allow-blob-public-access false \
                --output none 2>/dev/null \
```

```
              && echo ☒" Storage account recreated: $DEPLOY_STORAGE_ACCOUNT" \
              || echo ⚠"  Could not create storage account (name globally taken or permissions)"
          else
            echo ☒" Deployment storage account exists: $DEPLOY_STORAGE_ACCOUNT"
          fi

          # Get connection string for the deployment storage account
          DEPLOY_STORAGE_CONN=$(az storage account show-connection-string \
            --name "$DEPLOY_STORAGE_ACCOUNT" \
            --resource-group "$RESOURCE_GROUP" \
            --query connectionString -o tsv 2>/dev/null || echo "")

          if [ -n "$DEPLOY_STORAGE_CONN" ]; then
            # Create deployment container if needed
            az storage container create \
              --name "${DEPLOY_STORAGE_CONTAINER:-deployments}" \
              --connection-string "$DEPLOY_STORAGE_CONN" \
              --output none 2>/dev/null \
              && echo ☒" Container ${DEPLOY_STORAGE_CONTAINER:-deployments} ready" || true

            # Update the app setting that Kudu reads for the connection string
            az functionapp config appsettings set \
              --name $FUNCTION_APP \
              --resource-group $RESOURCE_GROUP \
              --settings "${DEPLOY_CONN_KEY}=${DEPLOY_STORAGE_CONN}" \
              --output none 2>/dev/null \
              && echo ☒" ${DEPLOY_CONN_KEY} updated to point to $DEPLOY_STORAGE_ACCOUNT"
          else
            echo ⚠"  Could not get connection string for $DEPLOY_STORAGE_ACCOUNT"
          fi
        else
          echo ⚠"  No FC1 deployment storage URL found — using ARM PATCH fallback"
          DEPLOY_CONTAINER="app-package-${FUNCTION_APP}"
          CONTAINER_URL="https://${FUNCTIONS_STORAGE_NAME}.blob.core.windows.net/${DEPLOY_CONTAINER}"
          az storage container create \
            --name "$DEPLOY_CONTAINER" \
            --account-name "$FUNCTIONS_STORAGE_NAME" \
            --auth-mode login \
            --output none 2>/dev/null || true
          az rest \
            --method PATCH \
            --url
"https://management.azure.com/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites
\
            --headers "Content-Type=application/json" \
            --body
"{\"properties\":{\"functionAppConfig\":{\"deployment\":{\"storage\":{\"type\":\"blobContainer\",\"value\":\"${CONTAINER_URL}
\
            2>/dev/null \
            && echo ☒" FC1 deployment storage updated via ARM PATCH" \
            || echo ⚠"  ARM PATCH also failed"
        fi
        sleep 15

        echo ☒" Deploying function app package (with retry logic)..."

        # Retry deployment up to 3 times to handle Kudu restarts
        MAX_RETRIES=3
```

```
          RETRY_COUNT=0
          DEPLOY_SUCCESS=false

          while [ $RETRY_COUNT -lt $MAX_RETRIES ]; do
            echo "Attempt $((RETRY_COUNT+1))/$MAX_RETRIES..."

            # Run deployment and capture output
            # GitHub Actions runner is ubuntu-latest (x86_64), matching Azure Functions
            # linux/amd64 runtime. No need for --build-remote false on FC1 plans.
            # Note: --build-remote false on FC1 prevents Kudu from completing async extraction.
            az functionapp deployment source config-zip \
              --resource-group $RESOURCE_GROUP \
              --name $FUNCTION_APP \
              --src services/api/function-app.zip \
              --timeout 600 \
              2>&1 | tee deploy_log.txt || true

            # Check for "another deployment in progress" error (retry with longer wait)
            if grep -q "another deployment is in progress" deploy_log.txt; then
              echo ⏳" Another deployment in progress, waiting 60s before retry..."
              RETRY_COUNT=$((RETRY_COUNT+1))
              sleep 60
              continue
            fi

            # Check for Kudu restart
            if grep -q "Kudu has been restarted" deploy_log.txt; then
              echo ⏳" Kudu restart detected, retrying..."
              RETRY_COUNT=$((RETRY_COUNT+1))
              sleep 30
              continue
            fi

            # Check for successful deployment (in order of reliability)
            # 1. Explicit success message (most reliable for Flex Consumption)
            if grep -q "Deployment was successful" deploy_log.txt; then
              echo ✅" Deployment successful!"
              DEPLOY_SUCCESS=true
              break
            # 2. Deployment steps completed (for other plan types)
            elif grep -q "UploadPackageStep.*completed" deploy_log.txt || \
                 grep -q "SyncTriggerStep" deploy_log.txt; then
              echo ✅" Deployment steps completed!"
              DEPLOY_SUCCESS=true
              break
            # 3. Flex Consumption async deployment: zip uploaded, Kudu processing in background
            # "Deployment is still on-going" = deployment was accepted and is executing asynchronously
            elif grep -q "on-going\|still running\|still in progress" deploy_log.txt; then
              echo ⏳" Deployment accepted (async). Waiting 120s for Kudu to complete..."
              sleep 120
              DEPLOY_SUCCESS=true
              break
            # 4. Critical error (but not "partially successful" or "in progress")
            elif grep -q "ERROR:" deploy_log.txt && \
                 ! grep -q "partially successful" deploy_log.txt && \
                 ! grep -q "another deployment is in progress" deploy_log.txt; then
              echo ⚠️"  Deployment error on attempt $((RETRY_COUNT+1)), retrying..."
              cat deploy_log.txt
```

```
            RETRY_COUNT=$((RETRY_COUNT+1))
            sleep 30
          else
            echo ⚠"  Deployment status unclear, retrying..."
            RETRY_COUNT=$((RETRY_COUNT+1))
            sleep 30
          fi
        done

        if [ "$DEPLOY_SUCCESS" = false ]; then
          echo ⚠"  Deployment uncertain after $MAX_RETRIES attempts, checking function health..."
        else
          echo ☒" Deployment completed successfully!"
        fi

        # Restart function app to ensure new package is loaded.
        # FC1 Flex Consumption may keep old code in memory after async zip extraction.
        echo ☒" Restarting function app to load new deployment..."
        az functionapp restart \
          --name $FUNCTION_APP \
          --resource-group $RESOURCE_GROUP \
          --output none 2>/dev/null && echo ☒" Function app restarted" || echo ⚠"  Restart failed (non-fatal)"
        sleep 30

        # Configure always-ready instances to prevent FlexConsumption cold starts
        echo ☒" Configuring always-ready instances (cold start prevention)..."
        ALWAYS_READY_COUNT=$([[ "$STACK_NAME" == "staging" ]] && echo 0 || echo 1)
        az functionapp scale config always-ready set \
          --name $FUNCTION_APP \
          --resource-group $RESOURCE_GROUP \
          --settings http=$ALWAYS_READY_COUNT \
          2>/dev/null && echo ☒" always-ready http=$ALWAYS_READY_COUNT configured" || echo ⚠"  Could not set always-ready (non-fatal)"

      - name: Ensure Azure CORS Origins
        # Safety net: ensures both the app setting and platform-level CORS (Kestrel intercept)
        # always include https://<customDomain> regardless of what the deploy step set.
        # customDomain comes from the committed config file (.github/config/azure.<stack>.env).
        run: |
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"
          FUNCTION_APP="${{ steps.pulumi_outputs.outputs.function_app_name }}"
          FRONTDOOR_URL="${{ steps.pulumi_outputs.outputs.frontdoor_hostname }}"

          # Custom domain from config file (single source of truth)
          CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"

          if [[ -z "$CUSTOM_DOMAIN" ]]; then
            echo ☒"  No custom domain set — skipping CORS safety net"
            exit 0
          fi

          echo ☒" Ensuring CORS origins include https://$CUSTOM_DOMAIN (stack: $STACK_NAME)..."

          # Build correct CORS_ORIGINS
          CORS_ORIGINS="https://${FRONTDOOR_URL},https://${CUSTOM_DOMAIN},http://localhost:5173"

          # 1. Ensure app setting CORS_ORIGINS includes custom domain
```

```
          CURRENT_CORS=$(az functionapp config appsettings list \
            --name "$FUNCTION_APP" --resource-group "$RESOURCE_GROUP" \
            --query "[?name=='CORS_ORIGINS'].value" -o tsv 2>/dev/null || echo "")
          if echo "$CURRENT_CORS" | grep -q "$CUSTOM_DOMAIN"; then
            echo ☒" CORS_ORIGINS app setting already includes $CUSTOM_DOMAIN"
          else
            echo ⚠"  CORS_ORIGINS missing $CUSTOM_DOMAIN — restoring: $CORS_ORIGINS"
            az functionapp config appsettings set \
              --name "$FUNCTION_APP" --resource-group "$RESOURCE_GROUP" \
              --settings "CORS_ORIGINS=${CORS_ORIGINS}" > /dev/null
            echo ☒" CORS_ORIGINS restored to: $CORS_ORIGINS"
          fi

          # 2. Ensure platform-level CORS (Kestrel) includes custom domain
          CURRENT_PLATFORM=$(az functionapp cors show \
            --name "$FUNCTION_APP" --resource-group "$RESOURCE_GROUP" \
            --query "allowedOrigins" -o json 2>/dev/null || echo "[]")
          if echo "$CURRENT_PLATFORM" | grep -q "$CUSTOM_DOMAIN"; then
            echo ☒" Platform-level CORS already includes $CUSTOM_DOMAIN"
          else
            echo ⚠"  Platform CORS missing $CUSTOM_DOMAIN — adding..."
            az functionapp cors add \
              --name "$FUNCTION_APP" --resource-group "$RESOURCE_GROUP" \
              --allowed-origins "https://${CUSTOM_DOMAIN}" > /dev/null
            echo ☒" Platform CORS updated to include https://$CUSTOM_DOMAIN"
          fi

        - name: Enable Static Website Hosting
          run: |
            STORAGE_ACCOUNT="${{ steps.pulumi_outputs.outputs.frontend_storage_name }}"
            RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"

            echo ☒" Enabling static website hosting on $STORAGE_ACCOUNT..."

            # Get storage account key
            STORAGE_KEY=$(az storage account keys list \
              --resource-group $RESOURCE_GROUP \
              --account-name $STORAGE_ACCOUNT \
              --query '[0].value' -o tsv)

            az storage blob service-properties update \
              --account-name $STORAGE_ACCOUNT \
              --account-key $STORAGE_KEY \
              --static-website \
              --index-document index.html \
              --404-document index.html

            # Configure Blob Storage CORS to allow direct image uploads from the app origins.
            # The frontend uploads images directly to $web/images container using SAS tokens,
            # so Blob Storage must allow CORS from both the custom domain and FrontDoor hostname.
            echo ☒" Configuring Blob Storage CORS for image uploads..."
            FRONTDOOR_HOSTNAME="${{ steps.pulumi_outputs.outputs.frontdoor_hostname }}"
            # Custom domain from config file (single source of truth)
            AZURE_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"

            # Clear existing CORS rules and re-add them fresh
            az storage cors clear \
              --account-name $STORAGE_ACCOUNT \
```

```
            --account-key $STORAGE_KEY \
            --services b

          # Add FrontDoor hostname rule
          az storage cors add \
            --account-name $STORAGE_ACCOUNT \
            --account-key $STORAGE_KEY \
            --services b \
            --origins "https://${FRONTDOOR_HOSTNAME}" \
            --methods PUT GET HEAD OPTIONS DELETE \
            --allowed-headers "*" \
            --exposed-headers "*" \
            --max-age 3600

          # Add custom domain rule if configured
          if [ -n "$AZURE_CUSTOM_DOMAIN" ]; then
            az storage cors add \
              --account-name $STORAGE_ACCOUNT \
              --account-key $STORAGE_KEY \
              --services b \
              --origins "https://${AZURE_CUSTOM_DOMAIN}" \
              --methods PUT GET HEAD OPTIONS DELETE \
              --allowed-headers "*" \
              --exposed-headers "*" \
              --max-age 3600
            echo ☒" Blob Storage CORS: FrontDoor + custom domain (${AZURE_CUSTOM_DOMAIN})"
          else
            echo ☒" Blob Storage CORS: FrontDoor only (no custom domain set)"
          fi

          # Add localhost for local development
          az storage cors add \
            --account-name $STORAGE_ACCOUNT \
            --account-key $STORAGE_KEY \
            --services b \
            --origins "http://localhost:5173" \
            --methods PUT GET HEAD OPTIONS DELETE \
            --allowed-headers "*" \
            --exposed-headers "*" \
            --max-age 3600

      - name: Build and Deploy Frontend
        run: |
          STORAGE_ACCOUNT="${{ steps.pulumi_outputs.outputs.frontend_storage_name }}"
          RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"
          FRONTDOOR_HOSTNAME="${{ steps.pulumi_outputs.outputs.frontdoor_hostname }}"
          API_URL="${{ steps.pulumi_outputs.outputs.api_url }}"
          # Config file is authoritative for stable AD values — no Pulumi runtime dependency
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
          AZURE_CLIENT_ID="${{ steps.cloud_config.outputs.azure_client_id }}"
          AZURE_TENANT_ID="${{ steps.cloud_config.outputs.azure_tenant_id }}"
          AZURE_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          AZURE_PUBLIC_HOST="${AZURE_CUSTOM_DOMAIN:-${FRONTDOOR_HOSTNAME}}"

          if [ -z "$AZURE_CLIENT_ID" ]; then
            echo ☒" AZURE_CLIENT_ID is empty. Check .github/config/azure.${STACK_NAME}.env"
            exit 1
          fi
```

```
963
964         echo ⊠"  Building React frontend for Azure..."
965         echo ⊠" Using public host for redirect URIs: ${AZURE_PUBLIC_HOST}"
966         echo ⊠" Azure AD Client ID: ${AZURE_CLIENT_ID}"
967         echo ⊠" Azure AD Tenant ID: ${AZURE_TENANT_ID}"
968         cd services/frontend_react
969
970         # Install dependencies
971         npm ci --prefer-offline
972
973         # Build with Azure-specific env vars
974         VITE_AUTH_PROVIDER=azure \
975         VITE_API_URL="${API_URL}" \
976         VITE_AZURE_TENANT_ID="${AZURE_TENANT_ID}" \
977         VITE_AZURE_CLIENT_ID="${AZURE_CLIENT_ID}" \
978         VITE_AZURE_REDIRECT_URI="https://${AZURE_PUBLIC_HOST}/sns/auth/callback" \
979         VITE_AZURE_LOGOUT_URI="https://${AZURE_PUBLIC_HOST}/sns/" \
980         VITE_BASE_PATH=/sns/ \
981         npm run build
982
983         echo ⊠" Frontend build complete. Uploading to Azure Blob Storage..."
984         cd ../..
985
986         # Get storage account key
987         STORAGE_KEY=$(az storage account keys list \
988           --resource-group $RESOURCE_GROUP \
989           --account-name $STORAGE_ACCOUNT \
990           --query '[0].value' -o tsv)
991
992         # Upload built files to $web/sns/ (Front Door SPA route: /sns/* → $web/sns/)
993         # VITE_BASE_PATH=/sns/ means index.html references /sns/assets/...
994         # but the actual output files are in dist/ (flat), so upload to $web/sns/
995         az storage blob upload-batch \
996           --account-name $STORAGE_ACCOUNT \
997           --account-key $STORAGE_KEY \
998           --source services/frontend_react/dist \
999           --destination '$web/sns' \
1000          --overwrite
1001
1002        # Copy public SVG files to sns/assets/ as well (CSS references ./file.svg
1003        # relative to /sns/assets/, so they must exist at /sns/assets/file.svg)
1004        for svg_file in services/frontend_react/dist/*.svg; do
1005          [ -f "$svg_file" ] || continue
1006          svg_name=$(basename "$svg_file")
1007          az storage blob upload \
1008            --account-name $STORAGE_ACCOUNT \
1009            --account-key $STORAGE_KEY \
1010            --container-name '$web' \
1011            --file "$svg_file" \
1012            --name "sns/assets/${svg_name}" \
1013            --content-type "image/svg+xml" \
1014            --overwrite > /dev/null 2>&1 && echo ⊠" Uploaded sns/assets/${svg_name}"
1015        done
1016
1017        echo ⊠" Frontend uploaded to Azure Blob Storage"
1018        echo ⊠" Frontend URL: https://${FRONTDOOR_HOSTNAME}/sns/"
1019
1020      - name: Verify Deployment
```

```
        run: |
          FUNCTION_APP="${{ steps.pulumi_outputs.outputs.function_app_name }}"
          RESOURCE_GROUP="${{ steps.pulumi_outputs.outputs.resource_group_name }}"

          echo ⊠" Verifying API deployment..."
          echo "Getting Function App hostname..."

          # Get hostname - for Flex Consumption plan, use config hostname list
          # (defaultHostName is not reliable for Flex Consumption SKU)
          max_hostname_attempts=10
          hostname_attempt=0
          FUNC_HOSTNAME=""

          while [ $hostname_attempt -lt $max_hostname_attempts ]; do
            FUNC_HOSTNAME=$(az functionapp config hostname list \
              --webapp-name $FUNCTION_APP \
              --resource-group $RESOURCE_GROUP \
              --query '[0].name' -o tsv 2>/dev/null || echo "")

            if [ -n "$FUNC_HOSTNAME" ] && [ "$FUNC_HOSTNAME" != "None" ]; then
              echo ⊠" Got hostname: $FUNC_HOSTNAME"
              break
            else
              echo ⊠" Waiting for Function App to be ready... (attempt
$((hostname_attempt+1))/$max_hostname_attempts)"
              sleep 10
              hostname_attempt=$((hostname_attempt+1))
            fi
          done

          if [ -z "$FUNC_HOSTNAME" ] || [ "$FUNC_HOSTNAME" = "None" ]; then
            echo ⊠" Failed to get Function App hostname after waiting"
            echo "Try checking manually: az functionapp config hostname list --webapp-name $FUNCTION_APP
--resource-group $RESOURCE_GROUP"
            exit 1
          fi

          FUNC_URL="https://${FUNC_HOSTNAME}/api/HttpTrigger"
          echo "Testing: ${FUNC_URL}/health"

          # Wait for function to be ready (max 10 minutes for FC1 cold start + async deploy)
          max_attempts=60
          attempt=0
          health_check_passed=false

          while [ $attempt -lt $max_attempts ]; do
            if response=$(curl -sf "${FUNC_URL}/health" 2>&1); then
              echo ⊠" API is responding!"
              echo "$response" | jq .
              health_check_passed=true
              break
            else
              echo ⊠" Waiting for function to be ready... (attempt $((attempt+1))/$max_attempts)"
              sleep 10
              attempt=$((attempt+1))
            fi
          done

```

```
          if [ "$health_check_passed" = true ]; then
            echo ""
            echo ☒" Azure Function deployment verified successfully!"
            echo ☒" API URL: ${FUNC_URL}"
          else
            echo ""
            echo ☒" Health check failed after waiting for $((max_attempts * 10)) seconds"
            echo "This may be a transient issue. Check manually: ${FUNC_URL}/health"
            exit 1
          fi

      - name: Notify Success
        if: success()
        run: |
          echo ☒" Azure deployment succeeded!"
          echo ☒" API URL: ${{ steps.pulumi_outputs.outputs.api_url }}"
          echo ☒" Frontend Storage: ${{ steps.pulumi_outputs.outputs.frontend_storage_name }}"

      - name: Notify Failure
        if: failure()
        run: echo ☒" Azure deployment failed!"
```

6.3 deploy-gcp.yml - GCP デプロイ

```
name: Deploy to GCP

on:
  push:
    branches:
      - develop # 自動デプロイstaging
      - main # 自動デプロイproduction
    paths:
      - "services/**"
      - "infrastructure/pulumi/gcp/**"
      - ".github/workflows/deploy-gcp.yml"
  workflow_dispatch:
    inputs:
      environment:
        description: "Deployment environment"
        required: true
        default: "staging"
        type: choice
        options:
          - staging
          - production

env:
  GCP_REGION: asia-northeast1
  NODE_VERSION: "24"
  PYTHON_VERSION: "3.13"

# Prevent concurrent Pulumi updates on the same stack (causes 409 Conflict)
concurrency:
  group: deploy-gcp-${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment ||
     (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
  cancel-in-progress: false
```

```yaml

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    # →developstaging, →mainproduction, 手動→選択した環境
    environment: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.environment || (github.ref
     == 'refs/heads/main' && 'production' || 'staging') }}

    steps:
      - uses: actions/checkout@v4

      - name: Authenticate to Google Cloud
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${{ secrets.GCP_CREDENTIALS }}

      - name: Set up Cloud SDK
        uses: google-github-actions/setup-gcloud@v2
        with:
          project_id: ${{ secrets.GCP_PROJECT_ID }}

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install Pulumi CLI
        uses: pulumi/actions@v5

      - name: Install Pulumi Python Dependencies
        run: |
          cd infrastructure/pulumi/gcp
          pip install -r requirements.txt

      - name: Determine Pulumi Stack Name
        id: set_stack
        run: |
          if [ "${{ github.event_name }}" == "workflow_dispatch" ]; then
            STACK_NAME="${{ github.event.inputs.environment }}"
          elif [ "${{ github.ref }}" == "refs/heads/main" ]; then
            STACK_NAME="production"
          else
            STACK_NAME="staging"
          fi
          echo "stack_name=$STACK_NAME" >> $GITHUB_OUTPUT
          echo ⌧" Using Pulumi stack: $STACK_NAME"

      - name: Load Cloud Config
        # Single source of truth for stable per-environment values.
        # Eliminates dependency on repo-level secrets and inline case/esac patterns.
        id: cloud_config
        run: |
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"
```

```
          CONFIG_FILE=".github/config/gcp.${STACK_NAME}.env"
          if [ ! -f "$CONFIG_FILE" ]; then
            echo ☒" Config file $CONFIG_FILE not found — cannot proceed"
            exit 1
          fi
          source "$CONFIG_FILE"
          echo ☒" Loaded config: $CONFIG_FILE"
          echo "  custom_domain=$CLOUD_CUSTOM_DOMAIN"
          echo "custom_domain=$CLOUD_CUSTOM_DOMAIN" >> $GITHUB_OUTPUT

      - name: Initialize Pulumi Stack
        run: |
          cd infrastructure/pulumi/gcp
          pulumi login
          pulumi stack select ${{ steps.set_stack.outputs.stack_name }} 2>/dev/null || pulumi stack init ${{ steps.set_stack.outputs.stack_name }}
          pulumi config set gcp:project ${{ secrets.GCP_PROJECT_ID }}
          pulumi config set gcp:region asia-northeast1
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}

      - name: Remove SecretVersion from Pulumi state (IAM permission workaround)
        run: |
          cd infrastructure/pulumi/gcp
          STACK="${{ steps.set_stack.outputs.stack_name }}"
          # Remove SecretVersion from Pulumi state to prevent 403 on create/refresh
          # (Pulumi SA lacks secretmanager.versions.access permission)
          # This is idempotent: fails silently if resource is not in state
          pulumi state delete \
            "urn:pulumi:${STACK}::multicloud-auto-deploy-gcp::gcp:secretmanager/secretVersion:SecretVersion::app-secret-version" \
            --yes 2>/dev/null && echo ☒" Removed SecretVersion from state" || echo ☒" SecretVersion not in state (already clean)"
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}

      - name: Refresh Pulumi State (sync with actual GCP resources)
        # Prevents 400/412 errors on ManagedSslCertificate and URLMap during pulumi up.
        # Known issue: GCP Pulumi state drift causes pulumi up to fail with
        #   "ManagedSslCertificate 400" or "URLMap 412 precondition failed"
        # pulumi refresh syncs the state to match actual GCP resource state before making changes.
        run: |
          cd infrastructure/pulumi/gcp
          pulumi refresh --yes --stack ${{ steps.set_stack.outputs.stack_name }} 2>/dev/null || echo ⚠"  pulumi refresh encountered warnings — proceeding with up"
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          GOOGLE_PROJECT: ${{ secrets.GCP_PROJECT_ID }}

      - name: Deploy Infrastructure with Pulumi
        id: pulumi
        # continue-on-error: GCP staging has a known state drift issue where ManagedSslCertificate
        # (400) and URLMap (412 precondition failed) cause pulumi up to fail. The existing
        # infrastructure (Cloud Run, GCS buckets, CDN) continues to work fine and pulumi stack
        # outputs are still valid. Subsequent steps (Cloud Run update, frontend deploy) should
        # proceed regardless of this known non-blocking Pulumi error.
        continue-on-error: true
```

```
        uses: pulumi/actions@v5
        with:
          command: up
          stack-name: ${{ steps.set_stack.outputs.stack_name }}
          work-dir: infrastructure/pulumi/gcp
        env:
          PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
          GOOGLE_PROJECT: ${{ secrets.GCP_PROJECT_ID }}

      - name: Get Pulumi Outputs
        id: pulumi_outputs
        # Run even if Pulumi up failed (state drift) — stack outputs still valid for existing resources
        if: always()
        run: |
          cd infrastructure/pulumi/gcp
          echo "function_source_bucket=$(pulumi stack output function_source_bucket)" >> $GITHUB_OUTPUT
          echo "frontend_bucket=$(pulumi stack output frontend_bucket)" >> $GITHUB_OUTPUT
          echo "function_name=$(pulumi stack output function_name)" >> $GITHUB_OUTPUT
          echo "cdn_ip_address=$(pulumi stack output cdn_ip_address)" >> $GITHUB_OUTPUT
          echo "uploads_bucket=$(pulumi stack output uploads_bucket)" >> $GITHUB_OUTPUT
          echo "compute_sa_email=$(pulumi stack output compute_sa_email)" >> $GITHUB_OUTPUT
          echo "custom_domain=$(pulumi stack output custom_domain 2>/dev/null || echo '')" >> $GITHUB_OUTPUT

      - name: Update Firebase Authorized Domains
        run: |
          CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          if [ -z "$CUSTOM_DOMAIN" ]; then
            echo ☒"  No custom domain configured, skipping Firebase domain registration"
            exit 0
          fi

          echo ☒" Adding $CUSTOM_DOMAIN to Firebase authorized domains..."
          PROJECT="${{ secrets.GCP_PROJECT_ID }}"
          TOKEN=$(gcloud auth print-access-token)

          # Get current authorized domains
          curl -s \
            -H "Authorization: Bearer $TOKEN" \
            -H "x-goog-user-project: $PROJECT" \
            "https://identitytoolkit.googleapis.com/admin/v2/projects/$PROJECT/config" \
            -o /tmp/fb_config.json

          # Build updated domains list (add custom domain if not already present)
          CURRENT_DOMAINS=$(jq -r '.authorizedDomains // [] | @json' /tmp/fb_config.json 2>/dev/null || echo "[]")
          if echo "$CURRENT_DOMAINS" | grep -q "\"$CUSTOM_DOMAIN\""; then
            echo ☒" $CUSTOM_DOMAIN already in Firebase authorized domains"
            NEW_DOMAINS="$CURRENT_DOMAINS"
          else
            NEW_DOMAINS=$(echo "$CURRENT_DOMAINS" | jq --arg d "$CUSTOM_DOMAIN" '. + [$d]')
            echo ☒" Adding $CUSTOM_DOMAIN to authorized domains"
          fi
          jq -n --argjson domains "$NEW_DOMAINS" '{"authorizedDomains": $domains}' > /tmp/fb_patch.json

          # Apply patch
          TOKEN=$(gcloud auth print-access-token)
          curl -s -X PATCH \
            -H "Authorization: Bearer $TOKEN" \
            -H "x-goog-user-project: $PROJECT" \
```

```
            -H "Content-Type: application/json" \
      "https://identitytoolkit.googleapis.com/admin/v2/projects/$PROJECT/config?updateMask=authorizedDomains" \
            -d @/tmp/fb_patch.json -o /tmp/fb_result.json

          if jq -e '.error' /tmp/fb_result.json > /dev/null 2>&1; then
            echo "❌ Error: $(jq -r '.error.message' /tmp/fb_result.json)"
            exit 1
          fi
          echo "✅ Firebase authorized domains updated:"
          jq -r '.authorizedDomains[] | "  - " + .' /tmp/fb_result.json

      - name: Package Cloud Function
        run: |
          cd services/api

          echo "📦 Creating optimized deployment package..."

          # Create deployment directory
          rm -rf .deployment
          mkdir -p .deployment

          # Install GCP-specific dependencies only
          echo "Installing GCP-specific dependencies..."
          pip install --target .deployment --no-cache-dir -r requirements-gcp.txt

          # Aggressive cleanup of unnecessary files
          echo "Cleaning up unnecessary files..."
          find .deployment -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type d -name "tests" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type d -name "*.dist-info" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type d -name "*.egg-info" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type f -name "*.pyc" -delete 2>/dev/null || true
          find .deployment -type f -name "*.pyo" -delete 2>/dev/null || true
          # Note: .so files are C extension binaries and should NOT be deleted
          # find .deployment -type f -name "*.so" -delete 2>/dev/null || true

          # Remove test and documentation files
          find .deployment -type f -name "test_*.py" -delete 2>/dev/null || true
          find .deployment -type f -name "*_test.py" -delete 2>/dev/null || true
          find .deployment -type d -name "docs" -exec rm -rf {} + 2>/dev/null || true
          find .deployment -type d -name "examples" -exec rm -rf {} + 2>/dev/null || true

          # Copy application code
          cp -r app .deployment/
          # Rename function.py to main.py for Cloud Functions
          cp function.py .deployment/main.py
          cp requirements-gcp.txt .deployment/requirements.txt

          # Clean up app __pycache__
          find .deployment/app -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true

          # Create ZIP package with maximum compression
          cd .deployment
          zip -r9 -q ../function-source.zip .
          cd ..

          echo "✅ Package created:"
```

```
          ls -lh function-source.zip
          PACKAGE_SIZE=$(du -h function-source.zip | cut -f1)
          PACKAGE_SIZE_KB=$(du -k function-source.zip | cut -f1)
          echo "📦 Package size: $PACKAGE_SIZE ($PACKAGE_SIZE_KB KB)"

          # Show size breakdown
          echo ""
          echo "📊 Top 10 largest items in package:"
          unzip -l function-source.zip | sort -k4 -n -r | head -n 11

      - name: Deploy Cloud Function
        id: deploy_function
        run: |
          # Use values from Pulumi outputs
          PROJECT_ID=${{ secrets.GCP_PROJECT_ID }}
          FUNCTION_NAME="${{ steps.pulumi_outputs.outputs.function_name }}"
          FUNCTION_BUCKET="${{ steps.pulumi_outputs.outputs.function_source_bucket }}"
          REGION=${{ env.GCP_REGION }}

          echo "🚀 Deploying Cloud Function: $FUNCTION_NAME"
          echo "  Project: $PROJECT_ID"
          echo "  Bucket: $FUNCTION_BUCKET"
          echo "  Region: $REGION"

          # Upload source to GCS
          gsutil cp services/api/function-source.zip gs://$FUNCTION_BUCKET/function-source.zip

          # Get CDN IP from Pulumi outputs for CORS
          # GCP Classic HTTP LB serves over both HTTP and HTTPS
          CDN_URL="${{ steps.pulumi_outputs.outputs.cdn_ip_address }}"
          # Custom domain from config file (single source of truth)
          CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          CORS_ORIGINS="http://${CDN_URL},https://${CDN_URL},http://localhost:5173"
          if [ -n "$CUSTOM_DOMAIN" ]; then
            CORS_ORIGINS="${CORS_ORIGINS},https://${CUSTOM_DOMAIN}"
          fi

          echo "🌐 CORS Origins: $CORS_ORIGINS"

          # Determine environment
          DEPLOY_ENV="${{ steps.set_stack.outputs.stack_name }}"
          # Auth is enabled in both staging and production (Firebase for GCP)
          AUTH_DISABLED_VALUE="false"
          UPLOADS_BUCKET="${{ steps.pulumi_outputs.outputs.uploads_bucket }}"

          # Create env vars file to avoid escaping issues with special characters
          echo "ENVIRONMENT: ${DEPLOY_ENV}" > /tmp/env-vars.yaml
          echo "CLOUD_PROVIDER: gcp" >> /tmp/env-vars.yaml
          echo "GCP_PROJECT_ID: $PROJECT_ID" >> /tmp/env-vars.yaml
          echo "GCP_POSTS_COLLECTION: posts" >> /tmp/env-vars.yaml
          echo "GCP_PROFILES_COLLECTION: profiles" >> /tmp/env-vars.yaml
          echo "AUTH_DISABLED: \"$AUTH_DISABLED_VALUE\"" >> /tmp/env-vars.yaml
          echo "AUTH_PROVIDER: firebase" >> /tmp/env-vars.yaml
          echo "GCP_STORAGE_BUCKET: $UPLOADS_BUCKET" >> /tmp/env-vars.yaml
          echo "GCP_SERVICE_ACCOUNT: ${{ steps.pulumi_outputs.outputs.compute_sa_email }}" >> /tmp/env-vars.yaml
          echo "CORS_ORIGINS: \"$CORS_ORIGINS\"" >> /tmp/env-vars.yaml

```

```
          echo ☒" Environment variables file created (ENVIRONMENT=$DEPLOY_ENV, AUTH_DISABLED=$AUTH_DISABLED_VALUE, AUTH_PROVIDER=firebase, GCP_STORAGE_BUCKET=$UPLOADS_BUCKET)"

          # Deploy function with env vars from file
          gcloud functions deploy $FUNCTION_NAME \
            --gen2 \
            --region=$REGION \
            --runtime=python313 \
            --source=gs://$FUNCTION_BUCKET/function-source.zip \
            --entry-point=handler \
            --trigger-http \
            --allow-unauthenticated \
            --min-instances=$([[ "${{ steps.set_stack.outputs.stack_name }}" == "staging" ]] && echo 0 || echo 1) \
            --max-instances=10 \
            --memory=512MB \
            --timeout=60s \
            --env-vars-file=/tmp/env-vars.yaml

          # Get function URL
          FUNCTION_URL=$(gcloud functions describe $FUNCTION_NAME --region=$REGION --gen2 --format="value(serviceConfig.uri)")
          echo "api_url=$FUNCTION_URL" >> $GITHUB_OUTPUT

          echo ☒" Cloud Function deployed successfully"
          echo "  URL: $FUNCTION_URL"

      - name: Build and Deploy Frontend
        run: |
          FRONTEND_BUCKET="${{ steps.pulumi_outputs.outputs.frontend_bucket }}"
          API_URL="${{ steps.deploy_function.outputs.api_url }}"
          STACK_NAME="${{ steps.set_stack.outputs.stack_name }}"

          # Custom domain from config file (single source of truth)
          GCP_CUSTOM_DOMAIN="${{ steps.cloud_config.outputs.custom_domain }}"
          PUBLIC_HOST="${GCP_CUSTOM_DOMAIN}"

          echo ☒"  Building React frontend for GCP ($STACK_NAME)..."
          echo ☒" Public host: ${PUBLIC_HOST}"
          echo ☒" API URL: ${API_URL}"
          cd services/frontend_react
          npm ci --prefer-offline

          VITE_AUTH_PROVIDER=firebase \
          VITE_API_URL="${API_URL}" \
          VITE_FIREBASE_API_KEY="${{ secrets.FIREBASE_API_KEY }}" \
          VITE_FIREBASE_AUTH_DOMAIN="${{ secrets.FIREBASE_AUTH_DOMAIN }}" \
          VITE_FIREBASE_PROJECT_ID="${{ secrets.GCP_PROJECT_ID }}" \
          VITE_FIREBASE_APP_ID="${{ secrets.FIREBASE_APP_ID }}" \
          npx vite build --base=/sns/
          echo ☒" Build complete ($(du -sh dist | cut -f1))"
          cd ../..

          # Upload index.html with no-cache
          gsutil -h "Content-Type:text/html" \
            -h "Cache-Control:no-cache, no-store, must-revalidate" \
            cp services/frontend_react/dist/index.html \
            "gs://${FRONTEND_BUCKET}/sns/index.html"
```

```
369
370         # Upload assets with immutable cache
371         gsutil -m -h "Cache-Control:public, max-age=31536000, immutable" \
372           cp -r services/frontend_react/dist/assets \
373           "gs://${FRONTEND_BUCKET}/sns/"
374
375         # Upload SVGs
376         gsutil -m -h "Cache-Control:public, max-age=3600" \
377           cp services/frontend_react/dist/*.svg \
378           "gs://${FRONTEND_BUCKET}/sns/" 2>/dev/null || true
379
380         echo "✅ Frontend deployed to gs://${FRONTEND_BUCKET}/sns/"
381
382         # Invalidate Cloud CDN cache
383         URLMAP_NAME=$(gcloud compute url-maps list --format="value(name)" | grep "${STACK_NAME}" | head -1)
384         if [ -n "$URLMAP_NAME" ]; then
385           gcloud compute url-maps invalidate-cdn-cache "$URLMAP_NAME" \
386             --path "/sns/*" --async 2>/dev/null || true
387           echo "✅ CDN cache invalidated: $URLMAP_NAME"
388         fi
389
390       - name: Notify Success
391         if: success()
392         run: |
393           echo "✅ GCP deployment succeeded!"
394           echo "✅ Function URL: ${{ steps.deploy_function.outputs.api_url }}"
395           echo "✅ Frontend Bucket: ${{ steps.pulumi_outputs.outputs.frontend_bucket }}"
396
397       - name: Notify Failure
398         if: failure()
399         run: echo "❌ GCP deployment failed!"
```

Chapter 7

フロントエンド (React) 主要コード

このセクションでは、React + TypeScript + Viteベースのフロントエンドの主要ファイルを掲載します。

7.1 App.tsx - メインアプリケーション

```
import { Routes, Route } from "react-router-dom";
import { AuthProvider } from "./contexts/AuthContext";
import Layout from "./components/Layout";
import HomePage from "./pages/HomePage";
import PostPage from "./pages/PostPage";
import ProfilePage from "./pages/ProfilePage";
import LoginPage from "./pages/LoginPage";
import CallbackPage from "./pages/CallbackPage";
import SolverPage from "./pages/SolverPage";

function App() {
  return (
    <AuthProvider>
      <Layout>
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/post/:postId" element={<PostPage />} />
          <Route path="/profile" element={<ProfilePage />} />
          <Route path="/login" element={<LoginPage />} />
          <Route path="/auth/callback" element={<CallbackPage />} />
          <Route path="/exam" element={<SolverPage />} />
          {/* Fallback: send everything else to home */}
          <Route path="*" element={<HomePage />} />
        </Routes>
      </Layout>
    </AuthProvider>
  );
}

export default App;
```

7.2 main.tsx - エントリーポイント

```
import { StrictMode } from "react";
```

```
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./App.css";
import App from "./App.tsx";

// www なしドメインへのアクセスを www 付きに強制リダイレクト
// 古いブックマーク・(キャッシュ・DNSSafe Browsing 警告対策)
// azure.ashnova.jp は一時的にに設定されにクロールされたため警告が発生した実績あり。AFDGoogle
// AWS/GCP も同様のリスクに備えて予防的に対処する。
const NON_WWW_REDIRECT: Record<string, string> = {
  "azure.ashnova.jp": "www.azure.ashnova.jp",
  "aws.ashnova.jp": "www.aws.ashnova.jp",
  "gcp.ashnova.jp": "www.gcp.ashnova.jp",
};
if (typeof window !== "undefined") {
  const wwwHost = NON_WWW_REDIRECT[window.location.hostname];
  if (wwwHost) {
    window.location.replace(
      "https://" +
        wwwHost +
        window.location.pathname +
        window.location.search +
        window.location.hash,
    );
  }
}

// basename from Vite's base config (e.g. /sns/ → /sns)
const basename = import.meta.env.BASE_URL.replace(/\/$/, "") || "/";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter basename={basename}>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

7.3 AuthContext.tsx - 認証コンテキスト

```
import React, {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
} from "react";
import { getLogoutUrl, authProvider } from "../config/auth";

// ---- Types ----
interface AuthState {
  /** Decoded JWT token string stored in localStorage */
  token: string | null;
  /** Whether the user is authenticated */
  isLoggedIn: boolean;
```

```
}

interface AuthContextValue extends AuthState {
  /** Save tokens returned from auth provider callback / Firebase SDK */
  setTokens: (params: {
    idToken?: string;
    accessToken?: string;
    expiresIn?: number;
  }) => void;
  /** Clear tokens and redirect to provider logout */
  logout: (currentOrigin: string) => void;
}

// ---- Context ----
const AuthContext = createContext<AuthContextValue | null>(null);

const TOKEN_KEY_ACCESS = "access_token";
const TOKEN_KEY_ID = "id_token";

function readToken(): string | null {
  return (
    localStorage.getItem(TOKEN_KEY_ACCESS) ||
    localStorage.getItem(TOKEN_KEY_ID) ||
    null
  );
}

// ---- Provider ----
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [token, setToken] = useState<string | null>(readToken);

  // Stay in sync across tabs
  useEffect(() => {
    const onStorage = () => setToken(readToken());
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, []);

  const setTokens = useCallback(
    ({
      idToken,
      accessToken,
      expiresIn = 3600,
    }: {
      idToken?: string;
      accessToken?: string;
      expiresIn?: number;
    }) => {
      if (accessToken) {
        localStorage.setItem(TOKEN_KEY_ACCESS, accessToken);
        // Auto-clear after expiresIn seconds
        setTimeout(() => {
          localStorage.removeItem(TOKEN_KEY_ACCESS);
          setToken(readToken());
        }, expiresIn * 1000);
      }
      if (idToken) {
        localStorage.setItem(TOKEN_KEY_ID, idToken);
```

```
        if (!accessToken) {
          setTimeout(() => {
            localStorage.removeItem(TOKEN_KEY_ID);
            setToken(readToken());
          }, expiresIn * 1000);
        }
      }
      setToken(readToken());
    },
    [],
  );

  const logout = useCallback((currentOrigin: string) => {
    localStorage.removeItem(TOKEN_KEY_ACCESS);
    localStorage.removeItem(TOKEN_KEY_ID);
    setToken(null);

    // Firebase: no hosted logout URL; the SDK sign-out is handled in LoginPage
    if (authProvider === "firebase") {
      return;
    }

    const snsTop = `${currentOrigin}/sns/`;
    const logoutUrl = getLogoutUrl(snsTop);
    if (logoutUrl && logoutUrl !== snsTop) {
      window.location.href = logoutUrl;
    }
  }, []);

  const value = useMemo(
    () => ({
      token,
      isLoggedIn: Boolean(token),
      setTokens,
      logout,
    }),
    [token, setTokens, logout],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// ---- Hook ----
export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;
}
```

7.4 api/client.ts - APIクライアント

```
import axios from 'axios';

// VITE_API_URL: backend API base (https://api.example.com)
// Empty string in dev → Vite proxy forwards to localhost:8000
const API_URL = import.meta.env.VITE_API_URL ?? '';
```

```

const apiClient = axios.create({
  baseURL: API_URL,
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000,
});

// Attach stored auth token to every request
// id_token is preferred because the Lambda JWT verifier validates audience against the Cognito client_id,
// which matches the id_token aud claim. Cognito access tokens may omit the aud claim.
apiClient.interceptors.request.use((config) => {
  const token =
    localStorage.getItem('id_token') ||
    localStorage.getItem('access_token');
  if (token) {
    config.headers['Authorization'] = `Bearer ${token}`;
  }
  return config;
});

export default apiClient;
```

7.5 api/posts.ts - 投稿API

```
import apiClient from "./client";
import type {
  Post,
  PostsResponse,
  CreatePostInput,
  UpdatePostInput,
} from "../types/message";

export const postsApi = {
  async getPosts(
    limit = 20,
    nextToken?: string | null,
    tag?: string | null,
  ): Promise<PostsResponse> {
    const res = await apiClient.get<PostsResponse>("/posts", {
      params: {
        limit,
        ...(nextToken ? { next_token: nextToken } : {}),
        ...(tag ? { tag } : {}),
      },
    });
    return res.data;
  },

  async getPost(postId: string): Promise<Post> {
    const res = await apiClient.get<Post>(`/posts/${postId}`);
    return res.data;
  },

  async createPost(data: CreatePostInput): Promise<Post> {
    const res = await apiClient.post<Post>("/posts", data);
    return res.data;
```

```
  },

  async updatePost(postId: string, data: UpdatePostInput): Promise<Post> {
    const res = await apiClient.put<Post>(`/posts/${postId}`, data);
    return res.data;
  },

  async deletePost(postId: string): Promise<void> {
    await apiClient.delete(`/posts/${postId}`);
  },
};
```

7.6 vite.config.ts - Vite設定

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  // VITE_BASE_PATH: must be set to /sns/ when building for AWS/Azure/GCP staging deployment
  // e.g.  VITE_BASE_PATH=/sns/ npm run build
  // Omit (defaults to '/') for local dev server only
  base: process.env.VITE_BASE_PATH || "/",
  server: {
    // dev server: proxy API paths to local backend
    proxy: {
      "/v1": { target: "http://localhost:8000", changeOrigin: true },
      "/posts": { target: "http://localhost:8000", changeOrigin: true },
      "/profile": { target: "http://localhost:8000", changeOrigin: true },
      "/uploads": { target: "http://localhost:8000", changeOrigin: true },
      "/health": { target: "http://localhost:8000", changeOrigin: true },
      "/storage": {
        target: "http://localhost:9000",
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/storage/, ""),
      },
    },
  },
  // Remove tailwind/postcss — we use app.css from frontend_web directly
  test: {
    globals: true,
    environment: "jsdom",
    setupFiles: ["./src/test/setup.ts"],
    include: ["src/**/*.{test,spec}.{ts,tsx}"],
    coverage: {
      provider: "v8",
      reporter: ["text", "html", "lcov"],
      exclude: ["src/test/**", "src/main.tsx", "src/vite-env.d.ts"],
    },
  },
});
```

7.7 tsconfig.json - TypeScript設定

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

7.8 package.json - パッケージ設定

```
{
  "name": "frontend_react",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest run --coverage"
  },
  "dependencies": {
    "@tanstack/react-query": "^5.90.21",
    "@types/katex": "^0.16.8",
    "@types/react-router-dom": "^5.3.3",
    "axios": "^1.13.5",
    "katex": "^0.16.33",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.13.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@tailwindcss/postcss": "^4.1.18",
    "@testing-library/jest-dom": "^6.6.3",
    "@testing-library/react": "^16.3.0",
    "@testing-library/user-event": "^14.6.1",
    "@types/node": "^24.10.1",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "@vitest/coverage-v8": "^3.2.4",
    "@vitest/ui": "^3.2.4",
    "autoprefixer": "^10.4.24",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "jsdom": "^26.1.0",
```

```
    "postcss": "^8.5.6",
    "tailwindcss": "^4.1.18",
    "typescript": "~5.9.3",
    "typescript-eslint": "^8.48.0",
    "vite": "^7.3.1",
    "vitest": "^3.2.4"
  }
}
```

Chapter 8

スクリプト

このセクションでは、デプロイやテストに使用される主要なスクリプトを掲載します。

8.1 build-lambda-layer.sh - Lambda Layer ビルド

```
#!/bin/bash

# Lambda Layer ビルドスクリプト
# 依存関係をに分離して関数のサイズを削減LayerLambda

set -e

# カラー定義
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

# 設定
PROJECT_NAME="${PROJECT_NAME:-multicloud-auto-deploy}"
ENVIRONMENT="${ENVIRONMENT:-staging}"
LAYER_NAME="${LAYER_NAME:-${PROJECT_NAME}-${ENVIRONMENT}-dependencies}"

# ディレクトリ設定
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
API_DIR="$PROJECT_ROOT/services/api"
BUILD_DIR="$API_DIR/.build-layer"

echo -e "${GREEN}========================================${NC}"
echo -e "${GREEN}Lambda Layer ビルドスクリプト${NC}"
echo -e "${GREEN}========================================${NC}"
echo ""
echo "レイヤー名: $LAYER_NAME"
echo ""

# 前提条件チェック
echo -e "${YELLOW}1. 前提条件のチェック...${NC}"

if ! command -v python3 &> /dev/null; then
    echo -e "${RED⌧} がインストールされていませんPython3${NC}"
```

```
    exit 1
fi

echo -e "${GREEN⊠} 確認完了Python3${NC}"

# ビルドディレクトリのクリーンアップ
echo -e "${YELLOW}2. ビルドディレクトリのクリーンアップ...${NC}"
rm -rf "$BUILD_DIR"
mkdir -p "$BUILD_DIR/python"
echo -e "${GREEN⊠} クリーンアップ完了${NC}"

# 用の最小限の依存関係をインストールAWS
echo -e "${YELLOW}3. 依存関係のインストール...${NC}"

# requirements-layer.txt を使用（API から）dir
if [ -f "$API_DIR/requirements-layer.txt" ]; then
    echo "既存の requirements-layer.txt を使用します"
    cp "$API_DIR/requirements-layer.txt" "$BUILD_DIR/requirements-layer.txt"
else
    echo "requirements-layer.txt が見つかりません。デフォルト設定を使用します"
    # Lambda Layer requirements (専用、AWSboto3/除外botocore)
    cat > "$BUILD_DIR/requirements-layer.txt" <<EOF
# Type checking (Python 3.12 compatibility)
typing_extensions==4.12.2

# FastAPI Core
fastapi==0.115.0
pydantic==2.9.0
pydantic-settings==2.5.2

# Lambda adapter
mangum==0.17.0

# Auth
python-jose[cryptography]==3.3.0
pyjwt==2.9.0

# HTTP client
requests==2.32.3

# Note: とはランタイムに含まれているため除外boto3botocoreLambda
EOF
fi

echo "インストール対象の依存関係:"
cat "$BUILD_DIR/requirements-layer.txt"
echo ""

# Lambda の構造に従ってインストールLayer (python/ ディレクトリに配置)
# Suppress non-critical warnings for cleaner CI/CD logs
pip3 install -r "$BUILD_DIR/requirements-layer.txt" \
    -t "$BUILD_DIR/python/" \
    --upgrade \
    --platform manylinux2014_x86_64 \
    --python-version 3.12 \
    --only-binary=:all: \
    --disable-pip-version-check \
    --quiet 2>&1 | tee /tmp/pip-install.log || { \
```

```
      echo -e "${RED✗} pip install failed${NC}"; \
      cat /tmp/pip-install.log; \
      exit 1; \
    }

# 不要なファイルを削除してサイズを削減
echo -e "${YELLOW}4. 不要なファイルの削除...${NC}"
cd "$BUILD_DIR/python"

# テストファイルとドキュメントを削除
find . -type d -name "tests" -o -name "test" -exec rm -rf {} + 2>/dev/null || true
find . -type d -name "__pycache__" -exec rm -rf {} + 2>/dev/null || true
find . -type f -name "*.pyc" -delete 2>/dev/null || true
find . -type f -name "*.pyo" -delete 2>/dev/null || true
find . -type f -name "*.dist-info" -exec rm -rf {} + 2>/dev/null || true

# .so ファイル以外の開発用ファイルを削除
find . -type f -name "*.c" -delete 2>/dev/null || true
find . -type f -name "*.h" -delete 2>/dev/null || true
find . -type f -name "*.txt" -delete 2>/dev/null || true
find . -type f -name "*.md" -delete 2>/dev/null || true
find . -type f -name "*.rst" -delete 2>/dev/null || true

echo -e "${GREEN✗} クリーンアップ完了${NC}"

# パッケージの作成ZIP
echo -e "${YELLOW}5. パッケージの作成ZIP...${NC}"
cd "$BUILD_DIR"
zip -r9q ../lambda-layer.zip python/ || { \
  echo -e "${RED✗} Failed to create ZIP file${NC}"; \
  exit 1; \
}
cd "$API_DIR"
LAYER_ZIP="$BUILD_DIR/../lambda-layer.zip"
if [ ! -f "$LAYER_ZIP" ]; then
  echo -e "${RED✗} ZIP file not created: $LAYER_ZIP${NC}"
  exit 1
fi

LAYER_SIZE=$(du -h "$LAYER_ZIP" | cut -f1)
echo -e "${GREEN✗} パッケージ作成完了Layer サイズ(: $LAYER_SIZE)${NC}"

# サイズ確認 (Linux/互換macOS)
LAYER_SIZE_BYTES=$(stat -c%s "$LAYER_ZIP" 2>/dev/null || stat -f%z "$LAYER_ZIP" 2>/dev/null || du -b "$LAYER_ZIP"
     | cut -f1)
LAYER_SIZE_MB=$((LAYER_SIZE_BYTES / 1024 / 1024))

echo ""
echo -e "${GREEN}========================================${NC}"
echo -e "${GREEN}Layer ビルド完了${NC}"
echo -e "${GREEN}========================================${NC}"
echo "Layer パスZIP: $BUILD_DIR/../lambda-layer.zip"
echo "Layer サイズ: ${LAYER_SIZE} (${LAYER_SIZE_MB}MB)"
echo ""

if [ $LAYER_SIZE_MB -gt 50 ]; then
    echo -e "${YELLOW⚠} 警告: Layer サイズが50を超えています (MB${LAYER_SIZE_MB}) MB${NC}"
    echo "   Lambda の最大サイズは (解凍後) ですLayer250MB"
```

```
    echo -e "${YELLOW}   ⚠  アップロードが必要になり、デプロイが遅くなりますS3${NC}"
else
    echo -e "${GREEN⊠} Layer サイズOK (${LAYER_SIZE_MB}MB - 50以下MB)${NC}"
fi

# GitHub 環境での出力設定Actions
if [ -n "$GITHUB_OUTPUT" ]; then
    echo "layer_size_mb=$LAYER_SIZE_MB" >> "$GITHUB_OUTPUT"
    echo "layer_zip_path=$LAYER_ZIP" >> "$GITHUB_OUTPUT"
fi

echo ""
echo "⊠ Lambda Layer ビルド完了！"
```

8.2 deploy-aws.sh - AWS デプロイスクリプト

```
#!/bin/bash
# ========================================
# Script Name: deploy-aws.sh
# Description: AWS Terraform Deployment Script
# Author: PLAYER1-r7
# Created: 2025-12-15
# Last Modified: 2026-02-15
# Version: 1.0.0
# ========================================
#
# Usage: ./scripts/deploy-aws.sh [environment]
#
# Description:
#   Deploys infrastructure to AWS using Terraform.
#   Creates Lambda, API Gateway, DynamoDB, and S3 resources.
#
# Parameters:
#   $1 - Environment name (default: staging)
#
# Deployment Components:
#   - Lambda Functions (Python 3.12)
#   - API Gateway v2 (HTTP)
#   - DynamoDB (PAY_PER_REQUEST)
#   - S3 Static Website
#   - CloudFront Distribution
#
# Prerequisites:
#   - Terraform 1.5+ installed
#   - AWS CLI configured
#   - Valid AWS credentials
#
# Exit Codes:
#   0 - Deployment successful
#   1 - Deployment failed
#
# ========================================

set -e

ENVIRONMENT=${1:-staging}
```

```
PROJECT_NAME="multicloud-auto-deploy"
AWS_REGION="ap-northeast-1"

echo "⊠ Starting AWS deployment for environment: $ENVIRONMENT"

# 色付き出力
GREEN='\033[0;32m'
BLUE='\033[0;34m'
RED='\033[0;31m'
NC='\033[0m' # No Color

# 1. フロントエンドのビルド
echo -e "${BLUE⊠} Building frontend...${NC}"
cd services/frontend
npm install
VITE_API_URL="https://api-${ENVIRONMENT}.example.com" npm run build
cd ../..

# 2. バックエンドのパッケージング
echo -e "${BLUE⊠} Packaging backend...${NC}"
cd services/backend
pip install -r requirements.txt -t package/
cp -r src/* package/
cd package && zip -r ../lambda.zip . && cd ..
cd ../..

# 3. によるインフラデプロイTerraform
echo -e "${BLUE⊠}  Deploying infrastructure with Terraform...${NC}"
cd infrastructure/terraform/aws

# 用のプレースホルダーを作成（初回のみ）LambdaZIP
if [ ! -f lambda_placeholder.zip ]; then
    echo "print('placeholder')" > lambda_placeholder.py
    zip lambda_placeholder.zip lambda_placeholder.py
    rm lambda_placeholder.py
fi

terraform init
terraform plan -var="environment=$ENVIRONMENT" -var="project_name=$PROJECT_NAME" -out=tfplan
terraform apply -auto-approve tfplan

# の出力を取得Terraform
S3_BUCKET=$(terraform output -raw frontend_bucket_name)
CLOUDFRONT_ID=$(terraform output -raw cloudfront_distribution_id)
LAMBDA_FUNCTION=$(terraform output -raw lambda_function_name)
API_ENDPOINT=$(terraform output -raw api_endpoint)

cd ../../..

# 4. フロントエンドをにデプロイS3
echo -e "${BLUE⊙}  Deploying frontend to S3...${NC}"
aws s3 sync services/frontend/dist/ "s3://${S3_BUCKET}/" --delete

# 5. キャッシュを無効化CloudFront
echo -e "${BLUE⊠} Invalidating CloudFront cache...${NC}"
aws cloudfront create-invalidation --distribution-id "$CLOUDFRONT_ID" --paths "/*"

# 6. 関数を更新Lambda
```

```
echo -e "${BLUE⊠} Updating Lambda function...${NC}"
aws lambda update-function-code \
    --function-name "$LAMBDA_FUNCTION" \
    --zip-file fileb://services/backend/lambda.zip

echo -e "${GREEN⊠} Deployment completed successfully!${NC}"
echo -e "${GREEN⊠} Frontend URL: https://$(aws cloudfront get-distribution --id $CLOUDFRONT_ID --query
     'Distribution.DomainName' --output text)${NC}"
echo -e "${GREEN⊠} API Endpoint: $API_ENDPOINT${NC}"
```

8.3 deploy-azure.sh - Azure デプロイスクリプト

```
#!/bin/bash
# ========================================
# Script Name: deploy-azure.sh
# Description: Azure Deployment Script
# Author: PLAYER1-r7
# Created: 2025-12-15
# Last Modified: 2026-02-15
# Version: 1.0.0
# ========================================
#
# Usage: ./scripts/deploy-azure.sh [environment]
#
# Description:
#   Deploys infrastructure to Azure.
#   Creates Functions, Cosmos DB, Blob Storage, and Front Door.
#
# Parameters:
#   $1 - Environment name (default: staging)
#
# Deployment Components:
#   - Azure Functions (Flex Consumption, Python 3.12)
#   - Cosmos DB (Serverless)
#   - Blob Storage ($web container)
#   - Azure Front Door (Standard)
#
# Prerequisites:
#   - Azure CLI installed and authenticated
#   - Resource group created
#   - Valid Azure subscription
#
# Exit Codes:
#   0 - Deployment successful
#   1 - Deployment failed
#
# ========================================

set -e

ENVIRONMENT=${1:-staging}
PROJECT_NAME="multicloud-auto-deploy"
AZURE_REGION="eastus"

echo "⊠ Starting Azure deployment for environment: $ENVIRONMENT"

```

```
GREEN='\033[0;32m'
BLUE='\033[0;34m'
NC='\033[0m'

# 1. フロントエンドのビルド
echo -e "${BLUE⌧} Building frontend...${NC}"
cd services/frontend
npm install
VITE_API_URL="https://api-${ENVIRONMENT}.azurewebsites.net" npm run build
cd ../..

# 2. リソースグループの作成Azure
echo -e "${BLUE⌧}  Creating Azure resources...${NC}"
RESOURCE_GROUP="${PROJECT_NAME}-${ENVIRONMENT}-rg"
STORAGE_ACCOUNT="${PROJECT_NAME}${ENVIRONMENT}st"
FUNCTION_APP="${PROJECT_NAME}-${ENVIRONMENT}-api"

az group create --name "$RESOURCE_GROUP" --location "$AZURE_REGION"

# 3. ストレージアカウント（フロントエンド用）
az storage account create \
    --name "$STORAGE_ACCOUNT" \
    --resource-group "$RESOURCE_GROUP" \
    --location "$AZURE_REGION" \
    --sku Standard_LRS \
    --kind StorageV2 \
    --allow-blob-public-access true

# 静的サイトを有効化Web
az storage blob service-properties update \
    --account-name "$STORAGE_ACCOUNT" \
    --static-website \
    --index-document index.html \
    --404-document index.html

# 4. フロントエンドをデプロイ
echo -e "${BLUE⌧}  Deploying frontend...${NC}"
az storage blob upload-batch \
    --account-name "$STORAGE_ACCOUNT" \
    --destination '$web' \
    --source services/frontend/dist/ \
    --overwrite

# 5. Azure をデプロイFunctions
echo -e "${BLUE⌧} Deploying backend...${NC}"
az functionapp create \
    --resource-group "$RESOURCE_GROUP" \
    --name "$FUNCTION_APP" \
    --storage-account "$STORAGE_ACCOUNT" \
    --runtime python \
    --runtime-version 3.11 \
    --functions-version 4 \
    --os-type Linux

cd services/backend
func azure functionapp publish "$FUNCTION_APP"
cd ../..

```

```
FRONTEND_URL=$(az storage account show \
    --name "$STORAGE_ACCOUNT" \
    --resource-group "$RESOURCE_GROUP" \
    --query "primaryEndpoints.web" \
    --output tsv)

echo -e "${GREEN⊠} Azure deployment completed!${NC}"
echo -e "${GREEN⊠} Frontend URL: $FRONTEND_URL${NC}"
echo -e "${GREEN⊠} API Endpoint: https://${FUNCTION_APP}.azurewebsites.net${NC}"
```

8.4 deploy-gcp.sh - GCP デプロイスクリプト

```
#!/bin/bash
# ========================================
# Script Name: deploy-gcp.sh
# Description: GCP Deployment Script
# Author: PLAYER1-r7
# Created: 2025-12-15
# Last Modified: 2026-02-15
# Version: 1.0.0
# ========================================
#
# Usage: ./scripts/deploy-gcp.sh [environment] [project_id]
#
# Description:
#   Deploys infrastructure to GCP.
#   Creates Cloud Run, Firestore, Cloud Storage, and Cloud CDN.
#
# Parameters:
#   $1 - Environment name (default: staging)
#   $2 - GCP Project ID (required for first run)
#
# Deployment Components:
#   - Cloud Run (Containerized FastAPI)
#   - Firestore (Native Mode)
#   - Cloud Storage (Static Website)
#   - Cloud CDN (via Load Balancer)
#
# Prerequisites:
#   - gcloud CLI installed and authenticated
#   - Valid GCP project with billing enabled
#   - Required APIs enabled
#
# Exit Codes:
#   0 - Deployment successful
#   1 - Deployment failed
#
# ========================================

set -e

ENVIRONMENT=${1:-staging}
GCP_PROJECT_ID=${2:-your-gcp-project-id}
GCP_REGION="us-central1"
PROJECT_NAME="multicloud-auto-deploy"

```

```
echo "⊠ Starting GCP deployment for environment: $ENVIRONMENT"

GREEN='\033[0;32m'
BLUE='\033[0;34m'
NC='\033[0m'

# 1. プロジェクトを設定
gcloud config set project "$GCP_PROJECT_ID"

# 2. フロントエンドのビルド
echo -e "${BLUE⊠} Building frontend...${NC}"
cd services/frontend
npm install
VITE_API_URL="https://${GCP_REGION}-${GCP_PROJECT_ID}.cloudfunctions.net/api" npm run build
cd ../..

# 3. Cloud バケットの作成Storage
echo -e "${BLUE☁}  Creating Cloud Storage bucket...${NC}"
BUCKET_NAME="${PROJECT_NAME}-${ENVIRONMENT}-frontend"
gsutil mb -p "$GCP_PROJECT_ID" -l "$GCP_REGION" "gs://${BUCKET_NAME}/" || true
gsutil web set -m index.html -e index.html "gs://${BUCKET_NAME}/"
gsutil iam ch allUsers:objectViewer "gs://${BUCKET_NAME}/"

# 4. フロントエンドをデプロイ
echo -e "${BLUE⊠} Deploying frontend...${NC}"
gsutil -m rsync -r -d services/frontend/dist/ "gs://${BUCKET_NAME}/"

# 5. Cloud をデプロイFunctions
echo -e "${BLUE⊠} Deploying backend...${NC}"
cd services/backend
gcloud functions deploy "${PROJECT_NAME}-${ENVIRONMENT}-api" \
    --gen2 \
    --runtime=python311 \
    --region="$GCP_REGION" \
    --source=. \
    --entry-point=handler \
    --trigger-http \
    --allow-unauthenticated
cd ../..

# 6. またはFirestoreCloud （オプション）SQL
echo -e "${BLUE⊠} Setting up database...${NC}"
gcloud firestore databases create --location="$GCP_REGION" || echo "Firestore already exists"

FUNCTION_URL=$(gcloud functions describe "${PROJECT_NAME}-${ENVIRONMENT}-api" \
    --region="$GCP_REGION" \
    --format="value(serviceConfig.uri)")

echo -e "${GREEN⊠} GCP deployment completed!${NC}"
echo -e "${GREEN⊠} Frontend URL: https://storage.googleapis.com/${BUCKET_NAME}/index.html${NC}"
echo -e "${GREEN⊠} API Endpoint: $FUNCTION_URL${NC}"
```

8.5 test-e2e.sh - E2Eテスト

```
#!/usr/bin/env bash
# ==============================================================
```

```
# test-e2e.sh — Multi-Cloud End-to-End Test Suite (v2)
# ==============================================================
#
# Runs a lightweight E2E health + CRUD smoke test against all
# three cloud staging environments and prints a consolidated
# pass/fail table.
#
# Public endpoints are tested without a token.
# Authenticated endpoints (POST/PUT/DELETE) require tokens.
#
# Usage:
#   # Public endpoints only:
#   ./scripts/test-e2e.sh
#
#   # Full authenticated run:
#   ./scripts/test-e2e.sh \
#     --aws-token   <cognito-access-token> \
#     --azure-token <azure-ad-id-token> \
#     --gcp-token   <firebase-id-token>
#
#   # URL overrides:
#   AWS_API_URL=https://... ./scripts/test-e2e.sh
#
# How to get tokens:
#   AWS (Cognito):
#     aws cognito-idp initiate-auth \
#       --auth-flow USER_PASSWORD_AUTH \
#       --client-id 1k41lqkds4oah55ns8iod30dv2 \
#       --auth-parameters USERNAME=user@example.com,PASSWORD=pw \
#       --region ap-northeast-1 \
#       --query 'AuthenticationResult.AccessToken' --output text
#   GCP (Firebase):
#     gcloud auth print-identity-token
#   Azure (Azure AD):
#     Copy id_token from browser DevTools Application > Local Storage
#
# Exit codes:
#   0 — All tests passed
#   1 — One or more tests failed
#   2 — Missing required dependency
# ==============================================================

set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'
YELLOW='\033[1;33m'; BLUE='\033[0;34m'; CYAN='\033[0;36m'
BOLD='\033[1m'; NC='\033[0m'

AWS_API_URL="${AWS_API_URL:-}"
AZURE_API_URL="${AZURE_API_URL:-}"
GCP_API_URL="${GCP_API_URL:-}"
AWS_TOKEN=""
AZURE_TOKEN=""
GCP_TOKEN=""
VERBOSE=false
_ENV_=staging
_READ_ONLY_=false
_WRITE_=false
```

```
READY_ONLY=false

command -v curl >/dev/null 2>&1 || { echo -e "${RED}ERROR${NC}: curl required" >&2; exit 2; }
command -v jq   >/dev/null 2>&1 || { echo -e "${RED}ERROR${NC}: jq required"   >&2; exit 2; }

usage() {
  cat <<EOF
Usage: $0 [OPTIONS]
  -e, --env   <env>       Target environment: staging|production  (default: staging)
                          --env production uses custom domain URLs and implies --read-only
  -r, --read-only         Skip all write tests (POST/PUT/DELETE/presigned-urls)
      --write             Allow write tests even when --env production is set
  --aws-token   <token>   Cognito access token
  --azure-token <token>   Azure AD ID token
  --gcp-token   <token>   Firebase ID token
  -v, --verbose           Print response bodies on failure
  -h, --help              Show this help

Examples:
  # Staging (default):
  $0 --aws-token eyJ...
  # Production - read-only smoke test:
  $0 --env production
  # Production - full test with writes:
  $0 --env production --write --aws-token eyJ... --gcp-token \$(gcloud auth print-identity-token)
EOF
}

while [[ $# -gt 0 ]]; do
  case $1 in
    -e|--env)
      case "$2" in
        production|prod) _ENV_=production; _READ_ONLY_=true ;;
        staging|stag)    _ENV_=staging ;;
        *) echo -e "${RED}Unknown env: $2${NC}"; exit 1 ;;
      esac
      shift 2 ;;
    -r|--read-only) _READ_ONLY_=true; shift ;;
    --write)        _WRITE_=true;     shift ;;
    --aws-token)   AWS_TOKEN="$2";   shift 2 ;;
    --azure-token) AZURE_TOKEN="$2"; shift 2 ;;
    --gcp-token)   GCP_TOKEN="$2";   shift 2 ;;
    -v|--verbose)  VERBOSE=true;     shift   ;;
    -h|--help)     usage; exit 0 ;;
    *) echo -e "${RED}Unknown option: $1${NC}"; usage; exit 1 ;;
  esac
done

# Resolve read-only flag
READ_ONLY=$_READ_ONLY_
[[ $_WRITE_ == true ]] && READ_ONLY=false

# Resolve URLs based on environment
if [[ $_ENV_ == production ]]; then
  AWS_API_URL="${AWS_API_URL:-https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com}"
  AZURE_API_URL="${AZURE_API_URL:-https://multicloud-auto-deploy-production-func-cfdne7ecbngnh0d0.japaneast-01.azurewebsites.ne
  GCP_API_URL="${GCP_API_URL:-https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app}"
```

```
else
  AWS_API_URL="${AWS_API_URL:-https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com}"
    AZURE_API_URL="${AZURE_API_URL:-https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net}"
  GCP_API_URL="${GCP_API_URL:-https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app}"
fi

declare -A RESULTS
PASS=0; FAIL=0; SKIP=0

rec_pass() { RESULTS["$1:$2"]="pass"; PASS=$((PASS + 1)); }
rec_fail() { RESULTS["$1:$2"]="fail"; FAIL=$((FAIL + 1)); }
rec_skip() { RESULTS["$1:$2"]="skip"; SKIP=$((SKIP + 1)); }

HTTP_STATUS=0
HTTP_BODY=""

http_req() {
  local method="$1" url="$2" token="${3:-}" data="${4:-}"
  local tmpfile
  tmpfile=$(mktemp /tmp/e2e_body_XXXXXX)
  local curl_args=(-s -o "$tmpfile" -w "%{http_code}" -X "$method" --max-time 25 --compressed)
  [[ -n "$token" ]] && curl_args+=(-H "Authorization: Bearer $token")
  [[ -n "$data"  ]] && curl_args+=(-H "Content-Type: application/json" -d "$data")
  curl_args+=("$url")
  HTTP_STATUS=$(curl "${curl_args[@]}" 2>/dev/null || echo "000")
  HTTP_BODY=$(cat "$tmpfile" 2>/dev/null || echo "")
  rm -f "$tmpfile"
}

test_cloud_suite() {
  local cloud="$1" api_base="$2" token="${3:-}"
  local cloud_up
  cloud_up=$(echo "$cloud" | tr '[:lower:]' '[:upper:]')

  echo ""
  echo -e
    "${BOLD}${YELLOW━━━
```

```
    [[ "$VERBOSE" == true ]] && echo "$HTTP_BODY" | head -c 300
  fi

  # 2. List posts (public)
  echo -e "\n  ${BLUE}[2]${NC} List posts (public)"
  http_req GET "$api_base/posts"
  if [[ "$HTTP_STATUS" == "200" ]] && echo "$HTTP_BODY" | jq -e '.items' >/dev/null 2>&1; then
    local count
    count=$(echo "$HTTP_BODY" | jq '.items | length' 2>/dev/null || echo "?")
    echo -e "  ${GREEN}[PASS]${NC}  GET /posts → 200  (items=$count)"; rec_pass "$cloud" "GET /posts"
  else
    echo -e "  ${RED}[FAIL]${NC}  GET /posts → $HTTP_STATUS or missing .items"; rec_fail "$cloud" "GET /posts"
    [[ "$VERBOSE" == true ]] && echo "$HTTP_BODY" | head -c 300
  fi

  # 3. Auth guard
  echo -e "\n  ${BLUE}[3]${NC} Auth guard (unauthenticated POST)"
  http_req POST "$api_base/posts" "" '{"content":"auth guard test"}'
  if [[ "$HTTP_STATUS" == "401" || "$HTTP_STATUS" == "403" ]]; then
    echo -e "  ${GREEN}[PASS]${NC}  POST /posts (no token) → $HTTP_STATUS"; rec_pass "$cloud" "auth guard"
  else
    echo -e "  ${RED}[FAIL]${NC}  POST /posts (no token) → $HTTP_STATUS (expected 401/403)"; rec_fail "$cloud"
     "auth guard"
  fi

  # 4-6. Authenticated CRUD
  if [[ $READ_ONLY == true ]]; then
    echo -e "\n  ${CYAN}[SKIP]${NC}  Read-only mode: skipping write tests (POST/PUT/DELETE/presigned-urls)"
    for lbl in "POST /posts" "GET /posts/:id" "PUT /posts/:id" "DELETE /posts/:id" "presigned-urls"; do
      rec_skip "$cloud" "$lbl"
    done
    return
  fi

  if [[ -z "$token" ]]; then
    echo -e "\n  ${CYAN}[SKIP]${NC}  Sections 4-6: no token provided"
    for lbl in "POST /posts" "GET /posts/:id" "PUT /posts/:id" "DELETE /posts/:id" "presigned-urls"; do
      rec_skip "$cloud" "$lbl"
    done
    return
  fi

  # 4. Create post
  echo -e "\n  ${BLUE}[4]${NC} CRUD (authenticated)"
  local ts
  ts=$(date +%s)
  http_req POST "$api_base/posts" "$token" \
    "{\"content\":\"[E2E] $cloud_up smoke test $ts\",\"isMarkdown\":false}"
  if [[ "$HTTP_STATUS" == "201" ]]; then
    local pid
    pid=$(echo "$HTTP_BODY" | jq -r '.postId // .post_id // ""' 2>/dev/null)
    if [[ -n "$pid" && "$pid" != "null" ]]; then
      echo -e "  ${GREEN}[PASS]${NC}  POST /posts → 201  (postId=$pid)"; rec_pass "$cloud" "POST /posts"
      created_ids+=("$pid")
    else
      echo -e "  ${RED}[FAIL]${NC}  POST /posts → 201 but no postId"; rec_fail "$cloud" "POST /posts"
      pid=""
    fi
```

```
  else
    echo -e "  ${RED}[FAIL]${NC}  POST /posts → $HTTP_STATUS (expected 201)"; rec_fail "$cloud" "POST /posts"
    [[ "$VERBOSE" == true ]] && echo "$HTTP_BODY" | head -c 300
    for lbl in "GET /posts/:id" "PUT /posts/:id" "DELETE /posts/:id" "presigned-urls"; do rec_skip "$cloud" "$lbl"; done
    return
  fi

  # 5. Get by ID
  http_req GET "$api_base/posts/$pid" "$token"
  if [[ "$HTTP_STATUS" == "200" ]]; then
    echo -e "  ${GREEN}[PASS]${NC}  GET /posts/$pid → 200"; rec_pass "$cloud" "GET /posts/:id"
  else
    echo -e "  ${RED}[FAIL]${NC}  GET /posts/$pid → $HTTP_STATUS"; rec_fail "$cloud" "GET /posts/:id"
  fi

  # 6. Update
  http_req PUT "$api_base/posts/$pid" "$token" \
    "{\"content\":\"[E2E] $cloud_up updated $ts\"}"
  if [[ "$HTTP_STATUS" == "200" ]]; then
    echo -e "  ${GREEN}[PASS]${NC}  PUT /posts/$pid → 200"; rec_pass "$cloud" "PUT /posts/:id"
  else
    echo -e "  ${RED}[FAIL]${NC}  PUT /posts/$pid → $HTTP_STATUS"; rec_fail "$cloud" "PUT /posts/:id"
  fi

  # 7. Presigned URL
  echo -e "\n  ${BLUE}[5]${NC} Presigned URL"
  http_req POST "$api_base/uploads/presigned-urls" "$token" \
    '{"count":1,"contentTypes":["image/jpeg"]}'
  if [[ "$HTTP_STATUS" == "200" ]] && echo "$HTTP_BODY" | jq -e '.urls[0].url' >/dev/null 2>&1; then
    echo -e "  ${GREEN}[PASS]${NC}  POST /uploads/presigned-urls → 200"; rec_pass "$cloud" "presigned-urls"
  else
    echo -e "  ${RED}[FAIL]${NC}  POST /uploads/presigned-urls → $HTTP_STATUS"; rec_fail "$cloud" "presigned-urls"
    [[ "$VERBOSE" == true ]] && echo "$HTTP_BODY" | head -c 300
  fi

  # 8. Delete
  echo -e "\n  ${BLUE}[6]${NC} Cleanup"
  for del_id in "${created_ids[@]}"; do
    http_req DELETE "$api_base/posts/$del_id" "$token"
    if [[ "$HTTP_STATUS" == "200" || "$HTTP_STATUS" == "204" ]]; then
      echo -e "  ${GREEN}[PASS]${NC}  DELETE /posts/$del_id → $HTTP_STATUS"; rec_pass "$cloud" "DELETE /posts/:id"
    else
      echo -e "  ${RED}[FAIL]${NC}  DELETE /posts/$del_id → $HTTP_STATUS"; rec_fail "$cloud" "DELETE /posts/:id"
    fi
  done
}

# ─── Run all clouds ──────────────────────────────────────────────────────────

echo ""
echo -e "${BOLD╭──────────────────────────────────────────────────────────
```

```
echo -e "${BOLD└──────────────
```

```
fi
echo ""

if [[ $FAIL -eq 0 ]]; then
  echo -e "${GREEN}${BOLD}  ☒ All executed tests passed!${NC}"
  exit 0
else
  echo -e "${RED}${BOLD}  ☒ $FAIL test(s) failed.${NC}"
  if [[ -z "$AWS_TOKEN" && -z "$AZURE_TOKEN" && -z "$GCP_TOKEN" ]]; then
    echo ""
    echo -e "  ${YELLOW}TIP:${NC} Re-run with auth tokens to test CRUD:"
    echo "    $0 \\"
    echo "      --aws-token   \"\$(aws cognito-idp initiate-auth --auth-flow USER_PASSWORD_AUTH --client-id
     1k41lqkds4oah55ns8iod30dv2 --auth-parameters USERNAME=YOU@example.com,PASSWORD=PW --region ap-northeast-1
     --query AuthenticationResult.AccessToken --output text)\" \\"
    echo "      --gcp-token   \"\$(gcloud auth print-identity-token)\" \\"
    echo "      --azure-token \"<paste id_token from browser DevTools>\""
  fi
  exit 1
fi
```

8.6 cloud_architecture_mapper.py - アーキテクチャマッパー

```
#!/usr/bin/env python3
"""Collect multicloud resources and render architecture diagrams.

MVP scope:
- AWS / Azure / GCP collection using Pulumi outputs + cloud CLI discovery
- Normalized JSON snapshot output
- Mermaid diagram generation
"""

from __future__ import annotations

import argparse
import json
import re
import subprocess
import sys
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

REPO_ROOT = Path(__file__).resolve().parents[1]
PULUMI_DIRS = {
    "aws": REPO_ROOT / "infrastructure" / "pulumi" / "aws",
    "azure": REPO_ROOT / "infrastructure" / "pulumi" / "azure",
    "gcp": REPO_ROOT / "infrastructure" / "pulumi" / "gcp",
}


@dataclass
class Resource:
    provider: str
    resource_type: str
```

```
    name: str
    resource_id: str
    region: str | None
    source: str
    metadata: dict[str, Any] = field(default_factory=dict)


@dataclass
class CloudSnapshot:
    provider: str
    pulumi_outputs: dict[str, Any] = field(default_factory=dict)
    resources: list[Resource] = field(default_factory=list)
    errors: list[str] = field(default_factory=list)


def run_json_command(command: list[str], cwd: Path | None = None) -> Any:
    result = subprocess.run(
        command,
        cwd=str(cwd) if cwd else None,
        capture_output=True,
        text=True,
        check=False,
    )
    if result.returncode != 0:
        raise RuntimeError(
            result.stderr.strip() or result.stdout.strip() or "Command failed"
        )
    try:
        return json.loads(result.stdout)
    except json.JSONDecodeError as exc:
        raise RuntimeError(
            f"Invalid JSON output from command: {' '.join(command)}"
        ) from exc


def run_text_command(command: list[str], cwd: Path | None = None) -> str:
    result = subprocess.run(
        command,
        cwd=str(cwd) if cwd else None,
        capture_output=True,
        text=True,
        check=False,
    )
    if result.returncode != 0:
        raise RuntimeError(
            result.stderr.strip() or result.stdout.strip() or "Command failed"
        )
    return result.stdout.strip()


def safe_collect(collector: callable, snapshot: CloudSnapshot) -> None:
    try:
        collector(snapshot)
    except Exception as exc:
        snapshot.errors.append(str(exc))


def normalize_name(value: str | None, fallback: str) -> str:
```

```
    base = value or fallback
    cleaned = re.sub(r"[^a-zA-Z0-9_-]", "-", base)
    cleaned = re.sub(r"-{2,}", "-", cleaned)
    cleaned = cleaned.strip("-_")
    if not cleaned:
        fallback_cleaned = re.sub(r"[^a-zA-Z0-9_-]", "-", fallback)
        fallback_cleaned = re.sub(r"-{2,}", "-", fallback_cleaned).strip("-_")
        cleaned = fallback_cleaned or "node"
    return cleaned[:80]


def append_resource(
    snapshot: CloudSnapshot,
    resource_type: str,
    name: str,
    resource_id: str,
    region: str | None,
    source: str,
    metadata: dict[str, Any] | None = None,
) -> None:
    snapshot.resources.append(
        Resource(
            provider=snapshot.provider,
            resource_type=resource_type,
            name=name,
            resource_id=resource_id,
            region=region,
            source=source,
            metadata=metadata or {},
        )
    )


def collect_pulumi_outputs(snapshot: CloudSnapshot, stack: str) -> None:
    outputs = run_json_command(
        ["pulumi", "stack", "output", "--json", "--stack", stack],
        cwd=PULUMI_DIRS[snapshot.provider],
    )
    snapshot.pulumi_outputs = outputs


def collect_aws(snapshot: CloudSnapshot, stack: str, region: str) -> None:
    collect_pulumi_outputs(snapshot, stack)

    prefix = f"multicloud-auto-deploy-{stack}"

    distributions = run_json_command(
        ["aws", "cloudfront", "list-distributions", "--output", "json"]
    )
    items = distributions.get("DistributionList", {}).get("Items", []) or []
    for dist in items:
        aliases = dist.get("Aliases", {}).get("Items", []) or []
        domain = dist.get("DomainName", "")
        if prefix in domain or any("ashnova.jp" in alias for alias in aliases):
            append_resource(
                snapshot,
                "cdn",
                dist.get("Id", "unknown-cf"),
```

```
                dist.get("ARN", dist.get("Id", "unknown-cf")),
                "global",
                "aws-cli",
                {"domain": domain, "aliases": aliases},
            )

    buckets = run_json_command(["aws", "s3api", "list-buckets", "--output", "json"])
    for bucket in buckets.get("Buckets", []) or []:
        name = bucket.get("Name", "")
        if prefix in name or "ashnova" in name:
            append_resource(snapshot, "object_storage", name, name, region, "aws-cli")

    apis = run_json_command(
        ["aws", "apigatewayv2", "get-apis", "--region", region, "--output", "json"]
    )
    for api in apis.get("Items", []) or []:
        name = api.get("Name", "")
        if prefix in name:
            append_resource(
                snapshot,
                "api_gateway",
                name,
                api.get("ApiId", name),
                region,
                "aws-cli",
                {"apiEndpoint": api.get("ApiEndpoint")},
            )

    lambdas = run_json_command(
        ["aws", "lambda", "list-functions", "--region", region, "--output", "json"]
    )
    for fn in lambdas.get("Functions", []) or []:
        fn_name = fn.get("FunctionName", "")
        if prefix in fn_name:
            append_resource(
                snapshot,
                "compute",
                fn_name,
                fn.get("FunctionArn", fn_name),
                region,
                "aws-cli",
            )

    ddb = run_json_command(
        ["aws", "dynamodb", "list-tables", "--region", region, "--output", "json"]
    )
    for table in ddb.get("TableNames", []) or []:
        if prefix in table or table == "simple-sns-messages":
            append_resource(snapshot, "database", table, table, region, "aws-cli")


def collect_azure(snapshot: CloudSnapshot, stack: str, region: str) -> None:
    collect_pulumi_outputs(snapshot, stack)

    prefix = f"multicloud-auto-deploy-{stack}"

    frontdoor_profiles = run_json_command(
        ["az", "afd", "profile", "list", "-o", "json"]
```

```
    )
    for profile in frontdoor_profiles:
        name = profile.get("name", "")
        rg = profile.get("resourceGroup", "")
        if prefix in name or f"{stack}-rg" in rg:
            append_resource(
                snapshot, "cdn", name, profile.get("id", name), "global", "az-cli"
            )

    storage_accounts = run_json_command(
        ["az", "storage", "account", "list", "-o", "json"]
    )
    for account in storage_accounts:
        name = account.get("name", "")
        rg = account.get("resourceGroup", "")
        if prefix in rg or name.startswith("mcad"):
            append_resource(
                snapshot,
                "object_storage",
                name,
                account.get("id", name),
                account.get("primaryLocation", region),
                "az-cli",
            )

    function_apps = run_json_command(["az", "functionapp", "list", "-o", "json"])
    for app in function_apps:
        name = app.get("name", "")
        rg = app.get("resourceGroup", "")
        if prefix in name or prefix in rg:
            append_resource(
                snapshot,
                "compute",
                name,
                app.get("id", name),
                app.get("location", region),
                "az-cli",
            )

    cosmos_accounts = run_json_command(["az", "cosmosdb", "list", "-o", "json"])
    for account in cosmos_accounts:
        name = account.get("name", "")
        rg = account.get("resourceGroup", "")
        if prefix in name or prefix in rg:
            append_resource(
                snapshot,
                "database",
                name,
                account.get("id", name),
                account.get("location", region),
                "az-cli",
            )


def collect_gcp(snapshot: CloudSnapshot, stack: str, region: str) -> None:
    collect_pulumi_outputs(snapshot, stack)

    project_id = snapshot.pulumi_outputs.get("gcp_project")
```

```
    if not project_id:
        try:
            project_id = run_text_command(["gcloud", "config", "get-value", "project"])
        except Exception:
            project_id = ""

    frontend_prefix = f"ashnova-multicloud-auto-deploy-{stack}"

    backend_buckets = run_json_command(
        ["gcloud", "compute", "backend-buckets", "list", "--format=json"]
    )
    for bucket in backend_buckets:
        name = bucket.get("name", "")
        bucket_name = bucket.get("bucketName", "")
        if stack in name or frontend_prefix in bucket_name:
            append_resource(
                snapshot,
                "cdn",
                name,
                name,
                "global",
                "gcloud-cli",
                {"bucketName": bucket_name},
            )

    url_maps = run_json_command(
        ["gcloud", "compute", "url-maps", "list", "--format=json"]
    )
    for item in url_maps:
        name = item.get("name", "")
        if "multicloud" in name and stack in name:
            append_resource(
                snapshot, "load_balancer", name, name, "global", "gcloud-cli"
            )

    run_services = run_json_command(
        [
            "gcloud",
            "run",
            "services",
            "list",
            "--platform=managed",
            "--region",
            region,
            "--format=json",
        ]
    )
    for service in run_services:
        name = service.get("metadata", {}).get("name", "")
        if stack in name or "multicloud-auto-deploy" in name:
            append_resource(snapshot, "compute", name, name, region, "gcloud-cli")

    buckets = run_json_command(
        ["gcloud", "storage", "buckets", "list", "--format=json"]
    )
    for bucket in buckets:
        name = bucket.get("name", "")
        if frontend_prefix in name:
```

```
            append_resource(
                snapshot,
                "object_storage",
                name,
                name,
                bucket.get("location", region),
                "gcloud-cli",
            )

    firestore_dbs = run_json_command(
        [
            "gcloud",
            "firestore",
            "databases",
            "list",
            "--project",
            project_id,
            "--format=json",
        ]
    )
    for db in firestore_dbs:
        db_name = db.get("name", "")
        db_id = db_name.split("/")[-1] if db_name else "default"
        append_resource(
            snapshot, "database", db_id, db_name or db_id, region, "gcloud-cli"
        )


def build_snapshot(
    stack: str, aws_region: str, azure_region: str, gcp_region: str
) -> dict[str, Any]:
    aws_snapshot = CloudSnapshot(provider="aws")
    azure_snapshot = CloudSnapshot(provider="azure")
    gcp_snapshot = CloudSnapshot(provider="gcp")

    safe_collect(lambda s: collect_aws(s, stack=stack, region=aws_region), aws_snapshot)
    safe_collect(
        lambda s: collect_azure(s, stack=stack, region=azure_region), azure_snapshot
    )
    safe_collect(lambda s: collect_gcp(s, stack=stack, region=gcp_region), gcp_snapshot)

    normalized = {
        "generatedAt": datetime.now(timezone.utc).isoformat(),
        "environment": stack,
        "clouds": {
            "aws": {
                "pulumiOutputs": aws_snapshot.pulumi_outputs,
                "resources": [asdict(resource) for resource in aws_snapshot.resources],
                "errors": aws_snapshot.errors,
            },
            "azure": {
                "pulumiOutputs": azure_snapshot.pulumi_outputs,
                "resources": [
                    asdict(resource) for resource in azure_snapshot.resources
                ],
                "errors": azure_snapshot.errors,
            },
            "gcp": {
```

```
                "pulumiOutputs": gcp_snapshot.pulumi_outputs,
                "resources": [asdict(resource) for resource in gcp_snapshot.resources],
                "errors": gcp_snapshot.errors,
            },
        },
    }
    return normalized


def first_resource(
    resources: list[dict[str, Any]], resource_type: str
) -> dict[str, Any] | None:
    for item in resources:
        if item.get("resource_type") == resource_type:
            return item
    return None


def render_cloud_lines(provider: str, resources: list[dict[str, Any]]) -> list[str]:
    lines = [f"  subgraph {provider.upper()}[{provider.upper()}]"]
    nodes: list[str] = []
    node_index: dict[tuple[str, str], str] = {}

    mapping = {
        "cdn": "CDN",
        "load_balancer": "LoadBalancer",
        "api_gateway": "ApiGateway",
        "compute": "Compute",
        "database": "Database",
        "object_storage": "ObjectStorage",
    }

    for resource in resources:
        kind = mapping.get(
            resource.get("resource_type", ""), resource.get("resource_type", "Resource")
        )
        resource_type = resource.get("resource_type", "resource")
        rid = normalize_name(
            f"{resource_type}-{resource.get('name')}",
            f"{provider}-{resource_type}-{kind}",
        )
        label = resource.get("name", kind)
        node_index[(resource_type, resource.get("name", ""))] = f"{provider}_{rid}"
        nodes.append(f'    {provider}_{rid}["{kind}: {label}"]')

    lines.extend(nodes)

    cdn = first_resource(resources, "cdn")
    lb = first_resource(resources, "load_balancer")
    storage = first_resource(resources, "object_storage")
    api = first_resource(resources, "api_gateway")
    compute = first_resource(resources, "compute")
    db = first_resource(resources, "database")

    def node_id(resource: dict[str, Any] | None, fallback: str) -> str:
        if not resource:
            return fallback
        key = (resource.get("resource_type", "resource"), resource.get("name", ""))
```

```
        return node_index.get(
            key,
            f"{provider}_{normalize_name(f'{resource.get("resource_type", "resource")}-{resource.get("name")}', fallback)}",
        )

    if lb and storage:
        lines.append(f"    {node_id(lb, 'lb')} --> {node_id(storage, 'storage')}")
    if cdn and storage:
        lines.append(f"    {node_id(cdn, 'cdn')} --> {node_id(storage, 'storage')}")
    if api and compute:
        lines.append(f"    {node_id(api, 'api')} --> {node_id(compute, 'compute')}")
    if compute and db:
        lines.append(f"    {node_id(compute, 'compute')} --> {node_id(db, 'database')}")

    lines.append("  end")
    return lines


def render_mermaid(snapshot: dict[str, Any]) -> str:
    aws_resources = snapshot["clouds"]["aws"]["resources"]
    azure_resources = snapshot["clouds"]["azure"]["resources"]
    gcp_resources = snapshot["clouds"]["gcp"]["resources"]

    lines = [
        "flowchart LR",
        "  User((User))",
    ]
    lines.extend(render_cloud_lines("aws", aws_resources))
    lines.extend(render_cloud_lines("azure", azure_resources))
    lines.extend(render_cloud_lines("gcp", gcp_resources))

    # フロー経路関連ノードの収集ID
    flow_node_ids: set[str] = set()

    for provider, resources in [
        ("aws", aws_resources),
        ("azure", azure_resources),
        ("gcp", gcp_resources),
    ]:
        entry = first_resource(resources, "cdn") or first_resource(
            resources, "load_balancer"
        )
        storage = first_resource(resources, "object_storage")
        compute = first_resource(resources, "compute")
        database = first_resource(resources, "database")

        if entry:
            entry_key = f"{entry.get('resource_type', 'resource')}-{entry.get('name')}"
            entry_id = f"{provider}_{normalize_name(entry_key, provider)}"
            lines.append(f"  User ユーザーアクセス-->|| {entry_id}")
            flow_node_ids.add(entry_id)

            # フロー経路: CDN/LB → Storage → Compute → Database
            if storage:
                storage_key = f"object_storage-{storage.get('name')}"
                storage_id = f"{provider}_{normalize_name(storage_key, provider)}"
                lines.append(f"  {entry_id} データ取得-->|| {storage_id}")
```

```
                flow_node_ids.add(storage_id)

            if compute:
                compute_key = f"compute-{compute.get('name')}"
                compute_id = f"{provider}_{normalize_name(compute_key, provider)}"
                if storage:
                    storage_key = f"object_storage-{storage.get('name')}"
                    storage_id = f"{provider}_{normalize_name(storage_key, provider)}"
                    lines.append(f"  {storage_id} -->|呼び出しAPI| {compute_id}")
                else:
                    lines.append(f"  {entry_id} -->|呼び出しAPI| {compute_id}")
                flow_node_ids.add(compute_id)

            if database:
                database_key = f"database-{database.get('name')}"
                database_id = f"{provider}_{normalize_name(database_key, provider)}"
                if compute:
                    compute_key = f"compute-{compute.get('name')}"
                    compute_id = f"{provider}_{normalize_name(compute_key, provider)}"
                    lines.append(f"  {compute_id} データアクセス-->|| {database_id}")
                flow_node_ids.add(database_id)

    # フロー関連ノードにスタイルを適用
    lines.append(
        "  classDef user_flow fill:#c7d2fe,stroke:#4f46e5,stroke-width:3px,color:#1e1b4b"
    )
    for node_id in flow_node_ids:
        lines.append(f"  class {node_id} user_flow")

    return "\n".join(lines) + "\n"


def validate_mermaid(mermaid: str) -> list[str]:
    errors: list[str] = []
    node_ids: set[str] = set()

    for line in mermaid.splitlines():
        node_match = re.match(r'^\s*([A-Za-z0-9_-]+)\["', line)
        if node_match:
            node_id = node_match.group(1)
            node_ids.add(node_id)
            if "--" in node_id:
                errors.append(f"Invalid node id contains '--': {node_id}")

    for line in mermaid.splitlines():
        edge_match = re.match(r"^\s*([A-Za-z0-9_-]+)\s*-->\s*([A-Za-z0-9_-]+)", line)
        if edge_match:
            source = edge_match.group(1)
            target = edge_match.group(2)
            if source not in node_ids and source != "User":
                errors.append(f"Edge source node not defined: {source}")
            if target not in node_ids and target != "User":
                errors.append(f"Edge target node not defined: {target}")

    return errors


def collect_command(args: argparse.Namespace) -> int:
```

```
    snapshot = build_snapshot(
        stack=args.environment,
        aws_region=args.aws_region,
        azure_region=args.azure_region,
        gcp_region=args.gcp_region,
    )
    output_path = Path(args.output)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(
        json.dumps(snapshot, indent=2, ensure_ascii=False), encoding="utf-8"
    )
    print(f"Snapshot written: {output_path}")
    return 0


def render_command(args: argparse.Namespace) -> int:
    input_path = Path(args.input)
    if not input_path.exists():
        print(f"Input snapshot does not exist: {input_path}", file=sys.stderr)
        return 1
    snapshot = json.loads(input_path.read_text(encoding="utf-8"))
    mermaid = render_mermaid(snapshot)
    if args.validate:
        errors = validate_mermaid(mermaid)
        if errors:
            print("Mermaid validation failed:", file=sys.stderr)
            for error in errors:
                print(f"  - {error}", file=sys.stderr)
            return 2
    output_path = Path(args.output)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(mermaid, encoding="utf-8")
    print(f"Mermaid diagram written: {output_path}")
    return 0


def all_command(args: argparse.Namespace) -> int:
    snapshot = build_snapshot(
        stack=args.environment,
        aws_region=args.aws_region,
        azure_region=args.azure_region,
        gcp_region=args.gcp_region,
    )
    snapshot_path = Path(args.snapshot_output)
    snapshot_path.parent.mkdir(parents=True, exist_ok=True)
    snapshot_path.write_text(
        json.dumps(snapshot, indent=2, ensure_ascii=False), encoding="utf-8"
    )

    mermaid = render_mermaid(snapshot)
    if args.validate:
        errors = validate_mermaid(mermaid)
        if errors:
            print("Mermaid validation failed:", file=sys.stderr)
            for error in errors:
                print(f"  - {error}", file=sys.stderr)
            return 2
    diagram_path = Path(args.diagram_output)
```

```
    diagram_path.parent.mkdir(parents=True, exist_ok=True)
    diagram_path.write_text(mermaid, encoding="utf-8")

    print(f"Snapshot written: {snapshot_path}")
    print(f"Mermaid diagram written: {diagram_path}")
    for provider in ["aws", "azure", "gcp"]:
        errors = snapshot["clouds"][provider]["errors"]
        if errors:
            print(f"[{provider}] warnings:")
            for error in errors:
                print(f"  - {error}")
    return 0


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Multicloud architecture mapper (MVP)")
    subparsers = parser.add_subparsers(dest="command", required=True)

    collect_parser = subparsers.add_parser(
        "collect", help="Collect normalized JSON snapshot"
    )
    collect_parser.add_argument(
        "--environment", default="staging", choices=["staging", "production"]
    )
    collect_parser.add_argument("--aws-region", default="ap-northeast-1")
    collect_parser.add_argument("--azure-region", default="japaneast")
    collect_parser.add_argument("--gcp-region", default="asia-northeast1")
    collect_parser.add_argument(
        "--output", default="docs/generated/architecture/snapshot.json"
    )
    collect_parser.set_defaults(func=collect_command)

    render_parser = subparsers.add_parser(
        "render", help="Render Mermaid from snapshot JSON"
    )
    render_parser.add_argument(
        "--input", default="docs/generated/architecture/snapshot.json"
    )
    render_parser.add_argument(
        "--output", default="docs/generated/architecture/architecture.mmd"
    )
    render_parser.add_argument(
        "--validate",
        action="store_true",
        help="Validate generated Mermaid node IDs and edges before writing output",
    )
    render_parser.set_defaults(func=render_command)

    all_parser = subparsers.add_parser("all", help="Collect and render in one command")
    all_parser.add_argument(
        "--environment", default="staging", choices=["staging", "production"]
    )
    all_parser.add_argument("--aws-region", default="ap-northeast-1")
    all_parser.add_argument("--azure-region", default="japaneast")
    all_parser.add_argument("--gcp-region", default="asia-northeast1")
    all_parser.add_argument(
        "--snapshot-output", default="docs/generated/architecture/snapshot.json"
    )
```

```
    all_parser.add_argument(
        "--diagram-output", default="docs/generated/architecture/architecture.mmd"
    )
    all_parser.add_argument(
        "--validate",
        action="store_true",
        help="Validate generated Mermaid node IDs and edges before writing output",
    )
    all_parser.set_defaults(func=all_command)

    return parser


def main() -> int:
    parser = build_parser()
    args = parser.parse_args()
    return args.func(args)


if __name__ == "__main__":
    raise SystemExit(main())
```

Chapter 9

設定ファイル

このセクションでは、プロジェクト全体の主要な設定ファイルを掲載します。

9.1 docker-compose.yml - ローカル開発環境

```yaml
services:
  # ========================================
  # FastAPI Backend (New Python implementation)
  # ========================================
  api:
    build:
      context: ./services/api
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - CLOUD_PROVIDER=local
      - CORS_ORIGINS=*
      - AUTH_DISABLED=true
      - DYNAMODB_ENDPOINT=http://dynamodb-local:8000
      - DYNAMODB_TABLE_NAME=simple-sns-local
      - AWS_ACCESS_KEY_ID=local
      - AWS_SECRET_ACCESS_KEY=local
      - AWS_DEFAULT_REGION=ap-northeast-1
      - MINIO_ENDPOINT=http://minio:9000
      - MINIO_ACCESS_KEY=minioadmin
      - MINIO_SECRET_KEY=minioadmin
      - MINIO_BUCKET_NAME=simple-sns
      - MINIO_PUBLIC_ENDPOINT=http://localhost:9000
      - LOG_LEVEL=INFO
    volumes:
      - ./services/api/app:/app/app
    depends_on:
      minio:
        condition: service_healthy
      dynamodb-local:
        condition: service_started
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  # ========================================
  # FastAPI SSR Web Frontend (Simple SNS)
```

```
  # ========================================
  frontend_web:
    build:
      context: ./services/frontend_web
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      - API_BASE_URL=http://api:8000
      - AUTH_PROVIDER=local
      - AUTH_DISABLED=true
    volumes:
      - ./services/frontend_web/app:/app/app
    depends_on:
      - api

  # ========================================
  # Reflex Web Frontend (New Python implementation)
  # ========================================
  frontend_reflex:
    build:
      context: ./services/frontend_reflex
      dockerfile: Dockerfile
    ports:
      - "3002:3002" # Frontend
      - "3003:3003" # Reflex backend (WebSocket)
    environment:
      - API_URL=http://api:8000
    volumes:
      - ./services/frontend_reflex:/app
      - /app/.web
    depends_on:
      - api

  # ========================================
  # Static Site + Simple SNS at /sns/
  # ========================================
  static_site:
    build:
      context: .
      dockerfile: static-site/Dockerfile
    ports:
      - "8090:8090"
    depends_on:
      - api

  # ========================================
  # React SPA Frontend (Vite + Tailwind, served via nginx)
  # ========================================
  frontend_react:
    build:
      context: ./services/frontend_react
      dockerfile: Dockerfile
      args:
        # Browser-side: uses host port (localhost:8000) not container hostname
        - VITE_API_URL=http://localhost:8000
    ports:
      - "3001:3001"
```

```
    depends_on:
      - api

  # ========================================
  # MinIO CORS setup (one-shot init container)
  # ========================================
  minio-setup:
    image: minio/mc:latest
    depends_on:
      minio:
        condition: service_healthy
    volumes:
      - ./minio-cors.json:/tmp/minio-cors.json:ro
    entrypoint: |
      /bin/sh -c "
        mc alias set local http://minio:9000 minioadmin minioadmin
        mc mb --ignore-existing local/images local/simple-sns 2>/dev/null || true
        mc anonymous set download local/images
        mc anonymous set download local/simple-sns
        mc cors set local/simple-sns /tmp/minio-cors.json 2>/dev/null ||
          aws --endpoint-url http://minio:9000 s3api put-bucket-cors \
            --bucket simple-sns --cors-configuration file:///tmp/minio-cors.json 2>/dev/null ||
          echo 'CORS setup skipped for simple-sns bucket'
        mc cors set local/images /tmp/minio-cors.json 2>/dev/null ||
          aws --endpoint-url http://minio:9000 s3api put-bucket-cors \
            --bucket images --cors-configuration file:///tmp/minio-cors.json 2>/dev/null ||
          echo 'CORS setup skipped for images bucket'
        echo 'MinIO setup complete'
      "
    restart: on-failure

  # ========================================
  # DynamoDB Local (NoSQL, mirrors AWS DynamoDB)
  # ========================================
  dynamodb-local:
    image: amazon/dynamodb-local:latest
    ports:
      - "8001:8000"
    command: "-jar DynamoDBLocal.jar -sharedDb -inMemory"

  # ========================================
  # MinIO (S3-compatible storage for local dev)
  # ========================================
  minio:
    image: minio/minio:latest
    ports:
      - "9000:9000"
      - "9001:9001" # Console
    environment:
      - MINIO_ROOT_USER=minioadmin
      - MINIO_ROOT_PASSWORD=minioadmin
    volumes:
      - minio-data:/data
    command: server /data --console-address ":9001"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 30s
      timeout: 20s
```

```
      retries: 3

networks:
  default:
    name: multicloud-network

volumes:
  minio-data:
```

9.2 Makefile - ビルドタスク

```
.PHONY: help install test-aws test-azure test-gcp test-all deploy-aws deploy-azure deploy-gcp build-frontend
     build-backend clean \
        hooks-install version version-major version-minor version-patch version-azure-afd-resolved

# Default target
help:
    @echo "Multi-Cloud Auto Deploy - Makefile"
    @echo ""
    @echo "Available targets:"
    @echo "  install        - Install all dependencies (frontend + backend)"
    @echo "  build-frontend - Build frontend application"
    @echo "  build-backend  - Package backend for Lambda (x86_64)"
    @echo "  test-aws       - Test AWS deployment"
    @echo "  test-azure     - Test Azure deployment"
    @echo "  test-gcp       - Test GCP deployment"
    @echo "  test-all       - Test all cloud deployments"
    @echo "  deploy-aws     - Deploy to AWS via CLI"
    @echo "  terraform-init - Initialize Terraform for AWS"
    @echo "  terraform-plan - Plan Terraform changes for AWS"
    @echo "  terraform-apply - Apply Terraform changes for AWS"
    @echo "  clean          - Clean build artifacts"
    @echo ""
    @echo "Version management:"
    @echo "  hooks-install              - git hooks を有効化 初回セットアップ時に実行()"
    @echo "  version                    - 現在のバージョン一覧を表示"
    @echo "  version-major COMPONENT=<name|all>  - メジャー(X)を手動インクリメント"
    @echo "  version-minor COMPONENT=<name|all>  - マイナー(Y)を手動インクリメント"
    @echo "  version-patch COMPONENT=<name|all>  - パッチ(Z)を手動インクリメント"
    @echo "  version-azure-afd-resolved - Azure AFD 解消後: 0.9.x → 1.0.0"
    @echo ""
    @echo "Components: aws-static-site  azure-static-site  gcp-static-site  simple-sns  all"

# Install dependencies
install:
    @echo "⊠ Installing frontend dependencies..."
    cd services/frontend && npm install
    @echo "⊠ Installing backend dependencies..."
    cd services/backend && python3.12 -m venv .venv && \
        .venv/bin/pip install --upgrade pip && \
        .venv/bin/pip install -r requirements.txt
    @echo "⊠ Dependencies installed"

# Build frontend
build-frontend:
    @echo "⊠  Building frontend..."
```

```
    cd services/frontend && \
        VITE_API_URL=https://52z731x570.execute-api.ap-northeast-1.amazonaws.com npm run build
    @echo "☒ Frontend built in services/frontend/dist/"

# Build backend (x86_64 for Lambda)
build-backend:
    @echo "☒  Building backend for AWS Lambda (x86_64)..."
    cd services/backend && \
        rm -rf package lambda.zip && \
        mkdir -p package && \
        pip install --platform manylinux2014_x86_64 --only-binary=:all: \
            --target package --python-version 3.12 --implementation cp \
            fastapi==0.109.0 pydantic==2.5.3 python-dotenv==1.0.0 mangum==0.17.0 && \
        cp -r src/* package/ && \
        cd package && zip -r ../lambda.zip . -q
    @echo "☒ Backend packaged in services/backend/lambda.zip"

# Test deployments
test-aws:
    @echo "☁  Testing AWS deployment (ap-northeast-1)..."
    @curl -s https://52z731x570.execute-api.ap-northeast-1.amazonaws.com/ | jq .

test-azure:
    @echo "☁  Testing Azure deployment (japaneast)..."
    @curl -s https://mcad-staging-api--0000003.livelycoast-fa9d3350.japaneast.azurecontainerapps.io/ | jq .

test-gcp:
    @echo "☁  Testing GCP deployment (asia-northeast1)..."
    @curl -s https://mcad-staging-api-son5b3ml7a-an.a.run.app/ | jq .

test-all: test-aws test-azure test-gcp
    @echo "☒ All deployments tested"

# Deploy to AWS
deploy-aws: build-backend
    @echo "☒ Deploying to AWS..."
    @cd services/backend && \
        aws lambda update-function-code \
            --function-name multicloud-auto-deploy-staging-api \
            --region ap-northeast-1 \
            --zip-file fileb://lambda.zip \
            --query 'FunctionName' --output text
    @echo "☒ AWS Lambda updated"

deploy-frontend-aws: build-frontend
    @echo "☒ Deploying frontend to AWS S3..."
    @aws s3 sync services/frontend/dist/ s3://multicloud-auto-deploy-staging-frontend/ --delete
    @aws cloudfront create-invalidation \
        --distribution-id E2GDU7Y7UGDV3S \
        --paths "/*" \
        --query 'Invalidation.Id' --output text
    @echo "☒ Frontend deployed to S3 and CloudFront invalidated"

# Terraform commands
terraform-init:
    @echo "☒ Initializing Terraform..."
    cd infrastructure/terraform/aws && terraform init

```

```
terraform-plan:
    @echo "⊠ Planning Terraform changes..."
    cd infrastructure/terraform/aws && \
        terraform plan -var="environment=staging" -var="project_name=multicloud-auto-deploy"

terraform-apply:
    @echo "⊠ Applying Terraform changes..."
    cd infrastructure/terraform/aws && \
        terraform apply -var="environment=staging" -var="project_name=multicloud-auto-deploy" -auto-approve

# Clean
clean:
    @echo "⊠ Cleaning build artifacts..."
    rm -rf services/frontend/dist
    rm -rf services/frontend/node_modules
    rm -rf services/backend/package
    rm -rf services/backend/lambda.zip
    rm -rf services/backend/.venv
    @echo "⊠ Clean complete"

# ==========================================================
# バージョン管理
# ==========================================================

# git hooks を有効化 初回セットアップ時に実行()
hooks-install:
    @echo "⊠ git hooks を .githooks に設定..."
    git config core.hooksPath .githooks
    chmod +x .githooks/pre-commit
    chmod +x scripts/bump-version.sh
    @echo "⊠ git hooks 有効化完了"
    @echo "   コミット時に自動でパッチ(Z)をインクリメントします"
    @echo "   プッシュ時は GitHub Actions がマイナー(Y)をインクリメントします"

# 現在のバージョン一覧
version:
    @chmod +x scripts/bump-version.sh
    @./scripts/bump-version.sh show

# メジャー(X)を手動インクリメント
# 使用例: make version-major COMPONENT=all
#         make version-major COMPONENT=aws-static-site
COMPONENT ?= all
version-major:
    @chmod +x scripts/bump-version.sh
    @./scripts/bump-version.sh major $(COMPONENT)
    @echo "⊠ 次のコマンドでコミットしてください:"
    @echo "   git add versions.json && git commit -m 'chore: bump major version [skip-version-bump]'"

version-minor:
    @chmod +x scripts/bump-version.sh
    @./scripts/bump-version.sh minor $(COMPONENT)
    @echo "⊠ 次のコマンドでコミットしてください:"
    @echo "   git add versions.json && git commit -m 'chore: bump minor version [skip-version-bump]'"

version-patch:
    @chmod +x scripts/bump-version.sh
    @./scripts/bump-version.sh patch $(COMPONENT)
```

```
    @echo "☒ 次のコマンドでコミットしてください:"
    @echo "   git add versions.json && git commit -m 'chore: bump patch version [skip-version-bump]'"

# Azure AFD 解消後に 0.9.x → 1.0.0 へ昇格
version-azure-afd-resolved:
    @chmod +x scripts/bump-version.sh
    @./scripts/bump-version.sh azure-afd-resolved
```

9.3 .gitignore - Git除外設定

```
# Node modules
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
ENV/
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg
package/

# Terraform
.terraform/
*.tfstate
*.tfstate.*
.terraform.lock.hcl
tfplan

# Pulumi
.pulumi/
Pulumi.*.yaml

# IDE
.idea/
*.swp
```

```
*.swo
*~
# Commit VS Code project config; ignore personal/machine-specific overrides
.vscode/*
!.vscode/settings.json
!.vscode/extensions.json
!.vscode/tasks.json
!.vscode/launch.json

# OS
.DS_Store
Thumbs.db

# Environment
.env
.env.local
.env.*.local

# Build outputs
dist/
build/
*.zip

# Local archive / backup
archive/
!docs/archive/
lambda.zip
lambda_placeholder.zip

# Logs
logs/
*.log

# Secrets
secrets/
*.pem
*.key
service-account-key.json
services/api/.build/
services/frontend_react/.env.production
.venv/
.deployment/
```